

SUCA 104

**UNDERGRADUATE COURSE
BCA - COMPUTER APPLICATIONS**

FIRST YEAR

SECOND SEMESTER

CORE PAPER - III

PROGRAMMING IN C



**INSTITUTE OF DISTANCE EDUCATION
UNIVERSITY OF MADRAS**

WELCOME

Warm Greetings.

It is with a great pleasure to welcome you as a student of Institute of Distance Education, University of Madras. It is a proud moment for the Institute of Distance education as you are entering into a cafeteria system of learning process as envisaged by the University Grants Commission. Yes, we have framed and introduced Choice Based Credit System(CBCS) in Semester pattern from the academic year 2018-19. You are free to choose courses, as per the Regulations, to attain the target of total number of credits set for each course and also each degree programme. What is a credit? To earn one credit in a semester you have to spend 30 hours of learning process. Each course has a weightage in terms of credits. Credits are assigned by taking into account of its level of subject content. For instance, if one particular course or paper has 4 credits then you have to spend 120 hours of self-learning in a semester. You are advised to plan the strategy to devote hours of self-study in the learning process. You will be assessed periodically by means of tests, assignments and quizzes either in class room or laboratory or field work. In the case of PG (UG), Continuous Internal Assessment for 20(25) percentage and End Semester University Examination for 80 (75) percentage of the maximum score for a course / paper. The theory paper in the end semester examination will bring out your various skills: namely basic knowledge about subject, memory recall, application, analysis, comprehension and descriptive writing. We will always have in mind while training you in conducting experiments, analyzing the performance during laboratory work, and observing the outcomes to bring out the truth from the experiment, and we measure these skills in the end semester examination. You will be guided by well experienced faculty.

I invite you to join the CBCS in Semester System to gain rich knowledge leisurely at your will and wish. Choose the right courses at right times so as to erect your flag of success. We always encourage and enlighten to excel and empower. We are the cross bearers to make you a torch bearer to have a bright future.

With best wishes from mind and heart,

DIRECTOR

BCA, COMPUTER APPLICATIONS
FIRST YEAR
SECOND SEMESTER

CORE PAPER - III
PROGRAMMING IN C

COURSE WRITER & EDITING

Dr. S. Sasikala

Assistant Professor in Computer Science
Institute of Distance Education
University of Madras
Chepauk Chennai - 600 005.

BACELLOR OF COMPUTER APPLICATIONS

FIRST YEAR

SECOND SEMESTER

Core Paper - III

PROGRAMMING IN C

SYLLABUS

Unit 1

Fundamental Character set - Identifier and keywords - data types - constants - Variables - Declarations - Expressions - Statements - Arithmetic, Unary, Relational and logical, Assignment and Conditional Operators - Library functions.

Unit-2

Data input output functions - Simple C programs - Flow of control if, if else, while, do-while, for loop, Nested control structures - Switch, break and continue, go to statements - Comma operator.

Unit 3

Functions –Definition - proto-types - Passing arguments – Recursions- Storage Classes - Automatic, External, Static, Register Variables – Multi-file programs.

Unit-4

Arrays - Defining and Processing - Passing arrays to functions – Multi-dimension arrays - Arrays and String. Structures - User defined data types - Passing structures to functions - Self-referential structures – Unions - Bit wise operations.

Unit-5

Pointers-Declarations- Passing pointers to Functions - Operation in Pointers - Pointer and Arrays - Arrays of Pointers - Structures and Pointers – Files- Creating, Processing, Opening and Closing a data file.

1. Recommended Texts:

1. E.Balaguruswamy, 1995,Programming in ANSI C, TMH Publishing Company Ltd.

2. Reference Books:

- i) H. Schildt, 2004, The Complete Reference, 4th Edition, TMH
- ii) Gottfried,B.S, 1996, Programming with C, Second Edition, TMH Pub. Co. Ltd., New Delhi
- iii) Kanetkar Y,1999, Let us C, BPB Publications., New Delhi.
- iv) Kamthane,2002,Programming with ANSI & Turbo C , First Edition,Pearson Education, New Delhi

BACELLOR OF COMPUTER APPLICATIONS

FIRST YEAR

SECOND SEMESTER

Core Paper - III

PROGRAMMING IN C

SCHEME OF LESSONS

Sl.No.	Title	Page
1.	Introduction to C	001
2.	Variables and Expression	017
3.	Operators and Expressions	052
4.	Basic Input-Output Statements	075
5.	Control structures – I Decision Making Statements	101
6.	Control structure – II Looping statements	115
7.	Preprocessor	156
8.	Functions	165
9.	Local and Global Variables	223
10.	Arrays	243
11.	Strings	269
12.	Pointers	284
13.	Advanced Concepts of Pointers	312
14.	File Handling	361

LESSON - 1

INTRODUCTION TO C

Structure

- 1.1 Introduction
- 1.2 Learning Objective
- 1.3 Generality
- 1.4 Introduction to C language
- 1.5 Summary
- 1.6 Check Your Answers
- 1.7 Model Questions

1.1 Introduction

All the programming languages can be divided into two categories (i) High level languages: (ii) low level language. But C stands in between these two categories. That's why it is often called a Middle level language, since it was designed to have both a relatively good programming efficiency and a relatively good machine efficiency.

1.2 Learning Objectives

After learning this lesson, you should be able to

- Generality
- Introduction to C language

1.3 Generality

A Computer is a sophisticated tool comprising of hardware and software elements. The key constituents of the computer hardware are:

- * **Arithmetic-logic Unit**

To perform calculations and comparisons

- * **Control Unit**

To ensure that the given sequence of operations are performed

2

- * **Memory Unit**

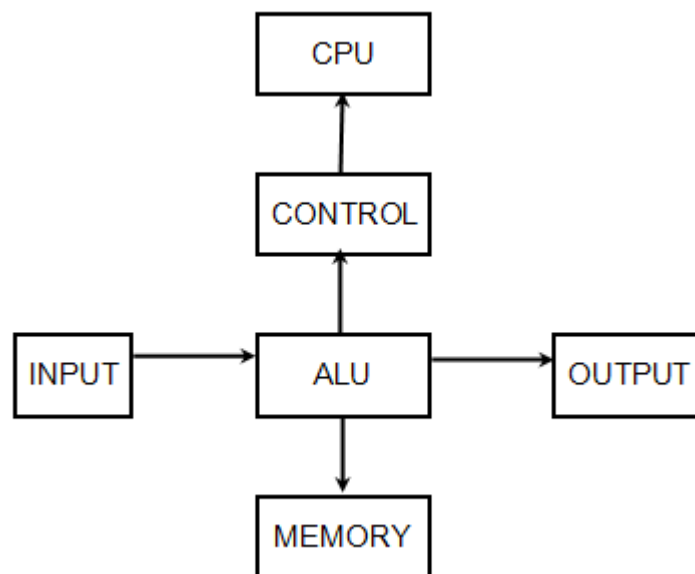
To store programs, input data, intermediated and final results

- * **Input Unit**

To obtain input data

- * **Output Unit**

To provide final results as output



A Computer program or software is a sequence of operations performed on the given input data to produce desired output. Any notation to represent the data and describe the sequence of operations on it is called **a programming language**. The stored program model of computing or Von Newumann computing methodology stipulates the entire program and the data to be placed in the main memory before the CPU starts executing the program.

Ever since the inception of computers, hundreds of programming languages have been designed and implemented. Most programmers never use more than a few languages.

However, a study of the broad concepts, underlying programming language design helps a programmer.

- To enhance the understanding of languages known.
- To easily remember useful programming constructs.
- To choose an appropriate language for a given application development.
- To make learning and designing new languages easier.

The design and implementation of programming languages has undergone rapid evolution. There are several aspects that determine the preference of programmers.

Some of the pertinent features of popular languages are:

- Clarity.
- Simplicity.
- Clear language objectives.
- Simple sentence forms.
- Ease of program verification.
- Specific supports for the application.
- Provisions for hiding mundane details.
- Good facilities to correct errors.
- Clear and simple documentation.
- Capability to execute the programs across different systems.
- Standards compliance.
- Optimal cost of execution, creation, translation and maintenance.

Those languages that are suitable for a wide range of applications are called general purpose languages. Specific purpose languages are tailor made for particular application domains. These languages have notations that cater to domain specific applications. Hardware Description Languages used to describe electronic circuits are examples for specific purpose languages. This lesson focuses on general purpose programming languages.

The design of programming languages is influenced by the following factors:

- Hardware
- Operating environment.
- Applications.
- Models for effective expression of the sequence of operations programming paradigms.
- Theoretical basis including formal proofs for verification.
- Standardisation.

Table 1.1 traces evolution of programming languages highlighting the major influences at each stage.

<i>Time frame</i>	<i>Influence</i>
1950-1955	Primitive computers, Assembly and machine languages, Subprograms, Data structures, Possibility of level languages
1956-1960	Small, Slow and Expensive computers. mass storage devices, Compilers, Interpreters code optimization Dynamic Storage management, Notations for grammars, List processing
1961-1965	Large Expensive computers, Magnetic disk mass storage, Operating systems, Multi-programming, Syntax directed compilers. Emphasis on general purpose languages
1966-1970	Instruction sets for computers with varying Size, Speed, expensive mass storage, Time sharing, Interactive operative environment, Micro programming, Optimizing compilers, Standard languages flexibility.
1971-1975	Microcomputers, Small, Cheap mass storage, Programming, Software engineering.
1976-1980	Powerful, Cheap computers, Distributed systems, Embedded systems, Concurrent programming, Formal semantics, Reliability
1981-1985	Database environments, Focus on Artificial object oriented programming

Table 1.1: Major influences on programming Languages

Today programming languages cater to wide ranging operating environment. Table 1.2 Outlines the influence of Operating Environment on language design.

<i>Factor</i>	<i>Batch Systems</i>	<i>Interactive systems</i>	<i>Embedded Systems</i>
Input/output	Sequential Files	Standard Files	Special Devices
Error Handling	Re-run	Good Diagnosis	Recovery is Vial
Timing Constraints	None	Good Response	Stringent (Real Time)
Structure	Main block	Collection of functions	Concurrent Blocks

Table 1.2: Influence of Operating Environment

The distributed and real-time environments are categorized under embedded systems. The software industry witnessed a rapid increase in the complexity of software problems. Today, thousands of programmers are needed to solve a given problem. As a consequence, different models for expressing the solution have been evolved. The object-oriented methodology caters to large programming have been evolved. The object-oriented methodology caters to large programming efforts. Also, special considerations play an important role in designing languages for interacting databases.

When a Central Processing Unit (CPU) comes off the assembly line, the built-in capabilities are arithmetic and logical operations, input and output and some control functions. A corresponding set of instructions is also supported. It is to be noted that every brand of CPU can have a distinct set of instructions.

A programming language is a systematic notation 'by which computational process are described. 'The computational process is a sequence of operations which a machine can perform for solving the problem at hand. Languages are classified into generations depending on the sophistication they offer to the programmer in expressing his solution.

1) First Generation Languages

These languages also known as machine languages are closest to the machine interpretable form. The interpretable of the machine was represented using binary codes called 'opcodes.' The data was required by the instruction is called an operand'. There are a few

standard formats to specify the requisite operand (s). The operands are also expressed as string of binary sequences.

A typical machine language instruction is thus a sequence of bits. It comprises of parts namely 'opcode' and 'operands'. The operands are usually address locations in the main memory internal registers of the CPU. Certain types of instructions permit the user to specify atleast one operand along with the opcode. A programmer expresses the sequence of operations as machine instructions. Each such instruction can be interpreted only by the computer.

Programming and usage of the machine was only for people well versed with the machine. Moreover it was highly error prone and tedious. For example a Machine language segment is as follows:

Address	Opcode	Operands	
0100	01010000		
0101	01010010		
0102	10110100	00001001	
0104	10110100	00000001	00001101
0107	11001101	00100001	
0109	01011010		

2) Second Generation Languages

It was a feeble attempt in making programming friendly. The 'opcode and the operands are represented as pre-specified symbolic names. These names which look like English words are called mnemonics. Every mnemonic has a one-to-one correspondence to the machine instructions (opcodes).

The operands are specified using symbolic addresses. A software program called assembler converts the program in assembly language into machine language. The mnemonics can be unique for a given brand of the CPU. For example an language segment is as follows:

Mnemonic	Operands
PUSH	AX
PUSH	DX
MOV	AH,9
MOV	DX, OFFSET
INT	21H

3) Third Generation Languages

These are usually called high level languages. Ever since the inception of computers, efforts were on to design programming language that resemble natural language. These language would permit ease of learning, natural ability to express solutions and team efforts. Moreover, they are hardware independent. A compiler or an interpreter translates the programs into machine language. Every high level language statement is translated into a sequence of machine language instructions. Some of the popular third generation languages are BASIC, FORTRAN, COBOL, PASCAL, C, ADA, C++ etc.

Every high level language programme is also a sequence of statements. Each statement has a specific format or syntax. The grammar of a language is a notation to specify the syntax of various legal sentences of the language. The meaning of the sentence in terms of the machine instruction sequence is called 'semantics' of the statement.

A compiler is a program to translate the entire sequence of statements into the corresponding machine code or object code. Optimization of object code is possible using compilers. Also once the object code is generated the entire memory is for this code only.

An interpreter translates the program, statement by statement. It is present in the main memory at run time also. It is extremely efficient to implement languages 'requiring sophisticated run time environment support.

Third generation languages opened up the area of computer programming to a vast multitude of programmers. Several advanced facilities to make programming easier were incorporated into high level languages. They are designed with the focus on the end-user to

the programmer. Standard compiler writing tools and techniques ensured that they get correctly translated into the machine code.

4) Fourth Generation Languages

Interactions with databases are typically in the form of short and specific enquiries. A standard sequence of operations are clubbed together in the form of a statement called 'Query'. A collection of such query formats is called a query languages. As the target applications are database oriented these languages have nice applications base.

5) Fifth Generation Languages

The problem in application domains such as Intelligence are complex and require special supporting features to efficiently express the solutions. Fifth generation languages are designed to cater to these needs. Some popular fifth generation languages are LISP, PROLOG, SMALLTALK etc.

6) Database Methodology

Third generation languages demand considerable time to develop proficiency in that language. The inertia in learning a new language and the high costs of retraining are often very prominent drawbacks. Moreover, they do not the aid solution to the problem. Owing to the complex concepts and syntax the computing power remains accessible to professional programmers.

The reduction in price of computer systems and availability of inexpensive mass storage mechanisms paved way for extensive database applications. These databases form the life-line of organizations across the globe.

A good database management system permit the user to

- CREATE a database
- ADD new data
- ORGANIZE the database into some meaningful order
- MODIFY data

- INSERT records at specified places
- REMOVE irrelevant data
- GENERATE reports
- SEARCH the database for specific information
- INT*TEGRITY checks on the data
- SECURITY of access

Interactions with database are typically in the form of short and specific enquiries. A standard sequence of operations are clubbed together in the form of a statement called 'Query'. Often the types of queries are also innumerable. A collection of such query formats is called a Query language. An easy interface called the front-end of a database helps naive users in constructing queries.

The languages confining to this paradigm are the fourth generation languages. The Chief characteristics of these languages are as follows:

- Non-procedural code
- Graded levels of user friendliness
- Self teaching software
- Interactive dialogues which clearly specify possible choices at every step.
- Limited functionality and quickly operational
- Fully and consistently defined default options
- Niche applications base
- Simplicity of query formats
- Easy report generation facilities
- Support for generation facilities
- Interfaces to other languages paradigms to harness their capabilities
- Automated documentation

- Extensive re-usable modules
- Fast prototyping
- Standards conformance
- Verifiable data models
- Simple code structures
- Automatic code structures
- Ease of maintenance

Check Your Progress - I

- 1) State whether the following are True or False.
 - a) A prescribed set of well - defined rules or processes for the solution of a problem in a series of step is Flow Chart.
 - b) A program written in machine language is called Object Program.
- 2) Fill in the blanks
 - a) _____ is a program to translate the entire sequence of statements into the corresponding machine code or Object code.
 - b) _____ translates the program, statement by statement.
 - c) C was originally developed in 1970's by _____ .

1.4 Introduction to C languages

C is a general-purpose structured programming language. Its instructions consist of terms that resemble algebraic expressions, augmented by certain English keywords such as if, else, for, do and while. In this respect C resembles other high-level structured programming languages such as Pascal and FORTRAN-77. C also contains certain additional features, however that allow it to be used at a lower level, thus bridging the gap between machine language and the more conventional high-languages

This flexibility allows C to be used for systems programming (e.g. for writing operating systems) as well as for applications programming (e.g. for writing a program to solve a complicated system of mathematical equations, or for writing a program to bill customers)

C is characterized by the ability to write very concise source programs, due to the large number of operators included within the language. It has a relatively small instruction set, though actual implementations include extensive library functions which enhance the basic instructions. Furthermore, the language encourages users to write additional library functions of their own. Thus, the features and capabilities of the language can easily be extended by the user.

C compilers are commonly available for computers of all sizes, and C interpreters are becoming increasingly common. The compilers are usually compact, and they generate object programs that are small and highly efficient when compared with programs compiled from other high-level languages. The interpreters are less efficient, though they are easier to use when developing a new program. Many programmers begin with an interpreter, and then switch to a compiler once the program has been debugged (once all of the programming errors have been removed).

Another important characteristic of C is that its programs are highly portable, even more so than with other high-level languages. The reason for this is that C relegates most computer dependant features to its library functions. Thus, every version of C is accompanied by its own set of library functions, which are written for the particular characteristics of the host computer. These library functions are relatively standardized. However, each manner from one version of C to another. Therefore, most C programs can be processed on many different computers with little or no alteration.

1) History of C

C was originally developed in the 1970's by Dennis Ritchie at Bell Telephone Laboratories. It is an outgrowth of two earlier languages, called BCPL and B, which were also developed at Bell Laboratories. C was largely confined to use within Bell Laboratories until 1978, when Brian Kernighan and Ritchie published a definitive description of the language (K&RC)

Following the publication of the K&R description, computer professionals, with C's many describable features, began to promote the use of the language. By the mid 1980's the popularity of C had become widespread. Numerous C compilers and interpreters had been written for

computers of all sizes and many commercial application programs had been developed. Moreover, many commercial software products that were originally written in other languages were rewritten in C in order to take advantage of its efficiency and its portability.

Most commercial implementations of C differ somewhat from Kernigham and Ritchie's original definition. This has created some minor incompatibilities among different implementations of the language, thus diminishing the portability that the language attempts to provide. Consequently, the American National Standards Institute (ANSI) has begun work on a standardized definition of the C language. Most commercial C compilers and interpreters are expected to adopt the ANSI standard once it has been completed. They may also provide additional features of their own.

2) Structure of a C Program

Every C program consists of one or more functions, one of which must be called main. The program will always begin by executing the main function. Additional function definitions may precede or follow main.

Each function must contain the following:

- (a) A function heading, which consists of the function name, followed by an optional list of argument enclosed in parentheses.
- (b) A list of argument declarations, if argument are included in the heading.
- (c) A compound statement, which comprises the remainder of the function.

The arguments are symbols that represent information being passed between the function and other parts of the program. (Arguments are also referred to as parameters).

Each compound statement is enclosed within a pair of braces, i.e.{and}. The braces may contain combination of elementary statements (called expression statements) and other compound statements. Thus, Compound statements may be nested, one within another. Each expression statement must end with a semicolon(;).

Comments may appear anywhere within a program, as long as they are placed within delimiters /*and*/

e.g. `/* this is a comment */`

such statements are helpful in identifying the program's principal features or in explaining the underlying logic of various program features.

These program components will be discussed in much greater detail later.

Example:

This is an elementary C program that reads the temperature in centigrade and converts it into Fahrenheit.

```
#include (stdio.h>                                /* LIBRARY FILE ACCESS */

/*PROGRAM TO CONVERT TEMPERATURE                   /*COMMENT LINE1*/

/* IN CENTIGRADE TO FAREAHELIT                     /* COMMENT LINE 2*/

main ( )                                           /* FUNCTION HEADING*/

float c, f;                                       /*VARIABLE DECLARATIONS*/

print f ("Enter the temperature in centigrade /n"); /* OUTPUT STATEMENT*/
```

The following features should be pointed out in the above program.

1. The program is typed in lower case. Most comments are typed in upper case.
2. The first line contains a reference to a special file (called `stdio.h`) which must be included in the program when it is compiled.
3. The second line is comment that identifies the purpose of the program.
4. The third line is a heading for the function `main`. The empty parentheses following the name of the functions indicate that this function does not include any arguments.
5. The remaining five lines of the program are indented and enclosed within a pair of braces. These five lines comprise the compound statement within `main`.
6. The first indented line is a variable declaration. It establishes the symbolic names `c` and `f` as floating point variables which are used to represent centigrade and Fahrenheit.

7. The remaining four indented lines are expression statements. The second indented line (printf) generates a request for information (namely centigrade).
8. The fourth indented line is a particular type of expression statement called an assignment statement. This statement cause the conversion of temperature in centigrade to temperature in fareheit.
9. The last indented line (printf) causes the calculated value for the fareheit to be displayed. The numerical value will be preceded by a brief label.
10. Note that each expression statement within the compound statement ends with a semicolon. This is required of all expression statements.
11. Finally, the liberal use of spacing and indentation separate different parts of the program into logically identifiable components, and the indentation indicates subordinate relationships among various instructions. These features are not grammatically essential but their presence is strongly encourages as a matter of good programming practice.

Execution of the program results in an interactive dialog such as that shown below:

Enter temperature in centigrade 10.0

Farenheit equivalent is 50.0

3) Desirable Program Characteristics

Before concluding this lesson let us briefly examine some important characteristics of well-written computer programs. These characteristics apply to programs that are written in any programming language. They can provide us with a useful set of guidelines.

1. **Integrity.** This refers to the accuracy of the calculations. It should be less if the calculations are not carried out correctly. Thus the integrity of the calculations is an absolute necessity in any computer program.
2. **Clarity.** This refers to the overall readability of the program, with particular emphasis on its underlying logic. If a program is clearly written, it should be possible for another programmer to follow the program logic without undue effort. It should also be possible for the original author to follow his or her own program after being away from the objectives in the design of C is the development of clear, readable programs through an orderly and disciplined approach to programming.

3. **Simplicity.** The clarity and accuracy of a program are usually enhanced by keeping things as simple as possible, consistent with the overall program objectives. In fact, it may be desirable to sacrifice a certain amount of computational efficiency in order to maintain a relatively simple, straight forward program structure.
4. **Efficiency.** This is concerned with execution speed and efficient memory utilization. These are generally important goals, though they should not be obtained at the expense of clarity or simplicity. Many complex programs require a trade off between these characteristics. In such situations, experience and common sense are key factors.
5. **Modularity.** Many programs can be broken down into a series of identifiable subtasks. It is good programming practice to implement each of these subtasks as a separate program module. In C, such modules are written as functions. The use of a modular programming design enhances the accuracy and clarity of a program, and it facilitates future program alterations.
6. **Generality.** Usually we want a program to be as general as possible, within reasonable limits. For example, we may design a program to read in the values of certain key parameters rather than placing fixed values into the program. As a rule, a considerable programming effort.

Check Your Progress - II

- 1) State whether the following are True or False.
 - a) Every C program consists of one or more functions, one of which must be called main.
 - b) C is a case sensitive language.
- 2) Fill in the blanks:
 - a) Each compound statement is enclosed within _____ .
 - b) Every C program must contain a _____.
 - c) A main () function in a C program should be located _____.
 - d) Statement in a C program should begin _____ .

1.5 Summary

C is a general - purpose structured programming language. Its instructions consist of terms that resemble algebraic expression, augmented by certain English keywords such as if, else, for, do and while. 'C' to be used for systems programming (as for writing operating systems) as well as for applications programming (e.g. for writing a program to solve a complicated systems of mathematical equations).

1.6 Check Your Answers

- I. 1) a) False b) True
- 2) a) Compiler b) Interpreter c) Dennis Ritchie
- II. 1) a) True b) True
- 2) a) a pair of braces b) main () function
- c) anywhere in the program d) in any column

1.7 Model Questions

- (1) Explain the main components of a computer?
- (2) Discuss the major influences on programming languages in different time frames?
- (3) Explain the structure of a C language with an illustration?
- (4) What are the desirable program characteristics that enhance the program development effectively?

LESSON - 2

VARIABLES AND EXPRESSION

Structure

- 2.1 Introduction
- 2.2 Learning Objectives
- 2.3 Character and variables
- 2.4 Constants
- 2.5 Summary
- 2.6 Check Your Answers
- 2.7 Model Questions

2.1 Introduction

In c, a quantity which may vary during program execution is called a variable. Variable names are names given to the locations in memory of a computer where different constant are stated. There locations can contain character, integer, real or any such constant.

2.2 Learning Objectives

After leaning this lesson, you should be able to

- Character set and variables
- Constants.

2.3 Character Set and Variables

The C character set consists of upper and lower case alphabets digits, special characters and white spaces. The alphabets and digits are together called the alphanumeric characters.

Alphabets:

A B C D E F X Y Z

a b c d e f x y z

Digits:

0 1 2 3 4 5 6 7 8 9

Special Characters:

,	comma	<	opening angle bracket	>	closing angle bracket
.	period	-	under score	(	left parenthesis
;	semicolon	\$	dollar sign	)	right parenthesis
:	colon	%	percent sign	[	left bracket
#	number	?	question mark sign	]	right bracket
'	apostrophe	&	ampersand	{	left brace
"	quotation	^	caret mark	}	right brace
!	exclamation	*	asterisk mark	/	slash
	vertical bar	-	minus sign	\	backslash
~	tide	+	plus sign		

White space characters:

blank space	newline	carriage return
formfeed	horizontal tab	vertical tab

1) Identifiers and Keywords

In a C program, every word is either classified as an identifier or a keyword. Identifiers, as the name suggests, are used to identify or name variables, symbolic constants, functions, etc. Keywords have predefined meanings and cannot be changed by the user. There are certain rules regarding identifier names, as enumerated below:

- * Identifier names must be a sequence of letters and digits, and must begin with a letter
- * The underscore character ('_') is considered a letter.
- * Name should not be the same as a keyword (such as int, char, void, if, for, while, etc.). Keywords are predefined by the language, and cannot be used by the programmer in any

way other than that specified by the syntax. The following keywords are predefined by the C language.

auto	double	int	struck
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsinged
continue	for	-singed	void
default	goto	sizeof	volatile
do	if	static	while

- * C is case sensitive (i.e. upper and lower case letters are treated differently). Thus, the names rate, Rate and RATE denote different identifiers. It is a general practice to use lower or mixed case for variable and functions names, and upper case for symbolic constants.
- * For any internal identifier name (an identifier declared in the same file or in a header included in the current file), atleast the first 31 characters are significant in any ANSI C, C compiler (it may be more in certain compilers, but never less)

2) Variables

A variable is an entity that has a value and is known to the program by a name. A variable definition associates a memory location with the variable name. A variable can have only one value assigned to it at any given time during the execution of the program. Its value may change during the execution of the program.

Example 2.1: $f = 1.8 * c + 32$

In the above example, f and c are variables.

Variable names are identifiers used to name variables. They are symbolic names given to memory locations. A variables name consists of a sequence of letter and digits, the first

character being a letter. The rules that apply to identifiers (given above) also apply to variable names. Examples of valid variable names are:

i rank MAX min class-mark
 student-name emp-num fact-recur Mark studentName

Examples of invalid variable names are:

a's illegal character (')
 fact recur blank not allowed
 5root first character should be a letter
 student, rec comma not allowed

3) Basic Data Types

The C language supports the following basic data types

char a single byte that can hold one character

int an integer

float a single precision floating point number

double a double precision floating point number

Note: Precision refers to the number of significant digits after the decimal point.

Further, applying qualifiers to the above data types yield additional data types. A qualifier alters the characteristics of the data type, such as its size or sign. The qualifiers that alter the size are short and long. These qualifiers are applicable to integers, and yield two more data types:

short int integer represented by a lesser number of bits (Usually 16)

long int integer represented by a greater number of bits (Usually 32)

The exact sizes of these data types depend on the compiler. Details are given later in this lesson. The qualifiers long can also be used along with double precision floating point type:

long double an extended precision floating point number

The sign qualifiers are signed and unsigned. The sign qualifiers are combined with the integer data types (int, short int and long int) resulting in six more data types given below:

signed short int

unsigned short int

signed int

signed long int

unsigned long int

These are also applicable to the char data type

signed char

unsigned char

Normally, size qualifiers (short and long) cannot be applied to the data types char and float, and sign qualifiers (signed and unsigned) cannot be applied to float, double and long double.

4) Declare Variables

In C, you always have to declare a variable before using it. Before going into the details of declaring and using the variable types, we need to understand how to input and output simple strings. The following example illustrates the use of printf to output an integer.

/*integer. c: An example illustrating an integer output. ***/**

```
# include <stdio.h >
```

```
void main ()
```

```
{int val;
```

```
val = 10;
```

```
printf ("%d", val);}
```

The output of the program is as follows: 10

The program is now explained line by line: `int val;` is the first statement in the function `main ( )`. it declares the identifier `val` to be a variable of integer type. In declare a variable of some type. Precede its name of the data type (float, char and so on). For instance, if a variable `f_val` is to be declared a floating point number, then declare `f_val` as:

```
float f_val;
```

Coming back to the program, having declared `val` an integer, assign a value to it with the statement:

```
val = 10;
```

Before this statement is executed, the value of `val` is undefined.

Let us now examine the third statement inside the function `main ( )`:

```
printf ( "%d", val);
```

This outputs, the value of the integer variable `val`. In the discussion that follows, we shall use the word parameter to refer to any variable or constant that is passed to a function. Thus, in the `printf` statement above, the first parameter is `"%d"` and the second parameter is `val`.

The first parameter passed to the function `printf` is always a string called the format string enclosed in double quotes. In the above case `"%d"` is the format string. It tells `printf` that the output is an int. The second parameter `val` provides the actual value to output.

Similarly, `%f` must be used as the format string to output floating point numbers, `%c` for single characters, and so on. All the format types are discussed in detail in the next lesson. Inside the format string, the `%` sign followed by a letter informs `printf` about the type of the parameter to be output. But the contents of the format string are not restricted to this kind of type-specifies. Let us examine the previous example again, this time adding something more to the message.

```
/* float.c: An example illustrating integer and floating point output. */
```

```
# include <stdio.h>
```

```
void main ( )
```

```

{int i_var;

float f_var;

i_var = 10;

f_var = 100.3125;

printf ("i_var is %d and f_var is %f", i_var, f_var);}

```

The output of this program is :

```
i_var is 10 and f_var is 100.3125
```

From the program, it is observed that more than one parameter can be present, following the format string in a printf statement. Inside the format string, %d and %f specify the type of the variables to be output. They are called format specifiers. The first format specifier, %d corresponds to the first parameter that follows the format string, i_var. It informs printf that i_var is an integer. Similarly, the second one, %f specifies that f_var is a floating - point number.

The above example illustrates the fact that the output produced by printf is the same as the format string, except that %d is replaced by the value of the parameter i_var, and %f by the value of f_var. The first parameter passed to printf, "i_var is the format string, while %d and %f inside it are the format specifiers.

To summarize, the parameters passed to the printf statement consist of a format string, followed by any number of parameters. However, each parameter must have a format specifier (%sign followed by a letter) corresponding to it, inside the format string.

5) printf Format Specifier

In addition to specifying the type of the parameter, the format specifier in printf can also perform the function of specifying how parameter must be output (for example, in decimal or hexadecimal). The specifiers %d and %i can be used interchangeably to output integers. Thus, the statement,

```
printf ( "%d", val );
```

can be replaced by,

```
printf ( "%i", val );
```

to get the same output. Also, note that % u and % x are used to output unsigned integers, but there is a difference, % u outputs it in decimal, while %x outputs it in hexadecimal. To illustrate this concept, a simple example that converts unsigned decimal numbers to hexadecimal is given below. Underlined data in this book indicate user input. The function scanf in this program is new, and is examined in the next section.

```
/* decimal.c: Conversion of unsigned decimal to hexadecimal number */
```

```
# include <stdio.h>
```

```
void main ()
```

```
{unsigned int inp_int;
```

```
scanf ( "%u," &inp_int);
```

```
printf ( "The input integer (in decimal) is %u/n", inp_int);
```

```
printf ( "The input integer (in hexadecimal) is % x/n", inp_int);}
```

RUN : 45

—

The input integer (in decimal) is 45

The input integer (in hexadecimal) is 2d

6) Reading Variables

The scanf function reads the values of variable from the standard input device (keyboard) and stores them in variables. The example below inputs the again years and computes the numbers of seconds you have lived. For simplicity, leap years have been ignored.

```
/* scanf.c: scanf example */
```

```
# include <stdio.h>
```

```
void main ( )
```

```
{float years, secs;
```

```
printf ( "Input your age in years: ");
```

```
scanf ( "%f", &years: ");
secs = years *34 * 24 * 60 * 60;
printf ( "you have lived for %f seconds /n", secs ); }
```

A sample session with the program is given below, where the user has input 21.

Input your age in years: 21

—

You have lived for 662256000.0 seconds

The statement

```
scanf ("%f ," & years);
```

performs the task of reading the value of the variable years. Like the printf function, the scan function accepts a format string as its first parameter. But, the second parameter years (the variable whose value we want to read) is preceded by an ampersand (&). An ampersand is not required in the case of printf. To understand this, it is necessary to consider a crucial difference between the two functions. The function printf, whose task is to output, needs only the value which has to be output. On the other hand, the scanf function stores the input value in a memory location. To perform this, it must know the location of the variable in memory (called the memory address of the variable) where the value has to be stored.

To understand the concept of memory address clearly, recall that the computer's memory consists of bytes. These bytes are numbered, starting with zero (i.e., the first byte is byte number 0, the second one is byte number 1, and so on). These numbers are called the address of the bytes.

Each variable in a program occupies a certain number of 1 bytes of the memory. Consider the floating point variable years in the above example. Assume that it occupies four consecutive bytes numbered 3064, 3065 and 3067 in the memory. So, the address of years is 3064 (the lowest numbered memory location). It is this address that scanf requires. In C. the address of any variable is obtained by preceding the variable name by an ampersand (&)

The statement

```
scanf ( "%f", &years );
```

The first parameter "%f" instructs scanf that a floating point numbers must be read. The second parameter & years informs scanf the location in the memory where the value of the floating point number entered by the user must be stored.

7) Characters and Character Strings

A character variable can hold one character, which can be any alphabet or symbol. Characters in C are enclosed within single quotes. The following example illustrates the task of assigning values to character variables.

```
/* char.c: Character assignment example. */
#include <stdio.h>

void main ( )
{
    /* Declare a character. */
    char c;

    /* Assign the symbol # to it . Note the single quotes.*/
    c = '#';

    printf ( " This is the hash symbol: c/n", c) ; }

```

The output is as follows:

```
This is the hash symbol: #
```

In the above example, c is a character variable. The character # assigned to c, is enclosed within a pairs of single quotes ('). Note that while strings are enclosed in double quotes, characters are enclosed in single quotes. The format specifier % c is used in the statement:

```
printf ( " This is the hash symbol: % c/n",c);
```

It specifies that a characters can be treated as integers. To understand this, we must clarify the concept of value of character variable. A character variable holds one character,

which may be a letter, a digit, a punctuation mark, and so on. These character are represented in the memory by a number, called the code of the character. For example, the code of the letter A may be 65, that of letter B may be 66, and so on.

Actually, any number can represent the letter A, any other number can be used for B and so on, but these numbers should be fixed by a coding convention. For example, when the computer wants the printer to print the letter A, it actually sends the number 65 to the printer. The important point here is that the printer must see the number 65 and print the letter A, and not something else. Hence, the printer must also use the same codes as the computer, for all the symbols. This led to the establishment of ASCII codes (ASCII: American Standard Code for Information Interchange). ASCII codes are widely used all over the world to represent various symbols in a computer. In C, the size of one character is 8 bits (one byte).

Here is a simple example that reads the ASCII code of a number and prints out the symbol associated with the code.

```
/* ascii.c: ASCII code example */

# include <stdio.h>

void main ()

{int code;

char symbol;

printf ( " Input an ASCII code (0 to 127): " );

scanf ( " % d " , & code);

symbol = code

printf ( " The symbol corresponding to % d is % c/n " , code, symbol ) ; }
```

Run 1 :

Input an ASCII code (0 to 127): 75

—

The symbol corresponding to 75 is K

Run 2 :

Input an ASCII code (0 to 127) : 50

—

The symbol corresponding to 50 is 2

The second run shows that just as the letter A is a symbol having as ASCII code, the digit 2 is also a symbol having an ASCII code of the digit 0 is 48, that of 1 is 49 and so on. To illustrate, let us say that the computer has to print the number 3210. It sends the following codes to the printer: 51 50 49 48

These are the ASCII codes of the digits 3,2,1 and 0, respectively.

Coming back to the program, statement

```
symbol = code;
```

Assigns the value of the integer variable code to the character variable symbol. In the printf statement,

```
printf ( " The symbol corresponding to %d is %c\n ", code, symbol);
```

%d is used to output its ASCII code contained in the integer code. The format specifier %c is used to output the actual symbol corresponding to the value contained in the character variable symbol.

We now come to the topic of strings. A string in C is a sequence of consecutive characters in the memory, the last one being the null character. A null character has an ASCII code 0, and is called the end - of - string marker in C. A string variable having 20 characters can be declared by using the statement:

```
char my_string [ 20 ]
```

Strings are enclosed within double quotes. Consider an example that assigns a string and prints it on the screen. The format specifier for a string is %s, which can be observed in the printf statement in the example below:

```

/* string.c: String example */

# include <stdio.h>

void main ( )

{char my_string [ 20 ] = " 13 is unlucky .";

printf ( "my_string ; %s\n" , my_string ) ; }

```

The format specifier % s is replaced by the value of the string constant. The output of the program is as shown below:

```
my_string: 13 is unlucky.
```

The variable my_string is just a sequence of bytes in memory whose values are the ASCII codes of each symbol of the string. The null character (a byte with value zero) is placed at the end of the string. Given below are the characters of my_string and their values in memory.

String :	1	3		i	s		u	n	l		u
ASCII Code	49	51	32	105	115	32	117	110	108	117	
String :	c	k	y	.	/0						
ASCII code :	99	107	121	46	0						

The numbers are the ASCII codes of the individual characters of the string (the ASCII code of the letters is 115, that of y is 121 and so on). The value 0 appearing at the end is the null character. It serves to terminate the string (i.e., marks the end of the string).

8) Qualifiers

Qualifiers modify the behavior of the variable type to which they are applied. We have seen declarations such as

```
int i ;
```

This declaration specifies that i is an integer which takes both positive and negative values, that is, i is a signed integer by default. The above declaration could also be written as

```
singed int i ;
```

but the qualifier signed is not necessary, since int is signed by default. If the variable i is used to hold only positive values (for example, to hold the number of students in a class), we declare:

```
unsigned int i ;
```

Here, the qualifier unsigned is applied to the data type int. This qualifier modifies the behavior of the integer so that the number contained in a variable of this type is unsigned.

Qualifiers can be classified into two types:

- Size qualifiers
- Sign qualifiers

Size Qualifiers, applied to int

Size qualifiers alter the size of the basic data type. There are two sizes of the basic data type. There are two size qualifiers that can be applied to integers (i.e., to the basic type int): short and long. In any ANSI C compiler, the sizes of short int, int and long int follow the restrictions given below:

- The minimum size of a short int is 16 bits
- The size of an int must be greater than or equal to that of a short int
- The size of a long int must be greater than or equal to that of an int
- The minimum size of a long int is 32 bits

In most compilers available on DOS, the size of a short int and an int are the same, both occupying 16 bits. A long int occupies 32 bits. But in 32-bit compilers such as GNU C, an int and a long int are of the same size (32 bits), while short int is 16 bits. On the other hand, almost all UNIX compilers have short int of size 16 bits, int and long int being 32 bits.

Size of Operator

To find out the size of each type on your compiler, try the example below. It uses the sizeof operator, that gives the size of any data type in bytes.

```

/* size.c: example to illustrate size of operator /

#include <stdio.h>

void main ( )

{ printf ( "Size of a short int is % d\n", size of (short int) );

printf ( "size of an int is % d\n", size of (int) );)

printf ( "size of a long int is % d\n", sizeof (long int) );}

```

The output of this program when compiled with Borland's compiler (Borland C Version 3.0) and run, gives the following output:

```

Size of a short int is 2

Size of an int is 2

Size of a long int is 4

```

On almost all UNIX systems, the program gives the following output:

```

Size of a short int is 2

Size of an int is 4

Size of a long int is 4

```

The name of the variable type is passed to the size of operator in closed parentheses. The number of bytes occupied by a variable of that type is given by the size of operator. The size of operator is very useful in developing portable programs. It is a bad practice to assume the size of a particular type since its size may vary from compiler to compiler.

The size of operator also takes variable names and returns the size of the variable given, in bytes.

```

For example, the statements,

int i;

printf ( " The size of i is %d\n", size of (i) );

will output (on a compiler where int is 16 - bit):

The size of i is 2

```

The result would have been the same if size of (int) had been used instead of size of (i), since i is an int. But passing the variable name to size of is a better practice. If the type of i has to be changed later, the rest of the program need not be modified.

What are the format specifiers used in the printf format string for short and long integers? For short integers, it is %hd (or %hi) and for long integers, it is %ld (or %li). The following example illustrates input-output statement for short and long integers.

```
/* inout.c: Input - output of short and long ints. */

#include main ()

{short int shorti;

long int longi;

printf ( "Input short int:");

scanf ( "%hd ", &shorti );

printf ( "Input long int:");

scanf ( "%ld", &longi );

printf ( "shorti = %hd and longi = %ld", shorti, longi ); }
```

A sample run is given below :

Input short int: 12

—

Input long int: 2000000000

—————

shorti = 12 and longi = 2000000000

limits.h

The header file limits.h contains the largest and smallest values that each of the various data types can hold. The name of the constant, the meaning, and the smallest acceptable magnitude is given in lesson 7.

Size qualifier for double

The type qualifier `long` can also be applied to the data type `double`. A variable of the type `long double` has more precision than a variable of the type `double`. Observe that C provides three data types for real numbers: `float`, `double` and `long double`. In any C compilers, you can be sure that the precision of a `double` is greater than or equal to that of `float`, and that the precision of a `long double` is greater than or equal to that of a `double`, i.e.,

precision of `long double` $\geq$ precision of `double` $\geq$ precision of `float`

Regarding the range that a floating point number can hold, the ANSI standard does not specify whether one type of floating point number can hold more than the other. So different compilers may provide different ranges for floating point numbers. The standard guarantees that the maximum floating point number is at least 10^37 .

Some compilers may provide a greater range. To find out the actual range that compiler offers, a program may refer to the constants defined in `float`.

Sign Qualifiers

The keywords `signed` and `unsigned` are the two sign qualifiers that specify whether a variable can hold both negative and positive numbers, or only positive numbers. These qualifiers can be applied to the data types `int` and `char` only.

An example of declaring an unsigned integer is :

`unsigned int it;`

As mentioned before, `signed int` is the same as `int` (i.e., `int` is signed by default). The type `char` may be the same as either `signed char` or `unsigned char`. This varies from compilers give the option of treating a `char` as signed or unsigned. In fact, many compilers give the option of treating a `char` as signed or unsigned. This fact (whether a `char` is signed or unsigned) does not matter when we use a variable of the type `char` to hold a single character. But when used as an 8-bit integer, it does matter whether the variable type dealt with is signed or unsigned.

9) typedef Statement

The `typedef` statement is used to give new names to existing types. For example, the statement

```
typedef unsigned, long ulong;
```

declares ulong a new type. Equivalent to unsigned long. It can be used just as any other type in a program For example the statement,

```
ulong u;
```

declares u to be of the type ulong. Also sizeof (ulong) gives the size of the new variable type in bytes.

10) Promotion and Typecasting

Consider the following example which assigns int to a float:

```
int i;
```

```
float f;
```

```
i = 5;
```

```
f = i;
```

The last statement assigns i to f. Since i is an integer while f is a floating point variable, the integer i is converted to float automatically. This type of conversion, when the variable of lower type (which holds lower precision) is converted to higher type (which holds higher range of values or has higher precision) is converted to a higher precision is called promotion.

Another example of promotion is the conversion of a character to an integer. This particular type of promotion is called integral promotion. For example,

```
int i;
```

```
char c;
```

```
c = 'a';
```

```
i = c;
```

The value present in the character variable c, i.e., the ASCII code of the character a is assigned to the character a is assigned to the integer i. If an integer is represented by sixteen bits, the lower eight bits will have the value of the while the upper eight bits will be filled with zeros. In some cases, the upper bits are filled with ones instead of zeros. To understand this, let us examine the concept of sign extension.

Conversion to a signed integer involves extending the sign. In case of signed numbers, the most significant bit is assumed to be the sign. If this bit is 1, the number is assumed to be negative, and if it is 0, it is assumed to be positive. When a character having its most significant bit set to one is assigned to a signed integer, the lower eight bits will be filled with 1's. On the other hand, if the character value has its most significant bit set property of filling the upper bits of the higher type with 0's. This of the lower type is called sign extension. Sign extension takes place only if the variable of the higher type is signed. If the variable of the higher type is unsigned, its upper bits will always be filled with 0's

Assignments of variable of higher types of lower types result in truncation. For example, the sequence,

```
f = 7.5;
```

```
i = f;
```

results in the fractional part (.5) discarded. The value 7 is assigned to the integer i.

In addition to automatic conversions, we can forcibly convert one type to another. The following example illustrates the need for forcible conversion.

```
int i, j;
```

```
float f;
```

```
i = 12;
```

```
j = 5;
```

```
f = i/j;
```

```
printf ( "%f \n" , f);
```

The output of this sequence of statement is as follow: 2

This is inspite of the fact that 12/5 is 2.4 and it can be assigned to f. Since both i and j are integers in the expression,

```
f = i/j
```

Integer division takes place, and the quotient 2 is assigned to f. Though the variable on the left-hand side of the assignment operator is a floating point variable, it does not cause

floating point division to take place. One of the operands involved in the division must be a floating point number. We can achieve this by temporarily converting either *i* or *j* to a floating point number. The division expression must be changed to

```
f = (float) i/j;
```

We are explicitly converting *i* to floating point number in the above expression. Explicit type conversions are called typecasting. The general syntax of typecasting given below:

```
(type) variable - name
```

The type name to which conversion is to be done is placed in closed parentheses preceding the variable name.

Typecasting can also be used to convert a higher type to a lower type. For example, if *f* is a float whose value is 2.7, the expression,

```
(int) f
```

evaluates to 2.

11) Value-initialized Variables

We can declare an integer variable *i* by the statement:

```
int i;
```

In order to initialize the variable *i* we can use two statements as shown below:

```
int i;
```

```
i = 5;
```

and then use the variable *i*. A simpler way to achieve this is to combine them into a single statement as given below:

```
int i = 5
```

A statement like the one above is preferred, since it is simpler, less prone to mistakes, and often leads to more efficient programs. When multiple variable are being declared in a single statement, initialization is done in the following way:

```
int i = 10, j = 5 ;
```

Also, the right side of the = symbol can have an expression. For following is a valid sequence of statement:

```
int i = 10, j = 5;
```

```
int k = i/j;
```

that will initialize the value of k to 2.

Check Your Progress - I

- 1) State whether the following are True or False.
 - a) A quantity which may vary during program execution is called variables.
 - b) The first character in the variable name must be a number.
- 2) Fill in the blanks:
 - a) The range for integer is _____.
 - b) The size of character is _____.
 - c) Real constants are often called _____.

2.4 Constants

A constant does not change its value during the entire execution of the program. Constants can be classified as integer constants, floating point constants, character constants and enumeration constants. Additionally, string literal can specify strings.

Example

Valid constants 48 6432 001

Invalid constants 964 (space not allowed)

1) Integer Constants

Integer constants can be specified in three ways: (i) Ordinary decimal numbers (base 10). For example, 175 is an integer constant specified in base 10. (ii) Octal numbers (base8). Octal numbers are specified with a leading zero, rest of the digits consists of those lying between 0 and 7. For example, 0175 is an integer consist specified in octal whose base 10 or decimal

value is 125. (iii) Hexadecimal numbers (base 16). Hexadecimal numbers are specified with 0x or 0X the beginning. The digits that follow 0x must be numbers in the range 0-9 or one of the letters a-f or A-F example, 0X175 is the same as 0x175, i.e., either a lower case or an upper case x can be used.

A size or sign qualifier can be appended at the end of the constant. The suffix u is used for unsigned int constants, l for long int constants and s for short int constants. Either of the cases can be used.

Example

1. unsigned integer constants

56789U

56789u

2. Long integer constants

7689909L

7689909l

0675434L (A long integer constant specified in octal)

0x34ADL (A long integer constant in hexadecimal with upper and lower case letters)

3. The suffixes can be combined and can be specified in any order

6578890994Ul

6578890994ul

We can use smaller integer constants that are stored with fewer bits for which the qualifiers used are s and us.

4. Short integer constants

120S - 1000s

5. Unsigned short integer constants

120US

1000us

Ordinarily, these suffixes are not needed. The compiler considers large integer constants to be of the type long automatically. These suffixes are useful in situations where the value may fit in an int but you want to treat it as long.

2) Floating Point Constants

Floating point constants have decimal point or an exponential sign or both.

Decimal notation: Here the number is represented as a whole number followed by a decimal point and a fractional part. It is possible to omit digits before or after the decimal point.

The following are examples of valid floating point constants:

241., .976, -.71 and + .5

Exponential notation: Exponential notation is useful in representing numbers whose magnitudes are very large or very small. The exponential notation consists of a mantissa and an exponent. The exponent is positive unless preceded by a minus sign. Consider the number 231.78. This can also be written as 0.23178e3 representing the number 0.23178 is the mantissa, and 3 is the exponent.

As another example, the number 75000000000 can be written as 75e9 or 75e11. Similarly 0,00000000045 can be written as 0.45e-9.

(i) The following examples are valid constants

2000.0434

3.4e4

3e8

(ii) The following are some invalid constants

3.4E.4 exponent must be an integer

3e 8 blank not allowed

Normalized exponential representation is the one in which the value of the mantissa is adjusted to a value between 0.1 and 0.99. For example, the number 75000000000 is written as 0.75e11 after normalization.

The mantissa exponential representation of real constants are given below.

The mantissa is either a real number expressed in decimal notation or an integer.

The mantissa can be preceded by a sign.

The exponent is an integer preceded by an optional sign.

The letter 'e' can be written in lowercase or uppercase.

Embedded white space is not allowed.

By default, real constants are assumed to be double. Suffixes for F can be used to specify the float values. For example, 0.257 is assumed to be a double constant, while 0.257f is a float constant. l or L can be used to specify long double values. For example, 0.257L is a long double constant.

3) Character Constants

A character constants is enclosed within single quotes.

Example 2.4

Valid character constants: 'a' '5' '\n'

Invalid character constants 'ab' '54'

Note: Multiple characters can also be enclosed within single quotes. The compiler will not report any error. The value of the constant however, depends upon the compiler used. This notation of having multiple characters within single quotes is practically never used.

In side the single quotes, a backslash character starts an escape sequence. \xhh specifies a character constant in hexadecimal, where hh is any hexadecimal digit. The hexadecimal digits hh give the ASCII value of the character. For example, the character constant \x07 represents the BEL character. Run the following program to hear a beep on the computer.

```
/* beep.c: example illustrating Beep sound */

# include <stdio.h>

void main ( )

{printf ( "%c", '\x07' ); }
```

\0ooo specifies a character constant in octal where, o is any octal digit. As before, ooo is the ASCII value of the number specified in octal. For example the ASCII code of K is 75. the character constant \0113 specifies this character in octal.

The complete list of escape sequences is given below

\a Beep (Alert)
 \b Backspace
 \f Formfeed
 \n Newline
 \t Horizontal tab
 \r Carriage return
 \\ Backslash
 \' Single quote
 \" Double quote
 \v Vertical tab
 \? Question mark
 \0 Null

\0oooCode specified in octal

\xhh Code specified in hexadecimal

Hence, \xo7 in the printf statement in the above example can be replaced by \a.

Example

(i) printf ("\ is a backslash. \n"); will print as follows:

\ is a backslash.

(ii) printf ("This \" is double quote.\n"); will print:

This " is a double quote.

4) Enumeration Constants

An enumeration is a user defined type with value ranging over a finite set of identifiers called enumeration constants. For example,

```
enum color {red, blue, green};
```

defines color as a new type having three values. red, blue and green. Each of these is an enumeration constant. In the program, color can be used as a new type. A variable of type color can have any one of the three values: red, blue or green. For example, the statement,

```
color c;
```

declares C to be of type color. Internally the C compiler treats an enum type (such as color) as an integer itself. The identifiers red, blue and green above represent the consecutive integer values of 0, 1 and 2 respectively. So, the statement,

```
c = blue;
```

```
printf ( "As an int, c has the value %d\n", c );
```

will print,

As an int, c has the value 1

Constant values can be explicitly specified for the identifiers. When the value of one identifier is specified in this manner, the value of the next element is the next higher integer.

For example, if the definition of color is

```
enum color {red = 10, blue, green = 34};
```

Then the statement `c = red` will assign the value 10 to c. `c = blue` will assign the value 11 to c, and `c = green` will assign the value 34. If no value is specified for green, it will assume the value 12.

Enumerations are a convenient way to associate constant integers with meaningful names. They have the advantage of generating the values automatically. Use of enumeration constants, in general, makes the program easy to read and change at a later date.

Names in different enum must be distinct. The following example is invalid.

```
enum emotion {happy, hot, cool};
```

```
enum weather {hot, cold, wet};
```

It is not difficult to see why the above declarations are invalid. It is because, the name hot has the value 1 in enum emotion and the value 0 in weather. In the program, if you use the name hot, there is an ambiguity as to which value to use. On the other hand, values need not be distinct in the same enumeration. For example, the following declaration is perfectly valid.

```
enum weather {hot, warm = 0, cold, wet};
```

The names hot and warm can be interchangeably used, since both represent the value 0.-

Consider the variable days_of_week. is declared as the declaration of this enum. The enumeration constants are Sun Mon Tues Wed Thu Fri Sat. These constants are separated by commas.

```
enum days_of_week {Sun, Mon, Tues, Wed, Thu, Fri, Sat}
```

5) String Literals

A string literal is a sequence of characters enclosed within double quotes. The character may consist of letters, numbers, escape sequences and space. To make it easier to write long strings, string constants are concatenated at the compile time. For example, the strings:

“C is the best” and

“C is “the best” are the same.

An Important Difference: ‘A’ and “A”

The notations ‘A’ and “A” have an important difference: the first (‘A’) is a character constant, while the second (“A”) is a string constant. The notation ‘A’ is a constant occupying a single byte containing the ASCII code of the character A. The notation “A” on the other hand, is a constant that occupies two bytes, one for the ASCII code of A and another for the null character with the value 0, that terminates all strings.

6) const Qualifier

To understand the necessity of const identifiers, consider the following program which calculates tax on the given salary, assuming that the rate of taxation is 30%.

```
/*tax.c: program to compute tax */  
# include <stdio.h>  
void main ( )  
{float sal, tax;  
printf ( " salary: " );  
scanf ( "%f", & sal );  
tax = sal * 0.3;  
printf ( "tax = %f\n", tax );}
```

The program works correctly. It uses the numeric constant 0.3 to calculate the tax since the rate of taxation is 30%. But assume that the program computes the tax at many different points. If the tax rate changes, all the statements that compute the tax will have to be modified. An obvious solution to this problem is to declare a variable called `tax_rate` that will have the current tax rate. In case the tax rate changes, we only need to update the variable `tax_rate`. The modified program is given below.

```
/*tax1.c: program to compute tax using a variable */  
# include <stdio.h>  
void main ( )  
{float tax_rate = 0.3;  
float sal, tax;  
printf ( " salary: " )  
scanf ( "%f", & sal );  
tax = sal * tax_rate;  
printf ( "tax = %f\n", tax );}
```

The statement,

```
tax = sal * tax_rate;
```

in the above program is more readable than

```
tax = sal * 0.3;
```

in the earlier one. This disadvantage is that any statement written carelessly can modify its value. Then, all subsequent calculations of the tax will yield wrong results. In a very long program, if such a problem is encountered, it could be very difficult to find where exactly it has been modified. To avoid suffering the consequences of such carelessness, declare the variable `tax_rate` as `const`. The statement that declared `tax_rate` would now be changed to:

```
const float tax_rate = 0.3;
```

This informs the compiler that `tax_rate` is a constant floating-point number, its value is 0.3 and it cannot be modified anywhere in the program. A statement, such as

```
tax_rate = 0.99;
```

Will cause a compile-time error, enabling you to detect the mistake early and save debugging time. The complete program using a constant identifier `tax_rate` is given below.

```
/* const.c: program to illustrate the use of the keyword const. */
```

```
#include <stdio.h>
```

```
void main ( )
```

```
{const float tax_rate = 0.3;
```

```
float sal, tax;
```

```
printf ( " salary: " );
```

```
scanf ( "%f", & sal );
```

```
tax = sal * tax_rate;
```

```
printf ( "tax = %f\n", tax );}
```

Another way to achieve the effect of `const` identifiers is by the use of `# define` preprocessor directive as shown below:

```
# include <stdio.h>

# define tax_rate 0.3

void main ( )

{float sal, tax;

printf ( " salary: " );

scanf ( "%f", & sal );

tax = sal * tax_rate;

printf ( "tax = %f\n", tax );}
```

Notice that in the statement,

```
# define tax_rate 0.3
```

There is no terminating semicolon (;) as in all preprocessor directives. The directive instructs the pre-processor to substitute 0.3 for all the occurrences of the identifier `tax_rate` in the succeeding part of the program.

This is the key difference between the use of constant identifiers and the `#define` directive. The compiler treats the former as a value that cannot change in the program, while the latter is handled by the preprocessor that replaces all instances of `tax_rate` by 0.3.

Note that conventionally, such constants are specified in the upper case. Normally the `#define` directive would be:

```
# define TAX-RATE 0.3
```

`TAX-RATE` is then used in the rest of the program.

Programming Examples

Example

```
/* interest.c: calculation of simple interest and compound
interest */
```

```

#include <stdio.h>

#include <math.h>    /* For the function pow */

void main ( )

{float p, t, r, ncmp_year;

float simple, compound, cmp_amount;

printf ( "principal: " );

scanf ( "%f" , & p );

printf ( "Rate: " );

scanf ( "%f" , & r );

printf ( "Time: " );

scanf ( "%f", & t );

printf ( "Compoundings per year: " );

scanf ( "%f", & ncmp_year );

simple = p * t * r/100;

/* Calculate the amount resulting from compound
interest. */

cmp_amount = p * pow ( 1+ r/ (ncmp_year*100)
ncmp_year * t);

/* Compute the compound interest from the amount and principal */ compound =
cmp_amount - p;

printf ( "The simple interest is: %f\n". simple);

printf ( "The compound interest is: %f\n", compound );}

Run:

principle: 1000

_____

Rate: 10

_____

Time: 2

```

Compoundings per year: 4

-

The simple interest is: 200.000000

The compound interest is: 218.402954

While compiling in UNIX, use `cc - 1m <filename>` instead of `cc <filename>`, since it uses the function `pow` from the `math.h` header file in the program. The function `pow` above needs some explanation. There is no operator in C which acts as the exponentiation (or to the power of) operator. The function `pow` is used for this purpose. Two floating-point numbers are passed to `pow`; it computes the first number raised to the power of the second, and returns the answer. In the above example, the function `pow` is called thus:

```
pow (1+ r/ (ncmp_year*100), ncmp_year* t)
```

The first number passed to `pow` is `1+r/ (ncmp_year*100)`. The second number is `ncmp_year * t`. The two are separated by a comma, as in the case of `printf` where the variables to be output that are passed to `printf` are separated by commas. Observe that the interest values are printed with 6 digits after the compiler. Whereas, it is sufficient if they are printed with the precision operations are given in the next lesson.

Example

```
/*temper.c: Conversion of centigrade to fahrenheit and vice-versa */
#include <stdio.h>

void main ( )
{float c, f;
printf ( "Enter temperature in celsius:");
scanf ( "%f", &c );
f = 1.8 * c + 32;
printf ( "Equivalent fahrenheit = %f\n", f);
printf ( "\nEnter temperature in fahrenheit:");
```

```
scanf ( "%f", &f );
c = (f - 32) / 1.8;
printf ( "Equivalent celsius = %f\n", c );}
```

Run 1:

```
Enter temperature in celsius: 10
—
Equivalent fahrenheit = 50.000000
```

Run 2:

```
Enter temperature in fahrenheit: 50
—
Equivalent celsius = 10.000000
```

Example

```
/*triangle.c: Computation of area of a triangle give the three sides*/
#include <stdio.h>
#include <math.h>      /* For the function sqrt */
void main ( )
{float a, b, c;
float peri, s, area;
printf ( "Enter the 3 sides: " );
scanf ( "%f %f %f", &a, &b, &c );
peri = a + b + c;
s = peri / 2;
area = sqrt (s* (s - a) * (s - b) * (s - c) );
printf ( "Area of the triangle is %f\n", area );}
```

Run:

```
Enter the 3 sides: 4 5 6
- - -
Area of the triangle is 9.921567
```

In the above example, the library function `sqrt` is used to find the square root. This function accepts a floating point number and returns its square root.

Example

```
/* triangle 2.c: Area of the triangle given the base and height */
#include <stdio.h>

void main ( )
{float b, h;
float area;

printf ( "Enter the base: " );
scanf ( "%f", &b );
printf ( "Enter the height: " );
scanf ( "%f", &h );
area = b * h / 2;
printf ( "Area of the triangle is %f", area );}
```

Run:

```
Enter the base: 3
-
Enter the height: 2
-
Area of the triangle is 3.000000
```

Check Your Progress - II

- 1) State whether the following are True or False.
 - a) A constant is a quantity which may vary during the program execution.
 - b) Floating point constants have decimal point or an exponential sign or both.
- 2) Fill in the blanks
 - a) _____ constants is enclosed within single quotation.
 - b) _____ is a sequence of characters enclosed within double quotes.

2.5 Summary

In this lesson we have covered variable and expression. We have also covered character set, special characters, Identifiers and keyword.

2.6 Check Your Answers

- I. 1) a) True b) False
- 2) a) -32, 768 to 32, 767 b) one character c) 4 bytes
- II. 1) a) False b) True
- 2) a) character b) String literal

2.7 Model Questions

- (1) Explain the four basic data types in C. What is the size in bytes of each type in DOS and Unix platforms
- (2) Explain the four basic types of constants in C
- (3) What is meant by explicit type casting? When is it useful?
- (4) What are the different types of qualifiers? Explain.

LESSON - 3

OPERATORS AND EXPRESSIONS

Structure

- 3.1 Introduction**
- 3.2 Learning Objectives**
- 3.3 Operators**
- 3.4 Summary**
- 3.5 Check Your Answers**
- 3.6 Model Questions**

3.1 Introduction

Operators are C's forte. It is extremely rich in operators. It has as many as 45 different operators. We are going to discuss most of these operators and their interaction with one another in this lesson.

3.2 Learning Objectives

After learning this lesson, you should be able to

- operators and expressions

3.3 Operators

An operator, in general, is a symbol that operates on a certain data type. For example, the operator is the addition operation. It can operate on integer, character and real (float, double and long double) numbers. An expression is a combination of variables, constants and operators written according to the syntax of the language. In C, every expression evaluates to a value, i.e., every expression results in some value of a certain type that can be assigned to a variable.

Let us take the simple example of an assignment operator and analyse the concept of evaluation. In C, the '=' symbol is the assignment operator. Its duty is to set the value of the variable on the left hand side to that of the right hand side. The left hand side of the assignment operator (=) and the right hand side together form an expression.

For example, if the variables *i* and *j* have been declared by the statement,

```
int i, j;
```

then a valid expression involving the assignment operator would be as follows:

```
i = 5;
```

The above statement assigns the value 5 to *i*. In addition, the whole expression evaluates to the value that has been assigned (i.e., 5). So, the statement,

```
j = i = 5;
```

Will first execute *i* = 5. This expression evaluates to 5. When we say that *i*=5 evaluates to 5, what we actually mean is that the *i* = 5 part (inside the expression *j* = *i* = 5) can be replaced by the number 5. So you can imagine that at the run time, the value 5 is first assigned to *i*. Then, the statement becomes

```
j = 5
```

and 5 is assigned to *j*. Thus, the value of both *i* and *j* after the above statement is executed, is 5.

1) Types of Operators

In C, operators can be classified into various categories based on their utility and action. A list of operator types is given below:

- Arithmetic operators
- Relational operators
- Logical operators
- Assignment operators
- Increment and decrement operators
- Conditional operators
- Comma operator

2) Arithmetic Operators

The arithmetic operators perform arithmetic operations and can be classified into unary and binary arithmetic operators. The arithmetic operators can operate on any built-in data type/ A list of arithmetic operators and their meanings are given below:

Operator	Meaning
+	addition or unary plus
-	subtraction or unary minus
*	multiplication
/	division
%	modulo division (remainder after division)

Note: .C has no operator for exponentiation. However, a function `pow(x,y)` exists in `math.h` which returns X^y . The unary minus operator has the effect of multiplying its operand by - 1

Integer arithmetic: When two operands such as `x` and `y` are declared integers, an arithmetic performed on these integers is called integer arithmetic. It always yields an integer value.

Example

Let `x` and `y` be declared by the statement: `int x = 16, y = 5`, (`x` and `y` are integers with values 16 and 5, respectively)

Then the integer arithmetic operations yield the following results:

$$x + y = 21$$

$$x - y = 11$$

$$x * y = 80$$

$$x / y = 3$$

The result is truncated, the decimal part is discarded.

$x \% y = 1$ The result is the remainder of the integer division and the sign of the result is always the sign of the first operand.

Note: In integer division operation, the result is truncated towards zero if both the operands are of the same sign, and is dependent on the machine if one of the operands is negative.

Example

$$6 / 8 = 0 \quad -6 / -8 = 0 \quad -6 / 8 = 0 \text{ or } -1$$

The result is machine dependent

Floating point arithmetic: Floating point arithmetic involves operands of real type in decimal or exponential notation. The floating point results are rounded off to the number of significant digits specified and hence, the final value is only an approximation of the correct result. The remainder operator % is not applicable to floating point arithmetic operands.

Example

Let a and b be declared by the statement `float a = 14.0, b = 4.0;` (a and b are float variables with values 14.0 and 4.0 respectively) and p, q and r be floating point variables: then the floating point arithmetic operations will yield the following results:

$$p = a / b = 3.500000 \quad q = b / a = 0.285714$$

$$r = a + b = 18.00000$$

Mixed mode arithmetic: In mixed mode arithmetic, if either of the operands is real, the resultant value is always a real value. For example, $35 / 5.0 = 7.0$. Here, since 5.0 is a double constant, 35 is converted to a double and the result is also a double.

3) Relational Operators

While arithmetic operators are used to evaluate arithmetic expressions, relational operators are used to compare arithmetic, logical and character expressions. We often compare two similar quantities and depending on their relation, take some decisions. These comparisons can be done with the help of relational operators. Each of these operators compares their left hand side with their right hand side. The whole expression involving the relational operator then evaluates to an integer. It evaluates to zero if the condition is false, and 1 (one) if it is true.

Operator	Meaning
<	less than
>	greater than

<=	less than or equal to
>=	greater than or equal to
= =	equal to
!=	not equal to

In order to study the relational operators in the right perspective, it is necessary to know the fundamentals of an if statement. Consider a simple example that uses a relational operator in an if statement. Details of the if statement are dealt with in the next lesson, but presently we need to have a basic idea of its function. The if statement evaluates an expression and executes the next statement only if the expression evaluates to true (a non-zero integer). The following example inputs two integers and tests for equality.

/* relation.c: Example of using relational operator in an if statement */

```
#include <stdio.h>
```

```
void main ( )
```

```
{ int i, j;
```

```
  print ( "Input two numbers: " );
```

```
  scanf ( "%d%d", &i, &j );
```

```
  if ( i = = j )
```

```
    printf ( "They are equal. \n" );
```

```
    printf ( "Got the message?\n" ); }
```

The if statement in the above example needs special mention. A pair of round braces follow the if statement. If the condition within these braces (which is `i = = j` here) is true, the statement just after the if statement is executed, otherwise it is skipped. Let us now run the program, enter two equal number and see what happens.

```
Input two numbers: 34 34
```

```
— —
```

```
They are equal.
```

Got the message?

since i and j are both 34, the condition `i == j` in the if statement

```
if ( i == j )
```

evaluates to 1, and statement following the if statement, which is

```
printf ( "They are equal.\n" );
```

is executed. Thereafter, the execution continues normally, and the last statement of the program

```
printf ( "Got the message?\n" );
```

is executed finally.

If the program is run with unequal numbers the output appears as follows:

Input two numbers: 34 45

— —

Got the message?

Since i and j are not equal, the condition `i == j` in the if statement evaluates to 0. The statement that follows the if statement is skipped, and the message They are equal is not displayed.

Note that in C, the operator for testing equality is `==` (two = signs placed together). One of the most common mistakes is to use single = sign to test for equality. For example, consider the following sequence of statements:

```
int i = 3, j = 70;
```

```
if ( i = j )
```

```
printf ( "They are equal.\n" )
```

```
printf ( "They are not equal. \n" )
```

As expected, the following message is printed:

They are not equal.

On the other hand, here is the wrong version that uses single = for comparison.

```

int i = e, j = 70;
if ( i = j )
printf ( "They are equal.\n" );
else
printf ( "They are not equal.\n" );

```

Since `i = j` evaluates to false (0), the statement following the `else` keyword is executed.

The above sequence of statements cause the following (erroneous) output to be printed:

They are equal

This happens because the result of an assignment operator is the assigned value itself. Here, since the condition used in the `if` statement is `i = j`, the value of `j` (70), is assigned to `i`, and the assignment expression evaluates to 70, which is non-zero. Since any non-zero value is considered to be true, the statement following `if` is executed, printing a message that the two values are equal.

Examples of other relational operators are given below. Suppose `i` and `j` are declared with

```
int i = 10, j = 20;
```

Here the expression,

```
i > j
```

is false (because `i` is not greater than `j`). Recall that every expression evaluates to a value in C. The above expression evaluates to false. But in C, there is no special data type to represent true or false. Integers are used for this purpose. The integer zero represents false and any non-zero integer represents true. So, the expression `i > j` evaluates to zero. The expressions `i < j` and `i <= j` evaluate to 1, since the relations are true.

For example, if the variable `t` is declared an integer, the statements,

```

t = (i < j);
printf ( "%d\n", t );

```

will print 1. More examples are given below:

When using relational operators, the signed or unsigned nature of the numbers being compared becomes important. Neglecting this fact can lead to hard-to find errors. For example, here is a program that uses the variable type char as an 8-bit integer. Whether char is signed or unsigned depends on the compiler used.

```
/* char.c: Using char as an 8-bit integer */
#include <stdio.h>

void main ( )
{ /* Integer value being assigned to a char */
  char c = 255;
  char d = -1;
  if ( c < 0 )
    printf ( "c is less than 0\n" );
  else
    printf ( "c is not less than 0\n" );
  if ( d < 0 )
    printf ( "d is less than 0\n" );
  else
    printf ( "d is less than 0\n" ); }
```

If the compiler used to compile the above program treats a char as signed, then the output of the program will be:

```
c is less than 0
d is less than 0
```

The second lined is true, since the number assigned to d, which is -1 is indeed less than 0. The first line is false, since the number we have assigned to c is 255, and is definitely greater than 0. But the catch is that both -1 and 255 have same binary representation in 2's complement arithmetic. Treating char as signed has the effect that both are treated as signed numbers (i.e., as -1 in this case).

On the other hand, if the compiler treats a char as unsigned, the output will be:

c is not less than 0

d is not less than 0

The first line is now correct, but the second is not. Again, when a char is treated as unsigned, both numbers (c and d) are assumed to be 255, and hence not less than 0. This is certainly an obstacle to writing portable programs. The solution lies in explicitly declaring variables of type char as signed or unsigned when using them as 8-bit integer values. Similarly, the signed or unsigned nature of numbers is important even in case of normal integers. Making the above program portable lies in changing the declaration of the characters.

```
/*char8.c: Using char as an 8-bit integer the right way */
```

```
#include <stdio.h>
```

```
void main ( )
```

```
{ /* Integer value being assigned to a char */
```

```
Unsigned char c = 255;
```

```
signed char d = -1;
```

```
if ( c < 0 )
```

```
printf ( "c is less than 0\n" )
```

```
else
```

```
printf ( "c is not less than 0\n" );
```

```
if ( d < 0 )
```

```
printf ( "d is not less than 0\n" );
```

```
else
```

```
printf ( "d is not less than 0\n" );}
```

Now, irrespective of the compiler, the output of the program is as follows:

c is not less than 0

d is less than 0

4) Logical Operators

A logical operator is used to compare or evaluate logical and relational expressions. There are three logical operators in C language. They are:

Operator	Meaning
&&	Logical AND
	Logical OR
!	Logical NOT

An expression involving && or || is sometimes called compound expression, since the expression involves two other expressions, that is, each of these operators (&& and ||) take two expressions, one to the left and another to the right.

Logical AND: For example, consider the following expression:

`a > b && x == 10`

The expression on the left is `a > b` and that on the right is `x == 10`. The whole expression evaluates to true (1) only if both expressions are true (if `a` is greater than `b` and the value of `x` is equal to 10)

Logical OR: Consider the following example involving the || operator:

`a < m || a < n`

The expression is true one them is true, or if both of them are true. That is, if the value of `a` is less than that of `m` or `n`. Needless to say, it evaluates to true in case `a` is less than both `m` and `n`.

Logical NOT: The ! (NOT) operator takes single expression and evaluates to true (1) if the expression is false (zero), and evaluates to false (0) if the expression is true. In other words, it just reverses the value of the expression. For example, consider:

`!(x > y)`

The expression after the ! operator, in this case is $x \geq y$. The not expression evaluates to true only if the value of x is neither greater than nor equal to y (i.e., only if x is less than y). Therefore, the whole expression including the ! operator is the same as:

$x < y$

So what is the use of this operator? The ! operator is convenient to use when you want to test whether the value of a variable is zero. For example,

```
if (!i )
```

```
printf ( "The value of i is zero. \n" );
```

The expression inside the braces of the if statement is: !i

This is true only if the value of i is zero.

5) Bitwise Operators

A bitwise operator operates on each bit of data. These operators are used for testing, complementing or shifting bits to the right or left. Usually bitwise operators are not useful in cases of float and double variables. A list of bitwise operators are given below.

Operator	Meaning
&	bitwise AND
	bitwise OR
^	bitwise XOR
<<	shift left
>>	shift right
~	bitwise complement

To experiment with these operators, assume that a, b and c are integers declared by the statement.

```
int a = 13, b = 7, c;
```

Also, for convenience, let us assume that an integer occupies 16 bits (2 bytes).

The binary representation of a is 0000 0000 0000 1101.

The binary representation of b is 0000 0000 0000 0111.

(The space appearing after every 4 bits are only for clarity. Actually, the integers are merely 16 continuous bits).

Bitwise AND: The statement,

$c = a \& b;$

make use of the bitwise AND operator. After this statement is executed, each bit in c will be 1 only if the corresponding bits in both a and b are 1. For example, the right most bit of both integers is 1, and hence the right most bit in c is 1. The next bit is 0 in a and 1 in b. Hence the second bit (from the right) in c is 0. Applying this reasoning to all the bits in each integer, the value of c after the above statement is executed, will be 0000 0000 0000 0101 which, in decimal is 5 and is illustrated below:

Bitwise AND operator: $a \& b$

a 0000 0000 0000 1101

b 0000 0000 0000 0111

a & b 0000 0000 0000 0101

Bitwise OR: The statement

$c = a \mid b;$

make use of the bitwise OR operator. After this statement is executed, a bit in c will be 1 whenever at least one of the corresponding bits in a or b is 1. In the above example, the value of c will be 0000 0000 0000 1111, i.e., decimal 15 and is illustrated below.

Bitwise OR operator: $a \mid b$

a 0000 0000 0000 1101

b 0000 0000 0000 0111

a $\mid$ b 0000 0000 0000 1111

Bitwise XOR: The statement

```
c = a ^ b;
```

make use of the bitwise XOR operator. After this statement is executed, a bit in c will be 1 whenever the corresponding bits in a and b differ. So, in the above example, the value of c will be 0000 0000 0000 1010 which, in decimal is 10 and is illustrated below.

Bitwise EX -OR operator: $a \wedge b$

```
a 0000 0000 0000 1101
```

```
b 0000 0000 0000 0111
```

```
a ^ b 0000 0000 0000 1010
```

Left shift operator: The left bit-shift operator, << is a binary operator. For example consider the statement

```
c = a << 3;
```

The value in the integer a is shifted to the left by three bit positions. The result is assigned to the integer c. Since the value of a is 0000 0000 0000 1101 the value of c after the execution of the above statement is 0000 0000 1010 1000 (140 in decimal) and is illustrated below:

Left-Shift <<

drop off <— 0000 0000 0000 1101 <— insert 0's

after left-bit shift by 3 places i.e., $a \ll 3$

```
0000 0000 0110 1000
```

The three left-most bits drop off due to the left shift (i.e., they are not present in the result). Three zeros are inserted in the right. The effect of shifting a variable to the left by one bit position is to multiply it by 2. In the above example, a is shifted left by 3 positions, which is equivalent to 3 multiplying a by $2 \times 2 \times 2$, i.e., 8. Since the initial value of a is 13, the value assigned to c is $13 \times 8 = 104$.

While multiplying a number by a power of 2, considerable saving in execution time can be achieved by using the left bit-shift operator instead of the multiplication operator, since shifting is faster than multiplication.

Right shift operator: The right bit-shift operator is also a binary operator. For example, consider:

```
c = a >> 2;
```

The value of a is shifted to the right by 2 positions. Since the value of a is 0000 0000 0000 1101, the value of c after the execution of the above statement is 0000 0000 0000 0011 (3 in decimal) and is illustrated below:

Right-Shift >>

insert 0's → 0000 0000 0000 1101 → drop off

after right shift by 2 places i.e., a >> 2

0000 0000 0000 0011

The 2 right-most bits drop off (are not present in the result), and zeros are inserted in the left. The effect of shifting a variable to the right by one bit position is to perform integer division by 2 (i.e., divide by 2 and truncate the result). Hence, shifting to the right by 2 bit positions has the effect of integer division by $2 \times 2 = 4$. Since the initial value of a is 13, shifting to the right by 2 bit positions yields the value 3 (the result of dividing 13 by 4 and truncating the result).

Shifting negative number: Note that zeros are not always inserted in the left, as shown in the above example. If the number is negative to begin with then 1 is inserted in the left for every bit shifted to the right. For example, if we assign a negative number, say -3 to a with the statement:

```
a = -3;
```

Then the binary representation of a would be the two's complement of the magnitude. To find the 2's complement of 3, take the binary representation of 3, complement it bitwise, and add 1 to the result. The binary representation of 3 is: 0000 0000 0000 0011

Complementing bitwise (i.e, taking each bit and inverting it, making it 0 if it is 1 and vice-versa), we get: 1111 1111 1111 1100

Add 1 to this sequence to get 2's complement.

```
1111 1111 1111 1101
```

This is the binary representation of -3. Now, the statement,

```
c = a >> 2;
```

will shift the value of a by 2 bit positions to the right and insert two 1's in the left. So, at end of this operation, the value of c will be:

```
1111 1111 1111 1111
```

This is a number which is in 2's complement form again. To see the magnitude of this negative number, take 2's complement again (2's complement of a 2's complement is the original number itself), resulting in

```
0000 0000 0000 0001
```

indicating that the number is -1 in decimal.

In the case of unsigned integers, the bit inserted on the left is always 0. Note that the value of the bit inserted on the left varies with the sign only in the case of a right shift. In case of a left shift, the bit on the right is always 0.

6) Bitwise Complement Operator

The bitwise complement operator (~) is a unary operator. It gives the value got by complementing each bit of the operand. To illustrate its usefulness, consider a situation where it is required to clear the least significant a can use the & operator to bitwise AND the integer with a constant having only the least significant four bits initialized to zero.

```
i = i & 0xfff0;
```

But this method assumes that the size of an integer is 16 bits. If an integer is of 32 bits, the above method clears the most significant sixteen bits also. Another way is to use the right and left shift operators and the fact that a left shift always shifts in zeros to the right. The statement below shifts the integer 4 bits to the right, and shifts it back 4 bits to the left.

```
i = (i >> 4) << 4;
```


This method though portable, involves two shifts. It can be done more efficiently using the `~` operator, as follows:

```
i = i & ~ 0xf;
```

The integer constant `0xf` has only its significant four bits set. When the bitwise complement (`~`) operator is applied to it, an integer that has all bits set except the least four significant bits, is obtained. Applying the bitwise AND now gives the desired result. Expressions such as above, where the variable being assigned to and the variable being operated on, are the same, permit an easier method. For example, the assignment expression `i = i & ~ 0xf` above can be written as:

```
i &= ~0xf;
```

The operator `&=` performs the bitwise AND operation on the operand on its left.

To illustrate the bitwise operators, a complete sample program is given below. This program reads an integer and prints the value of a specified bit in the integer. The bits are numbered from 0, starting from the right. For example, to find the value of the second bit of an integer `i`, it is necessary to shift `i` to the right by two bits, and take the least significant digit.

```
/* extract.c: Finishing the nth bit */

#include <stdio.h>

void main ( )

{ /* i is the input integer and n, the bit position to extract. */ int i, n;

  /* bit is the value of the bit extracted (0 or 1) */

  int bit;

  printf ( "Input an integer: " );

  scanf ( "%d", &i );

  printf ( "Bit position to extract: " );
```

```
scanf ( "%d", &n );
bit = (i >> n) & 1;
printf ( "The bit is %d" bit );}
```

The statement,

```
bit = (i >> n) & 1;
```

first shifts *i* to the right by *n* bits and then masks (clears) all bits of *i* except the least significant bit, giving us the value of the bit which we wanted. Parentheses are not required in the above statement, since the operator `>>` has a greater precedence than `&` (i.e., in an expression, if both `>>` and `&` are present, the `>>` operator is executed first). Since this fact is not so obvious, it is always better to use parentheses in such situation, for the sake of readability.

Arithmetic assignment operators: We have already used the assignment operator `=` (equal sign). As we know, this operator evaluates the expression on the right, and assigns the resulting value to the variable on the left. Other forms of assignment operators exist, that are obtained by combining various operators such as `+`, `-`, `*`, etc., with the `=` sign. For example, there is a `+=` operator that evaluates the expression to its right, and adds the resulting value to the variable on its left.

The statement `a += 5;` will add the number 5 to the value of *a*. Similarly, `a+=b-c` will evaluate *b-c* first, and adds the result to the value of *a*. The other assignment operators are:

```
- =    /=    *=    | =    &=    ^=
```

These have obvious meanings. They evaluate the expression on their right, and use the result to perform the corresponding operation on the variable on the left. Apart from the above assignment operators, bit-shift assignment operators also exist. They are:

```
>>=    <<=
```

For example, the statement,

```
a <<= 5;
```

will shift the bits of *a* to the left by 5 bit positions.

7) Increment and Decrement Operators

The increment and decrement operators are very useful in C language. They are extensively used in for and while loops. The syntax of the operators is given below:

```
++<variable name>      --<variable name>
<variable name ++      <variable name > --
```

The operator ++ adds 1 to the operand and -- subtracts 1 from the operand. These operators manifest in two forms: prefix and postfix. For example, the ++ operator can be used in two ways:

++m and m++

These are two different expressions. The first expression immediately increments the value of m by 1. For example, the statements,

```
int t, m = 1;
t = ++m;
printf ( "prefix operator: t = %d, m = %d\n", t, m );
will print,
prefix operator: t = 2, m = 2
```

The expression ++m will cause the value of m to be incremented first, and then this value is assigned to t, resulting in both having the same value. On the other hand, the statement,

```
int t, m = 1;
t = m++;
printf ( "Postfix operator: t = %d, m= %d\n", t, m );
will output
postfix operator: t =1, m =2
```

The expression `m++` will first evaluate the value of `m` (which is 1), resulting in 1 being assigned to `t` and then the value of `m` being incremented to 2.

8) Conditional Operator (ternary operator)

The conditional operator consists of two symbols: the mark (?) and the colon (:). This is the only operator in C that takes three operands. The syntax of the operator is given below:

expression 1 ? expression 2 : expression 3;

Let us examine the ternary operator through an example. This program reads two integers and displays the value of the larger of the two. If they are equal, then naturally either of them will be printed.

/*max.c: Example to find the maximum using the conditional operator */

```
#include <stdio.h>
```

```
void main ( )
```

```
{ int i, j, larger;
```

```
printf ( "Input two integers: " );
```

```
scanf ( "%d%d", &i, &j );
```

```
larger = i>j ? i : j;
```

```
printf ( "The larger of the two is %d\n", larger );}
```

The statement that uses the conditional operator is:

```
larger = i>j ? i : j;
```

It has three components; the expression `i>j` before the question mark, and the variable `i` and `j` after it, one on each side of the `:`. We shall first see a sample run of the above program, and then analyse what the conditional operator does.

```
Input two integers: 34 45
```

```
— —
```

```
The larger of the two is 45
```

Let us now see what the conditional statement does.

- First, the expression before? (which is $i > j$) is evaluated. In the above case, the expression is false.
- Since the expression before? is false, the whole expression involving the conditional operates to the expression following the colon (:). In this case, this is just the variable j .

Here is one more test run.

Input two integers: 5 -1

The larger of the two is 5

In this case, the conditional statement operates as follows:

- as before, the expression before? (which is $i > j$) is evaluated. The expression is true, since i is 5 and j is -1.
- since the expression before? is true, the whole expression involving the conditional operators evaluates to the expression before the colon (:) In this case, this is the variable i

9) Comma Operator

A set of expressions separated by commas is a valid construct in the C language. For example, i and j are declared by the statement: `int i, j;`

Consider the following statement that makes use of the comma operator:

```
i = (j = 3, j + 2);
```

The right hand side consists of two expressions separated by comma. The first expression is $j=3$ and the second is $j+2$. These expressions are evaluated from left to right. That is, first the value 3 is assigned to j and then the expression $j+2$ is evaluated, giving 5. The value of the entire comma-separated expression is the value of the right-most expression. In the above example, the value assigned to i would be 5.

10) Other Operators

The operator `sizeof` gives the size of the data type or the variable in terms of bytes occupied in the memory, as illustrated earlier. Another operator is the member selection operator (`.` and `->`) which is used with structures and unions. Pointer operators, `*` and `&` are explained in detail in later chapters.

11) Operator Precedence and Associativity

Consider the following two statements:

```
int a = 10, b = 15, c = 3, d;
```

```
d = a + b * c;
```

In the above expression, `b` is multiplied by `c` first. The result is added to `a` and the sum is assigned to `d`. Since multiplication is done before addition, the multiplication operator has a higher precedence than the addition operator. In order to override the precedence, braces can be used. For example, the statement,

```
d = (a + b) * c;
```

would add `a` to `b` first, multiply the result by `c` and assign the product to `d`. Associativity of an operator can be left-to-right or right-to-left. For example, in the expression.

```
d = a - b - c;
```

The first (left-most) minus is evaluated first. The variable `b` is subtracted from `a`. Then the second minus is evaluated, causing `c` to be subtracted from the result. Thus, in the case of many minus operators appearing in an expression without braces, the operators are evaluated starting from the left-most operator and proceed rightward. The minus operator thus associates from left to right. On the other hand, the equality operator associates from right to left. For example, in the statement,

```
d = a = c;
```

the second (right-most) equality operator is evaluated first. The variable `c` is assigned to `a`. The expression `a = c` evaluates to the value of `a` after the assignment has taken place. Then this value is assigned to `d`, when the left-most equality operator is executed.

All the operators have been listed here for the purpose of completeness. Many of the operators pertain to scop and pointers, and hence you may not be familiar with them. These will be explained in a later lessons. The operators are divided into 16 categories. The first category has the highest precedence; second category (unary operators) takes second precedence, and so on down to the comma operator, which has the lowest precedence. The operators within each category have equal precedence. The unary, and assignment operators associate right-to-left: all other operators associate left-to-right.

Order of evaluation of operands: The C language specifies the associativity of operators, but does not specify the order in which the operands of a operator are evaluated. For example, in a statement like

```
sum = (++a) - (--a);
```

the two operands are ++a and --a. It is left to the compiler as to which expression it evaluates first. If it evaluates ++a first, sum will be equal to -1, otherwise it will be 1.

Check Your Progress - I

- 1) State whether the following are True or False.
 - a) Operator is a symbol which represents a particular operation that can be performed same data.
 - b) Each relational operator needs are operand for comparison of the values.
 - c) ++ is the increment operator.
 - d) >= is the logical operator.
 - e) * is the multiplication operator.
- 2) Fill in the blanks
 - a) General form of conditional operator is _____ .
 - b) _____ is Modulo Division operator.
 - c) _____ is the right shift operator.
 - d) _____ operator needs two operands for comparison of their values.

3.4 Summary

In this Lesson we have covered operator and expression. We have also covered Arithmetic operators, logical operators, bitwise operator, relational operators and unary operators.

3.5 Check Your Answers

- I. 1) a) True b) False c) True d) False e) True
- 2) a) expression 1? expression 2: expression 3
- b) % c) >> d) Relational

3.6 Model Questions

- (1) It is a bad programming practice to rely on the order of evaluation of operands. Explain.
- (2) How does the use of the sizeof () operator lead to portable applications?
- (3) What is a normalized exponential expression?
- (4) What is an escape sequence? Explain.
- (5) What are the different types of operators available in C?
- (6) Explain the behaviour of the right shift operator for signed and unsigned numbers.
- (7) When is the const qualifier useful?
- (8) What do you understand by operator precedence? Explain the associativity of unary operators.

LESSON - 4

BASIC INPUT-OUTPUT STATEMENTS

Structure

- 4.1 Introduction
- 4.2 Learning Objectives
- 4.3 Input and Output functions
- 4.4 Summary
- 4.5 Check Your Answers
- 4.6 Model Questions

4.1 Introduction

C simply has no provision for receiving data from any of the Input device (like a keyboard, floppy etc.) nor for sending data to the output devices (like say VDU, floppy etc.). Then how do we manage I/O, and how is it that we were able to use printf () and scanf () if C has nothing to offer for I/O. This is what we intend to explore in this Lesson.

4.2 Learning Objectives

After learning this lesson, you should be able to

- discuss standard input and output functions.

4.3 Input and Output functions

As the names indicate, getchar reads a character from the standard input device, while putchar writes a character to the standard output device. First, we shall discuss an example involving both getchar and putchar.

```
/* getchar.c: Example involving both getchar and putchar */
```

```
#include <stdio.h>
```

```
void main ( )
```

```

{char c;
printf ( " Type one character : ");
c = getchar ( );
printf ( " The character you typed is : ")
putchar (c );}

```

A sample session with the above program is shown below. Note that the enter key is pressed after typing the character W.

Type one character : w

The character you typed is : w

getchar

In the above example, the first statement in the function main,

char c; declares c a character. In main, the third statement,

```
c = getchar ( );
```

uses getchar to input a character. Here, we observe the concept of return value returned by gerchar is value input from the keyboard. This is stored in the variable c.

putchar

The last statement of main is :

```
putchar ( c )
```

Here, the variable c is passed to putchar, which displays its value on the screen.

1) String Input and Output

The function gets receives the string from the standard input device, while puts outputs the string to the standard output device. Now, we shall see an example involving both gets and puts.

```
/*gets.c: Example involving both gets and puts */
```

```
#include <stdio.h>
```

```

void main ( )
{char s [ 80 ]
printf ("Type a string ( Less than 80 characters ):");
gets ( s );
printf ( " The string typed is : ");
puts ( s ) ; }

```

Run

Type a string (Less than 80 characters) : c Book

The string typed is : C Book

gets

From this example, you can observe that the methods of calling getchar and gets differ. In the previous section, notice that getchar does not take any parameter and returns the value of the input character; which is assigned to variable c. However, gets appears in the statement as shown below:

```
gets ( s );
```

The function gets accepts the name of a parameter, and fills the string with character is encountered (i.e., till we press the enter key). At the end, the function gets appends a null terminator as must be done for any string, and returns. The newline character is not added to the string.

puts

In the end, the statement,

```
puts ( s );
```

will display the string stored in s. In the above example, this is the same string that was read with gets previously.

2) General Output

The printf function is a very versatile output function. It can handle any basic data type, and offers several facilities with the data must be displayed. Briefly, the function printf performs formatted output device. This will become clear as we go through the examples.

The printf function does the following:

- Accept a format string followed by a series of arguments
- Apply to each argument, the corresponding format specifier contained in the format string
- Output the formatted data in the desired format

Recall that a format string is the first argument passed to printf. We also observed that a format specifier is a part of the format string which begins with a % sign (for example, %i corresponds to an integer). Every format specifier in the format string corresponds to a particular argument (any variable or constant passed to the function) that follows the format string. Format specifiers are used to specify two parameters:

- The type of the argument: for example, %d specifies an integer, %f a floating-point number, etc.
- The position and precision in which the output must be printed or displayed

Example

```
/* output.c: Example exploring the facilities offered by printf */

#include <stdio.h>

void main ( )

{float f_var = 10.12576893;

int wid_prec;

/*Output 1: Ordinary output. */

printf ("Output 1: %f\n ", f_var );

/*Output 2: output in a field of width 20 (Direct specification).*/
```

```

printf ("Output 2: %20f\n", f_var)

/* Output 3: %-20f\n", f_var);

printf ( "Output3: %-20f\n",f_var);

/*4: Explicit sign display. */

printf ( "Output 4:% + f\n " , f_var );

/*5: Indirect width specification (compare Output 2) */

printf ( "Input the width: " );

scanf ( "% i " , & wid_pre);

printf ( "output 5: % * F\n . f_var ) ;

/*Output 6: Precision specification 5 digits (Direct)

*/ printf ( "Output 6: %.5f\n, f_var );

/*Output 7: Indirect precision specification (compare output 6) */

printf ( "%i", & wid_prec);

printf ( " Output7:%20.*f\n ", wid_prec, f_var);

/* Output 8: A combination: Field width of 20, left justified, explicit sign, precision of 5 digits.*/

printf ( " Output 8: %- +20.5f\n " , f_var );}

```

The output is illustrated below in Figure 4.1 Several points can be observed here:

Output 1: This is the default way of writing a floating point number. Note that only 6 digits after the decimal point are printed. Here, s the default precision is 6. It can vary from system to system.

Output 2: When a field width (which is 20 here) greater than the actual width (in characters) of the output argument is specified, the output is right justified, with blanks added to the left of the output.

Output 3: The -sign in the format specifier left justifies the output, filling the right side with blanks.

Output 4: The + sign cause positive numbers to be preceded by a + sign. Note that negative numbers are anyway preceded by a - sign.

Output 5: The asterisk (*) in the format specifier enables us to determine the field width of the output during the execution of the program. An integer variable (wid_prec in the above example) whose value specifies the width precedes the actual variable to be output. This is called indirect width specification.

*/*Output 7: Indirect precision specification (compare Output 6) */*

```
printf ( " Input the precision : ");
```

```
scanf ( " %i", & wid_prec );
```

```
printf ( " Output 7: %20.*f\n " , wid_prec, f_var );
```

*/*output 8: A combination: Filed width of 20, left justified, explicit sign, precision of 5 digits.*/*

```
printf ( " output 8 : % - + 20.5f\n " , f_var );
```

Output	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Output 1	1	0	.	1	2	5	7	6	9											
Output 2												1	0		1	2	5	7	6	9
Output 3	1	0	.	1	2	5	7	6	9											
Output 4											+	1	0	.	1	2	5	7	6	9
Output 5					1	0	.	1	2	5	7	6	9							
Output 6	1	0	.	1	2	5	7	7												
Output 7										1	0	.	1	2	5	7	6	8	9	3
Output 8	:	+	1	0	.	1	2	5	7	7										

Fig. 4.1 Outputs

Output 6: A precision of 5 specified following the decimal point in the format specifier causes only 5 decimal digits to be printed after the decimal point.

Output 7: Similar to the width, the precision can also be specified during the execution of the program by specifying an asterisk (*) instead of a number. An integer variable (just preceding

the actual output variable) provides the precision.

Output 8: Here, the format specifier is `& -+20.5f`. The output is left justified. A sign precedes it. The width of the output field is 20. The number is output with 5 digits after the decimal point.

The components of the format specifier of the `printf` statement is as follows:

Beginning of the format specifier

Flags

Width specifier

Precision specifier

Type specifier

Variable

```
printf ("Output 8: % - + 20.5 f\n ", f_var);
```

Note: Always check whether the number of arguments specified after the format string is equal to the number of format specifiers in the format string. If more arguments are specified, they are ignored by the `printf` statement. The number of argument will be greater than the number of format specifiers if indirect width specification is used.

printf Conversion Type Characters for Numeric Data Types

Format speci	Type of argument	Output
<code>%d</code>	integer	signed decimal integer
<code>%i</code>	integer	signed decimal integer
<code>%o</code>	integer	unsigned octal integer
<code>%u</code>	integer	unsigned decimal integer
<code>%x</code>	integer	unsigned hexadecimal int (with a,b,c,d,e,f)
<code>%X</code>	integer	unsigned hexadecimal int (with A,B,C,D,E,F)
<code>%f</code>	floating point	signed value of the form [-] dddd.dddd
<code>%e</code>	floating point	signed value of the form [-] d.dddd or e [+/-] ddd

%g	floating point	signed value in the either e or from, based on given value and precision: Trailing zeros and the decimal point are printed if necessary
%E	floating point	same as e; with E for exponent
%G	floating point	same as g; with E for exponent if e format is used

Table 4.1 Format specifiers for printing numbers

--printf Conversion Type character for Qualified Data Types

Format specifier	Type of argument	Output
%ld, %li	long	decimal long integer
%lu,	unsigned long	decimal unsigned long integer
%hd, %hi	short	decimal short integer
%hu	unsigned short	decimal unsigned short integer
%le, %lf, %lg	double	signed, double
%Le, %Lf, %Lg	long double	signed, long double
%lo	long	octal long integer
%lx	long	hexadecimal long int

Table 4.2 Format specifiers for qualified numbers

Note that the letter l can be used with integer type -specifiers to specify the long forms of integers. Similarly, h is used to specify short integers. Also, l, when prefixed to a floating-point type character, e, f or g can be used to output double numbers. L is used for the long double type.

Note in floating point numbers:

- Infinite floating-point numbers are printed as + INF and -INF
- An IEEE Not-A-Number is printed as + NAN or - NAN

printf Conversion Types for character Datatype

<i>Format specifier</i>	<i>Type of argument</i>	<i>Output</i>
% c	char	single character
% s	Sequence of char	prints character until a null -terminator is pressed or precision is reached
% %	None	prints the % character

Table 4.3 Format specifiers for printing character

3) General input

Before going into the details of the scanf function, we will consider a couple of examples. In the previous lesson, single integer (the age in years) is accepted from the keyboard using the scanf function. Examples presented below illustrate the fact that a single scanf statement can be used to input multiple values into variable from the input device, just as a single printf statement can output multiple variable into the output device.

The following example inputs an integer and a floating -point number. The printf statements display these values.

```
/*scanf 1.c: program to illustrate the scanf function */

#include <stdio.h>

void main ( )

{ int int_var;

float float_var

printf ( " Input an integer followed by a floating - points number:");

scanf ( " %d %f" , &int_var, &float_var );

printf ( " The integer: % d\n: %d\n ", int_var );

printf ( " The floating - point number : %f\n ", float_var );}
```

Let us now experiment with our example by subjecting it to a number of trial runs, and try out various ways in which we can input the two numbers.

First, we shall try separating the two numbers by a space.

Input an integer followed by a floating -point

number : 34 56.57

— —————

The integer :34

The floating -point number : 56.570000

The output shows that scanf has input the two numbers properly. In fact, we can type any number of spaces between the two numbers and then type any number if spaces. Also, in place of one or more space, we may type one or more tab characters, and still scanf will input the numbers correctly.

Let us now try a second way of entering the numbers, by pressing the enter key after each value.

Input an integer followed by a floating -point number :34

—————

56.57

—————

The integer : 34

The floating -point number:56.570000

The function scanf works properly even if we separate the numbers with a newline. The concept of an input field is examined now. In the above program, consider the statement:

```
scanf ( " %d %f ", &int_var, &float_var );
```

As in printf, we shall use the term format string to refer to the string %d %f, passed to scanf as the first argument. Each of the format specifiers %d and %f in the string specifies the type of value to input when the program is run. when you type two numbers (one integer, and one floating-point) at the run time, each number is called an input field. Thus, in the example above, 34 is an input field and 56.57 is another.

Since scanf needs to know where one input field ends and where the next begins, the input fields (numbers in our case) can be separated by typing spaces, tabs, newline (enter) characters, or a combination of these. The characters that can separate different input fields are referred to as whitespace characters. There are 6 whitespace characters in all : space, tab, newline, vertical tab, carriage return and form feed. Any of these can be used to separate input fields.

The function scanf accepts a format string as its first parameter followed by a series of address of variable whose values are input. The scanf function has another important feature: it returns an integer that gives the number of input fields scanned successfully. To understand this concept, let us modify the previous example.

```

/*scanf 2.c: program to illustrate the scanf function */
#include <stdio.h>

void main ()
{
    /* int_var is an integer we will input, and success holds the value returned by scanf
(number of success-
fully scanned fields */
    int int_var, success;

    /*float_var is a floating -point number which we will
input. */
    float float_var ;

    printf ( " Input an integer followed by a floating - point number : " );
    success = scanf ( " % d % f" , & int_var, &float_var );
    printf ( " The integer : % d\n", int_var ) ;
    printf ( " The floating - point number : % f\n " , float_var );
    printf ( " scanf scanned % d field (s) successfully .\n", success); }

```

Let us first try ordinary case, separating the input fields by a space.

Input an integer followed by a floating -point
number : 34 56.57

— ———

The integer :34

The floating - point number: 56.570000

scanf scanned 2 field(s) successfully.

Next, let us try to type non-numeric data for the second input field.

Input an integer followed by a floating-point

number : 34 xyz

— ———

The integer :34

The floating - point number : 0.000000

Scanf scanned 1 field (s) successfully.

This time, scanf could scan the first input (34) successfully. It (scanf) expected a floating point number in the second input field, but we had typed xyz instead of a floating- point number. So, scanf failed to scanf this input field, and only one field was scanned successfully.

Let us another variation: entering a non - integer value for the first input field (the integer)

Input an integer followed by a floating - point

number: xyz 56.57

— ———

The integer :0

The floating- point number : 0.000000

scanf scanned 0 field (s) successfully.

This shows that scanf expected an integer in the first input field, but could not find one. So, it abandoned the rest of the input, and did not even attempt the next input field.

We now proceed to examine other features of scanf. The function scanf is a versatile input function. It can handle any basic variable type, and offers several facilities using which you can specify the way variables must be input. Briefly stated, the function scanf performs formatted input from the standard input device. The scanf function does the following:

- scans a series of input fields, one character at a time

- Converts each field according to a corresponding format specifier passed in the format string
- Stores the converted input at an address passed as an argument following the format string
- There must be one format specifier and address for each input field

4) Type of Characters in Format Strings

The format string is the first parameter passed to scanf. It controls the way in which scanf converts and stores the input fields. The format string is a character string that contains three types of objects:

- Whitespace characters
- Ordinary characters
- Format specifiers

Whitespace Characters

If scanf encounters a whitespace character in the format string, it reads, but does not store consecutive whitespace characters till the next non-whitespace character in the input.

Ordinary Characters

The ordinary characters are characters that are not whitespace characters, except the percent sign (%). If scanf encounters an ordinary characters in the format string, it will read, but not store a matching character.

Format Specifiers

The format specifiers direct scanf to read and convert characters from the input field into specific types of values, and then store them in the locations given by the address arguments. Format specifiers, as observed, begin with a & sign.

Here is an example that use all three types of characters.

/*scanf3.c: scanf using 3 types of characters in the format string. */

```
#include < stdio.h >

void main ()
{ char day [ 20 ];
  int date, month, year;

  printf ( " Input the day, followed by the date ( dd-mm-yyyy ) : " );
  scanf ( "%s %d -%d-%d", day , &date, &month, &year );
  printf ( " Day : %s\n ", day );
  printf ( " Date : %d\n ", date );
  printf ( " Month : %d\n ", month );
  printf ( "Year: %d\n ", year ); }
```

Note: that in the scanf statement, the string day is not preceded by the & symbol. This is because, the name of the string itself gives the address of the string. While reading strings, always pass the name of the string to scanf.

A sample session with the program is shown below:

Input the day, followed by the date (dd-mm-yyyy) : Sunday

27-1-1974

Day : Sunday

Date: 27

Month :1

Year : 1974

In the scanf statement

```
scanf ( "%d -%d -%d", &date, &month, &year );
```

the format string is %s %d- %d-%d. The format specifier are %s and the three %d specifiers. There is a whitespace character (a space in this case) after the first format specifier %s. We can type any number of whitespace characters to separate the first input (a string) from the rest of the input. The two hyphens (-) are ordinary characters. These instruct scanf to look for matching characters in the input stream. If any character other than a hyphen is entered to separate the three integers (the day, month and year), scanf would not have succeeded in scanning the input fields.

To illustrate, let us run the program again and type space instead of hyphens.

Input the day, followed by the date (dd-mm-yyyy) : Sunday

27 1 1974

Day : Sunday

Date : 27

Month :0

Year :2066

So, scanf has scanned the first two input fields (a string and an integer) successfully. But after the first integer (the date), scanf expected a hyphen (-), and could not find one. So, the variables month and year are not scanned at all. The values shown above (0 and 2066) are some values that the variables had when the program started. These values cannot be predicated, and hence, they may be different when you run the program.

Since scanf considers any whitespace characters to separate input fields, it is difficult to input strings by scanf. For example, consider the following example that inputs two strings.

```
/*scanf4.c: illustration of scanf function with strings */
```

```
#include <stdio.h>
```

```
void main ( )
```

```
{ char string 1 [ 20 ], string2 [ 20 ];
printf ( " Enter two strings : \n");
scanf ( " %s ", string 1 ) ;
scanf ( " % s ", string 2);
printf ( " You have input : \n% s\n %s\n", string1, string2 ); }
```

Note that the two statements,

```
scanf ( "% s", strings1 ) ;
```

```
scanf ( " % s ", string 2);
```

can be replaced by a single statement,

```
scanf ( " %s % s", string1, string2 ) ;
```

with exactly the same results. A sample session with no surprise is shown below.

Input two strings :

Hellow

world

You have input

Hellow

world

Next, let us try to include spaces in the input string.

Enter two strings:

A big hellow to all

You have input

A

big

We now see that string 1 is the string A, and string2 contains big. The space after the letter A in the input string is assumed to be the end of the first input field (the input field corresponding to string 1) and similarly, the space following big marks the end of the second input field by scanf. So, here is the moral of this story : in case you want to input strings that can have embedded spaces in them, use the function gets described earlier in this chapter.

5) scanf Specifier

In the above example, we declared two strings by using the statement:

```
char string g1 [ 20 ] , string2 [ 20 ]
```

Each string, string1 and string2 have 20 characters. By doing so, it is assumed that the user will not type more than 20 characters for each string. What happens if the user types more than 20 characters for one of them say string 1? The result are unpredictable, since string 1 has only 20 bytes in the memory , and scanf will store all inputs character into memory starting from the first byte belonging to string 1. It will overwrite some memory locations can be a part of another variable's memory, or a part of the program area itself.

To overcome such problems, scanf provides width specifiers that can be used in the format specifiers. Consider the example below.

/*scanf 5.c: illustration of scanf function with width specifiers */

```
# include < stdio.h >
```

```
void main ( )
```

```
{ char string 1 [ 5 ], string2 [ 6 ];
```

```
printf ( " Input two strings: " ) ;
```

```
scanf ( " % 4s %5s " , string1, string2);
```

```
printf ( " The two input strings are : \n % s\n % s\n " , string1, string2 ) ; }
```

In the scanf statement,

```
scanf ( " %4s %5s", string1, string 2 );
```

the format string is “ %4s %5s”, . It has two format specifiers %4s and %5s. The digits 4 and 5 are the width specifiers. string1 has 5 bytes and string 2 has even though their width is specified as 4 and 5, respectively, since every string has an extra null character at its end.

Let us run the program.

Input two strings: Morning comes

The two strings are :

Morn

ing

What scanf has done is quite obvious: it has put the first 4 characters (Morn) into string1. Then, it started to input characters and put them into string2. Upon encountering the space after the letter g, it stopped since a whitespace indicates the end of an input fields in all cases.

6) Search Sets

An example that enabled the user to input dates, separating the day, month and year by hyphens, was examined. Suppose, we want to have the option of separating these fields by either a hyphen (-) or a slash (/). For this, we use search sets. Search sets are characters enclosed in square brace, [and]. For search sets are parts of some format specifier. For example, the format specifier % [/ -] tells scanf to match any sequence of characters consisting of / and - All these characters are assigned to the corresponding argument. This argument must be the name of a string. Let us see how search are used.

```
/* scanf 6.c: illustration of scanf function with search sets */
```

```
# include < stdio.h>
```

```
void main ( )
```

```
{ int date, month year;
```

```
char separator [ 2 ];
```

```
printf ( “ Input the date :” );
```

```
scanf ( “ %d % [ / - ] % d % [ / - ] % d “ , & date, separator, & month, separator, & year );
```

```
printf ( " Input the date : " );
printf ( " Month : % d / n " , month );
printf ( " Year : % d / n " , year ) ; }
```

The scanf statement,

`scanf ( " % d % [ / - ] % d % [ / - ] % d %". & date, separator, & month, separator, & year);` is difficult to read. There are 5 arguments following the format string. The correspondence is shown in Table 4.4 below. The format specifiers are listed in the order in which they appear in the format string, from the left to the right.

A sample run of the above program follows:

Input the date : 31 - 12/1999

Date : 31

Month : 12

Year : 1999

While scanning the input, scanf will place the hyphen (-) after the date into the string separator and after scanning the month, it will place the slash (/) into separator again. Though the program works as expected, we are having the string separator without actually using it for any purpose other than temporarily strong the separator characters. To avoid this, a scanf format specifier can have an assignment suppression character in it. This is dealt with in the next section.

Format specifier	Corresponding argument
% d	& date
% [/ -]	separator
% d	& month
% [/ -]	separator
% d	& year

Table 4.4: Format Specifier

7) Assignment Suppression Character

Till now, we were having one argument for every format specifier in the format string. Sometimes, this is unnecessary, as shown in the example in the previous section. In such a case, using an assignment suppression character tells scanf that the input field should only be scanned and not assigned to any argument. The assignment suppression character is an asterisk (*), and it must be specified immediately after the % sign in the format **SPECIFIER**. Let us modify the example in the previous section using assignment suppression characters. Note that the string separator is absent in the example below.

/* scanf7.c: illustration of scanf function using assignment suppression */

```
# include < stdio.h >
```

```
void main ( )
```

```
{ int date, month, year;
```

```
printf ( " Input the date: " );
```

```
scanf ( " %d % * [ / - ] % d % * [ / - ] , & date, & month, & year ) ;
```

```
printf ( " Month : % d\n " , year ) ; }
```

```
printf ( " Year : % d\n " , year ) ; }
```

Here, the scanf statement has changed as follows :

```
scanf ( "% d% * [ / - ] % d % * [ / - ] %d", & date, & month, & year );
```

Still, there are 5 format specifiers, but only 3 arguments following the format string, Table 4.5 correspondence between the format specifiers and ARGUMENTS.

The program now behaves similar to the example in previous section. Only, we no longer need the string separator to hold the separator characters. The scanf function will scan, but not assign any input fields whose format specifiers have the assignment suppression character specifiers after the % character

Format specifier	Corresponding argument
%d	& date
% * [/ -]	(no arguments)
%d	& month

% * [/ -] (no arguments)

% * [/ -] & year

Table 4.5 Format specifiers in scanf 7.c

scanf Type Characters for Numbers

A list of type characters, which scanf expect at the input, and the type of argument (address to a particular type of variable) that must be specified corresponding to the format specifier having the type character, are given in Tables 4.6 and 4.7

Type character	Expected input	Type of argument
d	decimal Integer	address of an integer
D	decimal Integer	Address of a long int
o	octal Integer	Address of an integer
O	octal Integer	Address of long int
i	decimal, octal, or hexadecimal integer	Address of integer
I	decimal, octal, or hexadecimal integer	Address of a long int
u	unsigned decimal integer	address of an unsigned integer
U	unsigned decimal long	address of an unsigned long
x	hexadecimal integer	address of an integer
X	hexadecimal int -	address of a long int
e, E	floating point	address of a float
f	floating point	address of a float
g,G	floating point	address of a float

Table 4.6 scanf format specifiers for numbers

scanf Type Characters for Character Datatype

Type character	Expected input	Type of argument
S	character string	name of the string
C	character variable	address of a char
%	% character done; scanf looks for the % character in the input	no conversion

Table 4.7 scanf format specifiers for characters

8) Format Specifiers for scanf

We will not examine the details and variations (such as input width specifiers) that apply to the following format specifiers:

- Single character (%c)
- Character array (% [W] c)
- String (%s)
- Floating - point (%e, %E , %f , %g, %G)
- Unsigned (%d , %i, %x, %o, % x, %D, % I, %O, %X, %c, %n)
- Search sets (% [....] , % [^])

Single character (%c)

This specification reads the next character, irrespective of whether the character is a whitespace character or not.

Character array (%[w]c)

The address argument is a pointer to an array of characters: they array consists of w elements.

String conversion (%s)

The corresponding argument must be the name of a string. Do not precede this name with &. The function scanf copies the characters that are typed into a string variable

corresponding r to this particular format specifier, till a space or a newline character is encountered in the input field. A null terminator is automatically appended at the end of the string.

Floating-point conversions (%e, %E, %f, %g, %G)

Floating - point numbers can be input in one of the two forms:

- in the ordinary format : - 123.456
- using an exponential notation - 123456e-3 -123456 E -3 (E or e can be used)

Note: In addition, + INF, - INF, +NAN are recognized as floating - point numbers. The sign (+ or -) and capitalization are required.

Integer conversions (%d, %i, %o, %x, %D, %I, %O, %X, %u, %U)

These format specifiers can be used to read integers. The significance of each is given in next lesson, where a pointer to a character, integer or long respectively, is allowed.

Search Set Conversion (% [search - set])

The search_set specified above is a sequence of characters. While scanning the input, scanf scans all the characters encountered in the search_set. Square bracer surround a set of characters that define a search set of possible characters, making up the string (the input field). If the first character in the brackets is a caret (^) , the search set is inverted to include all ASCII characters except those within the brackets. The function scanf reads the corresponding input field up to the first character it meets that does not appear in the search set (or appears in the inverted search set.)

Example

To search the input field for any of the characters a,b,c and d % [abcd]

To search the input field for any character except a, b, c, and d % [^ abcd]

You can also use a range facility (using a hyphen to separate the lower and upper limits) to define a range of letters or numerals in the search set.

Example

To identify all decimal digits, you can define the search set in the following ways:

- (i) % [0123456789] to scan all decimal digits
- (ii) % [0-9] to scan all decimal digits using range

To identify alphanumeric characters, you can use the following syntaxes :

- (iii) % [A - Z] to scan all upper case letters
- (iv) % [0-9A- za-z] to scan all decimal digits and letters
- (v) % [A-FT-Z] to scan all upper case letters from A to F and from T to Z

Rules for Search Set Ranges

- The character prior to the minus sign (-) must be lexically less than the one after one after it, i, e., inside the search set, a-z is valid, but not z-a
- If hyphen is the first or last character in the set, it is considered to be the - character and not a range definer. Generally, the hyphen should not be the first or last character in the set
- The characters on either side of the hyphen must be the boundaries of the range and not part of some other range

9) Input Field for scanf

In a scanf function, any of the following is considered an input field:

- All characters up to (but not including) the next whitespace character
- All characters up to the first that cannot be converted under the current format specifier
- For a maximum of n characters, where n is the specified field width

When scanf stops scanning

This section examines situations where the function scanf either:

- Stops scanning a particular input field and skips to the next field
- Abandons scanning all remaining input fields and returns

scanf Stops and Skips to the Next Input Field

The scanf function stops scanning and storing the current input field: it proceeds to the next if any one of the following situations occurs:

- An assignment- suppression character (*) appears after the % in the format specifier: the current input field is scanned but not stored
- The number of characters specified by the width specifier have been read
- The next character read cannot be converted under the current format (for example, the character A is entered when the format is % i)
- The next character in the input field does not appear in the search set, or does appear in the inverted search set

When scanf stops scanning the current input field for one of these reason, it assumes that the next character is unread, and is either the first character of the character of the following input field, or the first character in a subsequent read operation.

The scanf function terminates under the following circumstance:

- The next character in the input field has a conflict with a corresponding non- whitespace character in the format string.
- The next character in the input field is EOF
- The format string specification has been exhausted
- If character sequence that is not a part of the format specifier occurs in the format string, it must match the current sequence of characters in the input field.

Check Your Progress - I

- 1) State whether the following are True or False.
 - a) printf() is the output function.
 - b) getch() is a single character output function.

2) Fill in the blanks

- General form of scanf () statement is _____.
- _____ is a string input statement.
- _____ is a single character input function.

4.4 Summary

In this lesson we have covered the use of `printf ( )`, `scanf ( )` functions, we have also examined the use of `gets ( )`, `getchar ( )`, `puts ( )`, and `putchar ( )` functions.

4.5 Check Your Answers

- I. 1) a) True b) False
- 2) a) scanf ("format string", list of address of variables);
b) gets () c) getch ()

4.6 Model Questions

- (1) What is the difference between scanf () and gets () with respect to data input with embedded white space characters
- (2) Write a program to read a sentence and display it.
- (3) For the printf () function, what are the format specifiers for specifying the type of arguments, the would occur in a format string of scanf ()
- (4) Describe the type of characters that would occur in a format string of scanf ()?
- (5) Describe the different format specifiers of scanf()
- (6) What are the components of a format specifier in a printf statement?
- (7) Explain search sets used in a scanf statement with examples.

LESSON - 5

CONTROL STRUCTURES - I

DECISION MAKING STATEMENTS

Structure

- 5.1 Introduction**
- 5.2 Learning Objectives**
- 5.3 If statement**
- 5.4 Summary**
- 5.5 Check Your Answers**
- 5.6 Model Questions**

5.1 Introduction

Many programs require that a logical test be carried out at some particular point within the program. An action will be carried out whose exact nature depends upon the outcome of the logical test. This is known finally, where is a special kind of conditional execution in which one group of statement is selected from several available groups. This is sometime referred to as selection.

5.2 Learning Objectives

After learning this lesson, you should be able to

- Explain decision making statements such as if statement, if - else statement

5.3 If Statement

Let us consider a simple example, where we required to execute or skip a statement depending on a certain condition. The program has to prompt for the user to input a number. If the user enters a number divisible by 3, a message has to be printed. If it is not divisible by 3, then the message need not be printed.

The following program uses an if statement to perform this action

```
/*if1.c: program to illustrate the if construct */

# include < stdio.h >

void main ( )

{ int i;

printf ( " Enter a number : " );

scanf ( " %d", & i);

if ( i%3 == 0 )

printf ( " Number entered is divisible by 3. \n " );}
```

First run:

Enter a number : 30

—

Number entered is divisible by 3

Note that the message is printed, since the number entered is divisible by 3.

Second run:

Enter a number:31

—

In the second run, no message is printed since 31 is not divisible by 3. The condition $i\%3 == 0$ in the statement is false. Hence, the statement following if is not executed.

Consider an example from the previous chapter, reproduced here for convenience.

```
/*scanf1.c: example to illustrate the necessity of the if statement */

# include < stdio.h>

voidmain ( )

{ float years, secs;

printf ( " Input your age in years : " );

scanf ( " %f", & years ) ;
```

```
secs = years * 365 * 24 * 60 * 60;
printf ( " You have lived for %f seconds \n ", secs );}
```

In the above program, the user is required to input a number as the age (a floating - point number, to be precise). What if someone enters a series of letters instead of a number? (Consider the following be precise). What if someone enters a series of letters instead of a number? Consider the following sample run:

Input your age in years: Deepak

You have lived for 0.000000 seconds

It is quite obvious: the function scanf failed to convert the input, sundeep, into a floating -point number. instead of printing the wrong answer (0.000000 seconds), as in the above example. Recall that scanf returns the numbers of input fields successfully scanned (illustrated in the second example scanf2.c. in the previous chapter). In this case, scanf would have returned zero, since there is only one input field (corresponding to the format specifier %f), and it is not successfully scanned. The program needs to execute the statement:

printf (" The input is not a floating - point number \n) ; only if scanf returns zero. For this purpose, an if statement is used.

```
/*if2.c example checks for a floating point input */
# include <stdio.h>
void main ( )
float years, secs;
int success;
printf ( " Input your age in years : " );
success = scanf ( " %f", & years : " );
if ( success == 0 )
printf ( " The input is not a floating -point number. \n");
secs = years * 365 * 24 * 60 * 60;
```

```
printf ( " You have lived for %f seconds \n " , secs ); }
```

Let us try a valid input first:

Input your age in years :75

—

You have lived for 2365199872.000000 seconds

Notice, try an invalid input:

Input your age in years: Deepak

The input is not a floating -point number.

You have lived for 0.000000 seconds

The function scanf fails to convert, and returns zero. This value is stored in the integer variable success. The expression,

```
success == 0
```

in the if statement evaluates to true and the program prints the message:

The input is not a floating-point number.

So far, so good. But the wrong answer is still getting printed. What we really want is that the two statements

```
secs = years * 365 * 24 *60 *60;
```

```
printf ( " You have lived for % f seconds s/n", secs );
```

at the end of main should get executed only if scanf successfully converts a valid input.

This can be done using another if statement, as shown below:

```
/* if 3.c: example-checkes for a floating point input */
```

```
#include < stdio.h>
```

```
void main ( )
```

```
{ float years, secs;
```

```
int success;
```

```
printf ( " Input your age in years")
```

```
success = scanf ( "%f", & years);
```

```

if ( success == 0)
printf ( " The input is not a floating -point number. \n");
if ( success != 0)
{ secs = years * 365 * 24 * 60 * 60;
printf ( "You have lived for %f seconds \n" secs ) ; } }

```

The second if statement is interesting: instead of one statement following the if statement, there are two of them. In such cases, where there are more than one statement to be executed depending on a condition, they must be enclosed within curly braces, {and}. A set of statements enclosed within the curly braces is referred to as a compound statement. Notice that main () is followed by a series of statements enclosed within curly braces. This also represents a compound statement. Thus a compound statement may consist of other compound statements.

Another example of the 'if' construct is given below. This program computes the largest of three numbers by using the if construct.

```

/*large.c: program to find the largest of three numbers */
#include <stdio.h>
main ( )
{ float a,b,c, big;
printf ( " Input a b c \n");
/* computing the largest of three numbers */
big = a;
if ( b > big ) big = b;
if ( c > big ) big = c;
printf ( " Largest of the three numbers = % 7.2 f\n ", big ); }

```

Run 1 :

```

Input a b c
10.00 20.00 45.00
— — — —
Largest of the three numbers = 45.00

```

Run 2:

Input a b c

78.00 45.00 23.00

Largest of the three numbers = 78.00

1) if -else Statement

The earlier example, if3.c prints the result (the age in seconds) only if scanf succeeds in converting the input, otherwise, it prints an error message. A more efficient way of writing the program would be to use an if -else statement, as shown below.

```
/*ifelse1.c: example checkers for a floating point input using
```

```
if -else*/
```

```
# include <stdio.h>
```

```
void main ( )
```

```
{float years, secs;
```

```
int success;
```

```
printf ( " Input your age in years: " );
```

```
success = scanf ( " %f", & years :");
```

```
if ( success ==0)
```

```
printf ( " The input is not a floating-point  
number. \n");
```

```
else
```

```
{ secs = years * 365 * 24* 60* 60;
```

```
printf ( " You have lived for % f seconds \n", secs ) ; } }
```

Vaild input:

Input your age in years: 100

You have lived for 31536000000.000000 seconds

Invalid input

Input your age in years: Deepak

The input is not a floating-point number.

The program is now working to our satisfaction. The statement under if executes only if the condition, `success == 0` is true, while the compound statement under else executes only if the condition is false.

One fact that has to be mentioned here, is that C treats any non-zero value as true, and zero as false. So, the expression `success == 0` can be re-phrased as `! success`. To make this clear, consider the expression `is success`.

This evaluates to true if the value of the variable `success` is non-zero, and false if it is zero. Hence, it is the same as the expression:

`success != 0`

When you apply the not operator (!) to an expression, its meaning is reversed. The expression,

`! success`

evaluates to true if `success` is zero, and false if `success` is non-zero. Hence, the statement, `if (success == 0)` in the above program can be replaced by `if`

`( ! success )`.

General form of if — else

`if (expression 1)`

`statement 1;`

`else`

`statement 2;`

Another example that uses the if-else construct is illustrated below.

/* ifelse2.c: illustration of if -else construct */

```
# include < stdio.h>

void main ( )

{ int input;

printf ( " Input an integer : " );

scanf ( " %d ", & input );

if ( input ! = 0 )

printf ( " It is non -zero. \ n");

else

printf ( " It is zero. \ n" ) ;}
```

Run 1 :

Input an integer : 12

—

It is non-zero.

Run 2 :

Input an integer : 0

-

It is zero.

In all above examples, the statement inside the if constructs are indented. The C language, however, does not require you indent statements. Indenting is done merely for the purpose of readability. The importance of indenting becomes evident during the use of nested if statements (if statements within other if statements: any number of nested if statements are allowed). For example, consider the if statement:

```
if ( precent == 100 ) if ( i > j ) z = i;

else z = j;
```

The above statement is perfectly valid as far as the compiler is concerned, but it is very difficult for reader to decipher it. An indebted version is presented below:

```

if ( percent == 100 ) {
    if ( i > j )
        z = i;
    else
        z = j;
}

```

The inner if statement is executed only if the variable percent is equal to 100. The inner if then causes either $z = i$ or $z = j$ to be executed. This indentation makes the logic of the statements explicitly clear. The syntax and the flowchart of the nested if -else statement are as follows:

```

if (expression 1)
    statement 1;
else if (expression 2)
    statement 2;
    |
else
    statement n;

```

An important fact to be perceived from the above example is that an else always associates itself with the closest (innermost) if that does not already have an else. In the previous example, else is combined with the inner if statement. If we want else to go with the outer if statement, braces should be used to enclose the inner if statement, as illustrated below:

```

if ( percent == 100 ) {
    if ( i > j )
        z = i; }
else
    z = j;

```

Often, wrong indentation results in hard-to-find bugs. For example the indentation in the if statement below is misleading. This indentation creates the else is associated with the outermost if statement, which is if (x == 50).

```
if ( x == 50 )
```

```
if ( i > j )
```

```
z = k;
```

```
else
```

```
z = 7;
```

Actually, else always gets associated with the nearest if statement, which in this case is : if (k > 7). The code to associated else with the outermost if is as follows:

```
if ( x == 50 )
```

```
{ if ( i > j )
```

```
if ( K > 7 )
```

```
z = K ; }
```

```
else
```

```
z = 7;
```

In general, when the logic is complex, it is a good idea to use braces to enclose each of the and else blocks.

2) Multi- ways Decisions

Multi-way decision arise when there are multiple conditions and different statements are to be executed under each condition. For example:

```
if ( i == j )
```

```
printf ( " i and j are equal. \n" );
```

```
else if ( i == k )
```

```
printf ( " i and k are equal. \n " );
```

```
else if ( l == 1 )
```

```
printf ( " i and l are equal. \n " );
```

```
else
```

```
printf ( " i is not equal. \n " );
```

It can be seen that the above structure is equivalent to a nested if- else structure. Use of nested if-else appears as:

Version 2

```
if ( i == j)
```

```
printf ( " i and j are equal. \n" );
```

```
else
```

```
if ( i == k )
```

```
printf ( " i and k are equal. \n");
```

```
else
```

```
if ( i == l )
```

```
printf ( " i and l are equal. \n " );
```

We shall consider the switch construct a little later in this lesson. This construct can be used to choose among several constant alternatives.

3) Compound Statements

A compound statement is a set of statements enclosed within a pair of curly braces, { and }. An example of a compound statement is an if statement in which a set of statement enclosed within { and } are executed when the condition is true, as shown in the previous illustration.

```
if ( success != 0 )
```

```
{ secs = years * 365 * 24 * 60 * 60;
```

```
printf ( " You have lived for % f seconds \n", secs ); }
```

The sequence of statements enclosed within curly braces,

```
{ secs = years * 365 * 24 * 60 * 60;
```

```
printf ( " You have lived for % f seconds \n ", secs );}
```

are considered as one compound statement. A compound statement can wherever a statement is allowed. This is simply another way of seeing the fact that whenever multiples

statements come under an if statement (or any other control structure that we describe in the rest of this lesson), they must be enclosed within curly braces.

4) Variable Declarations in Compound Statements

Compound statements are not simply sequences of executable statements. They can also contain variable declarations at the beginning. Variables can be declared at the beginning of a compound statement. A sample program to illustrate this is given below.

```
/*scope.c: example to illustrate the scope of the variables */
#include <stdio.h>

void main ()
{ int i = 10;

printf ( " In main. i is %d \n", i );

/* A compound statement.*/

{ /* Variable declared at the beginning of compound statement */
int i = 20;

printf ( " In compound statement. i is %d \n ", i );

i + +

printf ( " After incrementing : i is %d \n ", i ); }
```

The output of the program is as follows:

In main. i is 10

In compound statement. i is 20

After incrementing: i is 21

In main again. i is 10

Let us examine the reason for each of these outputs.

The function main begins with the variable i being declared an integer initialized to 10. Naturally, the first printf statement print:

In main. i is 10

This printf statement is followed by a compound statement. The compound statement begins with the declaration and initialization of an integer i again. But this time, i is initialized to 20. The interesting fact here is that the variable i declared first and that declared inside the compound statement, specify two different variables. Modification of the value of one does not affect the other. Inside the compound statement, two printf statements output the contents of the variable i. Which variable i, do these statements have an access of? They access declared inside a compound statement hides other variables outside statement prints the value of i as 20, increments it and prints the new value 21, giving the following output:

In compound statement > i is 20

After incrementing: i is 21

When the closing brace (}) of the compound statement is encountered, the variable i declared inside the compound statement ceases to exist. Hence, the last printf statement in main prints:

In main again i is 10

Check Your Progress - I

- 1) State whether the following are True or False.
 - a) The if-else statement is used to carryout a logical test.
 - b) The if-else statement is a looping control statement.
- 2) Fill in the blanks
 - a) The general form of simple if statement is _____.
 - b) _____ statement is a set of statements enclosed within a pair of curly braces.

5.4 Summary

In this lesson we have covered conditional statements and we also covered decision making statements such as if statement, if-else statement, nested if statement and switch statement.

5.5 Check Your Answers

- I.
- 1) a) False b) True
- 2) a) Compiler b) Interpreter c) Dennis Ritchie

5.6 Model Questions

1. What is condition execution?
2. Summarize the syntax rules associated with the if else statement

LESSON - 6

CONTROL STRUCTURE - II

LOOPING STATEMENTS

Structure

- 6.1 Introduction**
- 6.2 Learning Objectives**
- 6.3 Looping Statements**
- 6.4 Summary**
- 6.5 Check Your Answers**
- 6.6 Model Questions**

6.1 Introduction

Many programs require that a group of instructions are executed repeatedly, until some logical condition has been satisfied. This is known as looping. Sometimes the required number of repetitions will not be known in advance, the computation simply continue until the logical condition become true. Situation also arise, however, in which a group of consecutive instructions is repeated some specified number of times. This is another form of looping.

Many programs require that a logical test to be carried out at same particular point within the program. An action will be carried out whose exact nature depends upon the outcome of the logical test. This is known as conditional execution.

Decision making statement

- (a) if statement
- (b) if-else statement
- (c) switch statement

Loop constructs

- (a) for - loop
- (b) while - loop

(c) do - while loop

Discuss the usage of the control statements, break, continue and goto.

6.2 Learning Objectives

After learning this lesson, you should be able to

- explain different loop constructs such as for - loop, while - loop, do- while

6.3 Looping Statements

Loop in C case a section of the program to be executed repeatedly while an expression is true. When the expression becomes false, the loop terminates and the control passes on to the statement following the loop. A loop consists of two segments, one is known as the control statement and the other is the body of the loop. There are three kinds of loop in C:

- (a) for
- (b) while
- (c) do-while

1) for loop

The for loop is useful while executing a statement a number of times. For example, a program that display the first 10 multiples of 5 on a single line, is given below:

Example

```
/* for.c: illustration of the for loop */
# include <stdio.h>
void main ( )
{ int i;
  for ( i =1; i<= 10; i++)
    printf ("%i", 5*i);}
```

The output of the program is:

5 10 15 20 25 30 35 40 45 50

The for keyword is followed by three components enclosed within round braces,

(and). These three components are separated by semicolons. In the above example, the three components are:

`i = 1`

`i <= 10` and

`i++`

The first component, `i=1` is executed only once prior to the statements within the for loop. This is called the initialization expression. The second component `i <= 10` is evaluated once before every execution of the statements within the loop. It is called the test expression. If this expression is true, the statements, within the loop executes. If it is false, the loop terminates and the control of execution is transferred to the statements within the loop. In this case, this expression increments the value of `i` by 1. It is called the update expression.

The syntax of the for loop can be summarized as given below:

for (initial expression; test expression; update expression) single statement / compound statement

The general syntax, the flowchart, and the various syntax of the for loop are shown in Figure 6.1 and 6.2. Note that even in the case of a for loop, multiple statements can be enclosed within curly braces to form a compound statement.

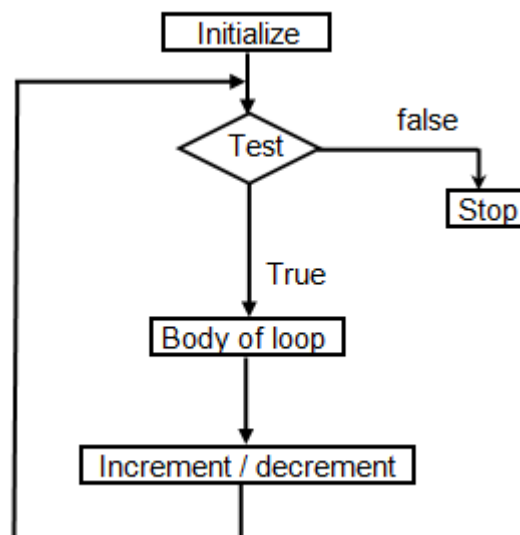


Fig. 6.1 flowchart for the loop

- (i) for (j = 0; (j < 25; j + +) —> No semicolon here
statement;————Single statement body
- (ii) for (j = 0; j < 15; j + +) ———> No semicolon here
{ statement; +
Compound statement body
statement; + }
- |
+ ———> x No semicolon here
- (iii) for (j = 0 ; j < 25 ; j + +) ; ———> Loop with no body
- (iv) for (i = 0, j = 0; j < 25; j + + —) —> print (“%d %d /n”, i, j) ;
Multiple initialization and multiple update by using comma operator
- (v) for (;; j + +) ———> Initialization and test expression not used
- (vi) for (;;) puts (“ I cannot stop .”) ; ———> Initialization and test expression not used
- (vii) for (;;) puts (“I cannot stop.”); —> Initialization, test, update expressions not used

Fig 6.2: Various Syntax of the for loop

- (i) **Initialization expression:** The control variable is initialized. This part is executed once before executing the body of the loop.
- (ii) **Test expression:** The value of the control variable is tested, and if found true, the body of the loops is executed; otherwise the loop is terminated. This expression is evaluated before every execution of the loop body. If this is not used, as in Figure 6.2 (iv) above, its value is assumed to be true for all iteration of the loop.
- (iii) **Update expression:** After executing the last statement in the for loop, the control is transferred back to the for statement. The control variable is updated using an expression, such as j + + and the test expression is evaluated to see if it is true before executing the body of the loop.

Note that the third construct in Figure 6.2(iii) ends with a semicolon, without a loop body. This construct simply performs the test and updates expression to be evaluated and executed for each iteration of the loop. For multiple initialization and update of loop variables, we use the

comma operator as shown in Figure 6.2 (iv). The variations in the for loop, without initialization expression, test expression and update expression are shown in (vii) of Figure 6.2.

Consider another example that finds the sum, and the sum of squares of the first 15 positive, natural numbers.

```
/* sumsq1.c: program to illustrate the sum of the
squares */

#include <stdio.h>

void main ( )

{ int i ;

int sum = 0, sum - of -squares = 0;

for ( i = 2 ; i <= 30 ; i + = 2)

{ sum + = i;

sum - of - squares + = i *i; }

printf ( " Sum of first 15 positive even numbers = % d /n", sum );

printf ( " Sum of their squares = % d /n ", sum - of - squares ) ; }
```

The output of the above program appears as follows:

sum of first 15 positive even numbers = 240

sum of their squares = 4960

Observe how, the loop variable is incremented by 2 using the update expression

i + = 2

The body of the loop consists of multiple statements and is, therefore, contained within braces, forming a compound statement.

The for loop in the above example counts from 2 to 30 in steps of two. As you might have understood already, it is just as easy for a loop to count down from 30 to 2. Consider the following variation of the previous program.

```

/* sumsq2.c: program to illustrate the sum of squares */

# include < stdio.h >

void main ( )

{ int i ;

int sum = 0, sum-of -squares = 0;

for ( i = 30; i > = 2; i - = 2 )

{ sum + = i;

sum - of - squares + = i* i; }

printf ( " Sum of first 15 positive even numbers = %d \n ", sum );

printf ( " Sum of the their squares = %d /n", sum - of -squares ) ; }

```

Notice the changes: the value of *i* is initialized to 30, the test expression involves the $\geq$ condition instead of $\leq$, and the update expression $i -= 2$ decrements the value of *i*. But the output in this case is identical to the previous program.

The comma operator is useful especially in for loops. Any of the initialization, test and update expression can have multiple expression separated by comma. Here is an example:

```

for ( j = 10, k = -5; j<25; j + +, k- )

{ printf ( " %d %d \n ", j,k ) }

```

In the initialization part, $j = 10$, $k = -5$ two variables *j* and *k* are assigned values. The test expression is $j < 25$. The update part is $j++$, $k--$. Two variables, *j* and *k* are updated.

Another interesting feature of the for loop is that any of the three components (initialization, test and update) may be left out. The separating semicolons, however, must be present and are shown in Figure 6.2. Consider a modified example that prints the first 10 multiples of 5, in which the for loop has only the test component.

```

/* for2.c: illustration of for loop without initialization

or update */

```

```
# include < stdio.h>

void main ( )

{ int i = 1;

for ( ; i<= 10;)

{ printf ( " %d ", 5 *i ); } }
```

Here, instead of the initialization expression, we have introduced the statement,

```
int i = 1;
```

before the loop. Also, instead of the update expression, we are incrementing *i* as a part of the loop. The output, of course, is the same as the previous example. Note again that the C language

does not require the user to indent statements in a for loop, or any other control structure. The lines are indented merely for the sake of clarity and readability

Example

The following example used the for loop to compute the sine series

```
/* sine. c: program to compute the sine series */

/* SIN ( X ) = X - X ^ 3 / 3! + X ^ 5 / 5! - ^ 7 / 7! + ..... X ^ N /N! * /

# include < stdio.h >

# include < math.h>

main ()

{ float x, t, sum ;

int i, n, sing = 1;

printf ( " Input X and N /n" );

scanf ( " %f %d", &x, &b );

/*Convert x into radians
```

122

```
x * = 3.1412 / 180;  
sum = t = x;  
for ( i = 0; i < n; i + + )  
{ sign * = - 1;  
t = sign * = x * x / (2 * i *; * (2 * i + 1) );  
sum + = t;  
printf ( " SIN ( X ) = %6.2 f/n ", sum ); }
```

Run 1:

Input X and N

90.00 10

SIN (X) = 1.00

Run 2:

Input X and N

60.00 12

SIN (X) = 0.86

2) While Loop

The for loop is more natural in places where the precise number of times the loop is to be executed is known before it is executed. The while loop is often used when the number of times the loop is to be executed is not known in advance. To see how the while loop can be used advantageously, consider the problem of calculating the average marks of a number of students.

One way is to input the number of students first, and then executed a for loop that inputs and sums the marks of all the students. Afterwards, the sum is divided by the number of students input, to get the average.

For the user of the program, this is an inconvenience, since the number of students must be counted before the average is computed. A while loop makes it easier for the user to input the marks. The user types in the marks one by one, and at the end of the list, enters a number that cannot possibly be valid marks (for example below). The program now recognizes that the list of marks is over, and prints the result. The complete program is given below :

Example

```
/* average.c: program to find the average of the marks */
#include <stdio.h>

void main ( )
{ /* i is the input variable sum is the sum of the marks, num is the number of marks . */
  int i, num = 0;
  float sum = 0, average;
  printf ( " Input the marks, -1 at the end \n"); scanf ( "%d", & i);
  while ( i != -1)
  { sum + = i;
    num + + ;
    scanf ( "%d ", &i );}
  average = sum / num;
  printf ( " The average is %6.2f", average ); }
```

A sample session with the program is given below:

Input the marks, - 1 at the end.

12

—

14

—

16

—

18

—

-1

—

The average is 15.00

The first scanf statement just before the while loop inputs the first marks and stores it in i, so that the statement inside the loop can have some valid data to work on. The scanf statement inside the loop reads the rest of the numbers one by one. When -1 is read, the condition in the while statement

`i != -1`

evaluates to false. So, the while loop terminates and the program execution proceeds with the statement immediately after the while loop, which in the above program is:

`average = sum / num`

Consider the case where the user inputs -1 as the first number. The condition in the while statement evaluates to false, and the statements inside the loop are not executed. The while loop, that is a top tested loop, since the test condition is checked prior to entering into the loop. Execution proceeds directly to the statement following the closing curly braces of the while loop, where the variable sum is divided by num. But initially, both sum and num are zero, and they have not been altered, and would result in a run time error. This can be prevented by using an if statement, as follows:

`if ( num != 0 )`

`average = sum / num;`

If a single statements are to be repeatedly executed based on a condition, the syntax of the while construct is :

`while ( test expression )`

`statement;`

If multiple statements are to be repeatedly executed based on a condition, the while - loop syntax is as shown below :

```
while ( test expression )
{ statement;
statement;.....}
```

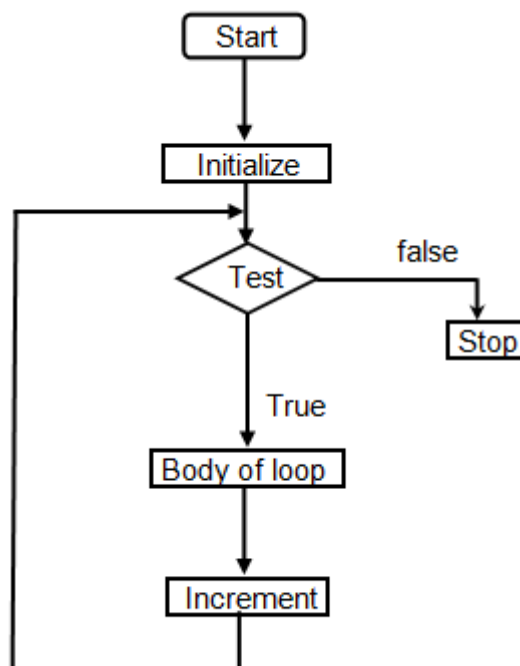


Fig. 6.3 Flowchart for while loop

- (ii) while (n < 45) → No semicolon here
 { statement ; + → Compound statement body statement ; }
- (iii) while (j - -) → Semicolon here —→ Loop with no body
- (iv) while (1) → No semicolon here —→ Infinite loop
 puts (" I cannot stop .");

Fig.6.4 Syntax of the while - loop

The while loop shown in Figure 6.4 (iii) has no body. It merely decrements the value of j till it is zero. The following example inputs a binary number as a string of 1's and 0's, converts it to decimal and displays the result.

Example

```
/* binary.c: program to convert binary number to a decimal number */

#include <stdio.h>

#include <string.h>

void main ( )

{ int bit , i, n;

  unsigned long decimal = 0;

  char binary [ 33 ] ; /* Maximum 32 bits*/

  printf ( " Input the binary number : " );

  scanf ( " %32s" , binary );

  n = strlen ( binary ) ; /*loop to convert binary to decimal */

  for ( i = 0; < n; i + + )

  { /* The statement below makes use of the fact that the ASCII value of the character '1' is
one more than that of the character '0', *

    bit = binary [ i ] - ' 0 ' ;

    decimal = ( decimal < < 1 ) + bit; }

  printf ( "The decimal equivalent of binary number %s is % lu/n", binary, decimal );}
```

Run 1:

Input the binary number : 1111

Decimal equivalent number 1111 is 15

Run 1:

Input the binary number : 1101

Decimal equivalent of binary number 1102 is 13

This program uses the while construct to convert a binary number to its decimal equivalent. The left shift operator, < < is used in this program.

3) do-while Loop

The while loop is top - tested, i.e., evaluates the condition before executing any of the statements in its body. The do-while loop, on the other hand, evaluates the condition after execution of the statements in its construct. The statements within the do-while loop are executed at least once. So, the do-while loop is called a bottom-tested loop.

If a single statement has to be executed repeatedly, the syntax is :

do

statement:

while (test expression);

Note the semicolon following the while statement at the bottom. The syntax of the do-while statement when a sequence of statements are to be executed repeatedly is as follows:

do

{ statement ;

|

statement;

.....} while (test expression) ;

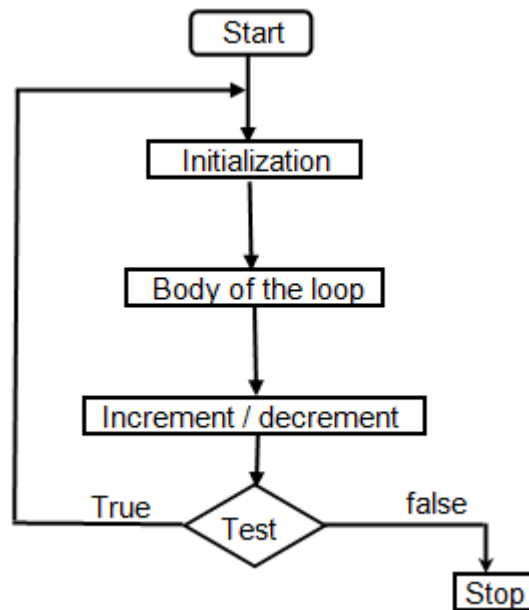


Fig. 6.5 Flowchart for do-while loop

- (i) do → No semicolon here
 statement ; ————— Single statement body
 while (n < 100) ; → Semicolon here
- (ii) do → No semicolon here
 { statement ;
 ;
 } > Multiple statement body
 statement n ; while (n < 100) ; → Semicolon here

Fig. 6.6 Syntax of do-while loop

To realize the usefulness of the do-while construct, consider the following problem. The user must be prompted to press y or n. In reality, the user can press any key other than y and n. In such a case, clearly, this is a loop construct. Also, it has to be executed at least once. This makes it ideal for the do-while loop. The following program illustrates the solution.

/* dowhile .c: program to illustrate the dowhile loop */

```
# include < stdio.h>
```

```
void main ( )
```

```
{ char inchar;
```

```
do
```

```
{ printf ( " Input y or n: " );
```

```
scanf ( " %c", &inclar ); }
```

```
while ( inchar != 'y' & & ubcgar != 'n' );
```

```
if ( inchar == 'y')
```

```
printf ( "You pressed y. /n " ); }
```

```
else
```

```
{ printf ( " You pressed n. /n" ); }
```

The do-while loop in the .above program,

```
do
```

```
printf ( " Input y or n: " );
```

```
scanf ( " %c" , & inchar ); }
```

```
while ( inchar != ' y ' & & inchar != ' n ' );
```

first prompts the user and then evelates the condition :

```
( inchar != ' y ' & & inchar != ' n ' );
```

It continues execution the body of the loop and prompts the user while the condition is true (i. e., while the input character is neither a y nor an n)

Example

A sample program that inputs an integer and checks whether it is a palindrome is given below. The program also sums up the digits of the given number.

```

/*palindrome.c: To check for a pallindrome */

# include < stdio.h >

main ( )

{ int n, num, digits, sum = 0, rev = 0;

printf ( " Input the number : " );

scanf ( " %d ", & num );

n = num

do

{ digits = num % 10 ;

sum + = digit;

rev = rev * 10 + digit;

num / = 10 ; } while ( num != 0 );

printf ( " Sum of the digits of the number = %4d \n ", rev );

if ( n == rev )

printf ( " Reverse of the number = % 7d \n ", rev );

else

printf ( " The number is not a palindrome \n " ); }

```

Run 1:

Input the number : 1234

Sum of the digits of the number = 10

Reverse of the number = 4321

The number is not a palilndrome

Run 2:

Input the number : 1221

Sum of the digits of the number = 6

Reverse of the number = 1221

The number is a palindrome

The program checks for a palindrome using a do - while construct. Note the use of mod operator %.

4) Break Statement

A break statement terminates the execution of the loop and the control is transferred to the statement immediately following the loop. During the discussion on the while loop, an example average.c that finds the average of a sequence of numbers entered by the user was illustrated. The main part of that program is the following loop:

```
scanf ( " d% ", & i );
while ( i != -1 )
{ sum + = i;
  num + +;
  scanf ( " d " , & i ) ; }
```

An alternate way of writing the above construct to eliminate the two scanf statements is the use of the break statement. The loop can be re- written as :

```
while ( 1 )
{ scanf ( " %d" , & i )
  if ( i == -1 )
    break;
  sum + = i;
```

```
num + + ; }
```

Whenever - 1 is input, the condition `i == -1` evaluates to true, and break statement is executed. This statement causes the execution of the loop to be terminated. Control passes to the statement following the while construct. Notice that the condition in the while loop has been specified as 1 (one) which is non-zero and hence, is always true. The condition specifies an infinite loop, but the break terminates its execution.

An infinite loop may be specified using any of the other two loop constructs. In particular, using the for loop, the same may be provided by :

```
for ( ; ; )
{ scanf ( " %d ", & i ) ;
  break;
  sum + = i;
  num + + ; }
```

Thus, none of the three components that go into a for construct are specified. No initialization or update is done, and the condition is implicitly assumed true, causing an infinite loop condition, again terminated by the break statement. In general, the break statement causes the control to pass to statement following the innermost enclosing while, do-while, for , or switch statement.

5) Switch Statement

A switch statement allows the user to choose a statement (or a group of statements) among several alternatives. The switch statement is useful when a variable is to be compared with different constants, a set of statements are to be executed. The constants in the case statement can be of char or int data type only.

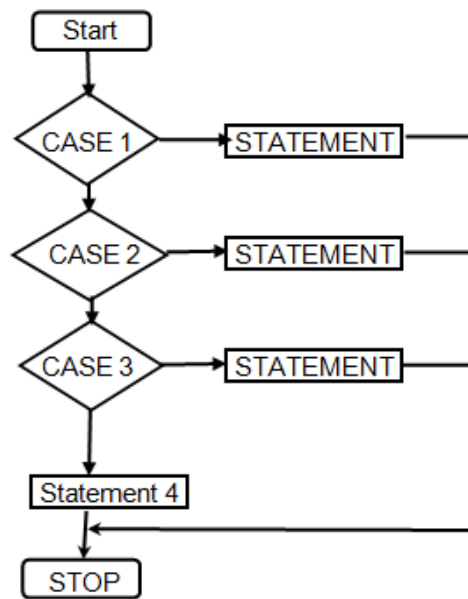


Fig. 6.6 Flowchart for switch statement

(i) switch (variable name) ➔ No semicolon here

{ + ———> Integer or character variable

Case 1:

statement 1;

statement n ; ➔ Body of case 1

break; +

Case 2:

statement 1 ;

statement ; > Body of case 2

break; +

.....

.....

default;

```

statement 1;
    > Body of default
statement n; ➔ No semicolon here
+—————> Integer valued expression
or character variable
+ — +
(ii)  switch ( n ) x No semicolon here
{ +—————> Integer or character variable
Case 1:
statement ; +
statement; > Body of case 1
break; +
Case 2:
Case 3:
statement; +
statement; > Body of case 2 or case 3
break; +
....
....
default;
statement; +
> Body of default
statement ; + } x No semicolon here

```

Fig. 6.7 Syntax of the switch statement

```

switch ( variable name )
{ case constant 1:
statements

```

case constant 2:

statements

...

default:

statements }

The switch statement will be explained using an example. Consider the following problem. It is required to automate an. interplanetary travel agent. The program must display a list of destinations and prompt the customer to press the number corresponding to his list. Depending on the number entered, it must give information about the destination. To see how a switch statement can make program- writing easier, first consider the solution to the above problem by using nested if - else constructs.

/ ifelse.c: Demonstration of if - else construct * /

```
#include <stdio.h>
```

```
void main ( )
```

```
{ int choice ;
```

```
printf ( " Choice of destinations : / n " ) ;
```

```
printf ( " / t1 - Mercury / n " );
```

```
printf ( " / t2 - Venus / n " )
```

```
printf ( " / t3 - Mars / n " )
```

```
printf ( " Enter the number corresponding to your choice : " );
```

```
scanf ( " ?% i " , & choice ) ;
```

```
if ( choice == 1 )
```

```
{ puts ( " Mercury is closed to the sun . " );
```

```
puts ( " So the weather may be quite hot there. " )
```

```
puts ( " The journey will cost you 5200 IGCs. " ); }
```

```

else
if ( choice == 2 )
{ puts ( " Venus is the second planet from the sun. " );
puts ( " The weather may be hot and poisonous. " );
puts ( " The journey will cost you 3200 IGCs. " ); }
else
if ( choice == 3 )
{ puts ( " Mars is the closest planet to the earth. " );
puts ( " The weather has been excellent till earth . " );
puts ( " The journey will cost you 1200 IGCs. " ); }
else
puts ( " Unknown destination. Unpredictable IGCs. " );
puts ( " / n Note : IGC = Inter -Galactic Currency " );}

```

The above nested if -else construct in essence, intends, to print information about the destination if a valid destination number is selected, or a message, unknown destination unpredictable IGCs. is printed if the number chosen is not valid. Finally, the message printed is :

Note : IGC = Inter - Galatic Currency

A sample session with the program is given below:

Choice of destinations:

1 - Mercury

2- Venus

3- Mars

Enter the number corresponding to your choice:2

Venus is the second planet from the sun.

The weather may be not and poisonous.

The journey will cost you 3200 IGCs.

Note: IGC = Inter - Galactic Currency

Example

The same example, using of the switch statement is given below:

/*switch.c: program to demonstrate the use of switch

statement */

#include < stdio.h >

void main ()

{ int choice ;

printf (" Choice of destination : / n");

printf (" t1 - Mercury / n");

printf (" t2 - Vecus / n ");]

printf (" t3 -Mars /n ");

printf (" Enter the number corresponding to your choice : ");

scanf (" %i ", & choice);

switch (choice) {

case 1 :

puts (" Mercury is closed to the sun . ");

puts (" So the weather may be quite hot there . ");

puts (" The journey will cost you 5200 IGCs.

case 2 :

puts ("Venus is the second planet from the sun . ");

puts (" The weather may be hot and poisonous . ");

```

puts ( " The journey will cost you 3200 IGCs. " ); break;

case 3:

puts ( " Mars is the closest planet to the earth. " );

puts ( " The weather has been excellent till now. " );

puts ( " The journey will cost you 1200 IGCs. " ); break;

default :

puts ( " Unknown destination. Unpredictable IGCs. " ); break; }

puts ( " / n Note : IGC = Inter - Galactic Currency" ); }

```

The above program generates exactly the same result as the program that used the nested if -else construct, but as you might have noticed, this program is easier to write and read. The following facts about the switch statement may be noted.

The expression following the switch keyword is an integer- valued of this expression decides the sequence of statements to be executed. Since we may treat characters to be of the type integer as and when required, the expression within the braces following the switch keyword may also be of the type character. The opening curly brace and its corresponding closing brace enclose a number of case lables.

Each sequence of statements is headed by the keyword case followed by constant integer. Note that constant characters may also be specified. Observe that the constant after the keyword case is followed by a colon, similar to a lable. Lables are examined later in this chapter under the goto statement. The constants must be unique within a given switch statement. Control is transferred to the statements under the case lable whose value is equal to the value of the expression in the statement.

The switch statement may have a group of statements labled with the keyword default. The statement under the default keyword is executed when the integer- valued expression or character variable assumes a value that does not correspond to any of the case lables. The default part is optional in the switch statement. The different cases and the default keyword may appear in any order. Braces are not necessary to group statements under a particular

case, even if multiple statements come under a case. The colon following the case expression constitutes a case statement delimiter.

You may be wondering, why the break statement is required at the end of each sequence of statements inside the switch construct. As discussed earlier, the break statement causes the control to break out of the innermost loop in which it is included. Similarly, here, it breaks out of the switch statement following the closing curly brace of not included at the end of a particular sequence of statements under some case label? The statements belonging to the next case label get executed.

This fact can be exploited in certain situations, for example, where a set of statements are to be executed in multiple conditions. This is illustrated in the following segment code.

```
char in - char;
puts ( " B - Blue ");
puts ( " R - Red " );
puts ( " Press the letter corresponding to your choice : " );
scanf ( " % c " , & inchar );
switch ( inchar )
{ case 'b' :
  case 'B' :
    puts ( " The colour is Blue " );
    break;
  case 'r' :
  case 'R' :
    puts ( " The colour is red " );
    break;
  default:
    puts ( " Invalid choice " );
    break ; }
```

The program allows the user to specify the letter either in the upper case or in the lower case. It is a good practice to put a break after the last case (which is default here), though it is logically redundant. Later, if you want to add another case statement, this kind of programming will save a lot of time.

Note : In case the default keyword is not included in the switch statement, and none of the case labels match the integer - valued expression, the switch statement will take no action. Control proceeds to the statement following the case structure.

Example

The following program computes the roots of a quadratic equation. Note the use of break at the end of each case statement to transfer the control out of the switch statement.

```
/*quad.c: Roots of a quadratic equation */
#include <stdio.h>
#include <math.h>

main ( )
{ float a, b, c, real , num, image, root1, root2, disc;
  int k;
  printf ( " Input a b c : / n ");
  scanf ( " % f % f ", &a, &b, &c );
  if ( a != 0 )
  { disc = b * b - 4 * a * c ;
    printf ( "Discriminate = % 5.2 f/n ", disc );
    if ( disc < 0 ) k = 1;
    else if ( disc == 0 ) k = 2;
    else if ( disc > 0 ) k = 3;
    switch ( k )
    { case 1 :
```

```

printf ( " Roots are imaginary / n " );
real = -b / ( 2*a );
disc = -disc;
num = pow ( double ) disc, ( double ) 0.5 );
imag = num / ( 2*a );
printf ( " Root 1 = % 5.2f + j % 5.2 f / n ", real, imag );
printf ( " Root 2 = % 5.2 f - j % 5.2 f / n ", real, imag ); break;
case 2 :
printf ( " Roots are real and unequal / n " );
root 1 = - b ( 2 *a);
printf ( " Root = Root2 = % 7.2 f / n", root1 ); break;
case 3 :
printf ( " Roots are real and unequal / n " );
root 1 = ( -b + sqrt ( ( double ) disc ) ) / ( 2 * a )
root 2 = ( -b - sqrt ( ( double ) disc ) ) / ( 2 * a )
printf ( " Root 1 = % 7.2f Root 2 = % 7.2 f / n ", root 1, root2);
break ;
} / * switch */
} / * end - if */
else
printf ( " Equation is linear / n " ); } / * main */

```

Run 1 :

Input a b c :

1.00 2.00 1.00

Discriminate = 0. 00

Roots are real and equal

142

Roots 1 = Root2 = - 1.00

Run 2 :

Input a b c :

1.00 2.00 7.00

Discriminate = - 24.00

Roots are imaginary

Root 1 = - 1.00 + j 2. 45

Root 2 = - 1.00 - j 2.45

Run 3:

Input a b c :

0.00 4.00 7.00

Equation is linear

6) Continue Statement

The continue statement is used to bypass the remainder of the current pass through a loop. The loop does not terminate when a continue statement is encountered. Instead, the remaining loop statements are skipped and the computation proceeds directly to the next pass through the loop. This statements is written as, continue; without any embedded statements or expressions.

Example

Consider the following program that finds the sum of five positive integers. If a negative number is entered, the sum is not performed since the remaining part of the loop (i.e., the statement `sum + = num`) is skipped using the continue statement.

```
/*continue.c : program to demonstrate the use of continue statement */
```

```
# include < stdio.h >
```

```
void main ( )
```

```

{ int i = 1, num, sum = 0;
for ( i = 0; i < 5; i + + )
{ printf ( " Enter an integer : " );
scanf ( " %i ", & num );
if ( num < 0 )
{ puts ( " You have entered a negative number " );
continue ; / * skip the remaining part of the loop * / }
sum + = num ; }
printf ( " The sum of the positive integers entered = %i / n " , sum ); }

```

Run 1 :

```

Enter an integer ; 7
-
Enter an integer ; 3
-
Enter an integer ; 10
—
Enter an integer ; 15
—
Enter an integer ; 30
—
The sum of the positive integers entered = 65

```

Run 2 :

```

Enter an integer ; 10
—
Enter an integer ; 15
—
You have entered a negative number

```

Enter an integer ; 15

—

Enter an integer ; - 100

——

You have entered a negative number

Enter an integer : 30

—

The sum of the positive integers entered = 55

Similarly, the continue statement can be used in other loops such as while and do- while loops.

7) goto Statement

The goto statement is used to alter the normal sequence of program execution by unconditionally transferring control to some other part of the program. This statement is written as goto label;

Here label is an identifier used to label the target statement to which the control would be transferred. Control may be transferred to any other statement within the current function. The target statement will appear as :

label : statement;

A note on the goto statement

World over, use of the goto statement is frowned upon, especially in languages such as C that have more elegant structures, such as while, for , etc. Any while or for loop can be replaced by an if statement coupled with a goto statement. But this makes the program less readable. The use of a while loop, for example, makes it clear that a condition. On the other hand, replacing it with if and goto statement obscures the fact to some extent.

On the other hand , there are situations where goto can make flow of control more obvious. This is especially evident when it is necessary to get out of deeply nested loops. For

example, here are for and goto structures that determine in common. The element x has n elements and y has m elements.

```

for ( i = 0 ; i < n; i + + )
for ( j = 0 ; < m ; j + + )
if ( x [ i ] = = y [ j ] )
goto found;
/* Element not found */
...
found :
/* Element found
....

```

The following version uses a variable fd to record the fact that a common element has been found, and is used as part of the condition expression in the for loop.

```

fd = 0;
for ( i = 0; i < n & & ! f; i + + )
for ( j = 0 ; j < m && ! f ; j + + )
if ( x [ i ] = = y [ j ] )
fd = 1;
if ( fd = = 1 )
/* Found */
else
/* Not found */

```

Except in cases such as the one cited above, the use of the goto statement must be avoided.

Programming Examples

Example

```

/* expo.c : program to compute exponential series */

/* e^x = 1 + x + x^2 / 2 ! + x^3 / 3 ! + .. x^n / n ! N represents number of terms */

# include < stdio. h >

main ( )

{ float x, t, sum ;

  int i, n, ifact

  printf ( " Input x and N : " )

  scanf ( " % f % d " , & x & n );

  t = 1;

  sum = 1;

  printf ( " E raised to power x = % 10. 4 f / n ", sum ); }

```

Run 1 :

Input X and N : 1.00 2

E raised to power x = 2. 00000

This program computes the exponential series and sample runs are shown above.

Example

```

/* fact.c: program to find factorial of a number, Non recursive method */

# include < stdio.h >

main ( )

{ int n, i, fact;

  printf ( " Enter the number : " );

  scanf ( " % d , &n );

```



```

if ( n == 0 )
printf ( " Factorial of 0 is 1 / n" );
else
{ fact = fact * i ;

```

Run 1:

Enter the number : 0

-

Factorial of is 1

Run 2:

Enter the number : 5

-

Factorial of 5 is 120

The program computes the factorial of a number using an if-else control structure.

Example

```

/* pascal.c: program to generate PASCAL'S triangle */
# include <stdio.h>

main ( )
{ int binom, p,q, r, x;
  binom = 1;
  q = 0;
  printf ( " Input the number of rows : " )
  scanf ( " %d ", & p );
  printf ( " Pascals triangle : / n " )
  while ( q < p )
  { for ( r = 40 - 3 * q; r > 0; -- r )

```

```

printf ( " " );
for ( x = 0; x <= q; ++ x )
{ if ( ( x == 0 ) || ( q == 0 ) )
    binom = 1 ;
    else
    binom = ( binom * ( q - x + 1 ) ) / x;
    printf ( " %6d " , binom ) ;
} / * for */
printf ( " / n " );
++ q;
} / * while */ }

```

Run:

Input the number of rows : 9

-

Pascals triangle

```

1
1   1
1   2   3
1   3   3   1
1   4   6   4   1
1   5   10  5   1
1   6   15  20  15  6   1
1   7   21  35  35  21  7   1
1   8   28  56  70  56  28  8   1

```

Pascal's triangle is useful in computing the coefficient of a polynomial. The program uses while, for and if structures. Proper spacing to get the shape of a triangle is manipulated by the loop;

```

for ( r = 40 - 3 * q; r > 0; -- r )

```

Example

```

/* area.c: program to find a area of a triangle */

# include <stdio.h>

# include <math.h>

main ( )

{ float a, b, c, base, height, area, s,p ;

int k;

printf ( " Enter 1 for triangle ( 3 sides ) & 2 for triangle (2 sides) : " );

scanf ( " % d ", & k );

printf ( " Enter 3 sides - a, b and c : " );

scanf ( " % f % f ", & a, & b , & c );

p = a + b + c ;

s = ( a + b + c ) / 2;

area = sqrt ( ( double ) ( s * ( s - a ) * ( s - b ) * ( s - c ) ) );

printf ( " Perimeter = % 7. 2 f / n ", area ) ; break ;

} / * switch * /

} / * main */

```

Run 1:

Enter 1 for triangle (e sides) & 2 for triangle (2 sides); 2

-

Enter two sides: 3.00 2.00

Area of a triangle = 3 . 00

Run 2:

Enter 1 for triangle (3 sides) & 2 for triangle (2 sides); 1

-

Enter 3 sides - a , b and c : 4. 00 5. 00 6.00

Perimeter = 15.00

Area of a triangle = 9.92

Run 3:

Enter 1 for triangle (3 sides) & 2 for triangle (two sides); 1

-

Enter 3 sides - a , b and c : 1. 00 2. 00 3.00

Perimeter = 6.00

Area of a triangle =0.00

The above program illustrates the use of switch statement.

Example

```
/* rev pyramid.c : program to print reversed pyramid */
```

```
# include <stdio.h>
```

```
main ( )
```

```
{ int p, q, r, t, s
```

```
printf ( " Enter the no of lines in the pyramid : " );
```

```
scanf ( " % d , & t );
```

```
printf ( " / t / t REVERSED PYRAMID OF DIGITS / n " );
```

```
for ( p = t ; p > 1 ; p -- )
```

```
{ printf ( "          " );
```

```

for ( r = 0 ; r <= p * 4 ; r + + )
printf ( " " );

for ( q = p ; q <= ( 2 * p ) - 1 ; q + + )
printf ( " % 4d " , q );

for ( s = ( 2 * p ) - 2 ; s >= p ; s - - )
printf ( " % 4d " , s );

printf ( " / n " ) ; }

printf ( " % d / n " , p ) ; }

```

Run 1 :

Enter the no of lines in the pyramid : 7

REVERSED PYRAMID OF DIGITS

```

7 8 9 10 11 12 13 11 10 9 8 7
6 7 8 9 10 11 10 9 8 7 6
5 6 7 8 9 8 7 6 5
4 5 6 7 6 5 4
3 4 5 4 3
2 3 2
1

```

The above program illustrates the manipulation of space to print numbers in the form of a reverse pyramid.

Example

```

/* cosine.c: program to compute consine series */

/* Cos (X) = 1-x^2/2! +x^4/4!-x^6/6!+.... x^n/n! */

#include <stdio.h>

```

152

```
#include <math.h>

main ( )

{float x,t,sum;

int i,n = 20;

printf ("Input X:");

scanf ("%f",&x);

/* converting x into radians */

x = x*3.1412/180;

t = 1;

sum = 1;

for (i = 1; i < n+1; i++)

{ t = t*pow (double) (-1), (double) (2*i-1) *x*x / (2*1* (2*i-1));

sum = sum + t;}

printf ("COS (X) = %7.3f\n", sum);}
```

Run 1:

Input X : 45.00

COS (X) = 0.707

Run 2:

Input X : 90.00

COS (X) = 0.000

This program computes the cosine series using the concept of for loop. Observe the use of power function pow whose arguments are of the type double.

Example

```
/* pyramid.c: To construct pyramid of digits */
#include <stdio.h>

main ( )
{ int p,m,q,n;
  printf ("Enter the number of lines:");
  scanf ("%d", &n);
  /* To print space */
  for (p = 1; p <= n; p++)
  {for (q=1; q <= n-p; q++)
    printf (" ");
    /* To print numbers */
    m = p;
    for (q = 1; q <= p; q++)
      printf ("%4d", m++);
    m = m - 2;
    for (q = 1; q < p; q++)
      printf ("%4d", m - - );
    printf ("\n\n");}}
```

Run 1:

Enter the number of lines: 7

-

1

2 3 2

3 4 5 4 3

4 5 6 7 6 5 4

5 6 7 8 9 8 7 6 5

6 7 8 9 10 11 10 9 8 7 6

7 8 9 10 11 12 13 12 11 10 9 8 7

For loops are extensively used in the manipulation of space and digits to form a pyramid of numbers as in the above example. The number stating in each row is incremented by one, then increased and decremented, to end the row with the same number.

Check Your Progress - I

- 1) State whether the following are True or False.
 - a) The general form of the for statement is for (exp1; exp2; exp3)
 - b) The do-while tests the condition before executing any of the statements within the do-while loop.
- 2) Fill in the blanks
 - a) _____ tests the condition after having executed the statements within the loop.
 - b) _____ statement is used to bypass the remainder of the current pass through a loop.

6.4 Summary

In this lesson, we have covered the looping control statements. Such as for, while, do-while, switch break statement.

6.5 Check Your Answers

- I. 1) a) True b) False
- 2) a) do-while b) continue

6.6 Model Questions

1. Explain the difference between while and do-while loops with respect to the minimum number of times the loop is executed.
2. Explain the significance of switch-case construct. Explain the use of break statement in this construct.
3. When the default keyword is useful in a switch-case construct?
4. Explain the role of initialization, test and update expressions in a for-loop. When are each of them executed.
5. Write a program to generate the first hundred terms of Fibonacci series and print their sum and average.
6. Write a program that determines whether or not the input string is a palindrome.
7. Write a program to input a string, sum the ascii values of all its characters and output the resulting value.

LESSON - 7

PREPROCESSOR

Structure

- 7.1 Introduction
- 7.2 Learning Objectives
- 7.3 Macro Definitions
- 7.4 Summary
- 7.5 Check Your Answers
- 7.6 Model Questions

7.1 Introduction

The C preprocessor is a collection of special statements, which are executed at beginning of the compilation process. Preprocessor directions usually appear at the beginning of a program, though this is not a firm requirement. Thus, a preprocessor direction may appear anywhere within a program. However, the direction will apply only to the portion of the program following its appearance.

7.2 Learning Objectives

After learning this lesson, you should be able to

- Macro definition and Macro functions

7.3 Macro Definitions

C macro definitions are offspring of the macros employed in assembly language programs. They work in the following manner: During compilation, the compiler checks the program for # define statements and creates a table that contains the symbol being defined, along with the values to be assigned; when the compiler later finds references to the symbol within the code, it simply locates the symbol in the table and exchanges the actual value for the symbol within the executable code.

Consider this example where MAXVAL is defined to be 200:

```
# define MAXVAL 200

main ( )

{int scores [MAXVAL];

int grades [MAXVAL];

int idnim [MAXVAL];

/* remainder of program code */}
```

During compilation, the compiler will store the symbol MAXVAL in a table along with the value 200. When the compiler later comes across the symbol MAXVAL, it substitutes the value 200. The executable code will eventually look like this.

```
main ( )

{ int scores [200];

int grades [200];

int idnum[200];

/* Remainder of the program code */}
```

Three advantages are gained with macro definitions: increased modifiability, portability and readability. For example, the following program will read and print characters until an EOL (end of line) or EOF (end of file) is encountered.

```
# include <stdio.h> /* character I/O */

# define EOL '\n' /* end of line */

# define EOF -1 /* end of file */

# define AND && /* logical AND */

# define NEWLINE '\n'

main ( )

{ int letter; /* read/print */}
```

```

while ( (letter=getchar ()) !=EOL AND letter!=EOF)

putchar (letter);

puthar (NEWLINE);}

```

By employing the constant definitions, our program becomes more readable. Constants in this case have increased the portability and modifiability of the program. If this program is running on a system that represents the end of file (EOF) as a -1 and it needs to be converted to a system that represents the end of file into -3, we need only change the line of code containing the `#define EOF` statement. By limiting the number of changes, we greatly reduce the possibility of introducing errors and decrease the time and effort required to get the program running on the new machine.

Again, it is important to note that definitions appear in UPPERCASE letters to distinguish them from lowercase variable.

1) Include Statement

As your programs grow in complexity, you may find that you are always using the same definitions throughout your routines. Rather than continually rewriting this list of definitions in each application, it is often convenient to develop a single file that contains all of your definitions. Then, when a new definition is required it can be added to the file to insure that it can be used in other routines. once you have created this file of definitions, C allows a program to reference its contents as long as an `#include` statement is placed prior to `main ( )` within the program.

For example, assume that you have created a file named `defn.h` that contains your macro definitions. All of your C routines can reference the definitions contained in this file as long as you provide the following statement at the top of each program.

```

#include "defn.h" /* note the quotes */

main ( )

{/* remainder of statement here */}

```

The quotation marks around the file name serve as delimiters. these are different from the brackets < > that enclose the file `stdio.h`. the difference between the quotation marks “ ” and the brackets < > lies in the directories the compiler will search to find the files. If we include a file with quotation marks

```
#include "myfile.h"
```

the compiler will first search the user's default or working directory and then the standard libraries. If we use the < >

```
#include <stdio.h>
```

the compiler will search only the standard libraries for the file.

2) Header Files

Each of the files we have included within our programs `stdio.h` and `defn.h` has the `.h` notation. the `.h` informs the compiler that the files are header files that contain definitions to be placed prior to `main ( )`. In addition to header files, C allows us to `#include` other C source files in a program. In this way, we need only specify one file at compilation time. We denote these files with a `.c`. Once a file has been included all of the routines it contains can be accessed throughout the code. For example, if we have already developed a function called `wordlen ( )` that we need to access within our program, we can include the file containing the function as follows:

```
#include "wordcnt.c"
```

```
main ( )
```

```
{/* remainder of instructions */ }
```

Most compilers will append the file `wordcnt.c` to the end of the program, and its contents are compiled with the program to produce the executable image.

3) Macro Functions

We have seen the use of the `#define` statement for symbol and constant declarations. C also provides a powerful tool in the macro function. The next program defines a macro called `cube ( )` to produce the cube of the values from 1 to 10.

```
#define cude (x) x*x*x

main ( )

(int count ; /* number to be cubed */

/* cube the numbers from 1 to 10 */

for (count=1;count <=10;++count)

printf(" count = %d cube = %d\n", count, cube (count);}
```

During compilation, each time the macro `cube ( )` is referenced, the compiler will replace it, in line, with the code for the macro. This is convenient for two reasons. First, the actual expression need be written only once and can then be assigned a meaningful name. For example, we can now use `cube (3.1459)` as opposed to continually writing the expression:

```
3.1459 * 3.1459 * 3.145
```

Second, because the code is placed in line, there is no overhead cost during execution, as would occur in a normal function call. Because of these advantages, you should consider using macros as often as possible in place of complex expressions.

The following statements perform an identical function:

```
b=feet * 0.3048;

b=fee_to_meters(feet);
```

The macro `FEET_TO_METERS ( )` increases readability with no cost to efficiency. Most C systems provide a file of macros called `ctype.h`. Print this file and examine the macros provided by C.

4) C Preprocessors

In addition to the `#include` and the `#define` statements, C provides other preprocessors, these preprocessors affect the compilation process and are generally termed conditional compilation operators. The pound sign (`#`) is used to designate these operators.

`#if`

`#ifdef`

`#undef`

`#ifndef`

`#else`

Each operator examines a specific condition. If the condition is true, the code that is contained between the `#condition` and the `#endif` or `#else` is compiled and included within the executable code. If the condition is false, the conditional code is not compiled. Applications requiring such preprocessors are rare. However, the implementation of each preprocessor will be presented here to make you aware of its experience.

The `#if` causes the code that is contained within the `#if` and the `#endif` (or `#else`) to be compiled only if the condition following the `#if` evaluates to true. In the following example, the statement

`printf ("Invalid string size \n");` will be compiled only if the constant `STRING_SIZE` is not equal to 512.

```
#if (STRING_SIZE !=512)
```

```
printf ("Invalid string size \n");
```

```
#endif
```

The `#IFDEF` will compile the code contained within `if` and the `#endif` only if the identifier referenced has been properly defined. The following example uses the `#ifdef` to test if `STRING_SIZE` has been defined

```
#ifdef STRING_SIZE  
  
printf ("STRING_SIZE properly defined \n");  
  
#endif
```

The statement `#ifdef` compiles the code only if the identifier referenced has not been previously defined. Periodically, we will attempt to define a constant that has already been defined in the standard library (`stdio.h`). Rather than redefining the value, we can test whether or not it is defined and if not define it:

```
#ifndef NULL  
  
#define NULL '\0'  
  
#endif
```

Finally, the statement `#undef` undefines the variable or constant specified from that point on in the program. If we want to redefine a constant that is defined in `stdio.h`, we can use `#undef` to undefine it before defining it with a `#define`. By undefining the constant first, we will not get a warning message during compilation:

```
#undef identifier
```

Example

```
#ifdef EOF  
  
#undef EOF  
  
#endif  
  
#define EOF -5
```

Check Your Progress - I

- 1) State whether the following are True or False.
 - a) C preprocessor is a collection of Special Statements.
 - b) A preprocessor direction may appear anywhere within a program.

- 10) Print out the file `stdio.h`. Which definitions have we used so far? Write a program that uses conditional compilation operators to determine whether or not the file `stdio.h` has defined `EOF` and `NULL`. If so, what are the values?

- 11) Write a program that creates the macros.

`max(a,b)` => the maximum of a and b

`average(a,b)` => the average of a and b

`min(a,b)` => the minimum of a and b

Use a for loop to print values returned from the macros for the numbers 0 through 100.

0,1 max =1, average =0.5, min =0

99,100 max =100, average=99.5 ,min=99

- 12) Write a program for the following?

- a) Write a program with a macro to find the roots of the quadratic equation.
- b) Write a program with a macro to find the smallest of three given numbers.

LESSON - 8

FUNCTIONS

Structure

- 8.1 Introduction
- 8.2 Learning Objectives
- 8.3 Function Main
- 8.4 Summary
- 8.5 Check Your Answers
- 8.6 Model Questions

8.1 Introduction

A number of statements grouped into a single logical unit is referred to as a function. As mentioned in the first Lesson, a C program is a collection of functions. The function `main ( )` in the program is executed first. This gives rise to the question: When are the other functions of the program executed? Normally, the other functions are executed when the function `main ( )` calls them directly or indirectly (i.e., `main ( )` calls some function which in turn calls other functions and so on: this can be nested to any level desired). It is necessary to have a single function `main ( )` in every C program, along with other functions used/defined by the programmer.

Actually, we have been using functions all along, `main ( )`, defined in the programs, is itself a function. Also, `printf` and `scanf` are two predefined functions that have been used so far. In this chapter, programs are made modular by breaking them up into functions.

8.2 Learning Objectives

After learning this lesson, you should be able to

- discuss how functions are useful.
- explain how functions accept more than one parameter.
- discuss various concepts like Function declaration, Function definition, passing arguments, return, Function call.

8.3 Function Main

The function main is also a user defined function except that the name of the function, the number of arguments, and the argument types are defined by the language. The individual statements and statement blocks are written by the programmer in the body of the function main (). The following, in a sense, is a complete C program.

```
Main ( ) {      }
```

The above statement is sufficient for the execution of the program, though the program does not do any useful operation. The function main 0 is executed first, when the program starts execution. Other user-defined functions can be called from the function main.

Getting started

To begin with, a simple C program with the function main given below:

```
/* main.c: program to illustrate the function main */
```

```
#include <stdio.h> /* header file */
```

```
main ( ) /* Control is passed to this statement */
```

```
{ printf ( "Welcome to functions" ); }
```

The above example has the following elements:

- * A preprocessor #include() directive
- * A function heading main ()
- * A function body, delimited by curly braces { and }
- * A printf statement to display a message

Compile the above program. When the name of the executable file is typed at the command prompt, the program runs. The function main is executed first. The statements within the body of main () are executed. In this case, there is only statement. When control is passed to the function main, the statement,

```
printf ( "Welcome to functions" \n);
```

is executed, that displays a message on the screen. After this statement is executed, the Closing brace of the function main () is encountered. The program then exits. The program has only one function, i.e., main.

As mentioned earlier, the function main is executed first. The following example has a function printMesg in addition to main. If a call to the function PrintMesg is included in the function main, the statement within the body of the function main. A prefix void is used before the function name, i.e., PrintMesg indicating that the function does not return any value.

```
/*prtmsg2c. function to print a message */
#include <stdio.h>

void PrintMesg ( )

{printf ( "PrintMesg, Welcome to functions \n" ); } void main ( )

{printf ( "Main, Welcome to functions \n" );

printMesg ( ); }
```

The output of the above program is:

Main, Welcome to functions

PrintMesg, Welcome to functions

Note that the function name in the above ex (PrintMesg) has mixed case, in contrast to the functions that we have used so far (such as printf and scanf) which consists entirely of lowercase characters. Using mixed case is a popular alternative to using underscores to make identifier names more readable. Function names are also identifiers. Hence, functions can have any name, as long as they satisfy the rules for identifiers given in Lesson 10.

1) Where are Functions Useful?

To appreciate the usefulness of functions and learn how to use them, we shall consider a simple program. Here, we shall define two functions; one of them computes the cube of a number, and the other is a special-purpose display routine. The function which computes the cube of a number is coded as:

```
int cube (int i)
{ int retval;
  retval = i * i * i;
  return retval; }
```

Notice that the first line of the function definition is:

```
int cube (int i)
```

The first `int` specifies that the function returns an integer (more precisely, it is the data type of the return value). The word `cube` is the name of the function. The next `int` inside the round braces specifies that the function accepts an integer. As far as the function `cube` is concerned, the integer which it accepts is `i`. Next, the body of the function follows (i.e., the actual statements that make up the function) between a pair of curly braces `{and}`, just as in the function `main ( )`.

We need to make a small digression here. The function definition varies from compiler to compiler. The older compilers require the function definition to be as follows:

```
int cube ( i )
int i;
(int retval;
  retval = i * i * i; return retval; }
```

In this scheme, only the names of parameters are mentioned within the parentheses following the function name. The data type of the parameters are given after the closing parenthesis, but before the opening curly brace of the function body. The K&R C uses this older standard. Many compilers built using the newer standard called ANSI C, can accept both old and new-style declarations (the `cube` function first follows the new ANSI C declaration). This book largely follows the ANSI C standard.

After the function `cube` has been defined thus, it can be called from `main ( )` (or any function, for that matter) by a statement such as:

```
cube (10);
```

The function is called by using the function name `cube` and an integer (in this case 10), enclosed within round braces. The above statement calls the function `cube`. Inside `cube`, the value of `i` is 10.

Inside the function `cube`

Let us now see what happens inside the function `cube`. Just after the Opening brace `{`, the statement

```
int retval;
```

declares the variable `retval` an integer. This integer is local to the function `cube`, that is, the variable `retval` can be used only inside the function `cube`. Any other function (for example `main`) cannot access it. However, if `main ( )` also defines a variable called `retval`, then the two are treated as different variables (more about this in a later chapter on scope).

The next statement,

```
retval = i * i * i
```

is the actual computation. After this statement is executed, the variable `retval` will have the cube of the integer `i`. But what is `i`? As mentioned before, the value of `i` is the value that is passed at the time of calling the function `cube` (10 in this case).

Return value

The last statement of the function,

```
return retval;
```

causes the function `cube` to return the value stored in `retval`.

But what happens to the returned value? In this example, the statement which calls the function is:

```
cube ( 10);
```

and the return value is discarded.

An example of how the return value can be utilized is given here:

```
int c;
```

```
c = cube (10);
```

```
printf ( "The cube of 10 is %d\n", c);
```

Here, c is declared an integer. The second line,

```
c = cube (10);
```

calls the function cube. The statements of cube are executed. When the statement,
return retval;

is encountered in the function cube, the value of integer retval in cube (which is now 1000) is assigned to the variable c in main and the control returns to the statement following the function call, which is

```
printf ( " The cube of 10 is %d\n", c);
```

Actually, the following statement has the combined effect of the above three lines.

```
printf ("The cube of 10 is %d\n", cube (10) );
```

The function call expression cube (10) is used in place of the integer variable passed to printf. This has the effect of first calling the function cube with the value of 10. The returned value (which is 1000) is passed to printf. Since the return value of cube is an integer, a call to cube can be used wherever an integer value is used.

Example

A complete program that defines and uses the function cube.

```
/* cube2.c: Program to illustrate a function */
```

```
#include <stdio.h>
```

```
int cube (int i)
```

```
{int retval;
```

```
    retval = i * i * i;
```



```

return retval;}

void main ( )

{int input_int;

/* Cube of a constant integer: */

printf ( "The cube of 10 is %d\n", cube (10) );

/* Cube of an input integer: */

printf ( "Input an integer: " );

scanf ( " %d", & input_int);

printf ( "The cube of %d is %d\n",. input_int, cube( input_int) ); }

```

Run:

The cube of 10 is 1000

Input an integer: 20

—

The cube of 20 is 8000

Placement of the function

One factor to notice here is the order of placement of the functions. The function cube is placed before the function main. This is appropriate, because the function main uses the function cube and hence, needs to know about it. However, in many cases, it is desirable to place the function main at the beginning. For example, in a large program having many functions, the function main is placed first in order to give the reader an overview of the logic. The reader can then perceive the logic in greater detail as he proceeds through the program.

The problem now is that while compiling main (), the functions called in it are not known to the compiler. To assist the compiler, we write the information about each function interface: its name, return type and the parameters which it accepts, prior to main itself. In main, these functions can be called. The actual code of these function can be placed after main.

Example

For example, if we want to place the function cube after the function main, we would do this:

```
/*cubecl.c: program illustrating function declaration */

#include <stdio.h>

/* Function cube... Declaration */

int cube (int);

void main ( )
{int input_int;

/* Cube of a constant integer */

printf ("The cube of 10 is %d\n", cube( 10) );

/*Cube of an input integer*/

printf ( "Input an integer: " );

scanf ( "%d", &input_int);

printf ( "The cube of %d is %d\n", input_int, cube( input_int ) );}

/* Function cube... Definition */

int cube( int i)
{ int retval;

retval = i * i * i

return retval; }
```

The output of this program is identical to that of the previous one. Observe the line,

```
int cube (int);
```

at the beginning of the program. It is called a function declaration. It informs the compiler that cube is a function that returns an integer value, and accepts an integer as an argument to the function. The identifier name i in the braces is unnecessary. A semicolon must always

follow the closing brace of the function declaration. Note that the function declaration informs the compiler about the function interface as it appears from outside. It says nothing about the code that comprises the function cube. The ANSI C standard uses this standard for declaration. If you have a compiler that conforms to the earlier K&R standard, it will generate an error. To rectify this, change the declaration to:

```
int cube ( ) ;
```

Though the function cube accepts an integer, the earlier K&R standard does not allow you to specify it in the function declaration.

Calls to cube are made as a part of the printf statements in main. After main, comes the definition of the function cube. The function definition contains the actual code that constitutes the function.

The function definition starts with the return data type, the function name, and the arguments it accepts (within round braces). However, after the closing braces, there is no semicolon. The function body follows, enclosed within a pair of curly braces.

In large programs having several functions, there is an added advantage in declaring all the functions at the beginning; we do not need to keep track of the order in which the functions are called.

You may now ask: what happens if the function declaration,

```
int cube (int );
```

(or `int cube (0;` in case of the older standard)

at the beginning of the above program is removed? Will the programme still be compiled without errors? This is a good question, and a tough one to answer. Whether the program compiles or not depends on the compiler that you use. There are three kinds of compilers in this respect: the early K&R compilers, ANSI compilers and the C++ compilers (remember that the C++ language is a superset of C and a C program can be compiled using a C++ compiler).

In any C compiler (either K&R or ANSI C compiler) there will be no compilation error. But a C++ compiler will report an error upon encountering a call to cube inside main. So, what is the advantage of declaring a function if a C compiler is used to compile the program? The answer is, type checking. In any compiler that follows the ANSI standard, the declaration.

```
intcube (int );
```

informs the compiler that the function cube accepts an integer. So calls to cube such as

```
cube ( );
```

having no parameter, or using a wrong parameter as.

```
cube ( "Hello");
```

will cause the compiler to report errors at the compile time itself. This may not seem very important in this example. But in case of functions accepting several parameters, a careless mistake may cause the program to produce unexpected results while running, requiring a lot of effort to be spent on debugging.

On the other hand, the older function declaration style

```
int cube ( );
```

causes any C compiler (either K&R or ANSI C) to assume that cube is a function that accepts any number of arguments. In other words, no type checking is done at the compile time at all. On the other hand, a C++ compiler assumes that the function cube accepts no parameter at all.

It is not necessary to remember all these details. Follow the new style of declaration,

```
int cube (hit);
```

if your compiler does not report errors due to the declaration (the compiler follows the new ANSI C standard). This will make the programs more reliable, and reduce the time spent on debugging. If the compiler reports errors on this line, it is a K&R style compiler. In this case, you have no option other than using the old style declaration.

```
int cube ( );
```

Be extra careful about the type and number of parameters passed in a function call.

Let us add one more feature to the program. Suppose the user enters an invalid input in response to the scanf statement (for example, he enters xyz, which is not a valid integer), the program must beep and output an error message.

For this purpose, the ASCII character set has a BEL character (in C, the BEL character is indicated by `\a`). When this character is output, it produces a beep sound. Let us now define a function called `beep`, that will emit a beep when it is called. Note two things about this function; it does not accept any value, nor does it need to return anything.

```
void beep ( )  
  
{ printf( "\a" );
```

There are several points of interest in the above function: First since the function does not need to return any value, the keyword `void` is used in place of the data type of the return value. Second, it does not accept any parameter. This is indicated by having a pair of closed round braces following the function name without any arguments within them (note that the braces are necessary). Lastly, a return statement is not included in the function `beep`. In case of a function not returning any value, it is optional to have a return statement at the end.

In the example below, the `beep` function contains a return statement at the end.

```
void beep ( )  
  
{ printf( "\a" );  
  
  return; }
```

This has the same effect as the previous `beep` function that did not use the return statement.

The function `beep` can be called by the statement:

```
beep ( );
```

Note that the empty braces are necessary, even in the calling statement.

Example

The program with the functions cube and beep, along with modifications in main () is given below.

```
/*cube3.c: Program to illustrate a function */
#include <stdio.h>

int cube (int i)
{int retval
  retval = 1 * 1 * i;
  return retval;}

void beep()
{printf ( "\a" ) ;}

void main ( )
{int input_int;

/* Cube of a constant integer: */
printf( "The cube of 10 is %d\n", cube( 10) );

/* Cube of an input integer: */
printf ( "Input an integer: ");
if ( scanf ( "%d", &input_int) < 1)
{beep ( );
  printf ( "Input is invalid!\n" );•
  else
    printf ( "The cube of %d is %d\n", input_int, cube( input_int)); }
```

The use of functions makes programming easier, since:

- * Repeated statement can be grouped into functions
- * Splitting the programs into separate functions makes the program more readable and maintainable

In the above program, for example, there is one major advantage in introducing the beep feature by using the function called beep. In future, if you want to modify the program to speak out a message instead of emitting a beep, all you have to do is to change the function beep and leave the rest of the program as it is.

People who are familiar with PASCAL or FORTRAN must remember that C makes no distinction between functions and procedures, which can return a value of some type (specified by int, float, etc., before the function name), or returns nothing (specified by void before the function name).

Example

The above example is re-written, with the function declarations at the beginning, and the definition of main before defining other functions.

```
#include <stdio.h>

/* Function declarations... */

int cube (int);

void beep( void);

void main ( )

{
    int input_int;

    /* Cube of a constant integer */
    printf ( "The cube of 10 is %d\n", cube (10) );

    /* Cube of an input integer */
    printf ( "Input an integer: " );

    if ( scanf ( "%d", &input_int) < 1)

    {beep ( );

    printf ( "Input is invalid\n" )

    else
```

```
printf ("The cube of %d is %d\n", input_int, cube( input_int)); }
```

```
/* Function cube... Definition */
```

```
int cube (int i)
```

```
{ int retval;
```

```
  retval = i * i * i;
```

```
  return retval; }
```

```
void beep (void)
```

```
{ printf( "\a");
```

Note the function declaration of beep at the beginning as shown below:

```
void beep (void);
```

The keyword void has been included within empty braces. This is done to inform the compiler that the function beep takes no argument. Recall that the declaration,

```
void beep ( )
```

informs a C compiler to assume that beep accepts any number of parameters But a C++ compiler assumes that the function beep takes no arguments. So the keyword void inside the braces is unnecessary for a C++ compiler.

Note, however, that the keyword void at the beginning which specifies that there is no return value, is always necessary. Without that, a declaration such as

```
beep (void);
```

causes any compiler to assume that the beep function returns an integer.

2) Functions Accepting more than One Parameter

The function cube accepted one parameter. A function can accept more than one parameter. The parameters are separated by commas. For example, consider the following program having a function maxfunc that accepts three parameters and computes their maximum.


```

#include <stdio.h>

int maxfunc (int i, int j, int k)

(int max;

if (i >= j && i >= k)

max = i;

else if (j >= k)

max = j

else

max = k;

return (max);

void main ( ) {int m, a, b, c;

printf ( "Input 3 numbers: " )

scanf( "%d %d %d", &a, &b, &c);

m = maxfunc (a, b, c);

printf ( "The maximum is %d\n", m );}

```

Run:

A sample run of the program is given below.

Input 3 numbers: 4 6 5

- - -

The maximum is 6

The statement

in = maxfunc(a, b, c);

in main calls the function maxfunc. Three integer variables are passed to it, separated by commas. The return value is assigned to m and is displayed. Since maxfunc comes before the

function main, separate function declaration and function definition are unnecessary. However, in case you want the function main to appear first, place the function declaration,

```
int maxfunc ( int, int, int);
```

before the function main.

The variables m, a, b and c are declared in main. They can be used only in the function main. An attempt to use them in maxfunc causes a compile time error. Similarly, the variables max, i, j and k (i, j and k being the parameters which the function accepts) belong to the function maxfunc. These variables cannot be accessed in main. The region of the program where an identifier can be accessed is called the scope of the identifier. Thus, the scope of is the function main, while the scope of i is the function maxfunc.

Note the variables with the same name can exist in both main and maxfunc. For example, the variable in main can be changed to max, and the program would still work perfectly. Even though variables in different functions can have the same name, they are treated as distinct variables.

3) User Defined and Library Functions

In the previous chapter, we used functions such as printf and scanf. These functions have already been written, compiled and placed in libraries and are called library functions.

In the present chapter, several functions have been coded, such as PrintMesg, cube, beep and maxfunc. Functions such as these, where the user has the freedom to choose the function name, return data type and the arguments(number and type), are called user defined functions.

One of the greatest features of C is that there is no conceptual difference between the user defined functions and library functions. For example, we can write a series of functions to process matrices (such as finding the inverse, calculating the determinant, etc.). These are user defined functions, since they are written by the user. Later, they can be compiled into libraries and distributed. To the users of the library, these functions will act as library functions. That is, these, functions are predefined and precompiled; the users need not worry about how

the functions work internally. The process of creating libraries is discussed in some other lesson.

4) Concepts Associated with Functions (review)

- (i) Function declaration or function prototype
- (ii) Function definition (function declarator and a function body)
- (iii) Combination declaration and function definition
- (iv) Passing arguments
- (v) Return statement
- (vi) Function call

Parts of a Function

A function declaration can appear outside all the other functions. In this case, all functions know about the other function declarations and can call these functions. Functions can also be declared within other functions. In that case, only the function within which the declaration is present will know about it. For example, the above declaration can appear inside function main:

```
void main ( )
{
    int i;
    double Cube (double);
}
```

Fig.8.1 Parts of a function

```
void main ( )
{
    + -> + — (write) Semicolon here
    void funcl ( ); ————— Function declaration
    + — +
    | ....
}
```

Return type < - + ...

...

func1 () ; ————— Function call

...

...

...}

Return type <—+ +-> X (wrong) No semicolon here + — +

void fund () ————— Declarator +

{

... + > Function

... >Function body Definition

... + +

}

+ - - > X (wrong) No semicolon here

Function Declaration

A function declaration provides the following information to the compiler:

- * The name of the function
- * The type of the value returned (optional, default is integer)
- * The number and the type of arguments that must be supplied in a call to the function

When a function call is encountered, the compiler checks the function call with its declaration, so that correct argument types are used. A function declaration has the following syntax:

```
ret_type function_name (type, ... , type );
```

ret_type specifies that data-type of the value in the return statement. A function can return any data-type; if there is no return value, the keyword void is placed before the function name. The function declaration terminates with a semicolon. For example:

```
double Cube (double);
```

The above declaration informs the compiler that the function Cube has one argument of type double (the list of data types separated by commas form the argument list). The function Cube returns a double value; the compiler knows how many bytes to retrieve and how to interpret the value returned by the function. Function declarations are also called prototypes, since they provide a model or blueprint of the function.

Function Definition

The function definition is similar to the function declaration but does not have the semicolon. The first line of the function definition is called a function declarator. This is followed by the function body. It is composed of the statements that make up the function, delimited by braces. The declarator and declaration must use the same function name, number of arguments, argument types, and the return type. No function definition is allowed within a function definition.

```
/*function1.c: illustrates function declaration within main */
void main (void)
{
    (int i, j; /* variable declarations */
    long fact (unsigned j, int num);
    /* function declaration or function prototype */ }
```

The definition of the function Cube is given below:

```
Double Cube (double dNum)
{
    return dNum * dNum * dNum;
}
```

As defined by the declaration, the function definition of Cube takes an argument of type double and returns a value of type double. A function can contain any number of statements. The statements are enclosed within curly braces, { and }. The function definition may include declarations of variables and declaration of other functions. Note that the return statement need not enclose the return value within parentheses.

Elimination of function declaration

C gives the programmer freedom to place function declarations anywhere in the program. If the functions are defined before they are called, then the declarations are unnecessary. The following example illustrates this concept.

```

/*funded.c: program to eliminate function declaration within main */

#include <stdio.h>

int Max (int a, int b)

{return a>b? a : b;}

void main (void)

{int i, j, iMax;

printf ("Enter two numbers :")

scanf ( "%d %d", &i, &j );

iMax = Max (i,j);

printf ("The maximum of %d and %d is %d", i, j, iMax );}

```

The function definition Max is defined before being called in the function main, eliminating the necessity of a function prototype or function declaration. In a program where there are several functions that call each other, the programmer has to arrange the functions, such that they are defined before being called by another function. However, in larger programs it is preferable to declare all the functions in the beginning to avail the flexibility of arranging the function definitions in any order.

Function Return Value

Functions in C may or may not have return values. People who are familiar with languages like FORTRAN and PASCAL must note that there are no separate entities as functions and procedures in C. In Pascal, for example, a function returns a value, and a function call can be used just like an expression. Procedures, on the other hand, do not return any value. In the world of C, both are referred to as function may or may not return any value. If a function does not return a value, the return type in the function definition and declaration is specified as void. Otherwise, the return type is specified as a valid data type.

Thus, the definition of a function that does not return anything is as follows:

```
void FunctionName (ParameterList)
```

```

{ statement/statements;

return;

/* optional, since it is at the end of the function anyway, and the function has no return
value. */}

```

There may be multiple return statements, return statements in the middle of a function (for example, in the body of an if statement) cause control to return to the caller without the rest of the function being executed. A return statement at the end is optional for functions without return values.

The definition of a function that returns a value of type Typename (for example, Typename can be int, char, float, etc.,) has the following syntax:

```

Typename FunctionName (ParameterList)

{ statement/statements;

return value;

/* return keyword must be used, and it must be followed by a value that matches the
return type specified by typename. */ }

```

Even in case of functions having return values, multiple return statements (all of them returning values of the correct type) can exist. Parameter List is the list of arguments separated by commas.

Function call

A function is a dormant entity, which comes to life when a call is made to the function. A function call is specified by the function name followed by the values of the parameters enclosed within parentheses, terminated by a semi colon (;) The following example illustrates the call to a function.

```

* / square1.c illustrates a typical function */

#include <stdio.h>

```

```

unsigned SquareInt (unsigned x)
{ return x * x;}

int main

(unsigned Sq = SquareInt (32);      /* function call */

printf ( ' The square of 32 is %u\n", Sq),

/* control returned here */}

```

The function SquareInt is called and the return value is assigned to the variable Sq. The control is returned to the printf statement after the statements within the function definition of SquareInt are executed. Here we see that the function SquareInt returns an unsigned int. A function call such as SquareInt (32) can be used to replace an unsigned integer. In fact, in the function main (), the two statements

```

unsigned Sq = SquareInt (32 );      /* function call */

printf ("The square of 32 is %u\n", Sq); /* control returned here */

```

can be merged into a single statement

```

printf ( " The square of 32 is %u\n", SquareInt (32 ) );

```

It may be useful to understand the sequence of steps that are executed when the above printf statement is executed. First, the function SquareInt is called then function accepts an unsigned integer as its parameter. The value 32 is passed to it. So, inside SquareInt, the value of x is 32. The function returns 32 * 32, which is 1024. Next, the function printf is called with the string,

```
"The square of 32 is %u\n"
```

as the format string, and 1024 (return value of SquareInt) as the second parameter.

Example

```

/* min.c: program to find the minimum of two numbers */

#include <stdio.h>

```



```

int main (void)

int min (int, int);

int a, b, Small,

printf ("Enter two numbers :");

scanf ("%d %d", &a, &b);

small = min (a, b );

print ("The minimum of two numbers is : '%d\n", small);

printf ("The minimum of two numbers is: %d\n", Min( a,b));

return 0; }

int min (int m, int n)

{ int minnum;

if (m> n)

MinNum = n;

else

MinNum = m;

return (MinNum) ; }

```

Take a look at the sample run of the above program:

Enter two numbers :32567 32567

The minimum of two numbers is : 32567

The minimum of two numbers is : 32567

The statement

```
int min ( int, int );
```

is the declaration or prototype of the function. The function definition includes statements that test the two input argument numbers and returns the smaller number. The function also

declares a local integer variable called minnum. The function call has two arguments and returns an integer. Function min takes two arguments of type integer.

5) Function Parameters

Function parameters are the means of communication between the calling and the called functions. They can be classified into formal parameters and actual parameters. The formal parameters (commonly called parameters) are the parameters given in the function declaration and function definition. The actual parameters, often known as arguments, are specified in the function call.

Consider the following program segment:

```
#include <stdio.h>

int sum (int a, int b)

/* This, and the following body (in curly braces) constitutes / the function definition*/

{ return a+b;}

void main (void)

{ int x, y, z,

z = sum (x, y); }
```

In the above program, the variables a and b defined in. function definition, are known as formal parameters or dummy parameters or placeholders. The variables x and y are actual parameters; they specify the values that are passed to the function sum. x and y are arguments in the function call.

The following conditions must be satisfied for a function call:

- * the number of arguments in the function call and function declaration must be the same.
- * the data type of each of the arguments in the function call should be the same as the corresponding parameter in the function declaration statement, i.e., the order in which we specify the arguments while calling the function must be the same as the order in which the parameters are specified: however the names of the arguments in the function call

and the names of parameters in the function definition are unrelated. They can be same or different.

C provides two mechanisms to pass arguments to a function.

- * Pass arguments by value, and,
- * Pass arguments by address or by pointers

An argument may take the form of a constant, variable, expression, pointer, structure etc.

Passing by the value

Functions in C pass all arguments by value. Passing arguments by value means that the contents of the arguments in the calling function are not changed, even if they are changed in the called function. This is because the content of the variable is copied to the formal parameter of the function definition, thus preserving the contents of the argument in the calling function. The following example illustrates the concept of passing arguments by value.

```
/*pval.c: program to demonstrate pass by value *1
#include <stdio.h>

void modifierfunc (int num)

printf ("\n In function, the value of Num is : %d\n", num); Num = 19;
printf ("In function, after changing, value of Num is: %d\n", num); }

void main ( )
{ int num;
num = 100;
printf ("In main, the value of Num is : %d\n", num);
modifierfunc( num);
printf ("\n After calling function, the value of Num is :%d\n", num); }
```

Examine the following sample run carefully:

In main, the value of Num is : 100

In function, the value of Num is: 100

In function, after changing, value of Num is: 19

After calling function, the value of Num is: 100

In the above program, the variable Num is assigned a value of 100 in the function main. During execution of the function, the value of the variable Num is changed to 19, but the value of the variable Num in the function main remains the same as prior to the execution of the function call (100).

Not knowing the fact that values in C are passed by value can be determined in the correct execution of programs. For example, suppose we want to write a function that accepts two parameters and swaps their values. An initial attempt to achieve this task is given in the program below.

```
/*swap1.c: program to swap two numbers /  
  
#include <stdio.h>  
  
void swap( int a, int b)  
{int temp;  
temp = a;  
a = b;  
b = temp;}  
  
void main ( )  
{int i, j;  
printf ( "Input two integers: ");  
scanf ( "d %d", &i, &j );  
printf ( "Before swapping: %d %d\n", i, j);
```

```
swap (i,j),
printf ( "After swapping: %d %d\n", i, j );}
```

A sample run of the above program is given below. It is obvious that the program does not work.

Input two integers: 3 5

Before swapping: 3 5

After swapping: 3 5

The reason becomes obvious when we examine the sequence of events that occurs when the statement,

```
swap( i, j ),
```

is executed. First, the values of the variables *i* and *j* are copied into the parameters of the function *swap*, *a* and *b* respectively. The values of *a* and *b* are *j* in the *swap* function. The function returns, and the values of *a* and *b* are discarded.

So what is the solution to this problem? To see this, remember that the function *scanf* also had to change the values of variables depending on what the user had input. While passing the variables to *scanf*, we always precede the name by an ampersand *&*, that is, the address of the variable is passed. So, instead of passing two integers to the *swap* function, we can pass the addresses of the integers that we want to swap. For this, two changes are required.

First, the function definition must be changed to accept the address of the two integers. This is done by specifying

```
void swap (int* a, int* b)
```

instead of

```
void swap (int a, int b)
```

Inside the function, instead of using *a*, which now means the memory address of an integer, we have to access the value that is address by *a*. This is done by using **a* instead of *a* and **b* instead of *b*. Second, the call in *main* must be changed to pass the addresses of *i* and *j* instead of their values. This is done by calling

```
swap (&i, &j);
```

Note that the variable names are preceded by an ampersand (&). The complete program for Swapping two numbers is given below. These concepts will be clear in the chapter on pointers.

```
/* Swap2.c: program to swap two numbers */

#include <stdio.h>

void swap (int *a, int *b)
{int temp;
 temp = *a
 *a = *b;
 *b= temp;}

void main ( )
{ int, i,j;
 printf( "Input two integers:");
 scanf ( "%d %d", &i, &j )
 printf ("Before swapping: %d %d\n", i, j );
 swap ( &i &j );
 printf ("After swapping: %d %d\n", i, j );}
```

6) Return Values

A function with a return value produces a value and returns it to the statement within the function that called it. Functions with return values require that a return statement be used, so that the value is returned to the calling statement. The return value can be a constant, a variable, an user-defined data structure, a general expression (reduced expressions), a pointer to function, or a function call (in which case the call must return a value). C does not place any restriction on the type of return values except that the return value cannot be an array. A pointer to an array can be returned. A function can return an array that is a part of a structure.

Example

```

/*fact1.c: illustration of a function that returns a value */
#include <stdio.h>

long fact (unsigned int ); / prototype*/

void main (void)
{
    unsigned int num;
    long factorial;
    printf ("Enter a positive number :");
    scanf ( "%u", &num);
    factorial = fact (num);
    printf ("\nThe factorial of %u is %li", num, factorial );}

long fact (unsigned int n)
{
    long fact ;
    int i;
    if (n == 0)
        fact = 1; /* factorial of zero is one */
    else
        {fact = 1; /* perform repeated multiplication */
        for (i = 2; i <= n; i++ )
            fact = fact * i;}
    return fact;}

```

Run:

Enter a positive number : 5

-

The factorial of 5 is 120

The function prototype before main defines that the function fact takes an integer argument and returns a long datatype. The function definition makes sure that the correct value is returned

by defining the appropriate data type (a long variable 1Fact) and placing it in the return statement. Suppose, the variable 1Fact is defined as an integer, the compiler would have performed the necessary type conversion (i.e., to type long) and returned a value of type long.

A function with a return value can be placed as an individual statement (i.e., the return value may not be assigned to any variable). Let us see a few examples.

Example

```
/*sum.c: sum of two numbers */

int sumtwo (int n1, int n2)

{/* n1 and n2 are the parameters */

return (n1 + n2);}

int sumthree (int n1, int n2, int n3)

{return (n1 + n2 + n3) ;}

void main (void)

{int i= 56, j =32;

...

sumtwo (3,4);

/* function call, return value is not assigned to any variable 3 and 4 are the argument to
the function */

sumtwo ( i, j ), sumthree ( 100, i, j );

/* two function calls in a single statement using a comma */

...}
```

Example

```
/*fibonacci.c: program to generate fibonacci series using iteration */

#include <stdio.h>

int main (void)
```



```

(void fib (unsigned int n); /* prototype */

unsigned int n;

printf ("Program to generate fibonacci numbers\n");

printf ("Enter the total number to be generated: ");

scanf ( "%d", &n);

fib (n ); /*function call, n is the argument to the function */ return (0);}

void Fib (unsigned int totnum)

/* ToNum is the parameter, and function does not return /*any value*/

{unsigned int flast = 1, slast = 0;

unsigned int curnum;

printf ("Fibonacci numbers :\n");

print ("0\n %u %u", flast, slast);

do

{curnum = flast + slast;

printf ( "%u\n", curnum)

slast = flast;

ifast = curnum

totnum -- ;

} while (TotNum> 1);

return; /* dummy return statement */}

```

The above program is an example of a function that does not return any value. The output statement in the function display each number in the series.

Function without arguments and a return value

A function can be used without any argument supplied to it, and still can be made to return a value. These type of functions find application where computations are performed based on some predefined values.

Example

In the following example, the function Sum20 takes no argument and returns an unsigned integer. Iteration is used to sum the first twenty natural numbers. An alternative is to use the summation formula $n * (n + 1) / 2$.

```
/*sum20c: program to sum first 20 numbers */

#include <stdio.h>

unsigned sum20 (void);

void main (void)

{printf ("Program to print the sum of first 20 numbers\n ");
printf ("The sum of first 20 numbers is :%u", sum20 ( ) );}

unsigned sum20 ( void)

{ unsigned Sum = 0;
for ( int i = 1; i <= 20; i++)
sum + i
return (sum); }
```

An example of a library function which takes no arguments but returns a value is `getc`. It waits for a keyword input and returns the character typed.

Functions that return a user defined data type

In addition to the predefined data types, function can return user defined data types. such as enumerated data types.

Example

The following example illustrates a function that returns a user defined data type.

```
/* palind.c: test given string is palindrome */

#include <stdio.h>
```

```

#include <string.h>

enum boolean { false, true };

enum boolean ispalindrome( char string ) [ ] )

{int left, right, len = strlen ( string);

enum boolean matched = true;

if( len == 0)

return true;

left = 0;

right = len - 1;

/* Compare the first and last letter, second and last but one.. */

while( left < right && matched)

{if ( string (left) != string (right))

matched = false;

else

{left++;

right -- ;}}

return matched;}

int main( void)

{char String [40];

printf ("Program to test the given string is a palindrome\n");

printf ("Enter a string :");

scanf( "%s", string);

if ( ispalindrome( string ) )

printf ("The given string %s is a palindrome\n", string);

```

```

else
printf ("The given string %s is not a palindrome\n", string);
return (0);}

```

Run:

Program to test the given string is a palindrome

Enter a string : ratsdrowninwordstar

The given string ratsdrowninwordstar is a palindrome

The function `IsPalindrome` takes string as an argument and returns an user defined data type, i.e., Boolean. The function compares the first with the last character, the second with the last but one character of the string, and so on.

7) Recursion

Expressing an entity in terms of itself is called recursion. In C, a function can call any function that has been defined, including itself. Recursion is an important concept which everyone in the software field must be familiar with. Let us start with a simple problem using recursion, the problem of computing the factorial of a number.

The definition of factorial can be stated in two ways:

- The factorial of 0 is 1, and the factorial of any positive integer n is the product of all integers from 1 to n
- The factorial of 0 is 1, and factorial of any positive integer n is the product of n and the factorial of the number $n-1$

The first definition is iterative whereas the second is recursive. In the second definition, you may ask, "What is the factorial of $n-1$?". The answer is that it is the product of $n-1$ and the factorial of $n-2$ (i.e., $(n-1) * (n-2)!$). It continues till n becomes 0, in which case the factorial is 1.

Let us first write a program to compute the factorial that uses the first definition (without recursion)

```

/*ifact.c: factorial of a number */

#include <stdio.h>

long calc_fact ( unsigned int num )
{long fact = 1;
 unsigned int i;
 for ( i = num; i > 1 ; i -- )
 fact * = i;
 return fact;}

void main ( )
{unsigned int n;
 printf ( "Enter an integer: " );
 scanf ( "%u", &n );
 printf ( "The factorial of %d is %li\n", n, calc_fact ( n ) ); }

```

The function is quite straightforward. Its central part is a loop that accumulates the product of the numbers beginning from the integer passed, down to 2, in the variable fact.

The function is now re-written according to the second definition using recursion.

```

long fact ( unsigned int num)
{if ( num == 0)
 return ( ) ;
 return (fact( num - 1) * num);}

```

Suppose, in main () , you call the function fact with the statement:

```
fact ( 0);
```

Then, inside the function fact, the value of num is 0. The if statement turns out to be true, and the statement.

return 1,

is executed, returning the correct value.

On the other hand, suppose it is called with the statement `fact (2);`

The value of `num` in the function `fact` is 2, the condition in the `if` statement is false, and the function `fact` gets called again with the value `num - 1`, which is 1. In the second invocation of `fact`, the value of `num` is 1. The condition in the `if` statement is false again, and a third invocation of `fact` results, passing 0 to it. In the third invocation, the value of `num` is 0, and the condition in the `if` statement is true. So the third invocation of `fact` returns 1 to the second invocation (because the second invocation of `else fact` called the third invocation). Remember that in the second invocation of `fact`, the value of `num` was 1. Hence, the statement

```
return fact ( num - 1) * num;
```

(which is now executing in the second invocation) multiplies 1 by `a` and causes the number 1 to be returned to the first invocation, where the value of `num` was 2. This brings us back to the statement

```
return fact( num - 1) * num;
```

which is now executing in the first invocation. The return value of the second invocation, 1 is multiplied by 2 and the value 2 is returned.

The definition of the function `fact` is based on, the principles explained above. This function definition where a statement within the body of the function calls itself is called recursion, and the function call is called recursive call. This type of definition exists not only for finding factorial of the number, but also for other computations such as finding the fibonacci sequence, binomial co-efficients, tree traversal, fractals and so on.

Recursive functions are those in which there is at least one function call to itself (there can be more than one call to itself, for example, tower of hanoi). Recursive functions may have an indirect form when a function, say `fn1` calls function `fn2`, and the function `fn2` in turn calls function `fn1`, thus forming loop like calls. This type of calling can be extended to any number of functions. The recursive approach of problem solving, substitutes the given problem with another problem of the same form in such a way that the new problem is simpler than the original.

Recursion is implemented through the use of functions. A function that contains a function call to itself, or a function call to a second function which eventually calls the first function, is known as a recursive function.

Two important conditions must be satisfied by any recursive function:

- Each time a function calls itself it must be closer, in some sense to a solution.
- There must be a decision criterion for stopping the process of computation.

For the function factorial, each time the function calls itself, its argument is decremented by one (condition 1). The stopping criterion is the if statement that checks for the zero argument.

It is not enough that there are recursive calls to a function. There must exist a criterion (also called the escape hatch) to stop the recursive calls. For example,

```
int Infinite( void)
{Infinite ( ) ;
return 1;}
```

will execute till the stack overflows. There is no stopping criterion by which the recursive calls can be terminated.

Example

```
/*rec.c: function to illustrate recursion */
#include <stdio.h>

void CountNumber( int n) (static mi 1;

printf ( "In the function, the value of n is :%d\n", n);

printf( "\tThe depth of the call is :%d\n", i++);

if( n> 1)

CountNumber( n - 1);

printf( "\n\t\tAfter recursive call, value of i si : " );}

int main( void)
```

```

int num = 3;

CountNumber( num);

return (0) ;}

```

Run:

In the function, the value of is :3

The depth of the call is :1

In the function, the value of i is :2

The depth of the call is :2

In the function, the value of i is :1

The depth of the call is :3

After recursive call, value of i is :1

After recursive call, value of i is :2

After recursive call, value of i is :3

The if statement evaluates the condition to true and the function call (to itself) is executed. The function CountNumber is called with an argument of 2. Again, the if statement is encountered, and a recursive call is made. The static variable i indicates the depth of the recursive calls. The number of times the function calls itself is called the depth of recursion.

Declaring variables as static within a function makes them retain their values. When a call is made with an argument of 1, the if statement evaluates or false. The function call CountNumber is skipped, and the statement following is executed, which prints the argument value. At this stage, the closing brace is encountered, the function does not return control to the main function, b the last call with an argument 1 was not called by the function main, but the statement with! P the body of the function, i.e., the call after the if statement. Similarly, the control is returned to the statement following the if statement as many times a call was issued to itself, and finally to the function main.

During each recursive call, a new environment corresponding to the function is created. All local variables and arguments are localized to the current recursive call and are not accessible outside.


```

/*fibrec1.c: compute fibonacci sequence using recursion */

#include <stdio.h>

unsigned uFibr( unsigned uNum )
{if( uNum == 0)
return (0);
else if ( uNum == 1 )
return (1);
else return (uFibr ( uNum - 1) + uFibr( uNum - 2 ) );}

int main( void)
{unsigned ulnum, ulIndex;

printf ("Enter how many numbers are to be generated :");
scanf ( "%u", &uTnum);

printf ( "The fibonacci sequences are :\n" );
for (ulIndex = 0; ulIndex < uTnum; ulIndex++)
printf( "\n%u", uFibr( ulIndex) );}

```

The recursive function given in the above program is not an efficient recursive principle, because for each number, the same set of sequences are computed repeatedly.

Example

The following program is built on the iterative principle, but again is not a good example of the recursive technique. If the static variables are used, its iterative counterpart is a better technique.

```

/*fiborec2.c: fibonacci sequence generation by iteration */

#include <stdio.h>

int main()

```

```

{ int n;

void fib ( unsigned int);

printf ( "Number of terms to be generated?");

scanf ( "%d", &n );

printf ( "\n\nFibonacci sequence upto %d terms :\n\n", n);

fib (n) ;

return (0) ;}

void fib (unsigned int n)

{static int f1 = 0, f2 = 1;

int temp;

if (n < 2)

{f1 = 0;

f2 = 1;}

else

{fib (n-i);

temp = f2;

f2 = f1 + f2;

f1 = temp;}

printf ( "%d\t", fi ); return;}

```

Run:

Number of terms to be generated? 10

—

Fibonacci sequence upto 10 terms

0 1 1 2 3 5 8 13 21 34

The function below is recursive and must be called using Fibonacci (0,1,n); where n is the number of terms.

```
void fibonacci (unsigned int n1, unsigned int n2, unsigned int num)

{if ( num != 0)

    (printf ( "%u\n", ni )

    fibonacci( n2, n2 + ni, num - 1 );}}
```

8) Comparison of Iteration and Recursion

Any iterative function consists of four parts:

Initialization: The decision parameter is set to an initial value, or more precisely, the loop variable is initialized. Other pre defined variable are also initialized.

Decision: The decision parameter is used to determine whether further looping is necessary. The loop variable is compared, and based on the outcome, the loop is executed again. The test performed on the loop variable may not be the only condition, other relational expression may also participate in the decision.

Computation: The necessary computation is performed within the loop.

Update: The decision parameter is updated and a transfer is made to the next iteration. The loop variable is incremented/decremented. It may also result in a set of variables begin modified.

For a recursive function

Initialization: The variables in the form of arguments are passed on to the function.

Decision: The argument values are used to determine whether further recursive calls are required.

Computation: The necessary computation is performed using the local variables and the parameters at the current depth.

Update: The update is done so that the variables can be used for further recursive calls.

Recursion exists because of the recursive nature of the process, or because of the recursive structure of the data to be processed. For some cases, the recursive solution may be simpler than iteration

For recursion, additional amount of work must be done in the form of saving return addresses, so that the correct statement is executed next when the return is executed. The function should also save formal parameters and local variables onto the stack upon entry and restore these parameters and variables on completion. But if the problem itself is recursive in nature, an iterative solution will have to simulate a stack. In such cases, recursion may turn out to be more efficient.

9) Variable Length Argument lists

All the examples that we have examined in this chapter accept a fixed number of arguments. But a function like `printf` accepts any number of parameters. How do we write functions such as `printf`? To do this, certain macros available in the header file `stdarg.h` can be used.

Let us write a function that accepts a string followed by any number of strings and integers. The last parameter will be `NULL`. For example, if this function is named `print_string_ints`, then we must be able to call it with a statement like

```
print_string_ints ( "Start", "Ondu", 1, "Eradu", 2, NULL );
```

The above call must result in the following output

Start

Ondu 1

Eradu 2

i.e., the first parameter is output, and the string pairs are output line by line till the `NULL` parameter is encountered.

Also, the function must be allowed to accept any number of string-integer pairs. Only, the last parameter must be NULL. To achieve this, the function prototype must be as follows:

```
void print_strings_ints ( const char *start, )
```

The three dots (...) above constitute the ellipsis operator. It allows the function to accept any number of parameters. These parameters, however, are not known to the function by name. So, how does the function access them? This is where the macros from `stdarg.h` come into picture. The following program uses variable length argument lists.

```
#include <stdio.h>

#include <stdarg.h>

void print_strings_ints (const char * start, ... )
{ va_list, arg_list;
  int i;
  char *s;
  printf ( "%s\n", start);
  va_start ( arg_list start),
  while (s)
  (s = va_arg ( arg_list, char* ));
  i = va_arg ( arg_list, int);
  if ( s)
  printf ( "%s %d\n", s, i ) }
  va_end ( va_list ) ; }

void main ( )
{print_strings_ints ( "First call", "Hello", 1, "World", 2, NULL);
 print_string "Second call", "Good",
 10, "Morning", 20, "to all", 30, NULL );}
```

Run:

First call

Hello 1

World 2

Second call

Good 10

Morning 20

to all 30

The function `print_strings_ints` declares a variable called `arg_list`, which is of type `va_list`. The type `va_list` is defined in `stdarg.h`, and is an array used in obtaining the arguments that come in place of the ellipsis (...).

The macro `va_start` initializes the `va_list` variable declared (which, in our case is `arg_list`).

The macro `va_start` is called in the above example 'with the statement

```
va_start ( arg_list, start);
```

The first parameter is the name of the `va_list` variable that has been declared. The second one is the name of the first parameter accepted by the function (which is `start` here).

The macro `va_arg` is the one which actually extracts the argument values. It accepts two parameters:

`va_list` which is initialized with the `va_start` function, and the type of the argument needed. In the above example, the first call to `va_arg` is:

```
s = va_arg ( arg_list, char * );
```

It instructs `va_arg` that an argument of type `char*` is expected, The next `va_arg` statement expects an `int`. The function assumes that atleast one string-integer pair follows the initial string.

There are some restrictions while using functions with variable number of arguments. The first parameter must be present, since its name has to be passed to `va_start`. Then, the type of the argument must be known while extracting it by using the `va_arg` macro. While calling the function, if the type of the variable passed does not match the type expected in the function, the results are unpredictable. The macro `va_end` performs cleanup operations. It releases any memory that might have been allocated when `va_start` was called.

Another function that is useful with a variable number of arguments is `vprintf`. To illustrate, let us develop a function that outputs an error and exits. Normally, we would write the function such that it accept a string and output it. But this function will accept the arguments which `printf` accepts, so that more detailed error message can be output. An illustration of this function is shown below.

```
error (const char *fmt, ... )
va_list arg_list;
va_start ( arg_list, lint);
vprintf ( fmt, arg_list);
va_end ( arg_list);
exit( 0 );}
```

An example of using the function is given below. This piece of code prompts the user to input an integer greater than 0, failing which it outputs an error message and exits.

```
printf ( "Input an integer greater than 0: " );

scanf ( "%d", &i )

if( i < 0)

error ("The integer i is %d. Its value must be greater than 0. \n", i);
```

Note that function `error` is called just like the function `printf`.

Example

```

/* gcdre.c: program to find gcd of two numbers */

#include <stdio.h>

int main( void)

{int a, b, iGcd;

unsigned gcd ( unsigned, unsigned);

printf ( "Type in any 2 numbers whose GCD is to be found:");

scanf ( "%d %d", &a, &b );

iGcd = gcd(a,b);

printf ( "GCD of %d and %d is %d\n", a, b, iGcd);

return (0);

unsigned gcd ( unsigned p, unsigned q)

{unsigned remainder;

remainder = p - (p / q * q);

if ( remainder 0)

return q;

else

gcd (q, remainder );}

```

Run:

Type in any 2 numbers whose GCD is to be found: 13 45

--

GCD of 13 and 45 is 1

The following is the recursive function for finding GCD:

```

unsigned Gcd( unsigned m, unsigned n)

{if(n>m)

```



```

return Gcd( ii, m);
jf( 0)
return m;
else
return Gcd( n, m %n );}

```

The above programs are examples where the execution leads to an unbranched chain of recursive calls and returns. A recursive function that calls no other function other than itself can lead to a chain of calls that splits into two or more chains in the form of a tree, The best example is the recursive function to compute the binomial co-efficient. The binomial co-efficient of the arguments of two numbers are defined as:

$$BC(n, k) = n! / (k! * (n-k) !)$$

The recursive approach of solving the problem is as follows:

1 if $k = 0$ or $n = k$

$BC(n, k) = \{BC(n-1, k) + BC(n-1, k-1) \text{ otherwise}$

Example

```

/*tohrec.c: tower of hanoi using redursion */
#include <stdio.h>

int main( void) {unsigned int nvalue;
char snvalue 'L', inval = 'C', dnvalue = 'R';
void hanoi (unsigned int, char, char, char);
printf ("Enter number of disks :"); scanf( "%u", &nvalue);
printf ("\n\nTower of Hanoi problem with %d disks\n nvalue);
hanoi ( nvalue, snvalue, inval, dnvalue); printf( "\n");
return 0;}

```

```

void hanoi ( unsigned n, char sndl, char mdi, char dndl)
{if (n != 0)

/* Move n-1 disks from starting to intermediate needle */
hanoi( n-1, sndl, dndl, mdi);

/* Move disk n from start to destination */

printf ( "Move disk %d from %c to %c\n", n, sndl, dndl);

/* Move n-i disks from intermediate to destination needle */
hanoi ( n-i, mdi, sndl, dndl );}}

```

Run:

```

Enter number of disks :3

Tower .of Hanoi problem with 3 disks Move disk 1 from L to R

Move disk 2 from L to C

Move disk 1 from R to C

Move disk 3 from L to R

Move disk 1 from C to L

Move disk 2 from C to R

Move disk 1 from L to R

```

Example

```

/* binsrch.cpp: Binary Search, all elements are unique */

#include <stdio.h>

int main( void)

{int a i, n, position, key; int binSrch ( inta int, int, int);

printf ( "Enter the size of the array :"); scanf( "%d", &n);

printf ( "\nArray elements (ascending order) ?\n\n");

```

```

for (i= 0; i< a; i++)

scanf ( "%d", &a );

printf ( "\nElement to be searched?: ");

else

scanf ( "%d", &key); position = binSrch( a, key, 0, n); if (position == 0)

printf ( "Unsuccessful search. %d not found.' key);

{printf ( "\nSuccessful search.\n");

printf ( "%d found at position %d", key, position );}

return 0;}

/* recursive function */

int binSrch( int b int key, int low, int high)

{static int mid;

if (low <= high)

(mid = (low + high) / 2;

if ( key < b [mid] )

{/* element in the lower half */

high = mid - 1;

binSrch( b, key, low, high );}

else if ( key> bEmid] )

{/* element in the upper half */

low = mid + 1;

binSrch( b, key, low, high );}

else

else if ( bEmid] == key) /* element found */

return (mid+1);}

return (0); /* element not found in the array */ }

```

Run:

Enter the size of the array:

Array elements (ascending order) ?

2 12 67 89 99 100

Element to be searched ? 99

Successful search.

99 found at position 5.

Example

```
/* minmax.c maximum and minimum of numbers by recursion */
/* Note: The size of the vector should be equal to a power of 2. */
#include <stdio.h>

int main( void)

( int i, ii, m, maxim, minim, x [25];

int max ( int, int, int);

int min ( int, int, int);

printf ("Finding the maximum and minimum recursively \n\n");

printf ( "Number of elements in vector? ");

scanf ( "%d", &n);

printf ( "\n\nVector elements ?\n" );

for (i = 0; i<n; i++)

scanf ( "%d", &x [i] );

i = 0;

m = n - 1;

maxim = max (i,m,x);
```

```

printf ( "\n Maximum of %d elements is %d\n", ii, maxim);

i = 0;

m = n - 1;

minim = min ( i, m, x);

printf ("\nMinimum of %d elements is ", n, minim);

return ( 0) ; }

/* Function max finds maximum of the vector recursively */

int max ( int j, int n, int a [25] )

{int m1, m2;

if (n == 0)

return (a [j]);

else

{m1 = max (j, n/2, a);

m2 = max (j + n/2 + 1, n/2, a);

if (m1 < m2)

return (m2);

else

return (m1);}

/* Function mm finds minimum of the vector recursively */

int min (int j, int n, int a [25] )

{int m1, m2;

if (n == 0)

return (a [j]).

else

```

```

{ml = mm (j, n/2, a);
m2 = mm (j ÷ n/2 +1, n/2,a);
if (ml < rn2)
return (m1);
else
return (m2);}}
```

Run:

Finding the maximum and minimum in vector recursively

Number of elements in vector? 8

Vector elements

12 34 56 78 54 -12 0 89

Maximum of 8 elements is 89

Minimum of 8 elements is -12

Example

The function main in a C program can accept arguments. The example below shows how the function main () with arguments can be used to read the command line arguments.

```

/*cmdargs.c: A C program to display command line arguments */

#include <stdio.h>

void main( int argc, char ** argv)

{printf ("Program to print the command line arguments\n");
printf ( "The number of command line arguments are: %d", argc);

for ( int i = 0; i < argc; i++ )

printf ("argv [%d]: %s\n", i, argv[i]); }
```

After you compile the above program to cmdargs.exe (in DOS) or into a.out (in UNIX), typing

cmdargs

or

a.out

will result in the following output (in DOS)

Program to print the command line arguments

The number of command line arguments are: 1

argv [0] : C:\RS\CMDARGS.EXE

and in UNIX,

Program to print the. command line arguments

The number of command line arguments are : 1

argv [0] : a.out

Observe that the full pathname is included in DOS, but not in UNIX.

On the other hand, typing:

cmdargs 1 two 3 four 5 six

or

a.out 1 two 3 four 5 six

will result in (in DOS)

Program to print the command line arguments

The number of command line arguments are :7

argv [0] : C:\RS\CMDARGS.EXE

argv [1] : 1

argv [2] : two

argv [3] : 3

argv [4] : four

argv [5] : 5

argv [6] : six

and in UNIX,

Program to print the command line arguments

The number of command line arguments are :7

argv [0]: a.out

argv [1]: 1

argv [2]: two

argv [3] : 3

argv [4]: four

argv [5]: 5

argv [6] : six

Example

```
/*cmdenv.c: command line arguments and environment variables. */
#include <stdio.h>

int main ( int argc, char ** argv, char **env)
{inti = 0;

printf ( "The number of command line arguments are: %d\n", argc);
printf ( "The command line arguments are as follows\n"); while( i < argc)
printf ( "argv : %s\n", i, argv );
printf ( "\nThe environment variables are :\n\n" );

i = 0;

while ( env )

printf ( "%d\n", env [i++] );
```



```
return 0;}
```

Now, entering the following on the command line

```
cmdenv 1 two 3 four 5 six
```

will result in (in DOS),

The number of command line arguments are: 7

The command line arguments are as follows

```
argvt [0] : C:\RS\CMDARGS.E
```

```
argv [1] : 1
```

```
argv [2] : two
```

```
argv [3] : 3
```

```
argv [4] : four
```

```
argv [5] : 5
```

```
argv [6] : six
```

The environment variable are:

```
COMSPEC=C:\NDOS.COM
```

```
PROMPT=$p$g
```

```
ORCADESP=C:\ORCADESP\
```

```
ORCADEXE=C:\ORCADESP\ORCADPROJ=C:\ORCADESP\
```

```
PAASM=C:\ORCADESP\PALSAM\
```

```
CMDLINE= 1 two 4 four 5 six
```

```
MSINPUT=C:\MSINPUT
```

```
PATH=C:\DOS;C:\UTIL\;C:\BORLANDC\BIN;C:\WINDOWS;C:\NC;C:\NU;  
C:\WS6;TEMPC:\WINDOWS\TEMP
```

```
NU=C:\NU
```

- ## 8.4 Summary

8.5 Check Your Answers

- ## 8.6 Model Questions

- (1) When are function declarations useful?
- (2) What is the necessity of a return statement? When is it necessary and when is it a must?
[consider the return type of the function]
- (3) What is the meaning of the following declarations?
 - a) `int f(float, double, char);`
 - b) `void g(int a, int b);`
 - c) `double h (void);`
- (4) Write a function that takes two int parameters, and returns the sum of all integers between them.

- (5) Assuming that the function written for question 3 has the declaration: `int g(int, int)`, what would the following statement print? Is it syntactically correct?

`printf ("Sum is : %d\n", g(3,15);`
- (6) List the benefits of using well designed functions that perform specific tasks in a large software project.
- (7) Clearly differentiate between function prototype, function definition and function call.
- (8) Can two functions in the same program have the same name, given that their return types are different, and that they accept different number and type of arguments? Don't you think this should be possible for the compiler?
- (9) How have the prototypes of the functions you have learnt so far (`getc 0`, `printf ( )` etc) been made available to your program? Where are the definitions for these functions?
- (10) Assume, function A calls function B in a program. Does the order of the function definitions make any difference? Explain.
- (11) Write a function that takes a string and a number between 0 9 as parameters, displays the string that many times, and returns its length.
- (12) How are libraries of functions created? What are their advantages?
- (13) What do you understand by passing arguments by value, and by pointer during function invocation? Explain both for function that takes an int argument and adds 10 to it if is and returns the same. Write the function prototype and definition in each case.
- (14) Write a function to accept 10 characters and to display whether each input character is a digit, or a lower case alphabet or an upper case alphabet.
- (15) What is recursion? What are its advantages if any? Trace the execution of each of the examples for recursion. Explained in this chapter by using a stack or plain logic.
- (16) How are the arguments and environment variables passed form the command line(operating system prompt) to the C program?
- (17) What is the difference between a function declaration and a function definition?
- (18) With an example, distinguish between the earlier (K&R) and new (ANSIC) styles of function declaration and definition.

- (19) Distinguish between actual and formal parameters.
- (20) When is the return statement mandatory in a function?
- (21) Illustrate the use of the elipsis operator with an example.
- (22) Distinguish between recursion and iteration.
- (23) What is the function of the keyword void in a function declaration when it is used:
 - (i) in place of the return data type (for example: void function (int i) ;) and
 - (ii) inside the braces specifying the parameter list

(For example: int func(void).

What is the effect of leaving out the void keyword in each of the cases above?

- (24) How do we access command line parameters and environment variables in a C program?

LESSON - 9

LOCAL AND GLOBAL VARIABLES

Structure

9.1 Introduction

9.2 Learning Objectives

9.3. Scope

9.4 Summary

9.5 Check Your Answers

9.6 Model Questions

9.1 Introduction

Every variable in a program has some memory associated with it. Memory for variables is allocated and released at different points in the program. For example, in case of normal local variables declared in functions, memory is allocated when the function starts executing and released when the function returns. The variables declared at a point outside all functions are called global variables. The period of time during which memory is associated with a variable is called the extent of the variable. For example, take an integer variable *i*. Allocation of memory to variable *i* is the process of deciding the memory locations to be occupied by *i*. Take for example, the following function.

```
void func ( )
```

```
{int i; i = 10;}
```

The function does nothing useful, but suffices for the purpose of illustration. The memory space occupied by the variable *i* is decided when the function is invoked or executed. So, we say that memory is allocated for *i* when the function `func` is called the memory that is allocated to *i* is released when the function returns, and that memory space is then available for use.

Identifiers declared in a function are not accessible outside that function. For example, in a program, along with the function `main`, we have the function `func`.

```
void func ( )  
{int i;  
i=10;}  
  
void main ( )  
{i 20;  
func ( );  
i = 30;}
```

When you try to compile this program, the two statement in main (0, namely, `i = 20;` and `i = 30;`

will give errors. This is because, the variable `i` is not visible inside The function `main`. So the declaration of the identifier `i` is valid only inside the function `func`. The region of source code over which the declaration of an identifier is visible is called the scope of the identifier.

If the variable `i` is declared using the statement,

```
int i;
```

at the beginning of `main`, no error occurs, but nevertheless, the variable `i` in the function `func` and that in the function `main` are different. Modifications done to one variable do not affect the other.

9.2 Learning Objectives

After learning this lesson, you should be able to

- distinguish between local variables and global variables.
- define the extend of the variable as the period of time during which memory is associated with a variable.
- define the scope of the variable as the region of source code over which the declaration of an identifier is visible.
- define the auto variable, register variable, static variable and extern variable.

9.3 Scope

Scope of a variable can be defined as the region over which the variable is visible or valid. To get an idea about the scope of a declaration, consider the following program.

```
/*scope1.c: illustration of variable declaration */

#include <stdio.h>

void func ( )

{int j = 20;

printf( "j is a variable is func.\n");

printf( "Value of j is %d\n", j );}

void main()

{int i = 10;

printf ( "i is a variable in main.\n");

printf ( "Value of i is %d\n", i);

printf ( "Calling func...\n" );

func ( );

/* This statement will cause compilation error, */

/* since there is no j in main.*/

j = 30;}
```

The statement `int i = 10;`, present inside the function `main`, declares `i` to be an integer variable and initializes its value to 10. On the other hand, the statement `int j = 20;` is contained inside the function `func` and declares `j` to be integer variable with its value initialized to 20. If you try to compile the above program, the last statement in `main`, which is

```
j = 30;
```

will cause an error. This is because, the variable `j` has not been declared in the function `main` and the declaration of `j` in the function `func` is not visible in the function `main`. We say that

the scope of j is restricted to the function func. So j cannot be accessed from any other function. Similarly, the scope of i is restricted to the function main. The variables i and j are local variables, since their visibility is restricted to the function in which they are declared.

Variables declared outside all the functions are called global variables and can be accessed by all the functions. Consider the following example. The integer g is a global variable.

```
/*global.c: illustration of global variables */
#include <stdio .h>
/* is a global variable */
int g;
void func ( )
{printf ( "In func. g is visible here, since it is global.\n" );
 printf ( "Incrementing g in func. . \n" );
 g+ +;}
void main ( )
{ printf ( "In main. g is visible here, since it is global.\n" );
 printf ( "Assigning 20 to g in main...\n" );
 g = 20;
 printf ( "Calling func.. \n" );
 func ( ) ;
 printf ( "func returned. g is %d", g );}
```

Run:

```
In main. g is visible, since it is global
Assigning 20 to g in main...
Calling func...
In func. g is visible here, since it is global
Incrementing g in func...
func returned. g is 21
```


The above program does not give any error, since `g` is global variable and hence, can be accessed from any function. Naturally, all functions work on the same variable `g`. This is the reason for `g` being printed as 21 in the function `main`, after `func` has been invoked.

Scope Resolution Operation (::)

When both global and local variables have the same name, a statement using that name will access the local variable (more precisely, it will access that variable of that name in the innermost block containing that statement).

The scope resolution operator `::` (a double colon) can be used to select the global variable explicitly. Consider the example below.

```
/*colcolon.c: Program illustrating the scope resolution operator.*/
#include <stdio.h>

int a = 10;

main ( )
{ int a = 20;
  printf ( "%d\n", a);
  printf ( "%d\n", ::a); }
```

The first `printf` statement above accesses the variable `a` as it is. It will access the local variable `a` (whose value is 20). The second `printf` statement uses the scope resolution operator with `a`. It will access the global variable (whose value is 10). Hence, the output of the above program is:

20

10

Irrespective of the block in which the statement is present, the scope resolution operator causes the program to access the global variable only.

1) Block Scope and File Scope

There are some more intricacies attributed to scope than those mentioned above. Hence, we shall deal with scope under two categories:

- Block scope
- File scope.

Block scope

A block is defined as a sequence of statements enclosed between curly braces, {and}. In other words, a block is a compound statement. We have already encountered many instances where we used blocks of statements. For example, a function body is a block, because it is simply a sequence of statements enclosed within curly braces. We also encountered blocks of statements in if statements and loops. All these blocks of statements actually follow the same rules. For the purpose of illustration, consider the following simple code.

```
{ char CollegeCode;  
...  
if ( CollegeCode == 'A' || CollegeCode == 'a' )  
printf ( "WISDOM COLLEGE OF TECHNOLOGY\n");  
...}
```

The variable CollegeCode is accessible within the block of code. The variables defined can be accessed only within the block in which they are declared. In cases of nested blocks, the variables declared in the outer blocks are accessible by statements in the inner blocks and not vice-versa. These variables are called local variables because they are localized to the block.

It helps to preserve the integrity of data (data in one function cannot be modified by another function, directly). It can be observed that if two variables of the same name are declared in many functions, they are distinct and unrelated variables. In the above example, CollegeCode is a local variable. The scope of the variables in the function parameter list is also confined to the function, i.e., they are also local variables.

Example

In the example shown below, the variable `j` is declared in both `main` and the user defined function `OtherFunc`. Accessing uses the local declaration in the called function.

```
/*block.c: illustration of block scope */

#include <stdio.h>

void OtherFunc( void)

{int j = 12;

printf( "In the function OtherFunc, the value of j is %d\n", j); }

int main( void)

{ int j = 144;

printf ( "In the function main, the value' of j is %d\n", j);

OtheFunc ( ); }
```

Variables in nested blocks

Blocks of code can be embedded within each other. This is referred to as nesting of blocks. The variables defined in these blocks can be accessed only within those blocks in which they are defined and not outside.

Example

```
/*nestblock.c: Illustration of the variables scope in */

/* nested blocks */

#include <stdio.h>

int main ( void)

{int i = 144, j =72;

printf ( "i = %d", i);

/* nested block */

int k = 12;
```

```

scanf ( "%D", &k);
i = 1% k;}
if( i == 0)
printf (" i is a divisor of %d\n", k);
return 0;}

```

If the variable `k` is accessed in the main block, a compile- time error results, because the variable `k` is declared inside the nested block within `main`. Memory space is allocated for the variable `k` when the statements within the nested block begin execution. When execution reaches the end of the nested block, memory space used by the variables in the blocks is released, as in the case of functions. Reference to any variable first checks the local declaration, and then checks the outer blocks around which the current block is built. The variable `i` can be accessed in the inner nested block since it is contained in the enclosing outer main block. The function can access the identifiers in the parameter list, local declarations and global declarations (if any).

Example

The program below demonstrates the accessing of variables in nested blocks. In the outer block of `main`, there are two variables declared `iNum` and `jNum`. In the inner block, a variable `iNum` is declared again. Since the name `iNum` is same as in the outer and inner blocks, any reference to the variable `iNum` will use the declaration of the current block and not that of the outer blocks. A reference to the variable `jNum` will cause the compiler to check the declaration in the current block first. Since there is no declaration for the variable `jNum`, the declarations in the outer blocks are searched, and if it is not found there, the global variables are searched. If there are no declarations even in the global list, it will result in an error. In the example `nestblk2.c`, the variable `jNum` is declared in the outer block, and hence, its declaration is used. After the nested block is executed, (i.e., after closing braces are encountered), a reference to the variable `iNum` will use the declaration in the main block and not the recent declaration (`iNum = 10`) in the inner nested block. This is because when the last `printf` is executed, the current block is the block `main`.

```

/*nestblk2.c: illustration of scope of variables in nested block*/

#include <stdio.h>

void main( void)

{int iNum = 5, jNum = 30;

  {int iNum = 10;

    printf ( "%d %d\n", iNum, jNum );}

    printf ( "%d %d\n", iNum, jNum );}

```

Run:

```
10  30
```

```
5   30
```

File scope

Variables declared outside all functions are said to have global scope. Global variables can be classified under two categories:

- Static or file static variables, whose scope is limited to the file in which it is declared
- Global variables, which can be accessed across files which are linked together

Example

In the following program, the variables iGlobal and Number declared before main are global variables. The variable iVal is a file static variable.

```

/*globdemo.c: Program that references global and local variables */

#include <stdio.h>

int iGlobal = 35, Number = 47;

static int iVal = 99;

void main ( void)

{int Number = 1000;

```

```
printf ( "The value of local variable Number is %d\n", Number);
printf ( "The value of global variable iGlobal is %d\n", iGlobal);
printf ("The value of global variable Number is %d\n", :: Number);
printf ( "The value of static global variable iVal is %d\n", iVal ) ; }
```

Run:

The value of local variable Number is 1000

The value of global variable iGlobal is 35

The value of global variable Number is 47

The value of static global variable iVal is 99

In the above program, reference to the variable Number, access the local declaration in main, since the variables in the innermost block are always considered first. In the above example, the keyword static used in the declaration of iVal does not make any difference, since only one file is being compiled. The usefulness of this keyword is discussed at the end of this lesson, where multiple files are compiled separately and then linked together.

2) Extent

The period of time during which memory is associated with a variable is called the extent of the variable. The file time of a variable is characterized by its storage class. The storage class of a variable indicates the allocation of storage space to the variable. Thus, storage classes define the extent of a variable. Storage classes are of four kinds.

- * auto
- * register
- * static
- * extern

Auto variables

All variables declared within a function are auto by default. The extent of a variable is defined by its scope. Variables declared auto can only be accessed only within the function or the nested block within which they are declared. They are created when the block is entered into and destroyed when it is e.

Register variables

C allows the use of the prefix register in primitive variable declarations. Such variables are called register variables, and are stored in the registers of the microprocessor. The number of variables which can be declared register are limited. If more variables are declared register variables, they are treated as auto variables. A program that uses register variables executes faster as compared to a similar program without. It is possible to find out the allocation of register variables only by executing and comparing the performance with respect to time taken by the program (perceptible in large programs). It is the responsibility of the compiler to allot register variables. In case the compiler is unable to do so, these variables are treated as auto variables. Loop indices, accessed more frequently, should be declared as register variables. For example, `i` index is register variable in the program given below.

```
/* regvar.c: illustration of register variables */
#include <stdio.h>
#include <stdio.h>
#include <string.h>
int main( void)
{char cName register; int iIndex;
 printf ( "Enter a string: " );
 gets ( cName );
 printf ( " The reverse of the string is : ");
 for( iIndex = strlen( cName) - 1; iIndex >= 0; iIndex - - )
 printf ("%c", cName[iIndex]);
 printf ( "\n" );
 return 0; }
```

Static variables

C defines another class of variables called static variables. Static variables are of two types:

- static variables that are declared within a function (function static variables). These variables retain their values from the previous call, i.e., the values which they had before returning from the function.
- file static variables. These variables are declared outside any function using the keyword `static`. File static variables are accessible only in the file in which they are declared. This case arises when multiple files are used to generate one executable code. This is illustrated at the end of this lesson.

Example

The following example illustrates static variables declared within a function.

```
/*count.c: use of static variables, defined inside functions*/
#include <stdio.h>
void printCount( void )
{static int Count = 1;
/* Count is initialized only on the first call */
printf( "Count = %d\n", Count);
Count = Count + 1;
/* The incremented value of Count is retained */}
int main( void)
{printCount ( );
printCount ( );
printCount ( );}
```

Run:

Count = 1

Count = 2

Count = 3

The output of the program is a sequence of numbers starting with 1, rather than a string of 1's. The initialization of static variable Count is performed only at the first instance of the function call. In successive calls to the function, the variable Count retains its previous value. However, these static variables are not accessible from other parts of the program.

Example

Following is a program which outputs the Fibonacci series. The function PrintNextFibo, which actually does the computation, has two static variables uF1 and uF2, that hold the value of the two Fibonacci numbers computed previously.

```
/*sfibo.c: variables not accessible in other parts of the
program */
#include <stdio.h>
void PrintNextFibo( void)
{static unsigned int uF1 = 0, uF2 = 1;
unsigned int uFpre;
uFpre = uF1 + uF2;
printf( "%u\n", uFpre);
uF2 = uF1;
uF1 = uFpre;}
int main ( void)
{int itotNum;
int ilIndex;
printf ( "Fibonacci sequences using static variables :\n" );
printf ("Enter how many numbers are to be generated: ");
scanf ("%d", &itotNum);
printf ( "The first %d fibonacci numbers are \n", itotNum );
for( ilIndex = 1; ilIndex <= itotNum; ilIndex++) PrintNextFibo 0;
return 0; }
```

The above example defines a function to generate fibonacci sequence. Two static variables, uF1 and uF2 are defined and initialized to zero and one respectively. After first function call to PrintNextFibo, the value of the variables uF1 and uF2 are both changed to one and retained for use the next call and so on. This would not be the case if variables were not declared static.

Role of static in File Scope

Consider the following of programs. There are three programs in all: bubble.c, linsrch.c and progl.c. The file bubble.c has a function to perform bubble sort, while linsrch.c performs linear search. The file progl.c has the function main, which calls the function BubSort in bubble.c and the function LinSrch in linsrch.c.

/*file bubble.c: This file should be linked with progl.c */

int Done; /* correction required here */

/* global variables should be declared in one file */

void BubSort (int *Arr, int Size)

{ Done = 0;

for (mti = 0; (i < Size - 1) && !Done; i++)

{ Done = 1;

for (int j = 0; j<(Size-1) – i; j++)

if (Arr [j] > Arr + 1))

{/*swap */

Arr =Arr +Arr

Arr [j] = Arr [j] + Arr [j+1]

Arr [j+1] = Arr [j] - Arr [j+1];

Arr [j] = Arr [j] – Arr [j +1];

Done = 0; }}

```

/*file linsrch.c This file must be linked with progl.c */

int Done; /* correction required here, global variables */

int LinSrch( int *Array, int Size, int Elem)

{int i = 0;

int pos;

Done = 0;

while ( i <Size && !Done)

{if (Array [i] == Elem)

{Done = 1;

pos = i;}

i + + ;}

return Done ? pos : 0;}

/*file progl.c: To compile use cc progl.c linsrch.c

bubble.c */

#include <stdio.h>

jut LinSrch( int *, int, int); /* prototype */,

void BubSort( jut 'i', int); /* prototype */,

void main ( void)

{int Arr [10];

int Size = 5, 1 = 0, pos;

printf ( "Enter five elements\n" );

while( i < Size)

scanf ( "%d", &Arr [i++] );

printf ( "Enter element to be searched: ");

```

```

scanf ( "%d", &i);

if ( (pos = LinSrch (Arr, Size, i) ) )

printf ( "element %d found at position %d", i, pos);

printf ( "element %d not found\n", i);

BubSort ( Arr, Size);

printf( "The sorted elements are :\n" );

i = 0;

while( i < Size )

printf( "%d\n", Arr [i+ = ] );}

```

These three programs are to be linked together now. To this, enter one of the following commands in DOS:

```
c:\cprgs>tcc progl.c bubble.c linsrch.c (if you use turbo C or C++)
```

```
c:\cprgs>bcc progl.c bubble.c linsrch.c (if you use Borland C++)
```

In UNIX, enter the command:

```
$ cc progl.c bubble.c linsrch.c
```

The programs are compiled, but you will get linker errors because the variable `done` appears in both files as global variables. By default, variables declared with the global scope (outside any function) by a statement such as `int Done;` are assumed to span across files (i.e., they can be accessed by functions in other files also). So, if such a declaration is present in two different files, there is a conflict as to which variable should be accessed. To make the program executable, the global declaration, `int Done;` in both the files `bubble.c` and `linsrch.c` should be changed to `static int Done;`

This declaration makes `Done` a file static variable, i.e., the variable `Done` can be accessed anywhere in the program or file as a global variable but not across different files. The identifier `Done` in each of the two files represents different variables. Functions in `bubble.c` cannot have

an access to the variable Done that is present in linsrch.c (this variable can be used by the functions in linsrch.c) and similarly in linsrch.c, you cannot have an access to the variable Done in bubble.c.

Extern Variables

Global static variables are global to the file in which they are defined. They are used when the same global variable is referenced in each of the files and these variables must be independent of each other across files. The use of global variables is not recommended, as function independence is one of the basic idea of modular programming. Global variables should be used only when it is inevitable (i.e., when all the functions need a particular variable).

When a program spans across different files and we want to have a global variable accessible to all functions. in these files, the keyword extern should be used before the data type name in the declarations in all the files where it is accessed except one. The linker requires that only one of these files have the definition of the identifier. Global variable definitions can occur in one file only.

For example, consider the following programs. The first program called sub_file.c contains a function called inc_global (). This function increments a global variable called globalVar. The second file is mainfile.c containing the function main. The variable globalVar is printed in main, before and after calling the function inc_global.

The code of the first file, sub_file.c is given below.

```
/* file sub_file.c: Defines the function sub_func.*/

static int globalVar;

void inc_global ( )
{globalVar+ + ; }
```

The code of the second file, mainfile.c is given below.

```
/*file mainfile.c: Defines the function main and uses */

/ * the function */
```

```

inc_global. Compile with cc mainfile.c sub_file.c */

static int globalVar;

/*Function prototype... */

void inc_global ( ) ;

void main ( )

{ /*Assign something to globalVar...*/

globalVar = 10;

printf ( "globalVar before calling = %d\n", globalVar);

/* Call the increment or */

inc_global ( ) ;

printf ( "globalVar after calling = %d\n", globalVar );}

```

Observe that the integer globalVar has been declared in the file sub_file.c as

```
extern int globalVar;
```

Upon encountering the above statement, the compiler knows that an integer globalVar exists. It goes ahead to produce the object file. In the mainfile.c, that actual definition of globalVar occurs with the statement

```
int globalVar;
```

Note that no special keywords are necessary to say that globalVar spans across files. This is because a declaration such as the one shown above tells the compiler that the integer globalVar is exported, and can be accessed in different files.

The linker is then invoked to link the two object files together. When the compiler produces the object file of sub_file.c, it does not know the exact location of the variable globalVar. So, it includes the information in the locations in the object file where the global variable is accessed. In the object file of mainfile.c, the compiler will store this information (that a global integer called global Var exists, that can be used across files).

When the linker links these two object files together, it knows the location of globalVar from the object file of mainfile.c. It uses this information to complete the object file of sut_file.c, where the compiler had encoded the information about the locations where globalVar is accessed in subfile.c .

Check Your Progress - I

- (1) State whether the following are True or False :
 - a) The variables declared at a point outside all functions are called local variables.
 - b) Static Variables that are declared within a function
- (2) Fill in the blanks:
 - a) _____ variables declared inside the function
 - b) A block is defined as a sequence of statements enclosed between _____

9.4 Summary

In this lesson we have discussed about global and local variables. The structure of programs that utilize multiple functions. For eg. In some situations, it may be desirable to introduce certain 'Global' variables that are recognized throughout the entire program. Such variables are defined differently than the usual 'local' variables, which are recognized only within a single function.

9.5 Check Your Answer

- I.
 - 1) a) False b) True
 - 2) a) Local b) Curly

9.6 Model Questions

- (1) Distinguish between local and global variables.
- (2) Mention the k used in defining the storage class of a variable and the significance of each.
- (3) Consider the segment of a code given below:

```
int f(int a, float b) { ... }
```

```
m c;
```

```
void main (void) { ... }
```

Evidently c is a global variable. Can c be accessed in function f? If not, Why?

- (4) If a variable a is both a local variable and a global variable as shown below, which a is actually accessed within the function?

```
int a;
```

```
void main (void)
```

```
{ int a;
```

- (5) What do you understand by a block scope for an identifier? How do nested blocks affect the accessibility of identifiers?
- (6) What do you understand by scope and extent of an identifier?
- (7) Mention the different storage classes. Does the storage class of a variable affect its scope or extent?
- (8) If a variable is declared static int outside any function, what is its accessibility? What is its storage class?
- (9) What are storage classes? What do they define?
- (10) When is the register storage class most useful? Think of a suitable example.
- (11) What is the difference between the function static and file static variables? Write a function that accumulates the sum of six numbers. The function accepts an .int as argument and uses a local static accumulator variable.

[Hint: invoke the function 6 times within a loop]

- (12) When is the extern qualifier used?
- (13) Write a program that consists of two files sum.c and main.c. The file sum.c has a function that takes two integer arguments and stores the result in a global shared variable called sum: Accept the two input in main.c, call the function sum.c and print the result in main.c.
- (14) What is the difference between using the static keyword with variables inside functions and global variables?
- (15) What is the use of the extern keyword? Illustrate with example.

LESSON - 10

ARRAYS

Structure

10.1 Introduction

10.2 Learning Objectives

10.3 Arrays

10.4 Summary

10.5 Check Your Answers

10.6 Model Questions

10.1 Introduction

An array is a sequence of data in memory, wherein all data are of the same type, and are placed in physically adjacent locations. For example, a sequence of 10 integers stored one after another in memory represents an array. Note that a string can be considered as a sequence of characters (an array of characters). The C language treats strings as an array of characters, the last of which is the null character (character with an ASCII value zero).

10.2 Learning Objectives

After studying this lesson, you should be able to

- declare array elements, access them for various purposes.
- discuss the declaration and accessibility of strings which are array of characters.

10.3 Arrays

To see the usefulness of arrays, consider the problem of reading the age of 5 persons, printing them out individually and computing the average. Here is a solution that does not use arrays.

```
#include <stdio.h> main( )

{int a1, a2, a3, a4, a5;

float sum = 0;

printf ("Age: ");

scanf( "%d", &a1);

sum += a1;

printf( "Age: ");

scanf( "%d", &a2);

sum += a2;

printf( "Age: ");

scanf( "%d", &a3 );

sum += a3;

printf( "Age: " );

scanf( "%d", &a4);

sum+=a4;

printf( "Age: ");

scanf( "%d", &a5 );

sum += a5;

printf( "The ages that were input are:\n" );

printf( "%d\n", a1 );

printf( "%d\n", a2 );

printf( "%d\n", a3 );

printf( "%d\n", a4 );

printf( "%d\n", a5);

printf( "Average= %d, sum / 5 );}
```

The program is very simple. It reads the ages of 5 persons, adds them up in a variable called sum, and at the end, divides sum by 5 to get the average age. Needless to say, this is a very clumsy way to write a program. If there are a large number of persons, the number of statements increases proportionately. To avoid this, use a loop. But the problem is that different variables are needed to store the ages of different persons and hence, the loop will have to store and print different variables each time. A more elegant way is to use an array of integers to store the ages, as shown in the example below. Notice the declaration of the array variable in the first line of main 0.

```
#include <stdio.h>

void main 0

{int age [5]; /*Array of 5 ages , declaration for an array */
int i, n;

float sum = 0;

printf ( "Number of persons: " );

scanf ( "%d", &n);

if ( n <= 0 : n > 5)

printf ( "Invalid number of persons entered.\n" );

else

{for (i = 0; i < n; i++)

{printf( "Age:");

scanf( "%d", &age [i] )

sum += age 1;}

printf ( "The ages input are:\n" );

for (i = 0; 1 < n; i++ )

printf( "%d\n", age[i]);

printf( "The average is: %f\n", sum);
```

```
} / end else /  
} / end main /
```

Run:

Number of persons: 5

—

Age: 10

—

Age: 20

—

Age: 30

—

Age: 40

—

Age: 50

—

The ages input are:

10

20

30

40

50

The average is: 30.000000

1) Declaration

The previous program is examined now. Recall that all variables in C must be declared before use. Hence, we must first declare the array. It is done at the beginning of the function main 0 with the statement

```
int age [ 5 ];
```

This statement states that age is an array of 5 integers. Note the use of square braces after the array name (i.e., age) enclosing the integer 5 that gives the number of elements in the array. This statement reserves enough memory for age to hold 5 integers. Once the declaration is made, age [refers to the first integer in the array, age [refers to the second integer and so on. Finally, age [refers to the last integer in the array. In C, array indices always begin with zero.

General form of array is

```
datatype Name of array [size];
```

For example

```
int age [5] ;
```

2) Accessing Array Elements

To access a particular element in an array, specify the array name, followed by square braces enclosing an integer, which is called the array index. The array index indicates the particular element of the array which we want to access. The numbering of elements start from zero(i.e., age is the age of the first person). Fig 10.1 depicts the elements of an array in memory. It is assumed that each element of the array (i.e., each integer) occupies two bytes.

Next, observe the loop used to input the elements. It is reproduced here for convenience.

```
for (i = 0; i < n ; i + +)
```

```
{ printf ( "Age: " ); .scanf( "%d", &age [ i ] );
```

```
sum + = age [ i ] ;}
```

The variable i varies from 0 to n-i (0 to 4 in the above sample run). The function scanf is called to input an integer value, the address of the ith element is passed to scanf. This makes scanf store the integer input into successive integer positions each time the loop is executed. Note that

```
&age [ i ]
```

in the scanf statement refers to the memory location of the integer at the ith position.

The Statement

```
sum += age [ i ];
```

adds the value of the i^{th} integer to sum.

The rest of the program illustrates the use of the array variable in various situations. The first for loop is used to add to the variable sum, and in the second, to print out the values of age that are input.

An element such as age [i] can be used just like any other ordinary integer. Statements such as

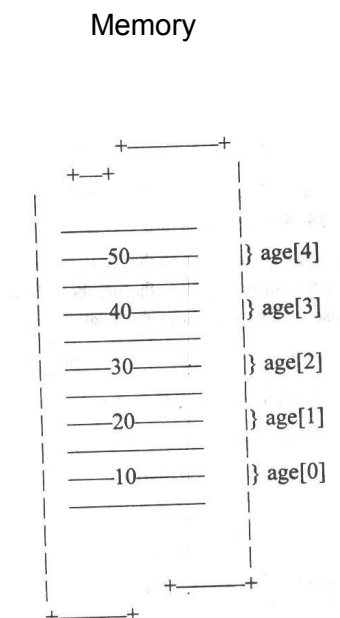


Fig.10.1. Elements of the array age

age [i] ++;, (to increment the value of the i^{th} integer in the array age) and age [i = 11]; are perfectly valid.

3) Initialization of Single Dimensional Array

Arrays can be initialized once they are declared. The declaration of an array can be done as follows:

```
int a 5] = { 13, 34, 2, 6, 234 };
```

Here, a is an array of 5 integers. The initial value of the five integers are specified in order, between the curly braces, separated by commas. The above declaration causes a to be initialized to 13, a to 34 and so on, till a a [4] which is initialized to 234. A semicolon always follows the closing brace.

The arrays size may be omitted during declaration, as shown below.

```
int a [ ] = {13, 34, 2, 6, 234};
```

This declaration is equivalent to the first one. In this case, the subscript is assumed to be equal to the number of elements in the array(5in this case). The initialization of an array is as follows.

List of constant, separated by commas delimited by braves

```
int a [5] = { 13,34,2,6,234 }; <- Semicolon here
```

4) Warning

C does not check the validity of the subscript when you use arrays. For example, take a look at the program below:

```
void main ( )
{int a [ 40 ];
a [ 50 ] 1 = 11;
a [ 50 ] 1 ++; }
```

In essence, the program declares a as an array of 40 integers, and then modifies the 51st element. When you compile the above program, you do not get any error. The executable code is produced. When it is run, the results are unpredictable. Hence, we must be very careful while using array subscripts. A wrong subscript in a large program can cause it to behave erratically, demanding a lot of debugging time.

Similar to an array of integers, we can have an array of any data type. For example, the declaration for an array of 50 floating point numbers is:

```
float f[50];
```

The individual floating-point numbers in this array are accessed as before, i.e., `f` is the first floating-point number and `f [49]` the last.

5) Name of the Array

Take the case where `f` is an array of floating point numbers. It is declared as:

```
float f [50];
```

The name of the array is `f`. If it is used as it is, without any subscript, then it refers to the memory location of the first floating-point number in the array. Recall that placing an ampersand (&) before a variable name refers to its memory location. Thus, the array name `f` is equivalent to `&f [0]`. The statement

```
scanf ("%f", &f [ 0 ] );
```

is equivalent to

```
scanf ( "%f", f);
```

Perhaps, this may not seem to be very useful now. But its potential will be made evident later, in the lesson on pointers.

Example

An example to find out the largest and smallest element in an array is illustrated below:

```
/* large1.c: largest and smallest element in the vector */
```

```
#include <stdio.h>
```

```
main ( )
```

```
{int i, n;
```

```
float a [50] ,large, small;
```



```

printf ("Size of vector ?");
scanf ("%d", &n);
printf ("\nVector elements ?\n");
for (i = 0; i < n; i++)
scanf ("%f", &a [i]);
/* Initialization */
large = a[0];
small = a [0]
/* largest and smallest elements */
for (i = 1; i < n; i ++)
{if (a [i] > large)
large = a [i] ;
else if (a < small)
small = a [i];}
printf ("\nLargest element in vector is %8.2f\n", large);
printf ("\nSmallest element in vector is %8.2f\n", small);}

```

Run:

Size of vector? 7

Vector elements?

34.00-9.00 12.00 10.00 -6.00 0.00 36.00

Largest element in vector is 36.00

Smallest element in vector is -9.00

Example

```

/* sort.c: sorting the vector elements in ascending order */
#include <stdio.h>

main ( )
{
    int i, j, k, n;
    float a [50], temp;

    printf ("Size of Vector?");
    scanf ("%d",&n); printf ("\nVector elements ?\n");

    for (i = 0; i < n; i++) /* reading */
        scanf ("%f",& a [i]);

    /* sorting vector A using selection sort */
    for (i=0; i<n-1; i++)
        for (j =1+ 1;j <n;j++)
            {/* To sort In descending order change the'>' *1
             /* operator '<'in the IF structure */
             if (a [i] > a [j])
                 temp = a [i ]
                 a [i] = a [j]
                 a [j] = temp;}}

    printf ("\nVector elements in ascending order r: \n");

    for (i = 0; i < n; i++)
        printf ("%8.2f", a[i]);}

```

Run:

Size of vector? 8

-

Vector elements?

91.00 20.00 1.00 7.00 34.00 11.00 -2.00 6.00

Vector elements in ascending order:

-2.00 1.00 6.00 7.00 11.00 20.00 34.00 91.00

Example

/ insert.c: inserting an element into an vector */*

```
#include <stdio.h>
```

```
main ( )
```

```
{int i, j, k, n, pos;
```

```
float a [50], item;
```

```
printf ("Size of vector? ");
```

```
scanf ("%d", &n);
```

```
printf ("\nVector elements ?\n");
```

```
for (i = 0; i < n; i++)
```

```
scanf ("%f",&a [i]);
```

```
printf ("\nElement to be inserted? ");
```

```
scanf ("%f", &item);
```

```
printf ("\nPosition of insertion ? ");
```

```
scanf ("%d",&pos);
```

```
/* Pushing down elements */
```

```
n++;
```

```
for (k =n; k >= pos ; k- - )
```

```
a [k] =a [k-1];
```

254

```
a [ - - pos] = item; /* item inserted */  
  
printf ("\nVector after insertion :\n");  
  
for (i = 0 ; i < n; i +=)  
  
printf ("%6.2f", a [i]);  
  
printf ("\n");}
```

Run:

```
Size of Vector?6  
  
Vector elements?  
  
1.00  2.00      3.00  4.00  5.00  6.00  
  
Element to be inserted?      10.00  
  
Position of insertion? 1  
  
Vector after insertion:  
  
10.001.00      2.00  3.00  4.00  5.00  6.00
```

Run2:

```
Size of vector? 5  
  
Vector elements?  
  
1.00  2.00      4.00  5.00  6.00  
  
Element to be inserted? 3.00  
  
Position of insertion? 3  
  
Vector after insertion:  
  
2.00  3.00      4.00  5.00  6.00
```

6) Multidimensional Arrays

Often, there is a need to store and manipulate two-dimensional data structures such as matrices and tables. Here, the array has two subscripts. One subscript denotes the rows and the other, the column. As before, all subscripts (row and column) start with 0.

Declaration

The declaration of a two-dimensional array is as follows:

```
float m [10] [ 20];
```

Here, m is declared as a matrix having 10 rows (numbered 0 through 9) and 20 columns (numbered 0 through 19). The first element in the matrix (the element at the first row and first column) is:

m [0] [0] and that at the last row and last column is:

```
m [9] [19]
```

Elements of multidimensional arrays

The first element is given by marks [0] [0] which contains 35.5, the second element is marks [0] [1] and contains 40.5 and so on.

Note: The declaration of a two dimensional array, for example, m[10] [20] may be confusing to those who are familiar with other languages such as FORTRAN, where the array subscripts are separated by a comma. Consider the following program typed by a confused programmer.

```
#include <stdio.h>

void main()

{intm. [10] [ 20];

ml [5] [16] = 13;

printf ( "%d\n", m [5], [16 ] );}
```

The first two lines in `main ( )` are fine, but the third line wrongly uses a comma inside the square braces, instead of enclosing the two subscripts separately. The danger lies in the fact that comma is an operator in C. Recall that the comma operator causes the expressions to be evaluated from left to right, and the value of the right most expression becomes the value of the entire expression containing the comma operators. In this case, the result of the expression with 5,16 inside the square braces, evaluates to 16 and hence, the `printf` statement shown above is equivalent to:

```
printf ( "%d\n", m [16] );
```

But since the array `m` is two dimensional, one would feel that `m` would give a compile time error. But this is not so, since `m` actually refers to the memory location of the first element in the 17th row. Even though there is no 17th row, C does not check this fact. so there will be no compile time errors on this account. At run time, instead of 13, some other number may get printed, or the program may crash. Needless to say, such errors made in a large program are difficult to detect.

Accessing two dimensional array elements

The elements of a two dimensional array can be accessed by the following expression:

```
marks [i] [j]
```

where `i` refers to the row number and `j`, the column number.

Initializing two dimensional arrays

Initialization of a two-dimensional array during declaration is done by specifying the elements in row major order (i.e., with the elements of the first row in a sequence, followed the those of the second, and so on). An examples given below

```
int two_dim_int [3] [4] = {{ 1,2,3,4 },
{2,3,4,5},
{5,6,7,8}};
```

The variable `two_dim_int` is declared as a two dimensional integer array with certain initial values. The first subscript can be omitted. For example, the declaration below wherein the number of rows (3) is not explicitly specified, is equivalent

the previous one.

```
int two_dim_int [ ] [ 4] =
{{ 1, 2, 3, 4 },
{2,3,4,5},
{ 5, 6, 7, 8 }};
```

Also, the inner braces can be omitted, giving the numbers in one continuous sequence. For example,

```
int two_dim_int [ 4 ] = { 1,2,3,4,2,3,4,5,5,6,7,8 };
```

has the same effect as the previous example, but is not readable as before.

Passing arrays to functions

Arrays can be passed as arguments to functions. Consider the following example:

```
/* arrfunc.c: passing arrays to function */

#include <stdio.h>

const int Rollno = 4;
const int Subject 3;

/* function declaration */

void display (float marks [Rollno] [Subject]);

main ( )

(float marks [Rollno] [ Subject] =

/* array initialization */

((35.5, 40.5, 45.5),
```

```

(50.5, 55.5, 60.5),
(65.0, 70.0, 75.5),
{80.0, 85.0, 90.0},};

/* function call with array as an argument */

display (marks);) /* end main */

/* function definition to display elements */

void display (float Studentmarks [Rollno] [Subjects])

(int r, s;

for (r=0; r<Rollno; r++)

{printf ("\n Rollno %d");

for (s=0; s<Subjects; s++)

printf("%10.2f", Studentmarks [r] [s];

} /* end for */

} /* end display */

```

The function declaration in the above program

```
void display (float marks [ [Rollno] [Subjects];
```

can also be written as,

```
void display (float marks [ ] [Subjects]);
```

The function display visualizes marks as an array, where the number of columns is Subjects. While calling the function, the name of the array is sufficient as an argument. Hence, the function call in main is of the form:

```
display (marks); /* function call /
```

The name marks represent the memory address of the array and is similar to using a reference argument. Elements of an array are not copied, but the function works with a different

name, on the original array itself. Hence, any modifications done to the array in the function are reflected in the array of the caller too. Passing by reference is efficient, as arrays can be very large and copying the entire array into a function consumes time and memory.

The function definition, void display (float StudentMarks [Rollon] [Subject] uses the name of the array, datatype and their sizes. inside the function the elements of the array are accessed by: Studentmarks [r] [s]

Example

```
/*addmat.c: program to add two matrices */

#include <stdio.h>

main ( )

(int a[10][10] b[10][10] c[10][10], i, j, m, n, p, q ;

printf ("Input row and column of A matrix \n");

scanf ("%d %d", &n, &m);

printf ("Input row and column of B matrix \n");

scanf ("%d %d", &p, &q);

/* checks if matrices can be added */

if((n == p) && (m == q))

{printf ("matrices can be added\n");

printf ("Input A-matrix\n");

for(i = 0; i < n; ++ i)

for(j = 0; j < m; ++ j)

printf ("Input B-matrix\n");

for(i = 0; i < n; ++i)

for(j = 0; j < m; ++j)

scanf ("%d", &b [i][j]);
```

```

/* Addition of two matrices */

for(i = 0; i < n; i++)
for(j = 0; j < m; ++j)
c [i] [j] = a [i] [j] + b [i][j]

printf ("Sum of A and B matrices :\n");

for(i = 0; i < n; ++i)
{for(j =0; j <m;++j)
printf("%5d",c [i][j]);
printf("\n") ; }
} /* endif /

else

printf(" matrices cannot be added\n");

} /* main */

```

Run 1:

Input row and column of A matrix

33

Input row and column of B matrix

12

matrices cannot be added

Run 2:

Input row and column of A matrix

33

Input row and column of B matrix

33

matrices can be added

Input A-matrix

1 2 3

- - -

4 5 6

- - -

7 8 9

- - --

Input B-matrix

1 2 3

- - -

4 5 6

- - -

7 8 9

- - --

Sum of A and B matrices :

2 4 6

- - --

8 10 12

- - -

14 16 18

- - -

Example

```
/*trace.c: Trace of the matrix */
```

```
#include <stdio.h>
```

```
main()
```

```
{int a [10] [10], i, j, n, trace
```

```
printf ("Enter the order of A matrix \n")
```

```

scanf ("%d", &n)

printf ("Input A-matrix\n")

for (i = 0; i <n; ++i
for (j = 0; j <n; ++j

scanf ("%d", &a [i][j]);

/* Trace = sum of the diagonal elements of the matrix */

trace = 0;

for (i = 0 ; j < n ; ++i)

trace + = a [i] [i] ;

printf("Trace =  %d \n", trace);

}/* main*/

```

Run:

Enter the order of A matrix

3

Input A-matrix 123

1 2 3

- - -

4 5 6

- - --

7 8 9

Trace = 15

Example

/***mult.c**: program to multiply two matrices */

#include <stdio.h>

main ()

```

{int a [10][10], b[10][10], c [10][10], i, j, m, n, ip, k, p, q;

printf ("Input row and column of A matrix: ");

scanf("%d %d", &m, &n);

printf ("Input row and column of B matrix: ");

scanf("%d %d", &k, &q);

if (n == k)

{printf(' matrices can be multiplied\n');

printf("resultant matrix is %d X %d\n",m,q);

printf ("Input A-matrix\n");

read_mat (a,m,n);

printf ("Input B-matrix\n");

/* function call to read the matrix */

read_mat (b,k,q);

/* Multiplication of two matrices */

for(i = 0; i < m; ++i)

for(j = 0; j < q; ++j)

{c [i][j] = 0;

for(ip = 0; ip < n; ++ip)

c [i] [j] = c[i] [j] + a [i] [ip] * b [ip] [j];}

printf("Resultant of A and B matrices :\n");

write_mat(c,m,q);

/* Multiplication of two matrices */

for(i = 0; i < m; ++i)

for(j = 0; j < q; ++j)

```

```

{c [i] [j] = 0;

for(ip = 0; ip < n; ++ip)

c [i][j] = c [i] [j] + a [i] [j] * b [ip] [j];}

printf("Resultant of A and B martices :\n");

write_mat (c,m,q);

} /* end if */

else

{printf(" col of matrix A must be equal to row of matrix B\n");

printf(" matrices cannot be multiplied\n");

} /* end else */

} / End of main */

/ function read matrix */

read_mat (a,m,n)

int a [10] [10],m,n;

{int i,j;

for(i = 0; i <m; i++)

for(j = 0; j <n; j++)

scanf ("%d",&a [i][j]);}

/* function to write the matrix */

write_mat(a,m,n)

int a [10][10],m,n;

{int i,j;

for(i = 0; i <m; i++)

{for(j=0; j < n; j++)

printf ("%5d", a[i][j]);

printf ("\n");}}
```

Run1:

Input row and column of A matrix: 3 3

Input row and column of B matrix: 3 3

matrices can be multiplied

resultant matrix is 3 x 3

Input A-matrix

1 2 3

4 5 6

- - -

7 8 9

- - -

Input B-matrix

1 2 3

- - --

4 5 6

- - -

7 8 9

- - --

Resultant of A and B matrices:

30 36 42

66 81 96

102 126 150

Run2:

Input row and column of A matrix : 1 2

Input row and column of B matrix: 3 4

col of matrix A must be equal to row of matrix B

matrices cannot be multiplied

Example

```
/*traflsc: Transpose of a matrix */

#include <stdio.h>

main()

{int  a [10] [10], b [10] [10],i,j,m,n;

printf ("input row and column of A matrix:");

scanf ("%d %d", &n, &m);

printf ("\nInput A-matrix\n");

for (i=0; i < n; ++i)

for (j = 0;j <m; ++j)

scanf ("%d", &a [i] [j]);

/* the rows and columns are interchanged: transpose */

for (i = 0; i < m; ++i)

for (j = 0; j < n; ++j)

b [i][j] = a [j][i];

printf ("\nTranspose of matrix A is :\n");

for (i= 0; i < m; ++i)

{for (j = 0; j < n; ++j)

printf("%5d",b[j][i];

printf("\n");}

}*/ main */
```


Run:

Input row and column of A matrix: 3 4

Input A-matrix

1 2 3 4

5 6 7 8

9 6 7 8

Transpose of matrix A is:

1 5 9

2 6 6

3 7 7

4 8 8

Check Your Progress – I

- 1) *State whether the following are True or False.*
 - a) Array is a collection of related data items
 - b) The two dimensional array is also called a matrix
- 2) *Fill in the blanks*
 - a) The General form of array is _____
 - b) The General form of the dimensional array is _____

10.4 Summary

In this lesson we have covered arrays and types of arrays. Array is a collection of related data items. And also we have covered some arrays related program in C language.

10.5 Check Your Answers

- I. 1) a) True b) True
- 2) a) data type name of array [size]

b) data type name of array [size] [size 2]

10.6 Model Questions

- (1) What are arrays? in C, can you have arrays of any data type?
- (2) In C, what is the index of the first element in an array?
- (3) What does the name of the array signify?
- (4) Can array indexes be negative?
- (5) Illustrate the initialization of a one dimensional array with an example.
- (6) Illustrate the initialization of a two dimensional array with an example.
- (7) Show the storage of a two dimensional array in memory with the help of a diagram.
- (8) Write a program to determine whether a matrix is singular or not ?
- (9) Write a program to find the inverse of a square matrix
- (10) Write a program to test for the orthogonality of a square matrix
- (11) Write a program to maintain a circular queue in an array
- (12) Implement the following sorting techniques:
 - (i) bubble sort
 - (ii) Selection sort
 - (iii) merge sort
 - (iv) quick sort
 - (v) heap sort
- (13) Write a program to find the minimum cost spanning tree.
- (14) Solve the 8 queens problem using backtracking.
- (15) Write program to perform polynomial addition.

LESSON – 11

STRINGS

Structure

11.1 Introduction

11.2 Learning Objectives

11.3. Strings

11.4 Summary

11.5 Check Your Answers

11.6 Model Questions

11.1 Introduction

Strings are arrays of characters, i.e., they are characters arranged one after another in memory. To mark the end of string, C uses null character. Strings in C are enclosed within double quotes. So, for example,

“My age is 2 (two)”

is a string. The string is stored in memory as ASCII codes of characters that make up the string, appended with 0 (ASCII value null). ASCII value of all characters are given in Chapter 14, with help of which we will examine the memory occupied by the string.

Normally, each character is stored in one byte, and successive characters of the string are stored in successive bytes.

Character: M Y a g e i s 2 (t w o) \0

ASCII code: 77 121 32 97 103 101 32 105 115 32 50 40 116 119 111 41 0 .rm65

The last character, as you can see, is the null character, having the ASCII value zero.

11.2 Learning Objectives

After learning this lesson, you should be able to describe the functions that are used for manipulating strings such as `strcat`, `strlen`, `strcpy` and `strlen`.

11.3. Strings

Following the discussion on character arrays, (i.e., a string, the initialization of a string must have the following form, (this is similar to initializing a one dimensional array):

```
char month 1 [ ] = { 'J', 'a', 'n', 'u', 'a', 'r', 'y', 0 };
```

Then the string `month` is initialized to `January`. This is perfectly valid, but C offers a special way to initialize strings. The above string can be initialized as:

```
char month 1 [ ] = "January";
```

The characters of the string are enclosed within a pair of double quotes. The compiler takes care of storing the ASCII codes of the characters of the string in memory and also stores the null terminator in the end. A simple program to display a string is listed below.

Example

```
/*string.c: string variable */
#include <stdio.h>

main
{char month [15];
printf ("Enter the string");
gets (month);
printf ("The string entered is %s", month);}
```

In this example, a string is stored in the character string variable `month`. The string is displayed by the statement:

```
printf ("The string entered is %s", month).
```

It is a one dimensional array. Each character occupies a byte. A null character ('\0') that has the ASCII value 0

Example

```
/*string2.c: illustration using a string literal *1
#include <stdio.h>

main ( )

{char strc [ ] = "This is a literal ";
printf ("%s", strc);}
```

Run:

This is a string literal

1) Arrays of Strings

An array of strings is a two dimensional array. Consider an example that initialize 5 names in an array of strings and prints them.

```
/*strarray.c: array of strings */
#include <stdio.h>

const int NUM = 5;
const int LENGTH = 9;

main ( )

{int j;

char Names [NUM] [LENGTH] =
{"Tejaswi", "Prasad", "Prasanth", "Prakash", "Anand" };

for (j = 0; j < NUM; j ++)
printf ("%s\n", Names [j]); }
```

Run:

Tejaswi

Prasad

Prasanth

Prakash

Anand

	0	1	2	3	4	5	6	7	8	
0	T	e	j	a	s	w	i	\0		Names [0]
1	P	r	a	s	a	d	\0			Names [1]
2	P	r	a	s	a	n	t	h	\0	Names [2]
3	P	r	a	k	a	s	h	\0		Names [3]
4	A	n	a	n	d	\0				Names [4]

Fig. 11.1 Array of strings

A string is an array; and Names is an array of strings. It can be represented by a two dimensional array of size [5] [9] as shown in Fig 11.1. The second dimension LENGTH specifies the length of the longest array string (9 characters for Prasanth including the terminal character '\0') and each name is accessed by Names [j]. Here, we do not need the second index. Names is sufficient to access string number j in array.

2) Function in strings.h

This section describes the functions that help in manipulating strings. To use these functions, the header file string.h must be included in the program with the statement:

```
#include <string.h>
```

strcat

This function concatenates two strings, i.e., it appends one string at the end of another. The function accepts two strings as parameters and stores the contents of the second string at the end of the first. An example is given below.

```

/*Strcat.c: Example program for strcat /
#include <stdio.h>

#include <string.h>

void main ( )

{ /* Declare two strings */

char oneString [10] = "Fish";

char twoString [ ] = "face";

/* Put twoString at the end of oneString. */

strcat( oneString, twoString)

printf( oneString ) ; }

```

The output of this program is:

Fish face

The central part of the program is the statement,

```
strcat (oneString, twoString);
```

which places the characters of twoString at the end oneString.

Observe that the declaration of oneString has the subscript 10 specifying the size of the string, while the declaration of twoString does not have any size specified. Recall that in the declaration of arrays, if the subscript is omitted, it is assumed to be of the size of the data with which the array is initialized. So, while the size of the string oneString is 10 (while means that oneString can hold a maximum of 10 characters), the size of twoString is equal to the size of the string face. This string has 4 characters plus a null character, which makes a total of 5. So, the size of the string twoString is 5.

If, we declare oneString with the statement,

```
char oneString [ ] = "Fish";
```

then oneString will be able to hold a maximum of 6 characters only. When the statement,

```
strcat (oneString, twoString);
```

appends the characters of twoString at the end of oneString, the size of oneString would not suffice. This leads to unpredictable results at run time. Hence, in the above program, 0 is declared by the statement,

```
char oneString [ 0 ] = "Fish ";
```

which is safe, since after concatenation, oneString becomes Fish face, which is 10 characters in length, including the null terminator. In general, whenever you use the strcat function, check to see 'whether the first string is large enough to hold the resultant concatenated string. Note, there is no '+' operator in C that can be applied to strings in order to concatenate them. The function strcat here exists for that purpose.

strcmp

The function strcmp compares two strings. This function useful while writing programs for constructing and searching strings as arranged in a dictionary. The function accepts strings as parameters and returns an integer, whose value is:

- * Less than 0 if the first string is less than the second
- * Equal to 0 if both are identical
- * Greater than 0 if the first string is greater than the second

The function strcmp compares the two strings, character by character, to decide the greater one. Whenever two characters in the string differ, the string which has the character with higher ASCII value is greater. For example, consider the strings hello and Hello!!. The first character itself differs. The ASCII code for h is 104, while that for H is 72. Since the ASCII code h is greater, the string hello is greater than the string Hello!!. Once a differing character is found, there is no need to compare the other characters of the strings. The following code fragment compares two strings and output the result.

```
char str1 [ 30 ], str2 [ ];
int cmpResult;
scanf ( "%s %s", str1, str2 );
cmpResult = strcmp( str1, str2 );
```



```

if (compResult < 0)
printf ( "stri < str2" );
elseif (cmpResult == 0 )
printf( "str1 == str2" );
else
printf( "str1 > str2" );

```

A sample run is shown below:

```
azcd abzzi
```

```
stri > str2
```

Again, note that the operators <, > and == cannot be used directly with strings. To compare strings, use strcmp.

strcpy

The strcpy function copies one string to another. The function accepts two strings as parameters and copies the second string character by character into the first one, upto and including the null character of the second string. As in the function strcat, you need to be careful to make sure that the size of the first string is greater or equal to the second.

For example the following code:

```

char str1 [ ] = "Will be overwritten";
char str2 [ ] = "Will overwrite";
printf( "Before strcpy stri is\n" );
printf( "%s\n", stri );
strcpy( sin, str2 );
printf( "After strcpy stri is\n" );
printf( "%s\n", stri );

```

will output:

Before strcpy stri is

will be overwritten

After strcpy stri is will overwrite

will overwrite

The operator = cannot be used to assign one string to another. Use strcpy for this purpose.

strlen

This function returns an integer which denotes the length the string passed. The length of a string is the number of characters present in it, excluding the terminating null character. For example,

```
char str "Hello";
```

```
printf( "%d\n", strlen(str) );
```

will output 5, since there are 5 characters in the string Hello.

More on string library functions is given in the chapter pointers.

Example

```
/*dup.c: Deleting duplicates in a vector */
```

```
#include <stdio.h>
```

```
main()
```

```
{int i, j, k, n, num, flag = 0;
```

```
float a [50]
```

```
printf (":Size of vector ? ");
```

```
scanf ("%d", &n);
```

```
num = n;
```

```
printf ("\nvector elements ?\n");
```

```

for (i = 0; i < n; i++)
scanf ("%t", &a [i]);
/* removing duplicates */
for(i=0; i<n-1; i++)
for (j=i + 1; j < n; j++)
{if (a[i] == a [j] )
{n = n - 1;
for (k = j; k < n; k++)
a[k] = a [k+1];
flag = 1;
j = j - 1;}}
/* use of IF and ELSE statement */
if (flag == 0)
printf ("\nNo duplicates found in vector \n");
{printf ("\nVector has %d duplicates ",num - n);
printf ("Vector after deleting duplicates \n");
for(i=0;i<n; i++)
printf ("%6.2f",a[i]);
printf ("\n") ;}}

```

Run:

Size of vector ? 6

Vector elements ?

1.00 2.00 1.00 3.00 2.00 4.00

Vector has 2 duplicates

Vector after deleting duplicates :

1.00 2.00 3.00 4.00

Example

/*grade.c: Calculating grades of 'N' students from 3 tests */

```
#include <stdio.h>
```

```
main ( )
```

```
{int i, j, k, n, m;
```

```
float a [50] sum,avg;
```

```
char grade [50]
```

```
printf ("Number of students ? ");
```

```
scanf ("%d", &n);
```

```
for(i=0, i < n; i++)
```

```
{sum=0.0;
```

```
printf ("\nEnter 3 scores of student-%d \n", i+1);
```

```
for (j= 0; j < 3; j++)
```

```
{scanf ("%f", &a [
```

```
sum += a [i] [j];}
```

```
avg = sum / 3.0;
```

```
a[i] [3] = avg;
```

```
if (avg <30.0) /* use of if.. else if.. else */
```

```
grade [i]= 'E';
```

```
else if (avg < 45.0)
```

```
grade [i] = 'D';
```

```
else if (avg <60.0)
```

```
grade [i] = 'C';
```

```
else if (avg < 75.0)
```

```

grade [i] = 'B';

else

grade [i] = 'A';}

printf ("\n Sl.no SCORE AVERAGE GRADE \n"); printf

("n                _\n\n");

for (l = 0; l < n; l ++ )

{printf ("%d.", i+l);

for(j = 0; j < 4; j++)

printf ("%8.2f", a [i] [j]);

printf ("%6c" < grade [i];

printf ("\n");

printf ("\n                \n\n");} }

```

Run

Number of students ? 5

Enter 3 scores of student-1

67.00 86.00 58.00

Enter 3 scores of student-2

20.00 30.00 23.90

Enter 3 scores of student-3

80.00 97.00 73.00

Enter 3 scores of student-4

56.80 47.90 62.00

Enter 3 scores of student-5

45.00 35.00 40.00

340

Sl.no	SCORES		AVERAGE	GRADE	
1.	67.00	86.00	58.00	70.33	B
2.	20.00	30.00	23.90	24.63	E
3.	80.00	97.00	73.00	83.33	A
4.	56.80	47.90	62.00	55.57	C
5.	45.00	35.00	40.00	40.00	D

```

printf ("Input A - matrix\n");

for (i = 0; i < n; ++i)
for (j = 0; j < n; ++j)
scanf ("%d", &a [i] [j]);

/* Transpose of the given Matrix */
for (i = 0; i < n; ++i)
for (j = 0; j < n; ++j)
b[i] [j] = a [i] [j];

printf ("Transpose of A matrix :\n");

for (i = 0; i < n; ++i)
{for (j = 0; j < n; ++j)
printf ("%5d", b[i][j]);
printf ("\n");}

/* Find symmetry of a matrix /
for (i = 0; i < n; ++i)
(for (j = 0; j < n; ++j)
if(a [i] [j] != b [i] [j])
flag = 1;}
```

```

if (flag)

printf ("Matrix is not symmetric\n");

else

printf ("Matrix is symmetric\n");

} */ main */

```

Run

Input order of A matrix: 2 2

Input A-matrix

1 2

3 4

Transpose of A matrix:

1 3

2 4

Matrix is not symmetirc

Run2:

Input order of A matrix: 3 3

Input A-matrix

1 2 5

2 3 4

5 4 9

Transpose of A matrix :

1 2 5

2 3 4

5 4 9

Matrix is symmetric

Check Your Progress – I

- 1) *State whether the following are True or False.*
 - a) Strings are array of characters
 - b) Strings in C are enclosed within single quotes.
- 2) *Fill in the blanks*
 - a) _____ function concatenates two strings.
 - b) _____ function compares two strings.
 - c) _____ function copies a string to another

11.4 Summary

In this lesson we covered strings and initializing strings. And also we have covered some string related examples programme in C language.

11.5 Check Your Answers

- I. 1) a) True b) False
- 2) a) Strcat b) Strcmp c) Strcpy

11.6 Model Questions

- (1) What is the null character and what is it used for, in the context of strings?
- (2) Point out the difference between:
 - (i) strcat and strncat
 - (ii) strcmp and strncmp
- (3) Write the name of the function in string.h which:
 - (i) performs case insensitive string comparison
 - (ii) searches for a character inside a string
 - (iii) searches for a string inside another string

- (4) What happens if the total size of the string after `strcat` becomes greater than the array size of the string used to hold the concatenated string? Does the compiler report an error?
- (5) Write a program to concatenate two strings.
- (6) Write a program to count the number of vowels, consonants and spaces in a line.

LESSON - 12

POINTERS

Structure

12.1 Introduction

12.2 Learning Objectives

12.3 Definition and Uses of Pointers

12.4 Summary

12.5 Check Your Answers

12.6 Model Questions

12.1 Introduction

The significance of pointers in C is the flexibility it offers in programming. This concept will become clear with the discussions that follow. Prior to understanding the importance of Pointers in C, it is necessary to consider some of the fundamental concepts about the organisation of memory in computers.

Memory is organized as a sequence of byte-sized (1byte = 8bits) locations. These bytes are numbered beginning with a zero.

The number associated with a byte is known as its address or memory location. A pointer is an entity which contains a memory address. Thus, a pointer is a number which specifies a location in memory. The key concepts and terminologies associated with memory organization are listed below,

- * Each byte in memory is associated with a unique address. An address is an integer having fixed number of bits, labelling a byte in the memory
- * Addresses are positive integer values that range from zero to some positive integer constant corresponding to the last location in the memory
- * Every object (data or program code) that is loaded into memory is associated with a valid range of addresses, that is, each variable and each function in a program starts at a

particular location, and spans across consecutive addresses from that point onwards depending upon the size of the, data item

- * The size of the data type referenced by the pointer is the number of bytes that may be accessed directly by using that address. For example, if an address A represents a float location, then the size of A is 4 bytes, beginning with the address A. The size of the data type to which the pointer refers to is relevant only when the address represents data rather than code.

Addressing is a technique used to access the contents of one or more memory locations. Two types of addressing modes relevant to programming are relative addressing and absolute addressing. Most of the conventional computers use the relative addressing mode to access memory. In the relative addressing mode, an address consists of two components, the base (or segment) address and the offset address. The base address designates a specific segment of the memory and the offset specifies the distance of the desired memory location from the beginning of that segment, as illustrated below. An example of a relative address on an 8086 machine (20 bit address bus) would be (in hexadecimal notation) as follows:

shift segment value 4 bits to Left. 108D:FFFE —————>

108D0 + FFPE = 208CE Seg:Off (Seg + Off Absolute address)

All relative addresses are first translated into the corresponding absolute addresses prior to physically accessing the memory. The use of relative addressing simplifies the managements of large memory

Note: Corresponding to a given physical address or absolute address, it is possible to have different relative addresses. For example, all relative addresses given below represent the same memory location.

Relative Addresses	Absolute Address
--------------------	------------------

0410 : 123C \	
---------------	--

040F: 124C ———>	00533C
-----------------	--------

040E: 125C /	
--------------	--

It is not always necessary to be aware of segments and offsets while programming in C. For example, for most C compilers in DOS, We should be aware of segments and offsets only if we have to manipulate more than 64KB of data at a time, or if the size of the executable part of the program is more than 64KB.

But on almost all UNIX systems, there is no need to worry about these limitations. On account of this, complex image? processing software and multimedia (applications requiring to handle large amounts of data) are generally developed on UNIX platforms. Note that compilers are available in the PC domain which do not have these limitations (such as compilers for OS2, Windows 95 and the GNU C compiler for DOS).

12.2 Learning Objectives

After learning this lesson, you should be able to

- define Pointers
- discuss how pointers are used in accessing array elements.
- discuss how pointers are passing arguments by reference.
- discuss how Pointers are used in passing arrays and strings to functions.
- explain how pointers are used in creating data structures such as linked lists, trees, graphs and so on.

12.3 Definition and Uses of Pointers

Pointer variables are defined as variables that contain the memory addresses of data or executable code. Normally, a pointer variable, like any other variable, is declared so that it will work only with data of a given type. Just as an integer variable can hold only integers, a character variable can hold the only characters and so forth, each pointer variable can point only to one specific type (int, char, float, double or any user defined data type) of data. Although, a lot of programming can be done without the use of pointers, their usage enhances the capability of the language to manipulate data. Dynamic memory allocation is programming concept wherein the use of pointers becomes indispensable. For example, for reading the marks of a set of students and processing them, an array can be declared with the statement, float marks [100]

But this limits the maximum number of students to 100. We are fixing the maximum number while writing the program. On the other hand, using dynamic memory allocation, we can design the program such that the limit for the maximum number of students is restricted only by the amount of available memory.

Variables can be created and defined when the program is being executed. Memory can be obtained from the system, if available, and allocated for the creation of a new variable or a data structure. This is called dynamic memory allocation. Dynamic variables so created can only be accessed by the use of pointers.

Pointers offer tremendous flexibility in the creation of dynamic variables which are no longer needed (a more detailed account of dynamic memory allocation is in Chapter 10).

The other uses of pointers are:

- Accessing array elements
- Passing arguments to functions by reference (i.e., the arguments can be modified)
- Passing arrays and strings to functions
- Creating data structures such as linked lists, trees, graphs, and so on
- Obtaining memory from the system dynamically

2)Address Operator &

All the variables defined in a program (including pointer variables) reside at specific addresses. It is possible to obtain the address of a program variable by using the address operator & (ampersand). When used as a prefix to the variable name, the & operator gives the address of that variable. The example given below demonstrates the use of the & operator.

```
/* getaddr.c: demonstrates the use of '&' address operator */
#include <stdio.h>

void main()

/* Define and initialize three integer variables */
int a = 100;
```

```

int b = 200;
int c = 300;

/* Print the address and contents of the above variables /
printf("Address: %x contains value: %i\n", &a, a);
printf("Address: %x contains value : %i\n", &b, b);
printf ("Address : %x contains value : %i/n", &c, c);}

```

This program defines three integer variables, initializes them and outputs their addresses and contents respectively. It should, however, be noted that the addresses printed by this program may be different systems. This is because, the actual addresses occupied by the program's variables depend on several factors, such as, the size of the operating system, and other programs residing in the memory, and so on.

The output of the program getaddr.c, when run in the DOS environment is given below.

Address : FFF4 contains value : 100

Address : FFF2 contains value : 200

Address : FFF0 contains value: 300

The output displays the addresses of variables in hexadecimal notation, It may be observed that the three hexadecimal addresses are in decreasing order, supporting the fact that all automatic variables are created in the program's stack area, and that the stack always grows from a higher address to a lower one. Further, each of the addresses differ from the other by exactly two bytes. It includes that integer variables are allocated 2 bytes of memory in this case. The size of operator of the C language can be used to determine the number of bytes allocated to each type of variable.

Adequate care must be taken to avoid any kind of confusion between the following:

- Unary address operator &, which precedes a variable name
- Binary bitwise logical operator &, which performs a bitwise

AND operation on its operands

3) Pointer Variables

Pointers are also variables and hence, must be defined in program like any other variable. The rules for declaring pointer variables names are the same as ordinary variables. This naturally gives rise to questions about the data type of a pointer, size of memory allocated to a pointer, and the syntax for declaring different types of pointers.

When a pointer is declared, the C compiler needs to know the type of variable the pointer points to. This rules out the necessity of having a special pointer data type in C. Instead, the format, `<data_type> *(<ptrvar name>)` is used, that supplies the required information about the pointer variable `<Ptrvar_name>` in the declaration, `<data_type>` could be any one of the primitive C data types like `int`, `char`, `float`, `double`, or a user defined structure (structures are explained in Lesson 10). The `ptrvar_name` could be any valid C name. It is the presence of an asterisk (*) following the `data_type` that makes the format different from the common C variable declarations. It indicates the compiler that the variable of the specified type (`<data_type>`). Some typical pointer declarations are:

```
int *int_ptr; /*int_ptr is a pointer to an integer */
char *ch_ptr; /*pointer to a character */
length *l_ptr; /*pointer to some user defined type (such as enum
called Length and so on. *,
```

Example

An example to access the address of a variable in the UNIX environment is given below. The addresses are printed in decimal. Hexadecimal output can be obtained by replacing `%u` by `%x` in the program.

```
/*ptradrl.c: POINTERS: ! the address of a variable
*/ #include <stdio.h>main (){int i = 7; floatf = 10.5;
char c = 'A' ;
```

```
/*%u (unsigned decimal) is used to print the location */
```

```
/* (address) in memory where the variable is stored */
```

*/The special symbol & is used to access the address of the variable. It is called the address operator */

```
printf ("%d is stored at address %u \n",i, &i);
```

```
printf ("%f is stored at address %u \n",f, &f);
```

```
printf ("%c is stored at address %u \n",c, &c);
```

Run

```
7 is stored at address 2147483476
```

```
10.500000 is stored at address 2147483472
```

```
A is stored at address 2147483471
```

Example

Here is a C program that assigns an integer pointer variable with the addresses of two different integer variables and prints the pointer value after each assignment. The printf statement here, uses the format specifier to output the pointer variable ptr.

```
/* initptr C: pointer (address variables) usage demonstration
```

```
*/#include <stdio.h>void main () {int *jptr;
```

```
/*pointer to integer, fig 8.3a */int var1,var2;
```

```
/*two integer variables, fig 8.3b */
```

```
var1 = 10; /* fig 8.3c */
```

```
var2 = 20; /* fig 8.3d */
```

```
iptr = var; /* fig 8.3e */
```

```
printf ("Address and contents of var1 is : %p and %d\n", iptr*iptr); = & var2; printf ("Address  
and contents of var2 is %p and %d\n",iptr,*iptr); *jptr = 125; var! = *jptr + 1;
```


The output when the program is run in the DOS environment is given below

Address and contents of var1 is :FFF4 and 10

Address and contents of var2 is :FFF2 and 20

Fig 12.3 gives a pictorial representation of the contents of memory during the execution of the program initptr.c:

The first statement in the function main (),

```
int *iptr;
```

specifies that iptr is a pointer to an integer. The asterisk (*) in the declaration specifies that iptr is pointer variable. It has no other significance here. The keyword int specifies that iptr can point to an integer in the memory.

The declaration could also be written as:

```
int* iptr;
```

This is valid, and makes no difference as far as the compiler is concerned. But there are certain advantages in following the former convention (i.e., placing the asterisk (*) closer to the variable name). The convention is to place the asterisk closer to the pointer variable name than the data type. The compiler always associates the asterisk (*) with the pointer variable name rather than the data type, thus allowing both pointer variables of a particular data type and non-pointer variables of the same data type to be declared in a single declaration. For example, we could replace the two separate declarations in the previous example initptr.c with a single equivalent declaration:

```
int *iptr, var1, var2;
```

To declare multiple pointers of the same data type in a single declaration, an asterisk must be prefixed to each pointer variable name in the declaration. For example,

```
float *f1, *f2, *f3;
```

declares f1, f2 and f3 as pointers to float variables.

The asterisk (*) in the last two statements of the function `main ( )` has an entirely different meaning; it is called the indirection operator and performs the operation of dereferencing. The term dereferencing means accessing the data in a memory location pointed to by a pointer. For example, if `iptr` is declared as a pointer to an Integer, `*iptr` is the integer which is pointed to by `iptr`.

In the statement,

```
*iptr = 125;
```

in the above program `iptr` contains the address of `avar2` (assigned by the statement `iptr &var2;`). Hence, `*iptr` refers to the contents of `var2`. The statement has the effect of setting `var2` to 125.

It is important that all pointers are suitably initialized with a spec (meaningful) address in the program prior their use, as they contain unpredictable values initially (after their declaration). Careless use of uninitialized pointers may cause the program to behave erratically.

For example, in the above program, the statement,

```
iptr = &varl;
```

is the first statement that assigns a meaningful value to `iptr`. Using `iptr` before this pointer is dangerous and can lead unexpected and unpredictable results when you run the program.

4) Dereferencing Pointers

Dereferencing is an operation performed to access and manipulate data contained in the memory location pointed to by a pointer. The operator `*` is used to dereference pointers. A pointer variable is dereferenced when the unary operator `*`, in this case called the indirection operator, is used as a prefix to the pointer variable or pointer expression. Any operation performed on the dereferenced pointer directly affects the value of the variable it points to. Here is an example that illustrates these concepts.

```
/*derefer.c illustrates dereferencing of pointer */
```

```
#include <stdio.h>
```

```
void main ( ) {
```

```

int *iptr, var1, var2;

iptr = &var1; /* initialize pointer iptr */

*iptr = 25;

/* dereference iptr, var1 = 25 */ *iptr += 10; /* var1 = var1 = var1+10; fig 8.4b */

/* next statement prints var1, it should be 35 */

printf ("Variable var1 contains : %i\n", var1);

var2 = *iptr; /* same as var2 = var1; fig8.34c */

/* next statement prints var2, it also contains 35 */

printf ("Variable var2 contains : %i\n", var2);

iptr = &var2 /* iptr now points to var2 /

*iptr += 20; /* increase contents of var2 by 20 */

/* next statement prints var2, it now contains 55 */

printf (Variable var2 now contains : %i\n", var2);}

```

Run

Variable var1 contains :35

Variable var2 contains :35

variable var2 now contains :35

This program highlights the following important facts about pointers.

1. The asterisk (*) used as an indirection operator has different meaning from the asterisk used to declare pointer variables.

```
int *ptr; /*This is a pointer declaration */
```

```
*ptr = 100 /*The LHS does pointer indirection */
```

2. Indirection allows the contents of a variable to be accessed and manipulated without using the name of the variable.

All variables that can be accessed directly (by their names) can also be accessed indirectly by means of pointers. Power of pointers becomes evident in situations where indirect access is the only way to access variables in memory.

Example

A complete program that shows the contents and addresses of the program variables is given below.

```
/* ptradr2.C Accessing a variable through its pointer
#include <stdio.h>

main ( )
{int a = 5,b = 10;
int *iptr;
/* pointer is initialised by assigning a variable to it */
ptr = &a;
printf ("initial values of 'a' and 'b' :\n");
printf ("a = %d \n", a);
printf ("b %d\n", b);
printf ("\n");
/*The symbol * placed before a pointer accesses the contents of the variable whose
address is
assigned to it */
b = *ptr;
printf ("Changed values of 'a' and 'b' :\n");
printf ("a = %d \n", a);
printf ("b = %d \n", b);
printf ("\n");
printf (Address of variable 'a' is %u \n", &a);
printf (Address of variable 'b' is %u \n", &b);
```

```
printf ("\n");
printf (Address assigned to pointer 'ptr' is %u \n", ptr);
printf ("Value pointer 'ptr' accesses is %d \n", * ptr);}
```

Run:

Initial values of 'a' and 'b'

a = 5

-b = 10

-Changed values of 'a' and 'b' :

a = 5

-b = 5

-Address of variable 'a' is 2147483476

Address of variable 'b' is 2147483472

Address assigned to pointer 'ptr' is 2147483476

Value pointer 'ptr' accesses is 5

Example

A program to swap two numbers using a function is given below. The function swap accepts 2 pointer variables. The dereferencing operator is used inside the function to access the data being pointed to.

```
/* Swpnums.c: program to swap 2 numbers using pointers */
```

```
#include <stdio.h>
```

```
void swap (float *, float *);
```

```
void main ()
```

```
{float data1, data2;
```

```
printf ("Enter 2 real numbers : \n");
```

```
scanf ("%f %f" &data1,&data2);
```

```
printf ("Data1 contains : %f\n", data1);
```

```

printf ("Data2 contains : %f\n", data2);

/* Pass address of the variables whose values are to be swapped.*/

Swap (&data1, data2);

printf ("After swapping ..... \n");

printf("Data1 contains : %f\n", data1 );

printf("Data2 contains : %f\n", data2 );}

Void swap (float *d1, float *d2) /* function to swap two numbers */

{float temp;

temp = *d1;

*d1 = *d2;

*d2 = temp; }

```

Run:

```

Enter 2 real numbers

Data1 contains : 23.55

Data2 contains : -65.73

After swapping

Data1 contains : -65.73

Data2 contains : 23.55

```

5) Void Pointers

Pointers defined to be of a specific data type cannot hold the address of any other type of variable to a pointer of type float as illustrated below:

```

float *f_ptr; / declare pointer to float */

int my_int; / declare an integer */

...

```

```
f_ptr = &my_int;
```

```
/* This statement results in error during compilation. */
```

An exception to this restriction is a general purpose pointer type called the void pointer. The format for declaring a void pointer is as follows:

```
void *v_ptr; /* declare a pointer to void */
```

This declaration uses the C reserved word void for specifying the type of the pointer. Pointers declared in this manner do not have any type associated with them and can contain the address of any type of variable (int, char, float etc.). The following are some valid C statements

```
...
```

```
void *vd_ptr; /* declare a pointer to void */
```

```
int *it_ptr; /* declare an integer pointer */
```

```
int invar; /* declare an integer variable */
```

```
char chvar; /* declare a character variable */
```

```
float flvar; /* declare a float variable */
```

```
double dvar; /* declare a double variable */
```

```
...
```

```
vd_ptr = &invar; /* a valid pointer assignment */
```

```
vd_ptr = &char; /* _____ ,, _____ */
```

```
vd_ptr = &flvar; /* _____ ,, _____ */
```

```
vd_ptr = &dvar; /* _____ ,, _____ */
```

```
it_ptr = &invar; /* _____ ,, _____ */
```

```
it_ptr = &chvar; /* an INVALID assignment */
```

Pointers to void cannot be directly dereferenced like other pointer variables operator. Prior to dereferencing ,a pointer to void, it must be suitably typecast to the required data type

A void pointer may be typecast by preceding its name with a relevant data type followed by an asterisk, both of which are enclosed within a pair of parentheses. Here is an example that explains the idea:

```
/*Voidptr.c: illustrates the use of void pointers */

#include <stdio.h>

void main ( )

{ int il = ; / declare and initialize int il to 100 */

float fi = 200.5; /* declare and initialize float fl to 200.50 */

void *yptr; /* declare pointer to void */

vptr = &il; /* valid pointer assignment

printf("il contains %i\n" *((int *) vptr) );

/* dereferences a void pointer using the typecast (float *) */}
```

Run

```
il contains 100

fl contains 200.500000
```

The expression *((float*) vptr) is explained below. When a function is designed to do similar operations on different data types, void pointers can be used to pass parameters to the function.

6) Pointer arithmetic

The size of the data type which the pointer variable points to is the number of bytes accessed in memory when the pointer variable is dereferenced (for a particular implementation of the C compiler):

- A pointer to an int accesses 2 bytes of memory
- A pointer to a char accesses 1 byte of memory
- A pointer to a float accesses 4 bytes of memory

- A pointer to a double accesses 8 bytes of memory

The C language allows arithmetic operations to be performed variables. It is, however, the responsibility of the programmer to see that the result obtained by performing pointer arithmetic is the address of relevant and meaningful data.

The arithmetic operators available for use with pointers can be classified as

- Unary operators : ++(increment) and —(decrement)
- Binary operators: + (addition) and - (subtraction)

Examples of pointer arithmetic

```
int a, b, *p,*q;

p = -q; / illegal use of pointer */

p <= 1; /* Illegal use of pointer */

p = p - b; /* Valid */

p = p - q; / Nonportable pointer conversion*/

p = (int *) (p - q); / Valid */

p = p - q - a; / Nonportable pointer conversion */

p = (int *) (p - q) -a; /* Valid */

p = p + a; /* Valid */

p = p + q; /* Invalid pointer addition */

p = p + q + a; /* Invalid pointer addition */

p = p * q; /* Illegal use of pointer */

p = p * a; /* Illegal use of pointer */

p = p / q; /* Illegal use of pointer */

p = p / b; /* illegal use of pointer */

p = a / p; /* illegal use of pointer */
```

The C compiler takes the size of the data type being pointed to into account, while performing arithmetic operations on a pointer. For example, if a pointer to an integer is incremented using the ++ operator (preceding or succeeding the pointer) then the address contained in the pointer is incremented by two and not one, assuming that an integer occupies two bytes in memory. Similarly, incrementing a pointer to a float causes initial address contained in the float pointer to be actually incremented by 4 and not 1 (if the size of the float variable is on the machine). In general, a pointer to some type, `d_type` (where `d_type` may be `int`, `char`, `float`, `double` or any user defined data type), when incremented by an integral value `i` will result the following value:

$$(\text{current address in pointer}) + i * \text{size of (d_type)}$$

to be the new incremented pointer value. This concept applies to all arithmetic operations performed on pointer variables in program. Pointer arithmetic becomes very significant accessing and processing array elements efficiently (a more detailed account of array processing with pointers is taken later in this chapter). Note that pointer arithmetic cannot be performed on void pointers, since they have no type associated with them.

Example

```
/* ptrarr1.c: Smallest in an array of 'n' elements using pointers */
#include <stdio.h>

main ( )
{int i, n, small,*ptr, a [50];
printf ("Size of the array ?");
scanf ("%d", &n);
printf ("\nArray elements ?\n");
for (i = 0;i < n; i++)
scanf ("%d", &a [i] );
/* assign address of a [ to pointer 'ptr'. This can be done in two ways:-
ptr = &a [0]; ptr = a; */
```

```

ptr = a;
/* contents of a assigned to small */
small = *ptr;
/* pointer points to next element in the array i.e., a [1] */+
ptr++;
/* loop n-1 times to search for smallest element in the array */
for (i) 1; i<n; i++)
{if (small > *ptr)
small = *ptr;
/* pointer is incremented to a [i+1] */}
/* Display smallest element stored in variable 'small' */
printf ("\nsmallest element is %5d \n", small);}

```

Run

```

Size of the array? 6
-Array elements?
-9 12 -23 5 0    -77
— — — - - —
Smallest element is    - 77

```

Example

```

/*pfrexp.c: program to illustrate pointer expressions in UNIX */
#include <stdio.h>

main ( )
{int a, b, c, d,*ptr1,*ptr2;
a = 5;
b = 10;
ptr1 = &a;

```

```

ptr2 = &b;
C = *ptr1 + *ptr2;
d = *ptr2/*ptr1 * 7
printf ("Addresses of variables 'a' and 'b' :-\n");
printf ("a : %u \n", &a);
printf ("b: %u \n",&b);
printf ("\n");
printf ("Addresses pointers 'ptr1' and 'ptr2' hold:-\n");
printf ("ptr1 = %u \n",ptr1);
printf ("ptr2 = %u \n",ptr2);
printf ("\n");
printf ("Values of 'a', 'b', 'c' and 'd' :-/n");
printf ("a =%2d b = %2d c = %2d d = %2d\n",a,b,c,d);
printf ("\n"); printf ("Values accessed by pointers 'ptr1' and 'ptr2':-\n");
printf (*ptr1 = %2d *ptr2 = %2d \n

```

Run:

Addresses of variables 'a' and 'b' :-

a : 2147483476

b : 2147483472

Addresses pointers 'ptr1' and 'ptr2' hold

ptr1 = 2147483476

ptr2 =2147483472

Values accessed by pointers 'ptr1' and 'ptr2':-

*ptr1 = 5 *ptr2 = 10

Subtraction of Pointers

When two pointers are subtracted, the result is of a type called `size_t`. This type is normally equivalent to the unsigned integer type. The result is in units of the size of the data type being pointed to. For example, consider the following piece of code:

```

short int *p, *q;

float *f, *g;

q = p + 1;

g=f+ 1

printf ("%ct\n", q - p );

printf ( "%d\n", g - p );

```

Both the above printf statements output 1. Even though the numerical values of the pointers differ by 2 (in the first case) by 4 (in the second case), this difference is divided by the of the type being pointed to. To get the difference in units bytes, typecast the pointers being subtracted, into char pointers. For example, the expression (char*)g - (char*)f will yield 4.

7) Memory Models in DOS

Pointer variables like other variables, are also allocated memory whenever they are declared. The size of the memory allocated (in bytes) to a pointer variable, in DOS, purely depends on whether the pointer just holds the offset part of the address, or both the, segment and offset values (recall the discussion presented in the beginning of this chapter for a detailed explanation of segment and offset addresses). The memory model in which the program is compiled also influences the of the pointer variables used in that program.

Memory			Segment		
<i>Model</i>	<i>Code</i>	<i>Data</i>	<i>Stack</i>	<i>Code</i>	<i>Data</i>
Tiny		64K		near	near
Small	64K	64K		near	near
Medium	1MB	64K		far	near
Compact	64K	1MB		near	far
Large	1MB	1MB		far	far
Huge	1MB	64 each	64 each	far	far

Table 12.1 Memory models

The DOS C compiler (such as Turbo C) support six memory models, each of which determines the amount of memory allocated to the program's data and code area. Table 12.1 summarizes the specifications for each of the models.

Other compilers in DOS, such as G NUC, do not have these memory models because in these compilers all addresses are 32 bits wide. This is advantageous for the user, since it does not have the 64 KB limit.

Typically, all pointers declared in a program in the small model contain only the offset part of the address. Such pointers are known as near pointers. Using near pointers limits the programmer to access only those memory locations which lie within one segment (the size of a segment is 64KB). This limitation can be overcome by the use of pointers which are capable of holding both the segment as well as the offset part of an address. Such pointers are called far pointers, far pointer variables are allocated 4 bytes of memory. It is possible to access any memory location by using far pointers, far pointers can be declared (even in a small model program) by using the keyword far following the data type in the pointer declaration. For example:

```
int far *iftrptr; /* declares a far pointer to int */
```

```
char far *cfarptr; /* declares a far pointer to char */
```

are valid far pointer declarations. The keyword far is not required in compact and large memory models, since all pointers are far by default in these models.

In the compact and large models, the data area can be more than 64KB but any single data structure (like arrays, structures) should be smaller than 64 KB. For example, if an array is defined as `int far *ary`, then `ary` will have both segment and offset parts but when pointer arithmetic is done, only the offset part is used and not the segment part. If `ary = 0x5437:0xffff`, and it is incremented, then `ary` will become `0x5437:0x0000`. The offset part wraps around and the segment part remains unchanged, hence any single data structure should be less than 64 KB. However, more than one such data structure can exist.

In the huge model or in case of a pointer declared with the huge keyword (as `int huge*ary;`) the segment of a pointer is updated properly and hence, any single data structure can be up to 1MB in size. In the above example `ary` will be `0x6437:0x0000` after incrementing in the huge model. In

other models, automatic increment of the of the segment is done only if the pointer is explicitly declared huge for example the statement `int huge *apg;` declares `ary` as a huge pointer.

DOS compilers normally provide three specialized and predefined macros viz., `MK_FP`, `FP_SEG`, and `FP_OFF` for use with far and huge pointers. The `MK_FP` macro takes two unsigned integer input arguments which are the segment and the offset addresses of the location to be accessed, and returns a values that can be used to initialize a far or huge pointer variable. here is an example for initializing a far pointer variable,

```
char far *cptr; /* declare a far pointer variables */

... cptr = (char *) MK_FP(0xB8000, 0x0000);

/* this causes the far pointer 'cptr' to point to a byte which must be typecast suitably
before use */
```

The macros `FP_SEG` and `FP_OFF` required a far pointer as their only input argument, and they return the segment and the offset parts of the address contained in the given far pointer. The three macros mentioned above, become available by including the header file `dos.h`.

An example p listed below declares a far pointer to a character, initializes it with an arbitrary address (say segment = `0xB800` and offset = `0x0000`), extracts and prints the segment and offset of `t1` same pointer. It also prints the ASCII character residing at the address `B800:0000`.

```
/*farptr.c: demonstrated the use of far pointers and related macros */

#include <dos.h>

#include <stdio.h>

void main ( )

{char far *cptr; /* declare far pointer to character */

unsigned int seg_val, off_val;
```

```

/* variables to hold segment and offset address respectively */

/* initialize far pointer */

cptr = (char far *) MK_FP(0xB800, 0x0000);

/* fetch segment address from far pointer */

seg_val = FP_SEG (cptr);

/* fetch offset address from far pointer */

printf( "character at 0xB800:0x0000 is : % c\n" , ;*cptr);

printf ("segment part of cptr = %X",seg_val);

printf ("offset part of cptr = %X" ,off_val) ; }

```

Run:

Character at 0xB800:0x0000 is : T

Segment part of cptr = B800

Offset part of cptr = 0

Note: The ASCII character printed by the above program will be same as the first character on the top left corner of the monitor. This is because, the address B800:0000 is a location in the video memory which holds the ASCII value of the character appearing in the top left corner in the text mode (except in HGA).

8) Pointers to Pointers

Pointer is type of data in C, hence we can also have pointers to pointers, just as we have pointers to integers. Pointers offer flexibility in handling arrays, passing pointer variables to functions, etc.

The general format for declaring a pointer to pointer is:

```
<data_type> **<ptr_to_ptr>;
```

which uses two asterisks (**) symbols placed one beside the other. Thus, the declaration implies that the variable <ptr_to_ptr> is a pointer to a pointer pointing to a data object of the

type <data_type>. This feature is often made use of while passing two (or higher) dimensional arrays back and forth among different functions. While a more detailed account of using arrays will be taken up in the next section, the following program illustrates the format for declaring and using a pointer to another pointer. Fig 15.6 illustrates the concept of a pointer of a pointer,

```
/* ptrptr.c: declaration and use of pointers to pointers,*/
#include <stdio.h>

void main(void)
{int *iptr;
 int **ptrjptr;
 int data;
 iptr = &data;
 ptrjptr = &iptr;
 *iptr = 100;
 printf ("Variable 'data' contains : %i\n", data);
 **ptrjptr = 200;
 printf("Variable 'data' contains : %i\n", data);
 data = 300;
 printf("ptrjptr is pointing to : %i\n", **ptrjptr; / *output 300*/}
```

Run:

Variable 'data' contains: 100

Variable 'data' contains: 200

ptrjptr is pointing to : 300

Check Your Progress – I

- 1) *State whether the following are True or False.*
 - a) The number associated with a byte is known as its address or memory location.
 - b) Address are positive real numbers

2) *Fill in the blanks*

- a) _____ are defined as variables that contain the memory addresses of data
- b) _____ operator gives the address of the variables

12.4 Summary

In this lesson we have covered the definition pointers and the use of pointers. We have also covered how the pointers are used in accessing array elements and how the points are used in creating data structures such as linked lists, trees, graphs and so on.

12.5 Check Your Answers

- I. 1) a) True b) False
- 2) a) Pointers b) &

12.6 Model Questions

- (1) What is the function of a pointer variable? What are its uses?
- (2) What are address operator and dereferencing operator symbols in C? What do they do?
- (3) Explain the difference between the address operator '&' and bitwise operator '^' with examples.
- (4) Do all pointer types (pointer to int, char, etc.) have the same size in a particular compiler?
- (5) Write a program to accept a string and print the address of its beginning location.
- (6) Write a function that accepts two integer arguments by reference, and performs a swap.
- (7) If ptr is a pointer to a user defined type or basic type, by how many bytes is ptr incremented when the statement ptr++ is executed? [**Hint:** a pointer variable always points to a specified type of variable]
- (8) What is meant by dereferencing? What is the difference between a pointer and a dereferenced pointer?
- (9) What is the value of a and b after the following statements are executed?

```
int *p1, pZ, a, b;

p1 = &a; p2 = &b;

*p1 = 5; *p2 = 10;

*p2 = (*p1)++ - - ;
```

[Hint: Determine the associativity of unary operations]

- (10) What is special about void pointers?
- (11) Why do you think a void pointer needs to be explicitly type cast before being dereferenced?
- (12) Which one of the following statements is true? (i) a void pointer can be assigned to any pointer type (ii) a pointer of any type can be assigned to a pointer
- (13) How do the arithmetic operators + and - behave in the following cases?
 - (i) both operands are pointers
 - (ii) one is an integer
- (14) What do you understand by a pointer to a pointer? Can this be extended to any level? Verify
- (15) Write a function that accepts two arguments: an array and its size n. It performs a bubble sort on the array's elements. Use the indirection operator instead of the array subscript operator '['].
- (16) Write a function to fill a two dimensional array of integers with user defined input by using the dereferencing operator "*" in place of the array subscript operator '[']. array is passed as an argument. [Note: a two dimensional array is of type 'pointer to a pointer']
- (17) Explain how three dimensional arrays are represented in memory.
- (18) Write a function that receives a three dimensional array and its dimensions as parameters, and populates it using the indirection operator "i" instead of the array subscript operator '['].
- (19) Explain the difference between an array and a pointer.
- (20) Write a function that accepts two strings as parameters and returns 1 if the first is a substring of the second, and 0 otherwise. [do not use strstr]

- (21) Explain the difference between an array of character as arguments and returns the number of occurrences of the character in the string.
- (22) Write a function that accepts two strings as parameters and prints the characters in the first that are not present in the second, and vice versa
- (23) What is the relationship between a function name and the function?
- (24) It is simple to declare pointers of type int, char, etc.. By what type is the function pointer characterized?
- (25) When are two function pointers f1 and f2 said to be of incompatible types? In other words, when is it not possible to executed the statement $f1 = f2$?
- (26) Explain the difference between a pointer to a constant and a constant pointer with examples.
- (27) Write a program to remove the trailing blanks in a string input by the user, and print the resulting string. Verify that the string has actually been trimmed, since spaces are invisible when printed.
- (28) Write a function that accepts characters into a string by using a pointer. Compute the length of the array using a pointer, given that the string is terminated by a null character. Finally print the string in reverse order using a Pointer.

LESSON - 13

ADVANCED CONCEPTS OF POINTERS

Structure

13.1 Introduction

13.2 Learning Objectives

13.3. Pointers and Arrays

13.4 Summary

13.5 Check Your Answers

13.6 Model Questions

13.1 Introduction

The elements of an array can be efficiently accessed by using a pointer. The following facts regarding the relation between pointers and arrays should be noted.

- Array elements are always stored in contiguous memory locations irrespective of the size of the array:
- The size of the data type which the pointer variable refers to, is dependant on the data type pointed to by the pointer. For example, if the pointer variable points to an integer data type, then it accesses two bytes in memory assuming that an integer occupies two bytes.

13.2 Learning Objective

After learning this lesson, you should be able to

- Explain pointers and arrays.

13.3 Pointers and Arrays

A pointer when incremented, always points to a location after skipping the number of bytes required for the data type pointed to by it. For example, if a pointer variable points to an integer data type at a certain address, say 1000, then on incrementing the pointer variable, it

points to the location 1002, If an integer data type occupies two bytes of data. One way of accessing the array elements by using pointers is illustrated below.

```
main ( )
{int iarray [5] = {1,2,3,4,5};
int l *ptr;
ptr = &iarray [0];
for (i=0; i<5; i++)
{printf ("%d - %d\n", ptr, *ptr);
ptr++;
/* increments the pointer to point to the next element
/* and not to the next memory location */ }}
```

In the above program, an integer array of size 5 is declared and initialized. Then, the address of the 0 element of the array is assigned to the integer pointer ptr. Within the loop, the address of the array elements is stored in ptr, and the contents of the memory location pointed to by ptr are printed. When ptr is incremented, the value stored in ptr is incremented by 2 to point to the next element of the array (as each integer occupies 2 bytes of memory). The array may be stored in memory as shown in Fig 13.1.

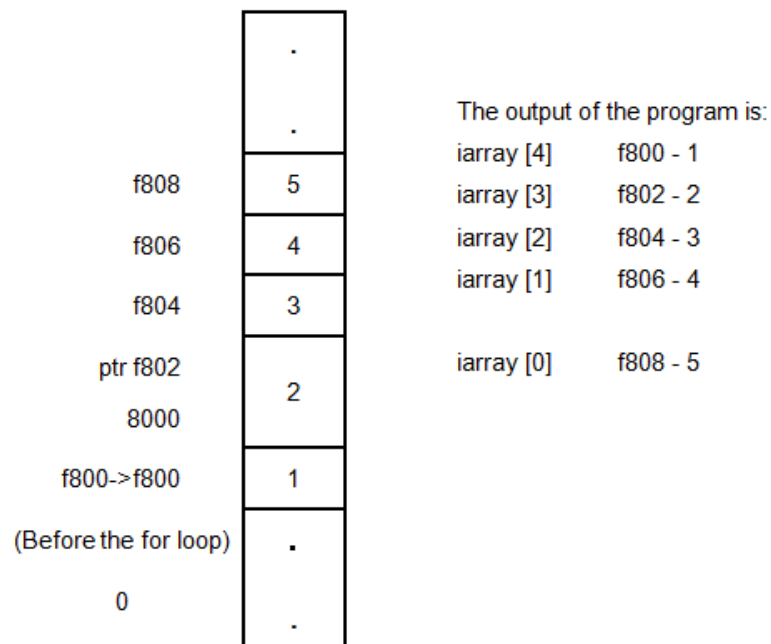


Fig. 13.1 Accessing array elements using pointers

In the above example, an integer pointer `ptr` is explicitly declared and assigned the starting address. The same result can be obtained by using one of the property concerning pointers and arrays. It is interesting to observe that the name of the array represents the starting address (base address) of the array. It is the address of the first element in the array. Thus, the name of the array can be used effectively as any other pointer to the array in accessing the elements of the array. This is illustrated in the following example.

```
main ( )
{int iarray = {1,2,3,4,5};
int i;
for (i = 0; i < 5; i++, iary++)
{printf ("%d %d ", iarray [i], * (array+i));
printf ("%d %d %d\n", *(i+iarray), i [array], *iai.y);}}
```

The output of the above program is given here (each column is output by accessing the array in a different way).

```
1 1 1 1 1
2 2 2 2 2
3 3 3 3 3
4 4 4 4 4
5 5 5 5 5
```

Note: In the above program, if we use the term `iarray[i]` more than once in the loop, then each time the array index `i`, is multiplied with the size of the data type (here it is the size of an integer) and then added to the base address (here `iarray`). The same sequence of operations take place in case of `*(iarray+i)`, `*(i+array)` and `I [iarray]`. The term `*iary` will require no arithmetic and hence, accessing the array elements by pointers is faster than accessing them by subscripts.

1) Passing Arrays to Functions

When a function requires to manipulate an entire array, it necessary to pass the entire array to the function during a single call rather than passing one element at a time. This can be achieved through the fact that the name of the array points to the base of the array. The following example illustrates this concept:

```

#include <stdio.h>

void sort_it (int , int); /* Function prototype */

void main ( )
{int a[20], i, n;

printf ("Enter the number of elements in the array (less than 21): ");

scanf ("%i", a+i);

printf ("Enter the elements : \n"); for(i = 0; i < n; i++)

scanf ("%i", a + i);

sort_it (a, n); /*call to function sort_it by passing array a */

printf ("\nThe sorted array : \n");

for (i = 0; i < n; i++)

printf ("%i", a + i);}

void sort_it (int th, int n) /* sorts the array in ascending order */
{int i = 0, j, f = 0, temp;

while (i ++ < n && !f)

{f = 1;

for (j = 0, j < n - i; j++)

if (b [j] > b [j + 1])

{ temp = b [i];

b [ j ] = b [j + 1];

b [j + 1] = temp;

f = 0; }}}

```

Run:

Enter the number of elements in the array (less than 21) : 5

Enter the elements:

- 4 7 4 0 1

-- - - - -

The sorted the array:

-4 0 1 4 7

In the above example, the function `main ( )` initializes an integer array and then calls the function `sort_it` to sort the array in ascending order using the bubble sort technique. The name of the array, which is a pointer to the base address of the array is pass as a parameter to the function `sort_it`. The function `sort_it` sorts the array `a` through formal parameter `b` which is a pointer to the base address of the array `a`.

The function `sort_it` actually sorts the original array `a` itself directly since the address of the array is passed as an argument to it. No temporary array is created in this case. This type of function call is referred to as call by reference. Thus, even without returning any value from the function `sort_it`, the sorted array is obtained directly.

Passing Array of Pointers to Functions

It is traditional style to sort numbers in ascending or descending order by data movement. This method of sorting incurs much overhead, as it needs temporary space for exchanging data, and has excessive data movement, especially if the size of the data being sorted is large. Pointers can be utilized to perform than same with more flexibility and less overhead. In this method, instead of data exchange, pointers are exchanged to accomplish the same. The program `sortptr.c` illustrates the method of sorting strings without swapping their contents. In `main`, person is an array of 100 char pointers. Each of these pointers have to point to valid memory locations. The function `malloc` is used to allocate 40 bytes, and assign pointer values to each of the 100 pointer variables.

Example

```
/* sortptr.c: sorting of strings by pointer movement */

#include <stdio.h>

#include <string.h>    //

#include <alloc.h> /* for malloc */

void SortBy PtrExchange (char ** person, t n)
```

```

{ int i, j;
char *temp;
for (i=0; i<n; i++)
for (j=0; j<n-1; j++)
if ( strcmp ( person [j] person [j+1] ) > 0)
{ /* exchange pointers */
temp = person [j]
person [j] = person [j+1];
person [j+1] = temp; }}}
void main ( )
{int i, n = 0;
char * person [100];
char choice;
do
person [n] = (char*) malloc(40);
printf ( "Enter name:");
scanf ( "%s", person [n++] );
printf ( "Enter another (y/n)? ");
scanf ( " %c", & choice)
} while (choice == 'y' )
printf ("Unsorted list: " );
for (i = 0; i<n; i + +)
printf ( "\n%s", person [i] );
SortByPtrExchange (person, n);
printf ( "\nSorted list: " );
for (i=0; i<n; i++)
printf ( "\n%s", person [i];

```

```
for (i=0; i<n; i++)  
free (person [i] );}
```

Run:

Enter Name : Tejaswi
Enter another (y/n)? y
Enter Name : Prasad
Enter another (y/n)? y
Enter Name : Prakash
Enter another (y/n)? y
Enter Name : Sudeep
Enter another (y/n)? y
Enter Name : Anand

Unsorted list:

Tejaswi
Prasad
Prakash
Sudeep
Anand
Sorted list:
Anand
Prakash
Prasad
Sudeep
Tejaswi

2) Pointers and Functions

We can pass arguments to a function by value or by a pointer. Modifications made to a parameter passed as a pointer to a function affects the variable in the calling program.

```

/* argref.c: Passing argument by pointer */

#include <stdio.h>

void Add 10 (int *studentmarks)

{ *Studentmarks += 10; /* Studentmarks is same as marks */}

main ( )

{ int Marks = 25;          /* marks has a value of 5 */

printf ("Value of Marks = %d", Marks);

Add 10 (&Marks);

printf ( "Value of Marks after execution = %d", Marks);}

```

Run:

Value of Marks = 25

Value of marks after execution = 35

The function Add 10 takes an integer pointer as an argument by the statement,

When main calls the function Add 10, it supplies the address of the variable Marks as an argument. It is given by the statement,

Add 10 (&Marks);

An indirection operator *Studentmarks in the function Add 10 is used to access the value stored in Marks in main (). The statement is as follows:

*Studentmarks += 10;

The pointer Studentmarks contains the address of the variable Marks and so any modification done to *Studentmarks will be reflected in the variable Marks. This is illustrated in Fig 9.9.

3) Accessing Arrays Inside Functions

Arrays can be accessed from a function. To illustrate this concept, consider the following example:

```

/* accfunc.c: Accessing array from a function */
#include<stdio.h>

const int Max = 5;

void Add10 (int *Studentmarks)

{ int i;
  for (i = 0; i< Max; i++)
    *Studentmarks +++ = 10;}

main ( )

({ int i;
  int Marks [Max] = [25, 43, 70, 80, 76];
  Add 10 (Marks);
  for (j= 0; j<Max; j++)
    printf ("Marks [ = %d\n]", j, Marks [ i ] ; }

```

Run:

```

Marks [0] = 35
Marks [1] = 53
Marks [2] = 80
Marks [3] = 90
Marks [4] = 86

```

The name of the array is sufficient to pass the array and an & is not required in this case as the name of the array itself is the address of the array and hence, the statement is as shown below:

```
Add 10 (Marks);
```

In the function Add 10() the address of the array Marks declared in main () is stored in the pointer variable Studentmarks. The elements of the array Marks can be accessed with the statement,

```
*Studentmarks +++ = 10;
```

The `*Studentmarks++` increments the pointer first and then accesses the contents pointer to by the pointer. It is equivalent to `*(Studentmarks++)`.

The prototype of the function Add10 may also be given as:

```
void Add10 (int Studentmarks [ ] )
```

If this prototype is used in the preceding program, the statement,

```
*Studentmarks +++ = 10;
```

in the function Add10 causes the compiler to report an error, since this statement increments Studentmarks. This is because the argument declaration syntax Studentmarks [] specifies Studentmarks as an array and not a pointer variable, hence, its value cannot be changed.

```
for (i = 0; i < Max; i++)
```

```
Studentmarks [i] = 10
```

The prototype affects only the code inside the function. The caller's code is the same, irrespective of the prototypes used. Also note that in either case, modifications to the array in the function affect the array in the caller function.

Note: `*` (indirection pointer) and `++` have the same precedence. The operators of the same precedence are distinguished by their associativity. Since both `*` (indirection operator) and `++` have right associativity, the compiler performs the operations on the right side of the expression and then goes to the left. Hence, `*Studentmarks++` can be interpreted as `*(Studentmarks++)`. That is, the pointer Studentmarks is incremented first and then the indirection operator is applied to the result. The result is the value which Studentmarks had before incrementing, since post—increment is used.

Example

```
/*ptrfunc.c: Pointers as Function Parameters SWAP */
```

```
#include <stdio.h>
```

```
void exchange ( int *x, int *y)
```

```

{ int temp;
temp = *x;
*x = *y;
*y = temp;}

main ( )
{int a, b;
printf ("Program swaps values of the variables 'a' and 'b'\n");
printf ("\nValues of a and b :");
scanf ("%d %d", &a, &b);
printf ("a = %5d b = %5d \n", a, b);
exchange (&a, &b); /* Addresses are passed as arguments */
printf ("\nAfter swapping :\n");
printf ("a = %5d b = %5d \n", a, b);}

```

Run:

Program swaps values of the variables 'a' and 'b'

Values of a and b : 78 196

a = 78 b = 196

After swapping:

a = 196 b = 78

4) Pointers and Two Dimensional Arrays

Two dimensional arrays are arrays with two dimensions, namely, rows and columns. Each elements of the array is accessed using two subscripts, the first subscript indicates the row number and the second indicating the column number. A two dimensional array can be thought of as a collection of a number of one dimensional arrays, each indicating a row. It is stored in memory in the row major form.

For example, the matrix

```

+-   -+
| 2  3  4 |
| 5  6  7 |

```

```
| 0  8  9 |
```

```
+ -      - +
```

is stored in the row major order in memory.

It is possible to access two dimensional array elements using pointers in the same way as in one-dimensional arrays. Each row of the two dimensional array is treated as a one-dimensional array. The name of the array points to the starting address of the array. The expression `arrayname [ i ]` points to the i^{th} row of the array. Therefore, `arrayname [ i ] + j` points to the j^{th} element in the i^{th} row of the array. The subscript `j` actually acts as an offset to the base address of the i^{th} row. The following example illustrates these concepts.

```
main ( )
{static int arrt [ ] [ ] = {{1,2}, {3,4}, {5,6}};
int i, j;
for (i = 0; i < 3; i++)
for (j = 0; j < 2; j++)
printf ("%d %d %d\n", *(arr + i) + j, arr [i] [j], (*(arr+i) +j)); }
```

In the above example, a two dimensional array of 3 rows and 2 columns is declared and initialized. Then, within the nested for loop, the address of the j^{th} element in the i^{th} row of the array and its contents are printed. Let the two-dimensional array be stored in the memory.

5) Pointers and Three Dimensional Arrays

A three dimensional array can be thought of as an array of two dimensional arrays. Each element of a three, dimensional array is accessed using three subscripts, one for each dimension.

As usual, the array name points to the base of the three- dimensional array. The array name with a single subscript `i`, contains the base address of the i^{th} two dimensional array. Hence, `arrayname [ i ]` is the address of the j^{th} row of the i^{th} dimensional array. Similarly, the expression `arrayname [ i ] [ j ]` is the k^{th} element in the j^{th} row of the i^{th} two-dimensional array. The following example illustrates these concepts. Note that expression `*(*(arr + i) +j) +k` is the same as `arr [i] [j] [k]`.


```

main ( )
{static int arr [2] [3] [2] ={{{2,1},{3,6},{5,3}},{{0,9},{2,3},{5,8}}};
int i, j, k;
printf ("%d\n", arr);
printf ("%d\n", *arr);
printf ("%d\n", **arr),
printf ("%d\n", ***arr);
printf ("%d\n", arr+1);
printf ("%d\n", *arr+1);
printf ("%d\n", **arr+1);
printf ("%d\n", ***arr+1);
for (i=0; i<3; i++)
{for (j=0; j<3; j++)
(for (k=0; k<2; k++)
printf ("%d ", *((*(arr+i) +j) +k) );
printf("\n") ;}
printf("\n") ; } }

```

The array `arr` will be stored in memory as shown in Fig 7.14. In the above program, the array name `arr` is the base address of the three-dimensional array. The expr `*arr` is the base address of the 0th two dimensional array, `**arr` is the 0th row of the 0th two dimensional array and `***arr` contains the value stored in the 0th column and the 0th row of the two dimensional array. The expressions `arr+1` is the base address of the 1st two dimensional array, `*arr+1` is the address of the 1st row in the 0th two dimensional array, `**arr+1` gives the address of the 1st element in the 0th row and the 0th column of the zeroth dimensional array. The expression `***arr+1` adds 1 to the value obtained from the 0th element in the 0th row of the 0th 2 dimensional array (which is 2), giving 3.

The expression within the for loop prints the Contents of the three-dimensional array in the order in which they are stored in memory. The output of the above program is as follows:

100
 100
 100
 2
 10c
 104
 102
 3
 2 1
 3 6
 5 3
 0 9
 2 3
 5 8

6) Array of Pointers

An array of pointers is similar to an array of any predefined data type. As a pointer variable always contains an address, array of pointers is a collection of addresses. These can be addresses of ordinary isolated variables or of array elements.

The elements of an array of pointers are stored in the memory just like the elements of any other kind of array. All rules that apply to other arrays also apply to the array of pointers. This is illustrated in the following example:

```

main ( )
{int arr[3] = {1,2,3};
int i, * ptr [3]
for (i=0; i<3; i++) /* Assigning addresses of array elements */
ptr [i] = arr+i;
for (i=0; i<3; i++)
printf ("%p %d\n", ptr [i], *ptr [i] );}
```

The variable ptr is an array of three pointers. The array of pointers and the integer array will be stored in the memory as shown in Fig 15.15. In the above program, the first loop prints the contents of memory locations whose addresses are stored in the array ptr. array of pointers is widely used for storing several strings in memory. The output of the above program is:

f800 1

f802 2

f804 3

Example

```
#include <stdio.h>

void main ( )

{int arr [5] = {1, 2, 3, 4, 5 }, i, *p;
  for (p = arr + 4, i = 0; i < 5; i ++)
    printf ("%i ", p[-i] ); }
```

In the above program, the integer pointer p is initially assigned the address of the 5th element of the array. The variable i is initialized to 0. The expression p is the same as the expression *(p-i). Hence, when i is 0, p [-i] refers to the 5th element; when i is 1, p [-i] = * (p-1) refers to the 4th element, and so on. The for loop is executed until all the elements of the array are printed.

Output of the program;

5 4 3 2 1

Example

```
#include <stdio .h>

void main ( )

{int * p, arr [5] = {-1, -2, . 4, -5 };
  for (p = arr + 4; p>= arr; p—)
    printf ("%i %i\n", arr [p - arr], *p );}
```

Let the elements of the array be stored in the memory

In the following discussion, assume that an int occupies 2 bytes in memory. The for loop in the above program initializes the pointer p to the address of arr (the last element of arr 0). The address contained in p is now 1008. The name of the array arr, denotes the base address of the array, which is 1000. Even though p-arr is numerically equal to 8, it evaluates to 4. This is because pointer subtraction divides the numeric difference by the size of the data type. Hence, initially, arr[p-arr] is equivalent to arr [4]. In the second iteration of the for loop, p is decremented. Its value becomes 1006. The expression p-arr is now 3, and hence arr is output. Of the three access methods in Example 12.9 & Example 12.10 above (p[-i], arr and *p, *p is the most efficient.

The output of example 12.10 is given below:

```
-5  -5
-4  -4
-3  -3
-2  -2
1   -1
```

7) Pointer Constants

As mentioned earlier, the name of an array holds the starting address of the array. Hence, if arr [3] is an array of any data type, then the name of the array arr is the address of the 0th element of the array, and arr+i is the address of the ith element of the array. If arr is a pointer, then arr+i can be replaced by incrementing arr, i times by using the increment operator with it (the name of the array) is incorrect as the starting address of the array has been placed in the code directly by the compiler, thus making the array name a constant. The array name does not have any storage location allocated unlike a pointer variable which itself has a storage location. Hence, performing an increment operation on the address of the array (which is a constant) is like performing the increment operation 5++, which is meaningless. Though, it is not possible to use the increment operation on an array variable, we can assign this address to a pointer variable and then increment the pointer variable. The following example illustrates these concepts.

```

#include <stdio.h>

main ( )
{static int ia [3] = {2,5,9};
  int i, *ptr;

  ptr = ia;
  for (i=0; i<3; i++)
  {/* printf ("%d",*(ia++)); is incorrect */
   printf ("%d",*ptr++ );}}

```

In the above program, the elements of the array are accessed using the pointer ptr which is assigned the starting address of the array ia. The pointer variable ptr is incremented every time to point to the next element. The expression ia++ is incorrect.

More on Incrementing Pointer Variables

When a pointer variable is incremented its value actually gets incremented by the size of the type being pointed to. For example, let pi be a pointer to an integer declared with statement:

```
int* pi;
```

Also, let pi point to the memory location 1020, i.e., number 1020 is stored in the pointer p1. Now, a statement increments p1, such as,

```
pi ++;
```

will add 2 to p1, making it 1022 (assuming that the size integer is 2 bytes). This makes p1 point to the next integer. Similarly, the statement,

```
pi - - ;
```

will decrement the value of pi by 2.

When a pointer variable is declared, a memory location the pointer is allocated, but will not be initialized. Before the pointer variable, it should be assigned a valid address. If this has

to be done in a function other than where pointer declared, then its address must be passed to the function. In function, the pointer variable is modified and not the location points to. In C, parameters are passed by value, hence, address of the pointer must be passed.

Example

```
/* big.c: program to find the largest number using pointers */
#include <stdio.h>

void FindBig (int *a, int *b, int **big)
{
    *big = a;
    /* assign a to the contents of big, i.e., the address of one*/
    /* the two variables (a itself is the address) is assigned to the pointer*/
    /* variable big and compare the contents of the two variables */
    if ( *b > *a)
        *big = b;}

void main ( )
{
    int a, b, *big;
    printf ("Enter two integers: ");
    scanf ("%i %i" &a, &b);
    FindBig (&a, &b, &big);
    printf ("Variable name address of the variable ; value:\n");
    printf (" a      %d    = %i\n",      &a, a);
    printf (" b      %d    = %i\n",      &b, b);
    printf ("The bigger of the two integers.\n")
    printf (" big      %d    =%p\n",      &big, big);
    printf ("The value as obtained from the pointer: %i\n", *big);}

```

Here, big is pointer to an integer. Here, the function FindBig will assign the address of the larger integer to big. M the pointer variable's content to be changed (and not the value where the pointer is pointing to), the address of big is passed to the function. *big is the integer value

pointed to by the pointer, big is the value of the pointer variable and &big is the location where this poi variable is stored.

8) Pointers and Strings

A string is an array of characters stored in consecutive memory locations. Strings are used to store text information and to perform manipulations on them. A string must be terminated with a null character denoted by '\0' indicating the end of string. Each character in the string occupies one byte of memory.

Strings can be initialized like any other array. For example, static char message [] = { 'h', 'e', 'l', 'l', 'o', '\0'};

They can also be initialized by specifying the set of characters within double quotes. For example,

```
static message [ ] = 'hello';
```

In this case, the null character '\0' is inserted by the compiler. The set of characters to the right of = is termed as string literal.

Since strings are character arrays pointers to strings are treated in a way similar to pointers to arrays of other data types. The name of the character array (string) points to base address of the string (i.e., the first character of the string). The expression (arrayname+i) points to the ith character in string. The expressions (arrayname), &arrayname and &0 [arrayname] are equivalent and pointer to the first character in the string. The following example illustrates the above concepts.

```
/* string. c: program to illustrate pointer to strings */
#include <stdio.h>

main ( )
{char city [ ] = "Chennai";
int i = 0;
printf ( "AddressContents\n" );
```

```

while (*(city+i) != '\0') /*check for end of string */
{printf ("%d %c %c %c\n", (city+i), city [i] , *(city+i), i [city]);
  i++;}}

```

The output of the above program is as follows:

Address Contents

```

801  B B B
802  a a a
803  n n n
804  g g g
805  a a a
806  1 1 1
807  0 0 0
808  r r r
809  e e e

```

In the example string.c, a string is declared and initialized. The addresses and the contents of the memory locations occupied by the characters are printed within the while loop. The expressions `city [i]`, `i [city]` and `*(city+i)` contain the *i*th character of the string. The while loop is used to check for the end of the string.

9) Pointers in Standard String Library Functions

Pointers are widely used in handling some of the standard string library functions include `strlen ( )`, `strcat ( )`, `strcpy ( )` and `strcmp ( )`. A pointer to the string is passed to these function instead of the entire string. In fact, the address of the first of string is passed. Since the string is stored in an array, it is enough to specify the name of the array while passing to these functions. If requires, the notation `&stringVariable[0]` can be used to specify the address. A pointer thus defined can be assigned the address of an existing string, or allocated memory space, and assigned the pointer returned by the allocation. For example:

```

/*charptr.c: program to illustrate character pointers */
#include <stdio.h>

```



```

void main (void)
{char chArr [ ] = "Pointer and Strings";
char *chPtr;
chptr = chArr; /* equivalent to chPtr = &chArr [0] */
printf ("contents pointed by the pointer chptr is: %d\n", chPtr);

```

In the above program, a string constant is defined. The address of the string is assigned to the pointer chPtr. chArr points to an array of 20 characters. It can be regarded as a constant pointer, the address which it points to, cannot be changed. However, the contents of the memory location to which it points to can be changed. A frequently used operation is to copy a string from one location to another. Suppose a string with length more than 20 is copied to the character array chArr. It would overwrite memory locations which do not belong to chArr, thus resulting in abnormal behaviour of the program. In simple terms, char points to a memory block of 20 characters. When chPtr is assigned the contents of chArr, the same discussion holds good. Suppose the number of bytes to be transferred is not known at compile time. The options available are:

Declare an array to hold the largest string possible. This may result in under utilization of memory. Allocate the memory dynamically (when the program executes).

strlen () Function

This function returns the length of a given string. It takes a single argument, namely, the pointer to the base address of the string. The length of the string *excludes the null* character. The following example illustrates the use of this function and a user defined function to find the length of the string. The user-defined is given only to illustrate how strlen may be implemented.

```

#include <stdio.h>
#include <string.h>
int usrstrlen (char *); /* function prototype */
main ( )
{int len;

```

```

static char s1 [ ] = "Good";

printf ("Length of string using standard library function \n");
printf ("%s\n", strlen(s1));

len = usrstrlen(s1); /* user defined function to find length */
Printf ("Length of string using user defined function %s\n", len);}

int usrstrlen (char *ptr)
while (*ptr != '\0')
{int 1n = 0;
while (*ptr != '\0')
{1n + = 1;
ptr++;}
return 1n;}

```

The output of the above program is as shown below:

Length of string using standard library function 4

Length of String using user defined function 4

Example

```

/* ptrstr1.c Pointers and Strings */
#include <stdio.h>

main ( )
{char string [80], * ptr;

ptr = string;

printf ("Enter the string to find it's length :");

/* reads in string */
while ((*ptr ++ = getchar ( ) != '\n');

*-ptr = '\0';

printf ("\nstring is          : %s\n", string);

printf ("\nIt's length is    : %d\n", ptr-string);}

```

Run:

Enter the string to find it's length: Vidya Mandir

String is : Vidya Mandir

It's length is : 12

strcpy () Function

This function copies the contents of one string to another. It takes two arguments, the first being the pointer to the beginning of the destination string and the second is a pointer to the first element of the source string. The source string is copied into the destination string. The following example illustrates the use of this function and a user defined function to copy a string.

```
#include <stdio.h>

int usrstrcpy(char 'i', char *); /* function prototype */

main ( )
{int len;
 static char s1 [ ] = "Good";
 char *s2;

 strcpy (s2,s1); /* call to standard library function to copy */
 printf ("Copied string using standard library function to copy */
 usrstrcpy (s2,s1); /* user defined function to find length */
 printf ("Copied string using user defined function is %s\n, s2);}

void usrstrcpy (char *p, char*q)
{while (*q != '\0')
{*p = *q;
 p++;
 q+ +; }
 * p = '0';}
```

The-function usrstrcpy can be reduced by using

```
while(*p++ = * q++);
```

The expression `*p++` will evaluate `p++`. The `++` operator is evaluated first, since unary operators associate right to left. The pointer is now dereferenced. The pointer `p`, is then incremented. Similar operations are done on `q`. The character pointed to by `q` is assigned to that pointed to by `p`. The while loop continues till the NULL pointer is encountered. The output of the above program is:

Copied string using standard library function is Good

Copied string using user defined function is Good

strcat () Function

This function concatenates two strings resulting in a single string. It takes two arguments, namely, the pointers to the strings. The resultant string is stored in the first string specified in the argument list. The following example illustrates the use of this function and a user defined function to concatenate two strings.

```
#include <studio .h>

void usrstrcat (char *, char *) /* function prototype */

main ( )

(static char s1 [ ] = "Good";
static char s2 [ ] = "Morning";
strcat (s1 ,s2); /*call to standard library function */
printf ("Concatenation using standard library function is");
printf("%s\n" s1);
usrstrcat (s2,s1); /* call to user defined fun to find length
printf ("concatenation using user defined function is %s\n", s1);}

void usrstrcat(char *p, char*q)

(while (*p != '\0')

p++;

while (*p != '\0')

{ *p = *q;
```

```

p++;
q++ ; }
* p = '0';}

```

The output of the above program is as follows:

Concatenation using user defined function is Good Morning

Concatenation using user defined function is Good Morning

strcmp () Function

This function compares two strings, character by character, and returns a value 0 if both the strings are equal, and a value 1 if the strings are different. `strcmp ( )` takes two arguments of the type pointer to character strings. The following example illustrates the use of this function and a user defined function to compare two strings.

```

#include <stdio.h>

int usrstrcmp(char *, char *); /* function prototype */

main ( )
{
    int cmp;
    static char s1 [ ] = "Good";
    static char s2 [ ] = "Morning";
    printf ("Comparison of strings using library function is" );
    printf ("%s\n", strcmp (s1 ,s2));
    cmp = usrstrcmp(s2,s1), /* user defined function to compare */
    printf ("Comparison of strings using user defined function:");
    printf ("%s\n", cmp);}

int usrstrcmp (char *p1, char *q1)
{
    int k=0;
    char*p, *q;
    for (p = p1, q = q1; ((*p == *q) && ((*p != '\0') && (*q!= '\0')));
    p++, q++);
}

```

```

if (( * p == '0') && (*q == '0'))
k=0;
else
k=1;
return k; }

```

The output of the above program is as follows:

Comparison of strings using library function is 1

Comparison of strings using user defined function: 1

10) Two Dimensional Array Characters

The two dimensional array of characters is an array of strings. The elements of the array of strings are accessed in, a manner similar to two dimensional arrays of other data types. The name of the array points to the base address, i.e., the first character of the string. Two subscripts are used to access the characters of the string. The expressions `*(arrayname+i)` and `arrayname[i]` point to the first character of the *i*th string and the expression `*(arrayname+i)+j` points to the *j*th character of the *i*th string in the array. The following example illustrates the above concepts.

```

#include <stdio.h>

main ( )
{char pet [2] [5] [ ={"Lion", "Cat"}
int i, j;
for (i=0; i<2; i++)
{printf( "\n%d", (pet+i) +j );
j=0;
while (*(pet+i)+j) != '\0')
{printf ( "%c", *(pet +i +j) );
j++;}}}

```

In this case, irrespective of the size of the strings, 5 bytes memory are allocated for each string. Hence, there is a wastage of memory that cannot be used for any other purpose. The output of the above program is:

801 Lion

806 Cat

11) Array of Pointers to Strings

An array of pointers to strings is an array whose elements are pointers to the base addresses of the string. An array pointers to strings is declared and initialized in the following manner:

```
static char**pets [ ] = {"Lion", "Cat"};
```

Here, pets [0] is a pointer to the base address of the string Lion, pets [1] is a pointer to the base address of the string cat and so on.

Advantages

Unlike in the two dimensional array of characters (array of strings), the strings do not have a fixed memory size for storage. The strings occupy only as many bytes as required, hence, there is no wastage of space. The strings are stored in memory in the manner.

Note that the strings need not be stored in consecutive memory locations. The individual characters within the string are accessed as usual using the indirection operator. The expression pets [i] +j points to the jth character of the string.

Example

```
main ( )
{static char st [ ] ="Hello";
int i;
for (i=0; st [i]; i++)
printf ("%c %c %c\n", st [i], *(st+i), i [st], *(i+st));}
```

This example shows the fact that the expressions St [i], i [st], *(st+i) and * (i+st) are all same and represent the constants of the ith element of the string. Each element of the string is printed by the for loop. The presence of the expression st in the for loop condition checks for the end of the string. The output of the program is as follows:

```

H H H H
e  e  e  e
1  1  1  1
1  1  1  1
0  0  0  0

```

Example

```

main ( )
{static char st [ ] = " Good Morning";
printf ("%s", st+3);}

```

In the above program, the expression `st+3` points to the element of the string `st` by using `%s` in the `printf` statement, entire string starting from the 3rd element gets printed (zeroth element is the character `G`). The output of the above program is:

```

d Morning

```

Example

```

main ( )

{static char *s[ ] = {"Read", "Blue", "Yellow"};
static char **ptr [ ] = {s+2, s+1, s};
char *** p = ptr;
printf ("%s", **++ p + 4);
printf ("%s", *—*+ + p + 4);
printf("%s\n", *p [-2] +9);}

```

In this example, `s` is declared an array of pointers whose elements are the base addresses of the three strings. Another array of pointers, `ptr` is declared and assigned the address of the elements of the array `s`. The pointer `p` is then assigned the base address of the array `ptr`.

The argument passed to the first printf statement is `**++p`. This expression has two indirection (`*`) operators and a `++` operator. The `++` operator evaluates first, and increments `p` to 502. The second `*` is evaluated now, and it gets the value stored in location 502, which is 802. Finally, the first `*` fetches the value in location 802, which is 104. The first printf statement prints Blue. The argument passed to the second printf statement is “— Keeping in mind that the value of `p` is now 502, and that unary operators evaluate right to left, the expression `*—*++p` evaluates first, giving 104. The `+` operator now adds 4 to this, yielding 108. Since the location 108 contains the null character, nothing is printed by the second printf statement. The next expression `*p[-2]+9` can be written as, `*(p-2)+9`. The current value stored in `p` is 504 and therefore, `(p-2)` results in 500 and the data contained is 800. A value of 9 is added to the address stored in 800 resulting in 109. The last printf statement prints the string at address 109.

The output of the above program is:

Blue

Yellow

12) String Library Functions

Few string library functions have been discussed in the chapter on arrays and strings. The rest of the string library functions (that use pointers) are discussed below.

13.13.1 strchr

The function `strchr` searches for a character in a string. It accepts two parameters. The first one is a string, while the second one is a character. The function searches for the specified character in the string passed and returns the memory address of the first matching symbol character found in the string. For example, consider the following program.

```
/*strchr.c: strchr example */

#include <stdio.h>

#include<string.h>

void main ( )

{ /*Declare and initialize the string.*/
```

```

char string [ 30 ]; char searchChar;
char* found;
/*found is a variable that can hold the address character*/
/*Note the asterisk (*) after char*/
/*Input the string.*/
printf ( "Input a string: " );
scanf ( "%s", string );
/*Input the, character to be searched.*/
printf ( "Input a character: " );
scanf ( "%c", &searchChar);
/*Search for the character 's' in string.*/
found = strchr (string, searchChar);
printf ( "%c was found in position %d\n", searchChar, found - string );}

```

A sample run of the program is given below:

Input a string: House

Input a character: s

s was found in position 3

In the above program, a variable called found is declared with the statement,

```
char* found;
```

The statement,

```
found = strchr (string, searchChar );
```

in the above example assigns the memory address of the letter s in the string House. For example, if the string was stored at the beginning, at the memory location 1000; it can be visualized in memory.

The \0 at the end is the null character, which is a byte containing the value zero. The address of s is 1003, while the address of the beginning of the string is 1000. Subtracting the

two character pointers gives an integer whose value is the difference between them. Hence, the expression:

```
found-string
```

in the printf statement returns the position of s in the string, i.e.,3. Note that though s is the fourth character in the string, its position is computed to 3, since array subscripts in C always begin with a zero.

But what happens if the character is not found in the string? The function returns a NULL pointer, which is a memory address having the value zero. So, if the character t were searched in the string House, the statement,

```
found = strchr ( string, searchChar);
```

would evaluate found to zero. to guard against such cases, printf state occurring after the strchr statement must be replaced by, the, following if-else construct:

```
if (found) /* If the values of found is non-zero...*/
printf ( "%c not found in position %d\n", searchChar, found - string);
else
printf ("%c not found\n", searchChar);
```

strrchr

The function strrchr accepts a string and a character parameter. It finds the last occurrence of a character in a (recall that the function strchr returned the first occurrence the character in the string). The memory address of the occurrence of the character in the string is returned. For example, consider the following code:

```
char str [ ] = "Hello world";
char* found;

/*Again, found can contain the address of a character.*/
found = strrchr ( str, 'o' );
```

```
printf ("The last o is at position %d\n", found - str);
```

The output of the above statement is:

The last o is at position 7

strcspn

The function `strcspn` accepts two strings as parameters. It returns the number of characters at the beginning of the first string that do not occur in the second string. Consider the following example:

```
/*strcspn.c: an example to illustrate strcspn /
#include <stdio.h>
#include <string.h>
int main (void)
{char *string1 = "0123456789";
char *string2 = "A47DCF8";
int length;
length = strcspn ( string1, string2 );
printf ( "Number of characters at the beginning of string\n"
"which are not in string2=%d\n", length);
return 0;}
```

The output of the program is:

Number of characters at the beginning of string 1

which are not in string 2=4

The first 4 characters of string 1, which is the first string passed to `strcspn` are 0,1,2 and 3. Notice that none of them are present in string 2.

strerror

The function `strerror` present in `string.h` has nothing to do with strings in general. It accepts an error code which is an integer, and returns an error message corresponding to that integer.

The file functions set a global integer called `errno` to a certain number depending on the error that has occurred. For example, you may be trying to access a file that does not exist. The possible sequence of statements that would retrieve the error message corresponding to the most recent error are:

```
char* msg;
msg = strerror (errno);
```

strpbrk

The function `strpbrk` accepts two strings as parameters. It searches the first string for any character occurring in the second string. If such a character is found, it returns the address of that character of the first string. Otherwise, an address of that character of the first string. Otherwise, an address having the value zero is returned.

A sample program is given below.

```
/*strpbrk.c: an example to illustrate the function strbrk */
#include <stdio.h>

printf ( "Character not found\n" );}
```

The above program inputs two strings `str1` and `str2`. It prints the first character of `str1` that occurs in `str2`. Notice the `printf` statement which prints this character. It has the expression `*found`.

The expression `*found` represents the character that occurs in a memory location Contained in `found`. For example, if the memory address contained in `found` is 1000 and the character occurs at this memory locations, then `*found` will correspond the character `a`.

strstr

The function `strstr` searches one string for the occurrence another. It accepts two strings as parameters and searches first string for an occurrence of the second. The function returns the memory address of the character in the first string, starting at which the first occurrence of the second string is found. Consider the following sample program. The declaration of the variable `found` and the computation of the character position where the match occurred are the same as in the `strchr` function defined before.

```

/* strstr.c: program to search for a string /
#include <stdio.h>
#include <string.h>
#include <string.h>
void main ( )
{char str1 [30], str2 [30];
char* found;
printf ( "Enter two strings:\n" );
gets( str1 );
gets (str2 );
/* Search for str2 in str1 */
found = strstr (str1, str2 );
printf ("string2 is found at position %d in string1\n",found-str1) ;}

```

A sample run of the above program is given here:

Enter two strings:

I saw the cat click the mouse.

the

string2 is found at position 6 in string1

Here, the first string passed to `strstr` is, I saw the cat click the mouse. The second string is the. The function searches the first string for the given second string. Though there are two instances of the in the first string, only the first one is considered, and a pointer to the beginning of the match is returned.

strtok

This function accepts two strings as parameters and tokenizes the first string. Tokenizing is simply a process of splitting up string into parts that are separated by certain characters. Here, the separating characters are given to `strtok` in the second string.

For example, if the separating character (also called the delimiting character) is a semicolon(;), then in the string abc;de;fghi, the tokens present are abc, de and fghi. In other words, they are strings within the string that are separated from each other by the delimiting character, which in this case is semicolon.

Now let us come to the part of using strtok for splitting into tokens. For the first time, call strtok with the input string as first parameter and a separator string of characters which you want to function as separators, as the second parameter. The strtok function returns the first token (a string inside a string passed as the first parameter separated by one of the characters in the second string).

The following example will make things clear.

```
/*strtok.c: program to illustrate stroke */

#include <string.h>

#include <stdio.h>

int main (void)
{ char str [30];
  char *token;

  printf ("Input a string, parts separated by, or ;\n");
  gets (str);

  printf ( "The tokens found were:\n");

  /* strtok places a NULL terminator in front of the token, if found */
  token = strtok( str, ",;");

  if ( token)

    printf ( "%s\n", token);

  /* A second call to strtok using a NULL as the first parameter returns a pointer to the
  character following the token */
```

```

while( token)
{token strtok ( NULL, "; " )
if ( token)
printf ( "%s\n", token );}
return 0;}

```

A sample run of the program is given below:

Input a string, parts separated by, or;
;abc, defg;;hi,jklmn;op

The tokens found were:

```

abc
defg
hi
klmn
op

```

strncat, strncpy and strncmp Functions

Functions such as strcat, strcpy and strncat, do not check the size of the target string. For example, the function strcat appends the contents of the second string passed to the first. If the first string does not have enough space to hold concatenated string, the results are unpredictable. In large programs, it may be very difficult to find the exact location this type of bug. For that purpose, there are string functions, strncpy and strncnpy, which consider the maximum length of the string they operate on.

strncat

Similar to strcat, this function appends one string to another. It accepts three parameters; the first two are string, while last one is an integer specifying the maximum length of the second string to be appended at the end of the first.

strncpy

Similar to strcpy, this function copies one string to another. It accepts three parameters; the first two are strings, while last one is an integer specifying the maximum length of the source string (second parameter).

strncmp

Similar to strcmp, this function compares two strings. It accepts three parameters; the first two are strings, while the last one is an integer specifying the maximum length to compare. The return values are same as the function strcmp.

13) Pointers to functions

Functions have addresses just like data items. A function's address is the starting address of the machine language code of the function stored in the memory. For example, it is possible to write a function that accepts the address of another function as an argument, and executes it. This approach may look unnecessary as the first function can directly call the second function without obtaining its address, it is possible to pass addresses of different functions at different times thus making the function more flexible and abstract.

Address of a Function

The address of a function can be obtained by only specifying the name of the function without the trailing parentheses. For example, let the function be defined as test(). Now, test is the address of the function. Hence, to pass the address of a function, it is sufficient to pass the name of the function.

The declaration of a pointer to a function requires the function's return type and the function's argument list to be specified along with the pointer variable. The general syntax for declaring a pointer to a function is as follows.

```
return type (*pointer variable) (function's argument list);
```

Example

```
int test(double); /* function prototype */
```

```
m test! (double); /* function prototype */
```

```
m (*ptr) (double); /* pointer to function with prototype of test */
```

Here, ptr is a pointer variable pointing to the function that returns an integer value, and takes a data item of type double as its argument. The pointer ptr can point either to the function test or test1. The expression *ptr replaces the function name in the declaration.

The addresses of the functions can be assigned to pointer variables, just like assigning addresses of variables and other data types to the corresponding pointer variables. In pointer to function assignments, the complete function type must match exactly the pointer type and argument list.

The following example illustrates the concepts mentioned.

```
#include <stdio.h>

void func1(int i, float f)
{printf ("%i %f\n", i, f);}

int func2 (char *.s)
{printf ("%s\n", s);}

void main ( )
{void (*p) (int, float); /* declaring pointers to functions */
int (*q) (char *), i = 5;
float f = 5.375;
char s [ ] = "String";
p = func1; /* assigning address of function fund to p */
q = func2; /* assigning address of function func2 to q */
q = func1; /* incorrect assignment, error */
p(i, f); /* Calling the functions using the pointers. */}
```

In the above program, after declaring the function prototypes and two pointers p and q to the functions; p is assigned the address of function fund and q is assigned the address of function func2. These two assignments are perfectly valid, as the complete function types match exactly with respect to their pointer type and argument list. In the next assignment, the address of the function fund is assigned to the pointer q, which is invalid as the function's

return type and pointer to function's return type do not match. Such as assignment will result in a compiler error.

Invoking a function by using pointers

In the pointer declaration to functions, the pointer variable along with the symbol * plays the role of the function name. Hence, while invoking functions by using pointers, the function name is replaced by the pointer variable. The following example illustrates this concept.

```
#include <stdio.h>

unsigned int fact (int m)
{
    unsigned int i, ans;
    for (i = m, ans = 1; i > 1; ans * = i-);
    return ans;}

void main ( )
{
    unsigned int (*ptr) (int);
    /* pointer to function with prototype of fact */
    unsigned int n, ft;
    ptr = fact; /* assigning address of function fact to ptr */
    printf ("Enter the integer- whose factorial is to be found: ");
    scanf ("%u", &n);
    ft = ptr(n); /* call to function fact using pointer ptr */
    /* ft = (*ptr) (n) is the same as ft = ptr(n) */
    printf ("\nFactorial of %u is %u.\n", n, ft);}
```

Run:

Enter the integer whose factorial is to be found : 8

—

Factorial of 8 is 40320.

In the above program, a pointer `ptr` is declared to point to a function that takes an integer argument and returns an integer value. Then, the address of the function `fact` is assigned to the pointer `ptr`. The function `fact` computes the factorial of a given positive integer. The function `fact` is invoked using the pointer variable `ptr`.

Passing function address to other functions

The address of function can be passed as an argument to other functions either by a function name or a pointer that holds the address of the function. The following example illustrates these concepts. It inputs a set of integers and outputs the largest and smallest of them.

```
#include <stdio.h>

/* function prototypes */

int large (int *c, int);
int small(int *c, int);

int large (int *b, int m) /* function to find the largest integer */
{int big, i;
  big = b [0];
  for(i = 1; i <m; i++)
    if(b [i] > big)
      big = b [i];
  return big;)

int small (int *b, int m) /* function to find the smallest integer */
{int sml, i;
  sml = b [0];
  for (i = 1; i <m; i++)
    if (b[i] < sml)
      sml = b [i];
  return Sml;)
```

/*The last parameter to the function select is a pointer to a function which accepts a pointer to an int and an int. main calls this function with the last parameter set to either the function largest or smallest */

```
void select (int *b1, int m1, int (*q) (int *d1, int m2) )
{int ans;
ans = q (b1 ,m1); /* call to function with address in q */
printf ("%i", ans) ;}

void main ( )
{int n, i, a [20], (*ptr) (int*d, int); /* declaring a pointer to function */
printf ("Enter the number of integers: ");
scanf ("%i", &n);
printf ("Enter the set of integers:\n");
for (i = 0; i <n; i++)
scanf ("%i", a + i);
ptr = large; /* assigning the address of function large to ptr */
printf ("\nThe largest integer is");
select (a, n, ptr); /* passing function address using pointer */
printf ("\nThe smallest integer is ");
select (a, n, small); /* passing function address directly */}
```

Run:

Enter the number of integers: 5

Enter the set of integers:

5 -7 4 0 1

The largest integer is 5

The smallest integer is – 7

In the above program, two functions `large` and `small` are called to find the largest and smallest integers respectively, from a given set of integers. The function `select` calls one of the two functions `large` or `small` depending on the address of the function passed to it. The address of the function `large` is passed to the function `select` through the pointer pointing to it and the largest integer is printed. The address of the function `small` is passed to the function `select` directly and the smallest integer is printed.

Examples on Pointer Definitions

```
* int (*data) [10]
```

`data`: It is a pointer to an array of size [10] of integers,

```
* int *data [10]
```

`data`: It is an array of size 10 of pointers to integers.

The difference between the above two declarations is as follows:

The first declaration `int (*data) [10]`, implies that there is only one pointer and it is a pointer to an array of 10 integers. This pointer is not initialized. Here, if `data++` is used, `data` will point beyond 10 integers, i.e., `10 * sizeof (int)` will be added to `data`.

The second declaration `int *data [10]`, implies that there are ten pointers and each of these is a pointer to integer and none of the ten pointers are initialized. Here, `data++` is invalid as `data` is an array. If `data` is used then `data` will point to the next integer in the 0th row, i.e., `10 * sizeof (int)` will be added to `data [0]`.

Several levels of indirection can be achieved using pointers. Some valid examples are given below:

```
* int ** po2po2i
```

`po2po2i` is a pointer to a pointer to integers

```
* float *power ( )
```

`power` is function returning a pointer to float.

```
* float (power) ( )
```

power is a pointer to a function returning float,

```
* float * (*power) ( )
```

power is pointer to a function returning pointer to float.

```
* long*(*fp0 [5] (int *))
```

```
* long*(*(*fp1 [5] (float*)) [10])
```

```
* long (* (*pf [12] ) ( ) [7] )
```

pf is an array [12] of pointers to a function each returning a pointer to array [7] of type long.

14) Pointers to Constant Objects

Consider the following declaration:

```
const int* pi;
```

The statement declares pi a pointer to a constant integer.

Let pi be initialized by the statements,

```
int i [20];
```

```
pi = i;
```

That is, *pi refers to the integer i [0]. Due to the declaration of pi (which, as mentioned above, is const int* pi;), statement, such as

```
*pi = 10; or even pi [10] = 20;
```

are not valid. Such statements result in compile time errors. But pi itself can be changed. Statements, such as

```
pi++.
```

are perfectly valid. These types of pointers are useful in character pointers, when pointers are to be passed to a function for printing. For example, suppose we are writing a function to print a character string. It is a good practice to code it as follows:

```
void print (const char* str)

{printf( str );}
```

The function accepts a pointer of type `const char*` (pointer to constant character). The string being pointed to cannot be changed. This is a safety measure, since it prevents accidental modification of the string passed to the function. In the function, the pointer `str` itself can be changed, for example, by the statement,

```
str++;
```

But this does not affect the calling procedure, since the pointer itself is passed by value.

15) Constant Pointers

The declaration

```
int const pi = i;
```

declares a constant pointer to an integer (assume that `i` is integer array). In this case, statements such as

```
*pi = 10;
```

are perfectly valid, but others which modify the pointer, such as

```
pi++;
```

are invalid, are result in compile time errors.

A pointer declaration, such as `const int* const pi = i;` will disallow any modification to `pi` or the integer which `pi` is pointing to (assume that `i` is an integer array).

Example

Arrays are never passed by value to functions in C. Only a pointer to an array is passed. To pass a copy of an array to the function, the array must be copied explicitly and the copy must be sent.

The program below has function `print_array` whose prototype is:

```
void print_array ( int a [ ], int n );
```


The first parameter is a pointer to an integer array. This is specified by having a pair of square braces after the name of the parameter `a`. These braces make `a` act as a constant pointer to an integer inside the function. (i.e., `a` cannot be modified, but the integers pointed to by `a` can be modified in the function). Observe that the size of the array need not be given in the square braces. The second parameter is the size of the array. Note that any change made to the array in the function will be reflected in the array passed by the calling function, since both refer to the same memory.

arrayfun.c: Program illustrating the passing of arrays to functions. */

```
#include <stdio.h>

#define MAX 20

/* Function to print the elements of the array */
void print_array ( int a [ ], int n)
{ int i;
  printf ( "The elements of the array are:\n" );
  for (i=0; i<n; i++)
    printf ( "%d\n", a [i] ); }

void main ( )
{ int a [ MAX];
  int i, n;
  printf ( "Enter the number of elements:" );
  scanf ( "%d", &n);
  printf ( "Enter the elements:\n");
  for ( i = 0; i < n; i++ )
    scanf ( "%d", a + i);
  if ( < 0 || n > MAX)
    printf ( "Invalid number of elements.\n");
  else
```

```
print_array (a, n ) ; }
```

Run:

Enter the number of elements: 5

Enter the elements:

1

2

3

4

5

The elements of the array are:

1

2

3

4

5

An alternate prototype for print_array is as follows:

```
void print_array( int *a, int n)
```

The old function body would still work.

Here, a is a variable integer pointer. An alternate, way of writing the function print_array is as follows

```
void print_array( int *a, int n)
```

```
{int i;
```

```
printf ("The elements of the array are:\n" );
```

```
for(i=0; i<n; i++)
```

```
{printf ("%d\n", *a);
```

```
/* Make a point to the next integer.*/
```

```
a++;}}
```

The program is executed in the same manner as before. This version is more efficient than the former one, since it causes the final address to be computed every time using `a` and `i`. A few good compilers recognize that the former and latter versions are the same and generate the same code for both. Note that in this case also, any change made to the array in the function will be reflected in the array passed by the calling function, since both refer to the same memory location. Since in our case, `print_array` function does not change the contents of the array, it is better to specify the prototype as

```
void print_array (const int [a], int n)
```

or

```
void print_array (const int *a, int n )
```

Example

The following program accepts a string and converts all its characters to upper or lower case. It uses the functions `toupper` and `tolower` defined in `ctype.h`.

```
/*lowup.c: upper case to lower case conversion and vice-versa */
#include <stdio.h>
#include <ctype.h>

/* function converts all characters in the string instr into uppercase. */
void upper( char instr [ ] )
{
    int i = 0;
    while ( instr[i] )
    {
        instr[i] = toupper ( instr[i] );
        i++;
    }
}

/*function converts all characters in the string instr into lowercase */
void lower(char instr [ ] )
{
    int i = 0;
```

```

while ( instr [i] )
{ instr [ i ] = tolower ( instr [ i ] );
i++;}}
void main ( )
{char instr [ 80], inchar;
printf ( "Input the string: " );
gets ( instr);
printf ( "To upper or lower case (u/l)?");
scanf ( "%c", &inchar);
switch ( inchar)
{case 'u':
case 'U':
upper ( instr);
break;
case 'l':
case 'L':
lower ( instr);
break;
default:
printf ( "Invalid input!\n" );}
printf( "The converted string is.. \n");
printf( "%s\n", instr );}

```

Run1:

Input the string: Chennai

To upper or lower case (u/l)? u

The converted string is...

CHENNAI

Run2:

Input the string: IDE, CHENNAI

To upper or lower case (u/l)? l

The converted string is...

ide, Chennai.

Check Your Progress – I

- 1) *State whether the following are True or False.*
 - a) Array elements are always stored in continuous memory locations.
 - b) The two dimensional array are array with three dimensions.
- 2) *Fill in the blanks*
 - a) _____ function returns the length of a given string
 - b) _____ function searches for a character in a strings
 - c) _____ function accepts two strings as parameters

13.3 Check Your Answers

- I.
 - 1) a) True b) False
 - 2) a) Strlen () b) Strchr
 - c) Strspn ()

13.4 Summary

In this lesson we have covered the advanced concepts of pointers. And also we have covered pointers and arrays, array of pointers and pointers to functions. In this lesson we have prepared some example programs.

13.6 Model Questions

- (1) How can a one-dimensional array of pointers be used to represent a collection of strings?
- (2) Consider a one-dimensional array of character pointers pointing to unsorted strings. If the strings need to be sorted, what is the most efficient technique that can be employed?

What are the alternatives?

- (3) Write a function that accepts a string as argument and returns 1 if it is a palindrome and 0 if it is not.
- (4) What is the difference between the following two declarations of a function fn?
 - (i) `void fn (int a[ ] );`
 - (ii) `void fn( int *a );`
- (5) Illustrate the main purpose of passing variables by pointer, with the help of a function which swaps two integers.
- (6) Distinguish between the following declarations:
 - (i) `const int *pi;`
 - (ii) `int *const pi;`

Which of them is a constant pointer and which one is a pointer to a constant object?

LESSON - 14

FILE HANDLING

Structure

14.1 Introduction

14.2 Learning Objectives

14.3 File Handling

14.4 Summary

14.5 Check Your Answers

14.6 Model Questions

14.1 Introduction

So far we have been writing and reading data to the standard output and from standard input respectively. In the case of small amounts of data an array can be defined to hold these data. But arrays are active only during the program execution and also if huge amounts of data is in process they are not advisable. However, huge amounts of data can be stored in files. To perform Input/Output operations from and to files, we require the functions in the standard I/O package.

File handling falls into four basic operations

open This allows access to a file

close This ends the access to a file

read This gets information from the file

write This adds information to the file or replaces information already in the file

There are two distinct ways to read and write files in C.

14.2 Learning Objective

After learning this lesson, you should be able to

- explain four basic operation of file handling.

14.3. File Handling

The read and write operations are done as a character at a time". This is also called buffered I/O because they provide their own buffer.

Sometimes called UNIX-like. In this read and write operations should be performed manually providing own buffers, counters and pointers.

Let us discuss the aspects of high level I/O.

1) Opening a File - fopen() Function

This function serves two things for us. First, it opens a disk file for use and second returns a file pointer which is useful in performing other operations on the file. The general form of fopen () is

```
file_pointer = fopen (filename ,mode)
```

where filename is user-defined name and mode is one of the following

r- opened for reading. The file should exist. The filepointer is positioned at the beginning of the file.

w - opened for writing. If the file does not exist it is created. If it exists then the contents are lost. The file-pointer is positioned at the beginning of the file.

a - opened for writing. If the file does not exist it is created. Otherwise the file pointer is positioned at the end of the file for appending. The contents of the file are not lost.

r+ opened for update. File is available for both reading and writing and the contents are not lost. The file pointer is positioned at the beginning of the file.

w+ opened for update. File is available for both writing and reading and the contents are lost if the file exists, If the file does not exist it is created and the file-pointer is positioned at the beginning of the file. a+ opened for update. File is available for both reading and writing and the contents are not lost. The file-pointer is positioned at the end of the file for appending. If the file does not exist it is created.

File-pointer is any user-defined variable of type FILE. FILE is a specific data type defined in the header file stdio.h. All file pointers are to be defined as type FILE.

The function fopen() returns a NULL if it is unsuccessful and , returns an address when successful. This successful return value is used to identify the file for performing input/output

Example:

```
FILE *fp;

fp=fopen ("test", "w");
```

Opens a file called test for writing. The return value of the function should generally be checked and the above statement will mostly look like

```
if (( fp = fopen ("test", 'w')) == NULL)
{printf ("cannot open file\n");
exit (0) ;}
```

This method detects any error in opening a file, such as write protected or a full disk.

2) Closing a File - fclose() Function

fclose() is used to close a file that was opened by a call to fopen()

All files must be closed before terminating the program. fclose() writes any data not yet written to the disk. The general form of fclose() is

```
fclose (file-pointer);
```

where file pointer is the return value of the call to fopen() and is used to identify which file is to be closed. fclose() returns NULL for a successful close and EOF (or) -1 for an unsuccessful close

3) Reading From a File

C supports the following functions which are used to read from a file. They are

fgetc ()

fgets ()

fscanf ()

fgetc () function

This function is used to read a character from a file which opened for reading. The general form is `variable = fgetc (file-pointer)`

Where filepointer is the return value of the fopen function and variable is any character variable which will store the character read from the file.

If the end of file is encountered during reading, fgetc() returns an end of file marker, so it is always advisable to check for the return value of this function for EOF i.e. -1 as follows

```
ch=fgetc(fp);
while(ch !=EOF)
{
    _____
    ch = fgetc(fp);
    _____
    _____}

```

Here **fp** is the file-pointer and **ch** is the character variable.

fgets() function

The fgets () function is used to read a string from a file opened for reading. This function requires the size of the destination array as an argument. The general form of this function is

`fgets (destination, no.of. bytes, filepointer);`

Where destination - the name of the array into which the string will be read

no.of.bytes - number of bytes to be read from the file

filepointer - the return value of the fopen () function

Example:

```

/* PROGRAM DEMONSTRATING THE USE OF FGETS ( ) */

#include <stdio.h>

main ( )

{char input [256],filename [20];

int ctr;

FILE *fp;

printf ("enter the file name \n");

gets (filename);

if( (fp=fopen (filename, "r")) == NULL)

{printf ("cannot open %s\n", filename);

exit (0) ;}

ctr =0;

while (fgets (input, 256, fp) != NULL)

{printf ("%s", input);

ctr + +; }

printf ("\n Number of bytes read = %d\n", ctr*256);

fclose(fp);}

```

fgets () function can also be also used to read information from the keyboard. This is achieved by replacing the file pointer with stdin which denoted the standard in - keyboard.

Example:

```
fgets (input, 256, stdin);
```

14.3.3. fscanf() function

fscanf () function is used to read various data formats from a file opened for reading. The general form of fscanf () function is

```
fscanf (filepointer, "control string", argument list);
```

where file pointer is the return value of `fopen ( )` function. The control string and argument list have the same format as in `scanf ( )` function.

Example:

```
fscanf (fp,"%s%d", str, &num);
```

This statement will read a string and an integer into the variable `str` and `num` respectively. The control string and the argument list should be equal as they are matched. `fscanf ( )` function can also be used to accept input from the keyboard by replacing the file pointer with `stdin`.

4) Writing to a File

Similar to reading from files, C supports the following functions which are used to write to a file. They are:

```
fputc ( )
```

```
fputc ( )
```

```
fprintf ( )
```

Now, we will see each function in detail.

fputc () function

This function writes a character to a file which is opened for writing. The general form of `fputc ( )` function is `fputc (character, filepointer);`

Where `character` is the character to be written to the file and the file pointer is the return value of the `fopen ( )` function.

Example:

```
fputc ('A', fp); /* writes A to the file */
```

```
fputc (ch,fp); /* writes the value contained in ch to the file */
```

fputs () function

The `fputs ( )` function is used to write a string to a file which is opened for writing. The general form of `fputs ( )` function is `fputs (string, filepointer)`

where string is the string to be written to the file and the file pointer is the return value of the `fopen ( )` function.

Example:

```
fputs ("DMS", fp) /* writes the string DMS to the file */
```

```
fputs(str, fp); /* writes the value contained in str to file */
```

fprintf() function

The `fprintf ( )` function is used to write various data formats to a file opened for writing. The general form of `fprintf ( )` function is.

```
fprintf (file pointer, control string, argument list);
```

where file pointer is the return value of the `fopen ( )` function and the control string and argument list have the same format and meaning as the `printf ( )` function.

Example

```
fprintf (fp, "%s %d", str, num);
```

This writes a string and an integer contained in the variables `str` and `num` respectively. The `fprintf( )` function can be used to write the output to the screen instead of the file, by replacing the file pointer with `stdout`, which denotes the standard output-the screen.

```
fprintf (stdout, "%s%d", str, num);
```

5) Reading and Writing Structures

File input/output frequently involves reading and writing structures from and to the file respectively. C provides two functions for this purpose. They are:

```
fread ( )
```

```
fwrite ( )
```

We will study them now in detail.

fread () function

This function reads a fixed number of bytes from a file which is opened for reading and stores into a structure whose address is supplied to the function. The general form of fread () function is:

```
fread (destination, size, no.of.units, file pointer);
```

Where destination is destination structure's address, size is the size of the to be read no.of.units is the number of structures to be read, usually 1 and the file pointer is the return value of the fopen fread () returns the no.of.units read if it is successful and a NULL if it is unsuccessful.

Example:

```
fwrite (&emp, sizeof (struct employee), 1 ,fp);
```

Will read a structure employee into the structure variable emp from a file. Use the size of operator to return the size of the structure.

fwrite() function

This function writes a fixed number of bytes to a file. It is normally used to write a structure to an output file. The general form of fwrite () function is:

```
fwrite (source, size, no.of.units, filepointer);
```

where source is the address of the source variable to be written, size is the size of the structure to be written, no.of.units is the no.of structures to be written, usually 1, file pointer is the return value of the fopen () function.

fwrite () returns no.of.units written to the file if successful and NULL if unsuccessful ,may be due to a full disk or some operating system problem..

Example:

```
fwrite (&emp, sizeof (struct employee), 1 ,fp);
```

6) Random Access - fseek () Function

Random read and write operations can be performed using high level I system with the help of fseek () function.

fseek () is used to set current (specified byte) position in a file. The general form of fseek () function is:

```
fseek (fp, offset, origin);
```

Where fp is the pointer returned by a call to fopen (), offset is number of bytes from the origin to access the current position, origin is one of the following to access the current position.

0 – from the start of the file

1 – from the current position

2 – from the end of file

Example:

To read the 234th byte in a file called "test"

```
fund 1 ( )
```

```
{FILE *fp;
```

```
if ((fp = fopen ("test", "r")) == NULL)
```

```
{Printf ("can not open file\n"); exit(0);}
```

```
fseek (fp,234,0); /* sets the pointer at 234th byte */
```

```
return getc(fp); /* reads and returns the character at 234th position */}
```

stdin and stdout

Normally, input and output I/O is directed to or from the terminal. C defines two file pointers in the file stdio.h that specify where the input is to be received from and where the output is to be sent. The pointers stdin and stdout represent standard input and standard output and are associated with your terminal by default.

If you are using UNIX or MS-DOS, you can use the pipe (|) to direct the output of one program to become the input of a second program. For example, the UNIX command is lists

the files contained in your directory. Rather than immediately writing the routines to the screen, the output is sent to the routine sort:

```
1s | sort
```

Once the files are sorted, they are written to the screen. When we issue the following command:

```
1s
```

the file pointer `stdout` is associated with our terminal. When we instead pipe the result of `1s` into! the routine `sort`, the file pointer `stdout` in the routine `1s` is modified so that all of the output is directed into `sort`. The file pointer `stdin` within the routine `sort` is updated to reference the result of the command `1s`.

UNIX also supports redirection operators that modify the pointers `stdin` and `stdout` in a manner similar to the pipe. In case, however, we are not redirecting out input or output from another program but rather to or front a file. For example, the following command runs the program `wc` (word count) the file `data.dat` and places the result into the file `result.data`

```
wc data.dat > result.data
```

The redirection symbol `>` specifies the destination of output of the program. We are assigning the file pointer to the file `result.dat` as opposed to our terminal. If we have program that reads to strings from the keyboard, we can redirect the program to read the strings from a file and not the user. The program will still use the routine `scanf ( )` to read the strings; however, the redirection operator will reassign the file pointer `stdin` to the file `string.dat` instead of to the keyboard:

```
getstrings < string.dat
```

If you list the file `stdio.h`, you will find that all of the routines are defined in terms of `stdin` and `stdout`. The routine `getchar ( )` is actually the following macro:

```
#define getchar ( ) getc(stdin)
```

It returns a character from the location referenced file pointer `stdin`. The routine `putchar ( )` is defined in terms `stdout` as follows:


```
#define putchar(x) putc (x, stdout)
```

Normally, then the pointers `stdin` and `stdout` are associated with the terminal, but, because all of the 1.0 routines are defined in terms of pointers, we gain added flexibility by redirecting input and output with the pipe or redirection operators

Format Conversions

When values are written or read formatted I/O, the values are converted by C to meet the format specified. For example, the variable `my_value` is read in from the keyboard with `scanf ( )`:

```
scanf ("%6.2", &my_value);
```

The ASCII value entered is converted to the equivalent floating point value automatically. Similarly, the routine `printf ( )` will perform formatted I/O.

Because C must provide these operations during I/O, C also allows us to convert values that are contained in memory. The two routines used in this process are `sscanf ( )` and `sprintf ( )`. The routine `sscanf ( )` converts a string to its numeric equivalent:

```
sscanf (string, control sequence, pointers);
```

The routine is similar to `sscanf ( )` in that it requires pointers to the variables it is to modify. The following statement, for example, assigns the variable `ivalue` the value 1123.

```
sscanf ("1123", "%d", &ivalue);
```

The routine `sscanf ( )` will return the value EOF if an error occurs.

The routine `sprintf ( )` performs the opposite function: it writes the value contained in the variable into a string. For example, if the variable `ivalue` contains the number 1007, and we want to write it out as

```
1 0 0 7
```

then we can convert the value to a string and perform the character I/O required

```
sprintf (string, "%d", ivalue);
```

The value returned from `sprintf ( )` is the number of characters written into the string. You should always test the status codes returned by these routines to verify success.

Examples:

A program using high-level I/O that will copy one disk file to another, using command line arguments to specify the files.

```
#include <stdio.h>

main (argc, argv)
int argc;
char *argv++; ++
{char ch;
FILE *fp1,*fp2;
if ((fp1 = fopen (argv [1], "r")) == 0)
{printf ('can not open file %s\n", argv[1]; exit(0);}
if ((fp2=fopen (argv [1], "w" )) == 0)
{printf ("can not open file %s\n", argv [2]) ;exit(0) ;}
while ((ch=getc(fp1)) !=EOF)
putc (ch,fp2);
fclose (fp1); fclose (fp2);}
```

Example

```
#include <stdio.h>

main (argc, argv)
int argc;
char *argv++;
FILE *fp;
char ch;
```

```

if(argc !=2)
{printf ("You forgot to enter the filename\n"); exit(0);}
if ((fp=fopen (argy [1] ,"r")) ==NULL)
{printf ("can not open file\n"); exit(0);}
ch = getc (fp); /*read one character*/
while (ch!= EOF)
{putchar(ch); ch=getc(fp);}
fclose (fp) ;}

```

Example

```

/*PROGRAM FOR DEMONSTRATING fprintf AND fscanf( ) FUNCTION */
#include <stdio.h>
main ( )
{FILE *fp;
char s [40];c
int t;
if((fp = fopen ("test", "w")) ==NULL)
{printf ("cannot open ffile\n");
exit(0);}
fscanf (stdin, "%s%d", s,&t);
fprintf (fp, "%s%d",s,t),
fclose (fp)
if( (fp=fopen"test","r")) ==NULL)
{printf ("cannot open file\n");
exit(1); }
fscanf (fp,"%s%d",s,&t); /* read from the file */
fprintf (stdout,"%s%d",s,t); /* write to the screen */
fclose (fp);}

```

Example

```

/* PROGRAM DEMONSTRATING fread( ) AND fwrite() FUNCTIONS */

#include <stdio.h>

struct employee
{
    int eno;
    char enam
    int sal;};
FILE *fp;

struct employee emp;

main ( )
{
    if ((fp=fopen ("emp.dat", "w")) == NULL)
    {
        printf ("cannot open file\n");
        exit (1); }
    while (1)
    {
        printf ("enter employee number\n");
        scanf ("%d", & emp .eno);
        if (emp.eno== 0) break;
        printf ("Enter Employee name\n");
        gets(emp.enam);
        printf ("Enter salary\n");
        scanf ("%d", & emp.sal);
        if( (fwrite (&emp, sizeof (struct employee), 1,fp) ) != 1)
        {
            printf( "Record not written\n");
            exit (2);}
    }
    /* r end of while loop */
    fclose (fp);
    if ((fp=fopen ("emp.dat", "r")) == NULL)
    {
        printf ("Error in opening file\n");
        exit (3);}
}

```

```

while (1)
{if ((fread(&emp,sizeof(struct employee)1,fp)) !=1)
{printf("Record not read\n");
break;}

printf ("%d\t%s\t%d\n", emp.eno, emp.enam, emp.sal) ;}

fclose(fp);}

```

Check Your Progress – I

- 1) *State whether the following are True or False.*
 - a) File is a collection of Records.
 - b) fputc
- 2) *Fill in the blanks*
 - a) The General form of array is _____
 - b) The General form of the dimensional array is _____

14.4 Summary

File is a collection of records. In this lesson we have covered the basic operations of file handling. And also we have prepared some programs using file concepts in c language.

14.5 Check Your Answer

- I.
 - 1) a) True b) False
 - 2) a) read b) fputs () c) fprintf ()

14.6 Model Questions

- (1) What is a file pointer?
- (2) What routine returns a file pointer? Explain the arguments associated with this routine?
- (3) What declarations are required to open the file data.dat?
- (4) What are the modes a file can be opened in? How does each affect the file pointer and any previous data contained within the file?

- (5) What value is returned from the routine `fopen ( )` if the file cannot be opened?
- (6) What functions support character I/O to files?
- (7) Why must we include the file `stdio.h` when we perform file manipulation?
- (8) What are the two routines that perform formatted I/O to files? How do they differ from `printf ( )` and `scanf ( )`?
- (9) Write a program that uses `getc( )` and `putc ( )` to create the routines `fgetc( )` and `fputc ( )` ?
- (10) Write a program that counts the number of characters, lines and words contained in a file.
- (11) Write a program that compares two files. Output the line and character location of the first difference in the files.

LESSON - 15

STRUCTURE

Structure

15.1 Introduction

15.2 Learning Objectives

15.3. Structures

15.4 Summary

15.5 Check Your Answers

15.6 Model Questions

15.1 Introduction

This lesson is convened with the use of structure within a C program. We will see how structures are defined, and how their individual member are accessed and processed within a program. The relationships between structure and pointers, arrays and functions will also be examined.

15.2 Learning Objectives

After studying this lesson, you should be able to

- explain use of structures, which allow us to combine several variables of different types into a single entity 'We have also examined the features of structures which lead to programming convenience.

15.3 Structures

Which mechanic is good enough who knows how to repair only one type of vehicle? None. Same thing is true about C language. It wouldn't have been so popular had it been able to handle only all **ints**, or all **floats** or all **chars** at a time. In fact when we handle real world data, we don't usually deal with little atoms of information by themselves/things like integers, characters and such. Instead we deal with entities that are collections of things, each thing

having its own attributes, just as the entity we call a 'book' is a collection of things such as title, author, call number, publisher, number of pages, data of publication etc. As you can see all this data is dissimilar, for example author is a string, whereas number of pages is an integer. For dealing with such collections, C provides a data type called 'structure'. A structure gathers together, different items of information that comprise a given entity

1) Why Use Structures

We have seen earlier how ordinary variables can hold one piece of information and how arrays can hold a number of pieces

For example, suppose you want to store data about a book. You might want to store its name (a string), its price (a float) and number of pages in it (an int). If data about say 3 such books is to be stored, then we can follow two approaches:

- (a) Construct individual arrays, one for storing names, another for storing prices and still another for storing number of pages.
- (b) Use a structure variable.

Let us examine these two approaches one by one. For the sake of programming convenience assume that the names of books would be single character long.

```
main ( )
{
char name [3];
float price [3] ;
int pages [3]
printf ("\nEnter names, prices and no. of pages of 3 books\n");
for (i = 0; i <= 2; i++)
scanf ("%c %f %d", &name [i], &price [i], &pages [i]);
printf ("\nAnd this is what you entered\n");
for (i = 0; i <= 2; i++)
```



```
printf ("%c%f %d\n", name [i], price [i] pages[i] ) ;}
```

And here is the sample run...

Enter names, prices and no. of pages of 3 books

A 100.00 354

C 256.50 682

F 233.70 512

And this is what you entered

A 100.000000 354

C 256.500000 682

F 233.700000 512

This approach no doubt allows you to store names, prices and number of pages. But as you must have realised, it is an unwieldy approach that obscures the fact that you are dealing with a group of characteristics related to a single entity: the book.

The program becomes more difficult to handle as the number of items relating to the book go on increasing. For example, we would be required to use a number of arrays, if we also decide to store name of the publisher, date of purchase of book etc. To solve this problem, C provides a special data type... the structure.

A structure contains a number of data types grouped together. These data types may or may not be of the same type. The following example illustrates the use of this data type.

```
main ( )
{
    struct book
    {
        char name;
        float price;
        int pages;
    };
    struct book b1, b2, b3;
```

```

printf ( names, prices & no. of pages of 3 books\n");
scanf ("%c %f %d", &b1.name, &b1.price, &b1.pages);
scanf ("%c %f %d", &b2.name, &b2.price, &b2.pages);
scanf ("%c %f %d", &b3.nanie, &b3.price, &b3.pages);
printf ("\nAnd this is what you entered");
printf ("\n%c %f %d", b1.name, b1.price, b1.pages);
printf ("\n%c %d %f", b2.name, b2.price, b2.pages);
printf ("\n%c %d %f", b3.name, b3.price, b3.pages);}

```

And here is the output ...

Enter names, prices and no. of pages of 3 books

A 100.00 354

C 256.50 682

F 233.70 512

And this is what you entered

A 100.000000 354

C 256.500000 682

F 233.700000 512

This program demonstrates two fundamental aspects of structures:

(a) declaration of a structure

(b) accessing of structure elements

Let us now look at these concepts one by one.

2) Declaring a Structure

In our example program, the following statement declares the structure type:

```
struct book
```

```
{char name;
float price;
int pages;};
```

This statement defines a new data type called **struct book**. Each variable of this data type will consist of a character variable called **name**, a float variable called price and an integer variable called **pages**. The general form of a structure declaration statement is given below:

```
struct <structure name>
{structure element 1;
structure element 2;
structure element 3;
....
....};
```

Once the new structure data type has been defined one or more variables can be declared to be of that type. For example the variables b1, b2, b3 can be declared to be of the type struct book, as, struct book b1, b2, b3;

This statement sets aside space in memory. It makes available space to hold all the elements in the structure: in this case, 7 bytes... one for **name**, four for **price** and two for **pages**. These bytes are always in adjacent memory locations.

If we so desire, we can combine the declaration of the structure type and the structure variables in one statement.

```
For example,
struct book
{ char name;
float price;
int pages;};
```

```
struct book b1, b2, b3;
```

is same as....

```
struct book
```

```
{char name;
```

```
float price;
```

```
int pages;}
```

```
b1, b2, b3;
```

or even...

```
struct
```

Like primary variables and arrays, structure variables can also be initialised where they are declared. The format used quite similar to that used to initiate arrays.

```
{char name; float price; int pages;}
```

```
b1, b2, b3;
```

```
struct book
```

```
{char name [0]
```

```
float price;
```

```
int pages;};
```

```
struct book b1 = {"Basic", 130.00, 550};
```

```
struct book b2 = {"Physics", 150.80, 800};
```

Note the following points while declaring a structure type:

- (a) The closing brace in the structure type declaration must be followed by a semicolon.
- (b) It is important to understand that a structure type declaration does not tell the compiler to reserve any space in memory. All a structure declaration does is, it defines the 'form' of the structure.

- (c) Usually structure type declaration appears at the top of the source code file, before any variables or functions are defined. In very large programs they are usually put in a separate header file, and the file is included (using the preprocessor directive **#include**) in whichever program we want to use this structure type.

```
char name; float price; int pages;}

b1, b2, b3;

struct book

{char name [0]

float price;

int pages;};

struct book b1 = {"Basic", 130.00, 550};

struct book b2 = {"Physics", 150.80, 800};
```

Note the following points while declaring a structure type:

- (d) The closing brace in the structure type declaration must be followed by a semicolon.
- (e) It is important to understand that a structure type declaration does not tell the compiler to reserve any space in memory. All a structure declaration does is, it defines the 'form' of the structure.
- (f) Usually structure type declaration appears at the top of the source code file, before any variables or functions are defined. In very large programs they are usually put in a separate header file, and the file is included (using the preprocessor directive **#include**) in whichever program we want to use this structure type.

3) Accessing Structure Elements

Having declared the structure type and the structure variables, let us see how the elements of the structure can be accessed.

In arrays we can access individual elements of an array using .a subscript. structures use a different scheme. They use a dot (.) operator. So to refer to **pages** of the structure defined in our sample program we have to use,

b1.pages

Similarly, to refer to **price** we would use,

b1.price

Note that before the dot there must always be a structure variable and after the dot there must always be a structured element.

4) How Structure Elements are Stored

Whatever be the elements of a structure, they are always stored in contiguous memory locations. This can be illustrated by the following program:

```
/* Memory map of structure elements */  
  
main( )  
  
{struct book  
  
{char name;  
  
float price;  
  
int pages;};  
  
struct book b1 = {'B', 130.00, 550};  
  
printf ("\nAddress of name = %u", &b1.name);  
  
printf ("\nAddress of price = %u", &b1.price)  
  
printf ("\nAddress of pages = %u", &b1.pages);}
```

Here is the output of the program....

Address of name = 1001

Address of price = 1002

Address of pages = 1006

Actually the structure elements are stored in memory as shown in the following:

b1.name	b1.price	b1.pages
'B'	130.00	550
1001	1002	1006

5) Array of Structures

Our sample program showing usage of structure is rather simple minded. All it does is, it receives values into various structure elements and output these values. But that's all we intended to do anyway... show how structure types are created, how structure variables are declared and how individual elements of a structure variable are referenced.

In our sample program, to store data of 100 books we would be required to use 100 different structure variables from **b1** to **b100**, which is definitely not very convenient. A better approach would be to use an array of structures. Following program shows how to use an array of structures.

```
/* Usage of an array of structures */
main ( )
{struct book
{char name;
float price;
int pages;};
struct book b [100];
int i;
for(i = 0; i<=99; i++)
{printf ("\nEnter name, price and pages");
```

```
scanf ("%c %f %d", &b[i].name, &b[i].price, &b[i].pages);}
for (i = 0; i <= 99; i++)
printf ("\n%c %f%d". b[i].name, b[i].price, b[i].pages);}
```

Now a few comments about the program:

- (a) Notice how the array of structures is declared...

```
struct book b(100);
```

This provides space in memory for 100 structures of the type **struct book**.

- (b) The syntax we use to reference each element of the array **b** is similar to the syntax used for arrays of **ints** and **chars**. For example, we refer to zeroth book's price as **b[10] price**. Similarly, we refer first book's pages as **b[1].pages**.
- (c) It should be appreciated what careful though Dennis Ritchie has put into C language. He first defined array as a collection of similar elements; then realised that dissimilar data types which are often found in real life cannot be handled using arrays, therefore created a new data type called structure. But even using structures programming convenience could not be achieved, because a lot of variables (**b1** to **b100** for storing data about hundred books) needed to be handled. And therefore he allowed us to create an array of structures; an array of similar data types which themselves are a collection of dissimilar data types. Hats off to the genius!
- (d) In an array of structures all elements of the array are stored in adjacent memory locations. Since each element of this array is a structure, and since all structure elements are always stored in adjacent locations you can very well visualise the arrangement of array of structures in memory. In our example, **b[10]'s name, price and pages** in memory is immediately followed by **b[1]'s name, price and pages**, and so on.

6) Additional Features of Structures

Let us now explore the intricacies of structures with a view of programming convenience. We would highlight these intricacies with suitable examples:

- (a) The values of a structure variable can be assigned to another structure variable of the same type using the assignment operator. It is not necessary to copy the structure elements

piece-meal. Obviously, programmers prefer assignment to piece-meal copying. This is shown in the following example.

```
main ( )
{struct employee
{char name [10];
int age;
float salary;};
struct employee e1 = "Sanjay", 30, 5500.50};
struct employee e2, e3;
/* piece-meal copying */
strcpy (e2.name, e1.name);
e2.age = e1.age;
e2.salary = e1.salary;
/* copying all elements at one go */
e3 = e2;
print ("\n%s %d %f", e1.name, e1.age, e1.salary);
printf ("\n%s %d %f", e2.name, e2.age, e2.salary);
print1 ("\n%s %d %f", e3.name, e3.age, e3.salary);}
```

The output of the program would be...

```
Sanjay 30      5500.500000
Sanjay 30      5500.500000
Sanjay 30      5500.500000
```

Ability to copy the contents of all structure elements of one variable into the corresponding elements of another structure variable is rather surprising, since C does not allow assigning the contents of one array to another just by equating the two. As we saw earlier, for copying arrays we have to copy the contents of the array elements by element.

This copying of all structure elements at one go has been possible only because the structure elements are stored in contiguous memory locations. Had this not been so, we would have been required to copy structure variables element by element. And who knows, had this been so, structures would not have become popular at all.

- (b) One structure can be nested within another structure. Using this facility complete data types can be created. The following program shows nested structures at work.

```
main ( )
{
    struct address
    {
        char phone [15];
        char city [25];
        int pin;
    };
    struct emp {
        char name[25];
        struct address a;
    };
    struct emp e = ("jeru" "531046", "nagpur" 10);
    print ("\nname = %s phone = %s", e.name, e.a.phone)
    print ("\ncity %s pin = %d", e.a.city, e.a.pin)}

```

And here is the output...

name = jeru phone = 531046

city = nagpur pin = 10

Notice the method used to access the elements of a structure that is a part of another structure. Here, the dot operator is used twice, as in the expression,

e.a.pin or e.a.city

Of course, nesting process need not stop at this level. We can nest a structure within a structure, within another structure, which is in still another structure and so on... till the time we can comprehend the structure ourselves. Such construction however gives rise to variable names that can be surprisingly self descriptive, for example:

maruti.engine.bolt.large.qty

This clearly signifies that we are referring to the quantity of large sized bolts which fit on an engine of a maruti car.

- (c) Like an ordinary variable, a structure variable can also be passed to a function. We may either pass individual structure elements or the entire structure at one go. Let us examine both the approaches one by one using suitable programs.

```

/* Passing individual structure elements */

main ( )

{struct book

{char name [25];

char author [25];

int callno;};

struct book b1 = {"Let us C", "YPK", 101};

display (b1.name, b1.author, b1.callno);}

display (char *S, char *t, int n)

{printt ('n%s %s %d", s, t, n);}

```

And here is the output...

Let US C YPK 101

Observe that in the declaration of the structure, **name** and **author** have been declared as arrays. 'Therefore, when we call the function **display ()** using,

```
display (b1.name, b1.author, b1.callno);
```

We are passing the base addresses of the arrays **name** and **author**, but the value stored in **callno**. Thus, this is a mixed call: a call by reference as well as a call by value.

It can be immediately realised that to pass individual elements would become more tedious as the number of structure elements go on j P. better way would be to pass the entire structure variable at a time. This method is shown in the following program.

```

Struct book

{char name [25];

char author[25];

int callno;};

```

```

main ( )

{struct book b1 = {"Let us C", "YPK", 101};

display (b1);}

display (struct book b)

{printf ("\n%s %s %d", b.name, b.name, b.author, b.callno);}

```

And here is the output...

Let us C YRK 101

Note that here the calling of function **display ()** becomes quite compact,

```
display (b);
```

Having collected what is being passed to the **display ()** function, the question comes, how do we define the formal arguments in the function. We cannot say,

```
Struct book b1;
```

because the data type **struct book** is not known to the function **display()**. Therefore, it becomes necessary to define the structure type **struct book** outside **main ()**, so that it becomes known to all functions in the program.

- (d) The way we can have a pointer pointing to an **int**, or a pointer pointing to a **char**, similarly we can have a pointer pointing to a **struct**. Such pointers are known as 'structure pointers'.

Let us look at a program which demonstrates the usage of a structure pointer.

```

main ( )

{struct book

{char name[25];

char author [25];

int callno;};

struct book b1 ={"Let us C", "YPK", 101}

struct book*ptr;

```

```
ptr = &b1;

printf ("\n%s%s%d", b1.name, b1.author, b1.callno);

printf ("\n%s%s%d", ptr->name, ptr->author, ptr->callno) ; }
```

The first **printf** is as usual. The second **printf ()** however is peculiar. We can't use **ptr.name** or **ptr.callno**

Because **ptr** is not a structure variable but a pointer to a structure, and the dot operator requires a structure variable on its left. In such cases C provides an operator **->**, called an arrow operator to refer to the structure elements. Remember that on the left hand side of the **'.'** structure operator, there must always be a structure variable, whereas on the left hand side of the **'->'** operator there must always be a pointer to a structure. The arrangement of the structure variable and pointer to structure in memory is shown in the fig given below.

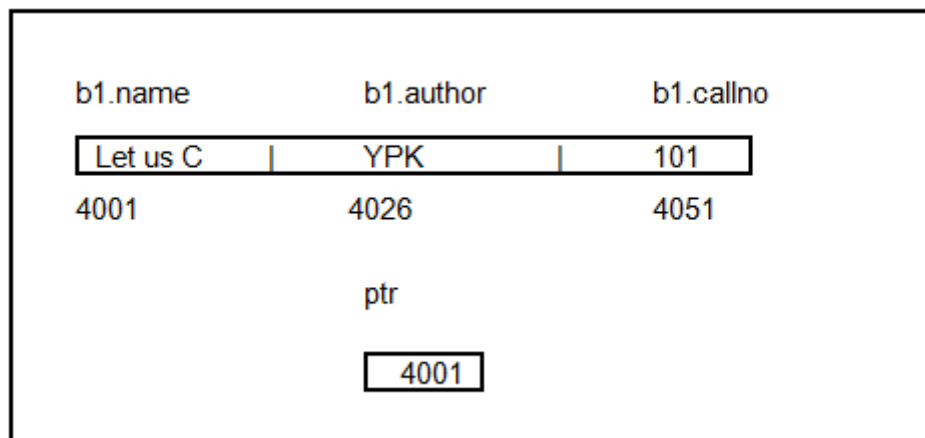


Fig 15.2

Can we not pass the address of a structure variable to a function? We can. The following program demonstrates this.

```
/ Passing address of a structure variable */

struct book

{char name [25];
```

```

char author [25];

int callno;};

main ( )

(struct book b1 ={"Let us "C", "YPK", 101};

display (&b1);}

display (struct book *b)

{printf {"\n%s%s%d", b->name, b->author, b->callno);}

```

And here is the output

Let us C YPK 101

Again note that to access the Structure elements using pointer to a structure we have to Use the '->' operator

Also the structure **struct book** should be declared outside **main()**

Such that this data type is available to **display ()** while declaring pointer to the structure

7) Uses of Structures

Where are structures useful? The, immediate application that comes to the mind is Database Management. That is, to maintain data about employees in an organisation books in a library, items in a store, financial accounting transactions in a company etc. But mind you, use of structures stretches much beyond database management. They can be used for a variety of applications like:

- (a) changing the size of the cursor
- (b) clearing the contents of the screen
- (c) placing the cursor at appropriate position on screen
- (d) drawing any graphics shape on the screen
- (e) receiving a key from the keyboard
- (f) checking the memory size of the computer

- (g) finding out the list of equipment attached to the computer
- (h) formatting a floppy
- (l) hiding a file from the directory
- (j) displaying the directory of a disk
- (k) sending the output to printer
- (1) interacting with the mouse

And that is certainly a very impressive list! At least impressive enough to make you realise how important a data type a structure is and to be thorough with it if you intend to program any of the above applications.

Check Your Progress – I

1) *State whether the following are True or False :*

- a) Structure is a collection of different type of data elements
- b) The closing brace in the structure type declaration must be followed by a semicolons

2) *Fill in the blanks*

- a) C Provides an operator $\rightarrow$, called an _____ operator to refer to the structure elements
- b) _____ is a structure operator.
- c) _____ contains a number of data types grouped together.

15.4 Summary

In this lesson we have covered the use of structures, which allow us to combine several variables of different types into a single entity. We have also examined the features of structures which lead to programming convenience. We concluded this lesson by making you aware of what different applications the structure can be put to.

15.5 Check Your Answers

- I. 1) a) True b) True
- 2) a) arrow b) • (Dot) c) Structure

15.6 Model Question

1. What is a structure? How does a structure differ from an array?
2. What is a member? What is the relationship between a member and a structure?
3. How can structure variable be declared? How do structure variable declaration differ from structure type declaration?

MODEL QUESTIONS
BACHELLOR OF COMPUTER APPLICATIONS
FIRST YEAR - SECOND SEMESTER
COMPUTER SEMESTER
Core Paper - III
PROGRAMMING IN C

Time : 3 Hrs.

Max marks 75

Section A (50 words)
Answer any ten questions

(10 x 2 = 20)

1. Define constant and variable.
2. State the Difference between do-while and while – do looping Statement.
3. What is Scope of the variable?
4. What is a function?
5. Define an array.
6. What is a recursive function?
7. Name any two String Function.
8. Explain the parameter used in the Main() function.
9. Define pointer.
10. What is a stream?
11. What is file?
12. What are the basic operations carried out in file?

Section B (250 Words)

Answer any five questions

(5 x 5 = 25)

13. Discuss about the operators available in C.
14. Write a program to find biggest no. among three numbers.
15. Write a C program to sort a list of numbers using array.
16. Discuss the features of C programming.
17. Discuss the difference between Structure and Union with example.
18. Write a C program to generate Fibonacci series using recursive function.
19. Explain the creation and processing of Student file.

Section C (500 Words)

Answer any three questions

(3 x 10 = 30)

20. Explain the different types of Control and looping statements with example.
 21. Discuss any five string functions with example.
 22. Write a program to process income tax information using Structure.
 23. Explain the definition of function and pass the parameters as call by value and call by reference.
 24. Write a C Programming in to implement the employee file with necessary operations
-